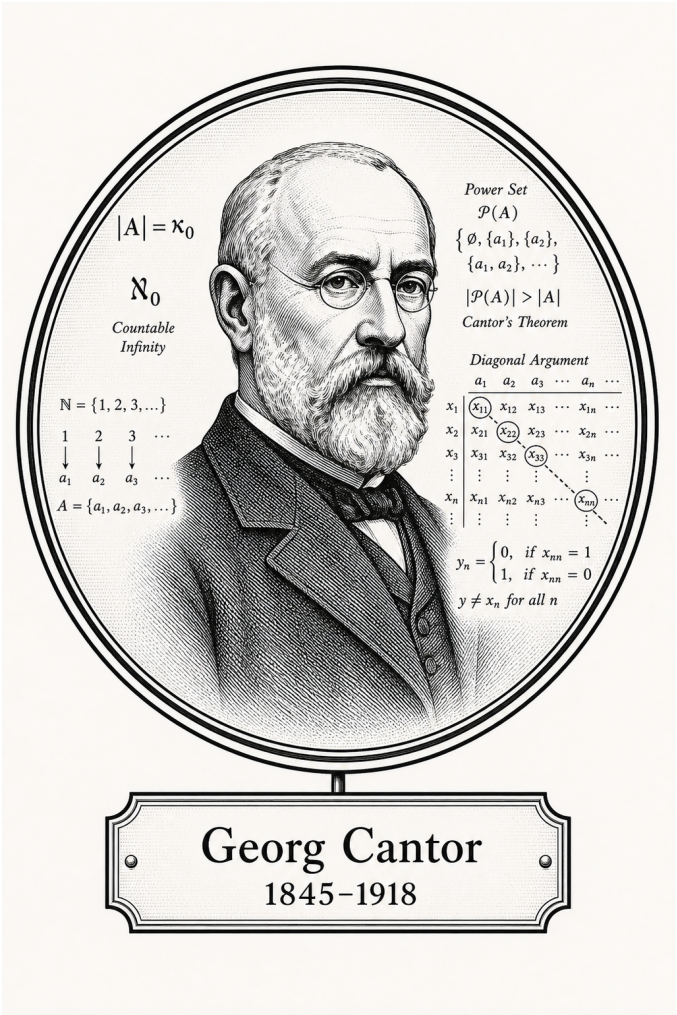


FROM CANTOR TO ITO

Logic, Sets, and Proof



Georg Cantor
1845-1918

Self-Study Notes

William Sollers

July 1, 2026

For every reader learning to make mathematics their own.

Contents

I	Logic, Sets, and Proof	1
1	Propositional Logic	2
1.1	Model-Theoretic View	2
1.2	Axiom Schemas	4
1.3	Notation and Conventions	4
1.4	Syntax	7
1.5	Semantics	20
1.6	Algebra of Propositions	32
1.7	Normal Forms	39
1.8	Functional Completeness	58
1.9	Argument Forms and Informal Proof Patterns	70
1.10	Compactness and Finite Satisfiability Preview	82
2	Predicate Logic	90
2.1	Model-Theoretic View	90
2.2	Axiom Schemas	92
2.3	Syntax	93
2.3.1	Syntax of First-Order Logic: Terms and Formulas	93
2.3.2	Syntax: Free Variables, Bound Variables, and Substitution	101
2.3.3	Syntax: Formula Depth	105
2.4	Semantics	106
2.4.1	Semantics: Structures and Variable Assignments	106
2.4.2	Semantics: Satisfaction and Truth	110
2.4.3	Semantics: Models, Theories, and Logical Consequence	114
2.4.4	Semantics: Open Formulas, Definable Relations, and Substitution Lemmas	117
2.5	Quantifiers	120
2.6	Proof Theory	134
2.6.1	Proof Theory: Quantifier Inference Rules	135
2.6.2	Proof Strategies for Quantified Statements	137
2.6.3	Proof Theory: Equality Rules	141
2.6.4	Proof Theory: Soundness and Completeness	143
2.7	Translation	145
2.8	Reference	154

3	Many-Sorted Logic	157
3.1	Introduction	159
3.1.1	Examples	159
3.1.2	Reduction to and comparison with first-order logic	159
3.1.3	Uses of many-sorted logic	159
3.1.4	Many-sorted logic as a unifier logic	159
3.2	Structures	159
3.2.1	Definition (signature)	159
3.2.2	Definition (structure)	159
3.3	Formal many-sorted language	159
3.3.1	Alphabet	159
3.3.2	Expressions: formulas and terms	159
3.3.3	Remarks on notation	159
3.3.4	Abbreviations	159
3.3.5	Induction	159
3.3.6	Free and bound variables	159
3.4	Semantics	159
3.4.1	Definitions	159
3.4.2	Satisfiability, validity, consequence and logical equivalence	159
3.5	Substitution of a term for a variable	159
3.6	Semantic theorems	159
3.6.1	Coincidence lemma	159
3.6.2	Substitution lemma	159
3.6.3	Equals substitution lemma	159
3.6.4	Isomorphism theorem	159
3.7	The completeness of many-sorted logic	159
3.7.1	Deductive calculus	159
3.7.2	Syntactic notions	159
3.7.3	Soundness	159
3.7.4	Completeness theorem (countable language)	159
3.7.5	Compactness theorem	159
3.7.6	Löwenheim–Skolem theorem	159
3.8	Reduction to one-sorted logic	159
3.8.1	The syntactical translation (relativization of quantifiers)	159
3.8.2	Conversion of structures	159
4	Standard Second-Order Logic	160
4.1	Introduction	162
4.1.1	General idea	162
4.1.2	Expressive power	162
4.1.3	Model-theoretic counterparts of expressiveness	162
4.1.4	Incompleteness	162
4.2	Second-order grammar	162
4.2.1	Definition (signature and alphabet)	162
4.2.2	Expressions: terms, predicates and formulas	162

4.2.3	Remarks on notation	162
4.2.4	Induction	162
4.2.5	Free and bound variables	162
4.2.6	Substitution	162
4.3	Standard structures	162
4.3.1	Definition of standard structures	162
4.3.2	Relations between standard structures established without the formal language	162
4.4	Standard semantics	162
4.4.1	Assignment	162
4.4.2	Interpretation	162
4.4.3	Consequence and validity	162
4.4.4	Logical equivalence	162
4.4.5	Simplifying our language	162
4.4.6	Alternative presentation of standard semantics	162
4.4.7	Definable sets and relations in a given structure	162
4.4.8	More about the expressive power of standard SOL	162
4.4.9	Negative results	162
4.5	Semantic theorems	162
4.5.1	Coincidence lemma	162
4.5.2	Substitution lemma	162
4.5.3	Isomorphism theorem	162
5	Languages, Axioms, and Models	163
5.1	Signatures	163
5.2	Axioms	167
5.3	Theories	170
5.4	Models	173
5.5	Consistency	179
5.6	Independence	181
5.7	Comparing Theories	182
5.8	Examples And Previews	182
5.9	Summary	182
6	Proof Techniques	183
6.1	Proof Architecture	183
6.2	The Proof Construction Algorithm	189
6.3	Proof Structures	192
6.4	Mathematical Induction	199
6.5	Algebraic Tactics	209
6.6	Proof Strategies Reference	212
6.7	Analysis: What Changes and Why	214

7	Set Theory	225
7.1	Model of Set Theory	225
7.2	Sets	229
7.2.1	Sets, Membership, and ZFC Axioms	229
7.2.2	Set Constructions and Operations	232
7.2.3	Algebraic Laws of Set Operations	237
7.3	Families	240
7.3.1	Indexed Families of Sets	240
7.3.2	Covers, Subcovers, and the Finite Intersection Property	243
8	Relations	247
8.1	Relations — Quick Reference	247
8.1.1	Ordered Pairs and Relations	248
8.1.2	Properties of Relations	257
8.2	Equivalence	263
8.2.1	Equivalence Classes and Partitions	263
9	Functions	268
9.1	Functions — Quick Reference	269
9.1.1	Functions	269
9.1.2	Composition, Inverses, and Set-Image Laws	277
10	Orderings	285
10.1	Order	285
10.1.1	Ordered Sets	285
10.1.2	Supremum and Infimum	292
10.1.3	Lattices	296
10.1.4	Hasse Diagrams and the Duality Principle	300
10.1.5	Order Extensions	304
10.1.6	Induced Orders and Order Embeddings	308
10.2	Zorn	312
10.2.1	Zorn’s Lemma and the Axiom of Choice	312
11	Cardinality	314
11.1	Cardinality	314
11.1.1	Cardinality and Infinite Sets	314
12	Euclidean Geometry	321
12.1	Foundations	321
12.2	Compass-and-Straightedge Constructions	328
13	Trigonometry	333
13.0.1	Trigonometric Functions from Euclidean Geometry	333

14 Analytical Geometry	339
14.1 Analytical Geometry	339
14.2 Lines	339
A Lean 4 Formalization	343

Part I

Logic, Sets, and Proof

Chapter 1

Propositional Logic



1.1 Model-Theoretic View

Toolkit

This section presents classical propositional logic as a syntax-plus-semantics model, with the syntactic language connected to the two-valued semantics by the shared free-algebra evaluation machinery.

Theory (Propositional Logic)

Depends on: Propositional language; formula; bridge:consequence; Cn_+

Language

$$\mathcal{L}_P = (P, \neg, \wedge, \vee, \rightarrow, \leftrightarrow), \quad \text{Form}_P.$$

Presented Theory

$$T := Cn_+(\Sigma), \quad \Sigma \subseteq \text{Form}_P, \quad \text{CPL} := Cn_+(\emptyset).$$

Theorems and Consequences

$$1. \quad \Gamma \vdash_{\text{CPL}} \varphi \iff \Gamma \models_{\text{CPL}} \varphi \quad (\text{completeness}).$$

Model (Classical Propositional Logic)

Model 1.1.1 (Classical Propositional Logic). **Depends on:** Propositional language; valuation; bridge:free-sigma-algebra-evaluation; bridge:consequence

Note

The seed is the model datum itself.

$$v : \mathbf{P} \rightarrow |\mathbf{2}|, \quad |\mathbf{2}| = \{0, 1\}.$$

Binding

$$\Sigma := \Omega, \quad X := \mathbf{P}, \quad \text{target} := \mathbf{2}, \quad \sigma := v, \quad \widehat{v} := \text{eval}(\mathbf{2})(v).$$

Syntax–Semantics Bridge

1.

$$v \models \varphi \iff \widehat{v}(\varphi) = 1.$$

Theorems and Consequences

1. Connective recursion is inherited from *Free Σ -Algebra and Evaluation*.
2. Semantic consequence, validity, and $\text{Th}(v)$ are inherited from *Consequence*.

Definitional Extension (Propositional Logic)

Depends on: theory:propositional-logic; truth-function; conservative extension

Admissibility

A connective $\star$ of arity n is admissible by a defining equivalence:

$$\star(\bar{\varphi}) \leftrightarrow \theta(\bar{\varphi}), \quad \widehat{v}(\star(\bar{\varphi})) = F_{\star}(\widehat{v}(\varphi_1), \dots, \widehat{v}(\varphi_n)).$$

There is no $\exists!$ obligation because formulas already evaluate to truth values.

Instances

$$\begin{aligned} \top &:= (p_0 \rightarrow p_0), & \perp &:= \neg \top, \\ \varphi \oplus \psi &:= ((\varphi \vee \psi) \wedge \neg(\varphi \wedge \psi)). \end{aligned}$$

Theorems and Consequences

1.

$$\text{CPL}^+ \vdash \sigma \iff \text{CPL} \vdash \sigma \quad (\sigma \in \mathcal{L}_{\mathbf{P}}).$$

2. Every extended formula has a base-language equivalent.

1.2 Axiom Schemas

Propositional Axiom Schemas

The propositional-logic development uses the following classical Hilbert-style axiom schemas as logical starting points. They are schemas: each substitution of formulas for the displayed schematic letters gives an axiom instance.

Axiom Schema (Propositional Weakening)

Axiom 1.2.1 (Propositional Weakening). For all formulas P and Q ,

$$P \implies (Q \implies P).$$

Remark (Interpretation). If P is available, then it remains available under any additional assumption Q .

Axiom Schema (Implication Distribution)

Axiom 1.2.2 (Implication Distribution). For all formulas P , Q , and R ,

$$(P \implies (Q \implies R)) \implies ((P \implies Q) \implies (P \implies R)).$$

Remark (Interpretation). Implications may be composed under a common hypothesis.

Axiom Schema (Classical Contraposition)

Axiom 1.2.3 (Classical Contraposition). For all formulas P and Q ,

$$(\neg Q \implies \neg P) \implies (P \implies Q).$$

Remark (Interpretation). This is the classical contrapositive principle. It is the axiom schema that marks the background logic as classical rather than intuitionistic.

1.3 Notation and Conventions

Toolkit

This notation ledger records the propositional symbols, metavariables, semantic notation, and normal-form abbreviations used in the chapter.

Remark (Exposition). The notation section fixes the symbolic vocabulary used throughout the chapter. It identifies which symbols are global conventions, which are local to propositional logic, and where each item is introduced by the formal development.

Symbol	Name	Meaning	Introduced by
$\mathbb{B}$	truth-value set	the set of truth values (global, canonical)	Truth Value
Prop	propositional variable set	the set of propositional variables (global, canonical)	Propositional Language
$\neg$	negation	unary propositional connective (global, canonical)	Propositional Language
$\wedge$	conjunction	binary propositional connective (global, canonical)	Propositional Language
$\vee$	disjunction	binary propositional connective (global, canonical)	Propositional Language
$\rightarrow$	conditional	binary propositional connective (global, canonical)	Propositional Language
$\leftrightarrow$	biconditional	binary propositional connective (global, canonical)	Propositional Language
P, Q, R	propositional variables	schematic propositional variables (chapter, canonical)	Propositional Variable
$P_1, \dots, P_n$	finite list of propositional variables	a finite schematic list of propositional variables (chapter, local)	Propositional Variable
$\circ$	generic binary connective	an arbitrary member of $\{\wedge, \vee, \rightarrow, \leftrightarrow\}$ (global, canonical)	Logical Connective
WFF	well-formed formula set	the set of propositional well-formed formulas (chapter, canonical)	Well-Formed Formula
$\varphi, \psi, \chi, \theta$	formula metavariables	metavariables ranging over propositional well-formed formulas (global, canonical)	Well-Formed Formula
α, β	additional formula metavariables	formula metavariables used in comparison statements (chapter, local)	Constructor Injectivity for Propositional Formulas
$\text{Tree}(\varphi)$	syntax tree	parse tree or formation tree of a formula (chapter, canonical)	Parse Tree
$\text{Var}(\varphi)$	variable set of a formula	the set of propositional variables occurring in a formula (chapter, canonical)	Variable Set of a Formula
$\text{Sub}(\varphi)$	subformula set	the set of subformulas of a formula (chapter, canonical)	Subformula
$\text{depth}(\varphi)$	formula depth	height, rank, or complexity of the formula formation tree (chapter, canonical)	Formula Depth

$\text{conn}(\varphi)$	connective count	the number of connective occurrences in a formula (chapter, canonical)	Connective Count
v	truth assignment	a truth assignment on propositional variables (chapter, canonical)	Truth Assignment
w	second truth assignment	truth assignment on propositional variables (global, canonical)	Truth Assignment
$\hat{v}$	extended truth assignment	unique extension of a truth assignment to formulas (chapter, canonical)	Unique Extension of a Truth Assignment
$\neg, \wedge, \vee, \rightarrow, \leftrightarrow$	standard connectives	standard propositional connectives (global, canonical)	Unique Extension of a Truth Assignment
$v \models \varphi$	satisfaction of a formula	truth assignment satisfies a formula (chapter, canonical)	Satisfaction of a Formula
$\models \varphi$	tautology notation	formula valid under every truth assignment (global, canonical)	Satisfaction of a Formula
$v \models \Gamma$	satisfaction of a set	truth assignment satisfies each formula in a set (chapter, canonical)	Satisfaction of a Set of Formulas
Γ	set of premise formulas	a set of propositional formulas (chapter, canonical)	Satisfaction of a Set of Formulas
$\Gamma \models \varphi$	semantic consequence	semantic consequence from premises to conclusion (chapter, canonical)	Logical Consequence
$\varphi \equiv \psi$	logical equivalence	same truth value under every truth assignment (chapter, canonical)	Logical Equivalence
$f : \mathbb{B}^n \rightarrow \mathbb{B}$	n-ary Boolean function	function from truth-value tuples to a truth value (chapter, canonical)	Boolean Function
f_φ	truth function induced by a formula	Boolean function represented by a formula (chapter, canonical)	Truth Function Induced by a Formula
$\equiv$	logical equivalence symbol	logical equivalence relation between formulas (global, canonical)	Propositional Equivalence Law
$\top$	tautological formula	a formula chosen as a true constant (chapter, canonical)	Identity and Domination Laws

$\perp$	contradictory formula	a formula chosen as a false constant (chapter, canonical)	Identity and Domination Laws
$C[-]$	formula context	a one-hole propositional formula context (chapter, canonical)	Formula Context
$C[\varphi]$	context substitution	formula obtained by filling a context with a formula (chapter, canonical)	Formula Context

1.4 Syntax

Toolkit

This section uses truth values, propositional variables, connectives, well-formed formulas, formation trees, and subformulas to build syntax.

Definitional Root (Truth Value)

Definition 1.4.1 (Truth Value). The set of truth values is

$$\mathbb{B} := \{\text{False}, \text{True}\}.$$

Some sources identify this set with $\{0, 1\}$, with 0 read as false and 1 read as true.

Remark (Standard quantified statement).

$$\mathbb{B} = \{\text{False}, \text{True}\}.$$

Remark (Interpretation). Truth values are the semantic targets of propositional formulas. Syntax determines which strings are formulas; semantics later assigns elements of $\mathbb{B}$ to formulas.

Definitional Root (Propositional Language)

Definition 1.4.2 (Propositional Language). A propositional language consists of:

1. a set Prop of propositional variables;
2. logical connectives;
3. punctuation symbols such as parentheses;
4. recursive formation rules specifying the well-formed formulas.

Remark (Standard quantified statement).

$$\mathcal{L}_{\text{prop}} = (\text{Prop}, \{\neg, \wedge, \vee, \rightarrow, \leftrightarrow\}, \text{punctuation}, \text{formation rules}).$$

Remark (Interpretation). The language supplies symbols and grammar. It does not yet assign truth values, truth conditions, or logical consequence.

Definition (Propositional Variable)

Definition 1.4.3 (Propositional Variable). A propositional variable is an atomic symbol intended to stand for a proposition. A standard supply of variables is

$$\mathbf{Prop} = \{P_1, P_2, P_3, \dots\}.$$

Informally, variables are often written $P, Q, R, S, \dots$

Remark (Standard quantified statement).

$$\text{PropositionalVariable}(P) \iff P \in \mathbf{Prop}.$$

Remark (Definition predicate reading). $\text{PropositionalVariable}(P)$ means $P \in \mathbf{Prop}$.

Remark (Interpretation). A propositional variable has no internal syntactic structure in propositional logic. It is a basic symbol from which more complex formulas are formed.

Remark (Examples). P, Q, R, P_1, P_2 are propositional variables when they belong to $\mathbf{Prop}$.

Remark (Non-examples). $\neg P, P \wedge Q$, and $(P \rightarrow Q)$ are not propositional variables; they are compound formulas.

Remark (Dependencies).

- [Propositional Language](#)

Definition (Logical Connective)

Definition 1.4.4 (Logical Connective). The standard primitive propositional connectives are

$$\neg, \quad \wedge, \quad \vee, \quad \rightarrow, \quad \leftrightarrow.$$

The connective $\neg$ is unary. The connectives $\wedge, \vee, \rightarrow, \leftrightarrow$ are binary.

Symbol	Name	Arity	Formation pattern	Reading
$\neg$	negation	unary	$\neg\varphi$	not φ
$\wedge$	conjunction	binary	$(\varphi \wedge \psi)$	φ and ψ
$\vee$	disjunction	binary	$(\varphi \vee \psi)$	φ or ψ
$\rightarrow$	conditional	binary	$(\varphi \rightarrow \psi)$	if φ , then ψ
$\leftrightarrow$	biconditional	binary	$(\varphi \leftrightarrow \psi)$	φ iff ψ

Remark (Standard quantified statement).

$\text{UnaryConnective}(\neg)$ and $\text{BinaryConnective}(\wedge) \wedge \text{BinaryConnective}(\vee) \wedge \text{BinaryConnective}(\rightarrow) \wedge \text{BinaryConnective}(\leftrightarrow)$

Remark (Interpretation). Arity records how many formulas a connective uses to form a new formula. The truth behavior of these connectives belongs to semantics, not to the syntactic definition.

Remark (Dependencies).

- [Propositional Language](#)

Definition (Well-Formed Formula)

Definition 1.4.5 (Well-Formed Formula). The set **WFF** of well-formed formulas is the smallest set of strings satisfying the following formation rules:

1. Every propositional variable $P \in \mathbf{Prop}$ belongs to **WFF**.
2. If $\varphi \in \mathbf{WFF}$, then $\neg\varphi \in \mathbf{WFF}$.
3. If $\varphi, \psi \in \mathbf{WFF}$ and $\circ \in \{\wedge, \vee, \rightarrow, \leftrightarrow\}$, then $(\varphi \circ \psi) \in \mathbf{WFF}$.
4. No string is a well-formed formula unless it is obtained by finitely many applications of the previous clauses.

Remark (Standard quantified statement).

$$\mathbf{WFF} = \bigcap \left\{ S : \mathbf{Prop} \subseteq S, (\forall \varphi \in S)(\neg\varphi \in S), (\forall \varphi, \psi \in S)(\forall \circ \in \{\wedge, \vee, \rightarrow, \leftrightarrow\})((\varphi \circ \psi) \in S) \right\}.$$

Remark (Definition predicate reading). $\text{WellFormedFormula}(\varphi)$ means $\varphi \in \mathbf{WFF}$, read as “ φ is a well-formed formula.”

Remark (Negated quantified statement).

$$\sigma \notin \mathbf{WFF} \iff \exists S [\mathbf{Prop} \subseteq S \wedge (\forall \varphi \in S)(\neg\varphi \in S) \wedge (\forall \varphi, \psi \in S)(\forall \circ \in \{\wedge, \vee, \rightarrow, \leftrightarrow\})((\varphi \circ \psi) \in S) \wedge \sigma \notin S]$$

Remark (Negation predicate reading). $\neg \text{WellFormedFormula}(\sigma)$ means the string σ is excluded by at least one formation-closed set containing **Prop**; equivalently, it is not generated by finitely many applications of the formation rules.

Remark (Failure modes). A string fails to be well formed when it violates the formation grammar: an operand may be missing, a connective may be missing, parentheses may be unmatched, or the string may not be generated by finitely many formation steps.

Remark (Failure mode decomposition).

$$\underbrace{P \wedge}_{\text{missing right operand}} \quad \underbrace{(P \ Q)}_{\text{missing binary connective}} \quad \underbrace{\neg \vee P}_{\text{negation applied to a non-formula}}.$$

Remark (Interpretation). Well-formed formulas are exactly the strings to which the semantic and proof-theoretic machinery of propositional logic applies.

Remark (Examples). P , $\neg P$, $(P \rightarrow Q)$, and $(P \rightarrow (Q \vee R))$ are well-formed formulas.

Remark (Non-examples). $P \wedge$, (PQ) , and $\neg \vee P$ are not well-formed formulas.

Remark (Dependencies).

- [Propositional Variable](#)

- Logical Connective

Lemma (Closure Under Formation)

Lemma 1.4.6 (Closure Under Formation). *The set WFF is closed under the propositional formation rules.*

That is, if $\varphi \in \text{WFF}$, then

$$\neg\varphi \in \text{WFF}.$$

If $\varphi, \psi \in \text{WFF}$ and

$$\circ \in \{\wedge, \vee, \rightarrow, \leftrightarrow\},$$

then

$$(\varphi \circ \psi) \in \text{WFF}.$$

Go to proof.

Remark (Standard quantified statement).

$$(\forall\varphi \in \text{WFF})(\neg\varphi \in \text{WFF})$$

and

$$(\forall\varphi, \psi \in \text{WFF})(\forall\circ \in \{\wedge, \vee, \rightarrow, \leftrightarrow\})((\varphi \circ \psi) \in \text{WFF}).$$

Remark (Interpretation). Closure says that every legal use of a formation rule produces another well-formed formula. It is the forward direction of the recursive grammar.

Remark (Dependencies).

- Well-Formed Formula

Theorem (Minimality of Well-Formed Formulas)

Theorem 1.4.7 (Minimality of Well-Formed Formulas). *Let S be a set of strings over the propositional alphabet. Suppose:*

1. $\text{Prop} \subseteq S$;
2. if $\varphi \in S$, then $\neg\varphi \in S$;
3. if $\varphi, \psi \in S$ and $\circ \in \{\wedge, \vee, \rightarrow, \leftrightarrow\}$, then $(\varphi \circ \psi) \in S$.

Then

$$\text{WFF} \subseteq S.$$

Go to proof.

Remark (Standard quantified statement).

$$(\forall S) \left[\text{Prop} \subseteq S \wedge (\forall\varphi \in S)(\neg\varphi \in S) \right. \\ \left. \wedge (\forall\varphi, \psi \in S)(\forall\circ \in \{\wedge, \vee, \rightarrow, \leftrightarrow\})((\varphi \circ \psi) \in S) \right] \rightarrow \text{WFF} \subseteq S.$$

Remark (Interpretation). Minimality is the formal content of the clause “nothing else is a wff.” It says that **WFF** is contained in every set of strings that contains the propositional variables and is closed under the formation rules.

Remark (Dependencies).

- [Well-Formed Formula](#)

Definition (Atomic Formula)

Definition 1.4.8 (Atomic Formula). An atomic formula is a well-formed formula that is a propositional variable.

Remark (Standard quantified statement).

$$\text{AtomicFormula}(\varphi) \iff \varphi \in \text{WFF} \wedge \varphi \in \text{Prop}.$$

Remark (Definition predicate reading). $\text{AtomicFormula}(\varphi)$ means that φ is a propositional variable viewed as a well-formed formula.

Remark (Interpretation). Atomic formulas are the base cases of the recursive grammar. They contain no connective structure.

Remark (Dependencies).

- [Well-Formed Formula](#)
- [Propositional Variable](#)

Definition (Molecular Formula)

Definition 1.4.9 (Molecular Formula). A molecular formula is a well-formed formula that is not atomic. Equivalently, it is a well-formed formula formed using at least one logical connective.

Remark (Standard quantified statement).

$$\text{MolecularFormula}(\varphi) \iff \varphi \in \text{WFF} \wedge \neg \text{AtomicFormula}(\varphi).$$

Remark (Definition predicate reading). $\text{MolecularFormula}(\varphi)$ means that φ has nontrivial connective structure.

Remark (Interpretation). Molecular formulas are exactly the formulas obtained by applying negation or a binary connective at least once.

Remark (Dependencies).

- [Well-Formed Formula](#)
- [Atomic Formula](#)
- [Logical Connective](#)

Definition (Main Connective)

Definition 1.4.10 (Main Connective). The main connective of a molecular formula is the connective introduced at the final formation step:

1. for $\neg\varphi$, the main connective is $\neg$;
2. for $(\varphi \circ \psi)$, the main connective is $\circ$.

Atomic formulas have no main connective.

Remark (Standard quantified statement).

$$\text{MainConnective}(\neg\varphi, \neg) \quad \text{and} \quad \text{MainConnective}((\varphi \circ \psi), \circ).$$

Remark (Definition predicate reading). $\text{MainConnective}(\varphi, c)$ means that c is the outermost connective introduced in the final formation step of φ .

Remark (Interpretation). The main connective identifies the outermost syntactic operation. It is the first connective used when analyzing a formula from the outside inward.

Remark (Dependencies).

- [Well-Formed Formula](#)
- [Molecular Formula](#)
- [Logical Connective](#)

Theorem (Structural Induction for Propositional Formulas)

Theorem 1.4.11 (Structural Induction for Propositional Formulas). Let $A(\varphi)$ be a property of well-formed formulas. Suppose:

1. $A(P)$ holds for every $P \in \text{Prop}$;
2. if $A(\varphi)$ holds, then $A(\neg\varphi)$ holds;
3. if $A(\varphi)$ and $A(\psi)$ hold, then $A((\varphi \circ \psi))$ holds for every $\circ \in \{\wedge, \vee, \rightarrow, \leftrightarrow\}$.

Then $A(\theta)$ holds for every $\theta \in \text{WFF}$.

Go to proof.

Remark (Standard quantified statement).

$$[(\forall P \in \text{Prop})A(P) \wedge (\forall \varphi \in \text{WFF})(A(\varphi) \rightarrow A(\neg\varphi)) \wedge (\forall \varphi, \psi \in \text{WFF})(\forall \circ \in \{\wedge, \vee, \rightarrow, \leftrightarrow\})((A(\varphi) \wedge A(\psi)) \rightarrow A(\varphi \circ \psi))]$$

Remark (Interpretation). To prove a statement for all formulas, it suffices to prove it for propositional variables and show it is preserved by each formula-forming operation.

Remark (Dependencies).

- [Well-Formed Formula](#)

Theorem (Structural Recursion for Propositional Formulas)

Theorem 1.4.12 (Structural Recursion for Propositional Formulas). *To define a function F on WFF, it suffices to specify:*

1. $F(P)$ for every $P \in \text{Prop}$;
2. $F(\neg\varphi)$ in terms of $F(\varphi)$;
3. $F((\varphi \circ \psi))$ in terms of $F(\varphi)$, $F(\psi)$, and $\circ$.

When these clauses respect the formation rules, they determine a unique function on WFF.

Go to proof.

Remark (Standard quantified statement).

recursive data on the formation clauses of WFF $\implies \exists! F : \text{WFF} \rightarrow X$ satisfying those clauses. ■

Remark (Interpretation). Structural recursion justifies recursive definitions such as variables of a formula, subformulas, depth, connective count, syntax tree, and truth evaluation.

Remark (Dependencies).

- [Well-Formed Formula](#)
- [Structural Induction for Propositional Formulas](#)

Lemma (Constructor Disjointness for Propositional Formulas)

Lemma 1.4.13 (Constructor Disjointness for Propositional Formulas). *The three outer formation cases for propositional formulas are disjoint:*

1. *no propositional variable is a negation formula;*
2. *no propositional variable is a binary formula;*
3. *no negation formula is a binary formula.*

Go to proof.

Remark (Standard quantified statement).

$$P \in \text{Prop} \implies [(\forall \varphi \in \text{WFF})(P \neq \neg\varphi) \wedge (\forall \varphi, \psi \in \text{WFF})(\forall \circ)(P \neq (\varphi \circ \psi))]$$

and

$$(\forall \varphi, \psi, \chi \in \text{WFF})(\forall \circ)(\neg\varphi \neq (\psi \circ \chi)).$$

Remark (Predicate reading).

$$\underbrace{P}_{\text{atomic}} \quad \underbrace{\neg\varphi}_{\text{negation case}} \quad \underbrace{(\varphi \circ \psi)}_{\text{binary case}}$$

belong to three distinct outer-form classes.

Remark (Interpretation). A formula cannot be parsed as two different outer formation types.

Remark (Dependencies).

- [Well-Formed Formula](#)
- [Atomic Formula](#)
- [Main Connective](#)

Lemma (Constructor Injectivity for Propositional Formulas)

Lemma 1.4.14 (Constructor Injectivity for Propositional Formulas). *The propositional formula constructors are injective:*

1. if $\neg\varphi = \neg\psi$, then $\varphi = \psi$;
2. if $(\varphi \circ \psi) = (\alpha \star \beta)$, then $\varphi = \alpha$, $\psi = \beta$, and $\circ = \star$.

Go to proof.

Remark (Standard quantified statement).

$$(\forall\varphi, \psi \in \mathbf{WFF})(\neg\varphi = \neg\psi \rightarrow \varphi = \psi)$$

and

$$(\forall\varphi, \psi, \alpha, \beta \in \mathbf{WFF})(\forall\circ, \star \in \{\wedge, \vee, \rightarrow, \leftrightarrow\}) [(\varphi \circ \psi) = (\alpha \star \beta) \rightarrow (\varphi = \alpha \wedge \psi = \beta \wedge \circ = \star)].$$

Remark (Interpretation). If two formulas are built by the same outer constructor and are equal as strings, then their immediate constituents are equal.

Remark (Dependencies).

- [Well-Formed Formula](#)
- [Logical Connective](#)

Theorem (Unique Decomposition for Propositional Formulas)

Theorem 1.4.15 (Unique Decomposition for Propositional Formulas). *Every $\varphi \in \text{WFF}$ has exactly one outermost form:*

1. $\varphi = P$ for a unique $P \in \text{Prop}$;
2. $\varphi = \neg\psi$ for a unique $\psi \in \text{WFF}$;
3. $\varphi = (\psi \circ \chi)$ for unique $\psi, \chi \in \text{WFF}$ and a unique binary connective $\circ \in \{\wedge, \vee, \rightarrow, \leftrightarrow\}$.

Go to proof.

Remark (Standard quantified statement).

$\forall \varphi \in \text{WFF}, \quad \text{exactly one of the following holds:}$

$\varphi \in \text{Prop}, \quad \exists! \psi \in \text{WFF} (\varphi = \neg\psi), \quad \exists! \psi, \chi \in \text{WFF} \exists! \circ \in \{\wedge, \vee, \rightarrow, \leftrightarrow\} (\varphi = (\psi \circ \chi)).$

Remark (Predicate reading).

$$\varphi \in \underbrace{\text{Prop}}_{\text{atomic case}} \quad \text{or} \quad \varphi = \underbrace{\neg\psi}_{\text{negation case}} \quad \text{or} \quad \varphi = \underbrace{(\psi \circ \chi)}_{\text{binary case}},$$

and exactly one of these alternatives holds.

Remark (Interpretation). Unique decomposition makes recursive definitions and induction on formulas unambiguous.

Remark (Dependencies).

- [Well-Formed Formula](#)
- [Constructor Disjointness for Propositional Formulas](#)
- [Constructor Injectivity for Propositional Formulas](#)

Definition (Parse Tree)

Definition 1.4.16 (Parse Tree). The parse tree, or syntax tree, of a well-formed formula is the tree representation of its recursive construction:

1. a propositional variable is represented by a single leaf;
2. a formula $\neg\varphi$ is represented by a root labeled $\neg$ with one child, the parse tree of φ ;
3. a formula $(\varphi \circ \psi)$ is represented by a root labeled $\circ$ with two children, the parse trees of φ and ψ .

Remark (Standard quantified statement).

$$\text{Tree} : \text{WFF} \rightarrow \{\text{finite rooted labeled trees}\}$$

is defined recursively by the three formation clauses for propositional formulas.

Remark (Interpretation). The parse tree records the same structure described by unique decomposition. It turns the recursive construction of a formula into a visible tree.

Remark (Dependencies).

- [Well-Formed Formula](#)
- [Unique Decomposition for Propositional Formulas](#)

Theorem (Unique Syntax Tree for Propositional Formulas)

Theorem 1.4.17 (Unique Syntax Tree for Propositional Formulas). *Every propositional well-formed formula has a unique parse tree.*

Go to proof.

Remark (Standard quantified statement).

$$\forall \varphi \in \text{WFF}, \quad \exists! T \ (T = \text{Tree}(\varphi)).$$

Remark (Interpretation). A formula has exactly one tree representation of its recursive construction. This is the tree form of unique decomposition.

Remark (Dependencies).

- [Parse Tree](#)
- [Unique Decomposition for Propositional Formulas](#)

Definition (Variable Set of a Formula)

Definition 1.4.18 (Variable Set of a Formula). The variable set $\text{Var}(\varphi)$ of a formula φ is defined recursively:

1. if $\varphi = P \in \text{Prop}$, then $\text{Var}(\varphi) = \{P\}$;
2. if $\varphi = \neg\psi$, then $\text{Var}(\varphi) = \text{Var}(\psi)$;
3. if $\varphi = (\psi \circ \chi)$, then
$$\text{Var}(\varphi) = \text{Var}(\psi) \cup \text{Var}(\chi).$$

Remark (Standard quantified statement).

$$\text{Var}(P) = \{P\}, \quad \text{Var}(\neg\psi) = \text{Var}(\psi), \quad \text{Var}((\psi \circ \chi)) = \text{Var}(\psi) \cup \text{Var}(\chi).$$

Remark (Interpretation). The variable set records exactly which propositional variables occur in a formula.

Remark (Dependencies).

- [Well-Formed Formula](#)
- [Structural Recursion for Propositional Formulas](#)

Definition (Subformula)

Definition 1.4.19 (Subformula). The set $\text{Sub}(\varphi)$ of subformulas of a formula φ is defined recursively:

1. if φ is atomic, then $\text{Sub}(\varphi) = \{\varphi\}$;

2. if $\varphi = \neg\psi$, then

$$\text{Sub}(\varphi) = \{\varphi\} \cup \text{Sub}(\psi);$$

3. if $\varphi = (\psi \circ \chi)$, then

$$\text{Sub}(\varphi) = \{\varphi\} \cup \text{Sub}(\psi) \cup \text{Sub}(\chi).$$

Remark (Standard quantified statement).

$$\text{Sub}(P) = \{P\}, \quad \text{Sub}(\neg\psi) = \{\neg\psi\} \cup \text{Sub}(\psi),$$

$$\text{Sub}((\psi \circ \chi)) = \{(\psi \circ \chi)\} \cup \text{Sub}(\psi) \cup \text{Sub}(\chi).$$

Remark (Interpretation). Subformulas are the formulas that occur at nodes of the parse tree.

Remark (Dependencies).

- [Well-Formed Formula](#)
- [Unique Decomposition for Propositional Formulas](#)

Definition (Proper Subformula)

Definition 1.4.20 (Proper Subformula). Let $\varphi \in \text{WFF}$. A *proper subformula* of φ is a formula ψ such that

$$\psi \in \text{Sub}(\varphi) \quad \text{and} \quad \psi \neq \varphi.$$

Remark (Standard quantified statement).

$$\text{ProperSubformula}(\psi, \varphi) \iff \psi \in \text{Sub}(\varphi) \wedge \psi \neq \varphi.$$

Remark (Interpretation). Proper subformulas are the strict syntactic parts of a formula. They exclude the whole formula itself.

Remark (Dependencies).

- [Subformula](#)

- [Well-Formed Formula](#)

Definition (Formula Depth)

Definition 1.4.21 (Formula Depth). The depth of a formula φ , denoted $\text{depth}(\varphi)$, is defined recursively:

1. if φ is atomic, then $\text{depth}(\varphi) = 0$;
2. if $\varphi = \neg\psi$, then $\text{depth}(\varphi) = \text{depth}(\psi) + 1$;
3. if $\varphi = (\psi \circ \chi)$, then

$$\text{depth}(\varphi) = \max\{\text{depth}(\psi), \text{depth}(\chi)\} + 1.$$

Remark (Standard quantified statement).

$$\text{depth}(P) = 0, \quad \text{depth}(\neg\psi) = \text{depth}(\psi) + 1,$$

$$\text{depth}((\psi \circ \chi)) = \max\{\text{depth}(\psi), \text{depth}(\chi)\} + 1.$$

Remark (Interpretation). Formula depth measures the height of the syntax tree. Some authors call this the rank, complexity, or formation depth of the formula.

Remark (Dependencies).

- [Well-Formed Formula](#)
- [Parse Tree](#)

Definition (Connective Count)

Definition 1.4.22 (Connective Count). The connective count of a formula φ , denoted $\text{conn}(\varphi)$, is defined recursively:

1. if φ is atomic, then $\text{conn}(\varphi) = 0$;
2. if $\varphi = \neg\psi$, then $\text{conn}(\varphi) = \text{conn}(\psi) + 1$;
3. if $\varphi = (\psi \circ \chi)$, then

$$\text{conn}(\varphi) = \text{conn}(\psi) + \text{conn}(\chi) + 1.$$

Remark (Standard quantified statement).

$$\text{conn}(P) = 0, \quad \text{conn}(\neg\psi) = \text{conn}(\psi) + 1,$$

$$\text{conn}((\psi \circ \chi)) = \text{conn}(\psi) + \text{conn}(\chi) + 1.$$

Remark (Interpretation). Formula depth measures the height of the syntax tree. Connective count measures the number of connective nodes in the syntax tree.

Remark (Dependencies).

- [Well-Formed Formula](#)
- [Parse Tree](#)

Lemma (Proper Subformula Depth)

Lemma 1.4.23 (Proper Subformula Depth). *If ψ is a proper subformula of φ , then*

$$\text{depth}(\psi) < \text{depth}(\varphi).$$

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi, \psi \in \mathbf{WFF}) [\psi \in \text{Sub}(\varphi) \wedge \psi \neq \varphi \rightarrow \text{depth}(\psi) < \text{depth}(\varphi)].$$

Remark (Interpretation). Every descent into a proper subformula lowers formula depth. This justifies induction on formula depth and recursive proofs over subformulas.

Remark (Dependencies).

- [Subformula](#)
- [Formula Depth](#)

Lemma (Finiteness of Variables in a Formula)

Lemma 1.4.24 (Finiteness of Variables in a Formula). *For every $\varphi \in \mathbf{WFF}$, the set $\text{Var}(\varphi)$ is finite.*

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi \in \mathbf{WFF}) \quad \text{Var}(\varphi) \text{ is finite.}$$

Remark (Interpretation). Every formula depends on only finitely many propositional variables, even though the language may contain infinitely many variables.

Remark (Dependencies).

- [Variable Set of a Formula](#)
- [Structural Induction for Propositional Formulas](#)

Lemma (Finiteness of Subformulas)

Lemma 1.4.25 (Finiteness of Subformulas). *For every $\varphi \in \mathbf{WFF}$, the set $\text{Sub}(\varphi)$ is finite.*

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi \in \text{WFF}) \quad \text{Sub}(\varphi) \text{ is finite.}$$

Remark (Interpretation). Every formula has only finitely many syntactic parts.

Remark (Dependencies).

- [Subformula](#)
- [Structural Induction for Propositional Formulas](#)

1.5 Semantics

Toolkit

This section uses truth assignments, recursive extensions, satisfaction, truth tables, and logical equivalence to assign meanings to formulas.

Remark (Exposition). The semantics section assigns truth conditions to the syntactic formulas introduced earlier. Truth assignments, recursive extensions, satisfaction, and truth tables provide the machinery for validity, consequence, equivalence, and semantic classification.

Truth Assignments

Definition (Truth Assignment)

Definition 1.5.1 (Truth Assignment). A *truth assignment* is a function

$$v : \text{Prop} \rightarrow \mathbb{B}.$$

For each propositional variable $P \in \text{Prop}$, the value $v(P)$ is the truth value assigned to P .

Remark (Standard quantified statement).

$$\text{TruthAssignment}(v) \iff v : \text{Prop} \rightarrow \mathbb{B}.$$

Remark (Definition predicate reading).

$$\text{TruthAssignment}(v)$$

means that v assigns exactly one truth value to each propositional variable.

Remark (Interpretation). A truth assignment gives semantic values to the atomic formulas. The recursive truth conditions for the connectives then determine the truth value of every well-formed formula.

Remark (Examples). If $\mathbf{Prop} = \{P_1, P_2, P_3, \dots\}$, then a truth assignment may satisfy

$$v(P_1) = \text{True}, \quad v(P_2) = \text{False}, \quad v(P_3) = \text{True},$$

with values assigned to every remaining propositional variable.

Remark (Non-Examples). A partial assignment that gives values only to P_1 and P_2 is not a truth assignment on $\mathbf{Prop}$. It may be useful for local computations, but it is not a function $\mathbf{Prop} \rightarrow \mathbb{B}$.

Remark (Dependencies).

- [Truth Value](#)
- [Propositional Variable](#)
- [Well-Formed Formula](#)

Definition (Domain of a Truth Assignment)

Definition 1.5.2 (Domain of a Truth Assignment). The *domain* of a truth assignment

$$v : \mathbf{Prop} \rightarrow \mathbb{B}$$

is the set $\mathbf{Prop}$ of propositional variables.

Remark (Standard quantified statement).

$$\text{TruthAssignment}(v) \implies \text{dom}(v) = \mathbf{Prop}.$$

Remark (Interpretation). A truth assignment is global: it assigns truth values to all propositional variables in the language, even when a particular formula uses only finitely many of them.

Remark (Dependencies).

- [Truth Assignment](#)

Extension of a Truth Assignment

Definition (Extension of a Truth Assignment to Formulas)

Definition 1.5.3 (Extension of a Truth Assignment to Formulas). Let

$$v : \text{Prop} \rightarrow \mathbb{B}$$

be a truth assignment. The *extension* of v to well-formed formulas is the function

$$\hat{v} : \text{WFF} \rightarrow \mathbb{B}$$

defined recursively as follows:

$$\hat{v}(P) = v(P) \quad \text{for every } P \in \text{Prop},$$

$$\hat{v}(\neg\varphi) = \text{True} \iff \hat{v}(\varphi) = \text{False},$$

$$\hat{v}(\varphi \wedge \psi) = \text{True} \iff \hat{v}(\varphi) = \text{True} \text{ and } \hat{v}(\psi) = \text{True},$$

$$\hat{v}(\varphi \vee \psi) = \text{True} \iff \hat{v}(\varphi) = \text{True} \text{ or } \hat{v}(\psi) = \text{True},$$

$$\hat{v}(\varphi \rightarrow \psi) = \text{False} \iff \hat{v}(\varphi) = \text{True} \text{ and } \hat{v}(\psi) = \text{False},$$

and

$$\hat{v}(\varphi \leftrightarrow \psi) = \text{True} \iff \hat{v}(\varphi) = \hat{v}(\psi).$$

Remark (Standard quantified statement). For every truth assignment $v : \text{Prop} \rightarrow \mathbb{B}$, its extension is a function

$$\hat{v} : \text{WFF} \rightarrow \mathbb{B}$$

satisfying the recursive truth clauses for propositional variables, negation, conjunction, disjunction, material implication, and biconditional.

Remark (Definition predicate reading).

$$\text{ExtendedTruthAssignment}(\hat{v}, v)$$

means that $\hat{v}$ is the recursively defined truth-value function on WFF induced by the atomic truth assignment v .

Remark (Interpretation). The original assignment v gives truth values only to propositional variables. The extension $\hat{v}$ evaluates every formula by following the formula's syntactic construction.

Remark (Examples).

φ	$\neg\varphi$	φ	ψ	$\varphi \wedge \psi$
True	False	True	True	True
False	True	True	False	False
		False	True	False
		False	False	False

φ	ψ	$\varphi \vee \psi$	φ	ψ	$\varphi \rightarrow \psi$
True	True	True	True	True	True
True	False	True	True	False	False
False	True	True	False	True	True
False	False	False	False	False	True

φ	ψ	$\varphi \leftrightarrow \psi$
True	True	True
True	False	False
False	True	False
False	False	True

Remark (Examples). The truth tables are not additional formation rules. They specify how each primitive connective acts on truth values once the formula has already been formed syntactically.

Remark (Dependencies).

- [Truth Assignment](#)
- [Well-Formed Formula](#)
- [Structural Recursion for Propositional Formulas](#)
- [Unique Decomposition for Propositional Formulas](#)

Theorem (Unique Extension of a Truth Assignment)

Theorem 1.5.4 (Unique Extension of a Truth Assignment). *Every truth assignment*

$$v : \mathbf{Prop} \rightarrow \mathbb{B}$$

has a unique extension

$$\hat{v} : \mathbf{WFF} \rightarrow \mathbb{B}$$

satisfying the recursive truth clauses for propositional variables and the primitive connectives.

Go to proof.

Remark (Standard quantified statement).

$$(\forall v : \mathbf{Prop} \rightarrow \mathbb{B})(\exists! \hat{v} : \mathbf{WFF} \rightarrow \mathbb{B})$$

such that $\hat{v}$ agrees with v on $\mathbf{Prop}$ and satisfies the recursive truth clauses for

$$\neg, \wedge, \vee, \rightarrow, \leftrightarrow.$$

Remark (Predicate reading).

$$\text{TruthAssignment}(v) \implies \exists! \hat{v} \text{ ExtendedTruthAssignment}(\hat{v}, v).$$

Remark (Interpretation). This theorem guarantees that semantic evaluation is well-defined. Once the truth values of the propositional variables are fixed, every well-formed formula has exactly one truth value.

Remark (Dependencies).

- [Truth Assignment](#)
- [Extension of a Truth Assignment to Formulas](#)
- [Structural Recursion for Propositional Formulas](#)
- [Unique Decomposition for Propositional Formulas](#)

Definition (Satisfaction of a Formula)

Definition 1.5.5 (Satisfaction of a Formula). Let v be a truth assignment and let $\varphi \in \text{WFF}$. We say that v satisfies φ , written

$$v \models \varphi,$$

if

$$\widehat{v}(\varphi) = \text{True}.$$

Remark (Standard quantified statement).

$$v \models \varphi \iff \text{TruthAssignment}(v) \wedge \varphi \in \text{WFF} \wedge \widehat{v}(\varphi) = \text{True}.$$

Remark (Definition predicate reading).

$$\text{SatisfiesFormula}(v, \varphi)$$

means $v \models \varphi$.

Remark (Negated quantified statement).

$$v \not\models \varphi \iff \widehat{v}(\varphi) = \text{False}.$$

Remark (Interpretation). The notation $v \models \varphi$ means that the formula φ is true under the truth assignment v .

Remark (Dependencies).

- [Truth Assignment](#)
- [Unique Extension of a Truth Assignment](#)
- [Well-Formed Formula](#)

Definition (Satisfaction of a Set of Formulas)

Definition 1.5.6 (Satisfaction of a Set of Formulas). Let $\Gamma \subseteq \text{WFF}$. A truth assignment v satisfies Γ , written

$$v \models \Gamma,$$

if $v \models \gamma$ for every $\gamma \in \Gamma$.

Remark (Standard quantified statement).

$$v \models \Gamma \iff \text{TruthAssignment}(v) \wedge \Gamma \subseteq \text{WFF} \wedge (\forall \gamma \in \Gamma)(v \models \gamma).$$

Remark (Definition predicate reading).

$$\text{SatisfiesSet}(v, \Gamma)$$

means v satisfies every formula in Γ .

Remark (Negated quantified statement).

$$v \not\models \Gamma \iff \exists \gamma \in \Gamma \text{ such that } v \not\models \gamma.$$

Remark (Interpretation). Satisfaction of a set means simultaneous satisfaction of every formula in the set.

Remark (Dependencies).

- [Satisfaction of a Formula](#)
- [Truth Assignment](#)

Definition (Truth Table)

Definition 1.5.7 (Truth Table). Let $\varphi \in \text{WFF}$ and suppose the variables occurring in φ are among $P_1, \dots, P_n$. A truth table for φ is a finite table with one row for each assignment

$$a : \{P_1, \dots, P_n\} \rightarrow \mathbb{B}$$

and a final column recording the value of $\widehat{v}(\varphi)$ for any truth assignment v extending a .

Remark (Standard quantified statement).

$$\text{TruthTable}(\varphi) \iff \varphi \in \text{WFF} \wedge \text{Var}(\varphi) = \{P_1, \dots, P_n\} \wedge \text{the table has } 2^n \text{ assignment rows.}$$

Remark (Interpretation). A truth table is finite because every formula contains only finitely many variables. It records the complete truth behavior of a formula.

Remark (Dependencies).

- [Variable Set of a Formula](#)
- [Finiteness of Variables in a Formula](#)

- [Unique Extension of a Truth Assignment](#)

Definition (Tautology)

Definition 1.5.8 (Tautology). A formula $\varphi \in \text{WFF}$ is a tautology, written

$$\models \varphi,$$

if every truth assignment satisfies φ .

Remark (Standard quantified statement).

$$\models \varphi \iff \varphi \in \text{WFF} \wedge (\forall v : \text{Prop} \rightarrow \mathbb{B})(v \models \varphi).$$

Remark (Definition predicate reading).

$$\text{Tautology}(\varphi)$$

means $\models \varphi$.

Remark (Interpretation). A tautology is true under every possible assignment of truth values to propositional variables.

Remark (Dependencies).

- [Satisfaction of a Formula](#)
- [Truth Assignment](#)

Definition (Contradiction)

Definition 1.5.9 (Contradiction). A formula $\varphi \in \text{WFF}$ is a contradiction if no truth assignment satisfies φ .

Remark (Standard quantified statement).

$$\text{Contradiction}(\varphi) \iff \varphi \in \text{WFF} \wedge (\forall v : \text{Prop} \rightarrow \mathbb{B})(v \not\models \varphi).$$

Remark (Definition predicate reading).

$$\text{Contradiction}(\varphi)$$

means that φ is false under every truth assignment.

Remark (Interpretation). A contradiction has no true row in its truth table.

Remark (Dependencies).

- [Satisfaction of a Formula](#)
- [Truth Assignment](#)

Definition (Satisfiable Formula)

Definition 1.5.10 (Satisfiable Formula). A formula $\varphi \in \text{WFF}$ is satisfiable if at least one truth assignment satisfies it.

Remark (Standard quantified statement).

$$\text{SatisfiableFormula}(\varphi) \iff \varphi \in \text{WFF} \wedge \exists v : \text{Prop} \rightarrow \mathbb{B} \text{ such that } v \models \varphi.$$

Remark (Definition predicate reading).

$$\text{SatisfiableFormula}(\varphi)$$

means that φ is true under at least one truth assignment.

Remark (Interpretation). A satisfiable formula has at least one true row in its truth table.

Remark (Dependencies).

- [Satisfaction of a Formula](#)
- [Truth Assignment](#)

Definition (Contingency)

Definition 1.5.11 (Contingency). A formula $\varphi \in \text{WFF}$ is a contingency if it is satisfiable but not a tautology.

Remark (Standard quantified statement).

$$\text{Contingency}(\varphi) \iff \text{SatisfiableFormula}(\varphi) \wedge \neg \text{Tautology}(\varphi).$$

Remark (Definition predicate reading).

$$\text{Contingency}(\varphi)$$

means that φ is true under some truth assignments and false under others.

Remark (Interpretation). A contingency has at least one true row and at least one false row in its truth table.

Remark (Dependencies).

- [Satisfiable Formula](#)
- [Tautology](#)

Definition (Logical Consequence)

Definition 1.5.12 (Logical Consequence). Let $\Gamma \subseteq \text{WFF}$ and $\varphi \in \text{WFF}$. The formula φ is a logical consequence of Γ , written

$$\Gamma \models \varphi,$$

if every truth assignment satisfying Γ also satisfies φ .

Remark (Standard quantified statement).

$$\Gamma \models \varphi \iff \Gamma \subseteq \mathbf{WFF} \wedge \varphi \in \mathbf{WFF} \wedge (\forall v : \mathbf{Prop} \rightarrow \mathbb{B}) [v \models \Gamma \rightarrow v \models \varphi].$$

Remark (Definition predicate reading).

$$\text{LogicalConsequence}(\Gamma, \varphi)$$

means $\Gamma \models \varphi$.

Remark (Negated quantified statement).

$$\Gamma \not\models \varphi \iff \exists v : \mathbf{Prop} \rightarrow \mathbb{B} [v \models \Gamma \wedge v \not\models \varphi].$$

Remark (Interpretation). Logical consequence means truth preservation from premises to conclusion. There is no truth assignment under which all premises are true and the conclusion is false.

Remark (Dependencies).

- [Satisfaction of a Set of Formulas](#)
- [Satisfaction of a Formula](#)

Definition (Logical Equivalence)

Definition 1.5.13 (Logical Equivalence). Formulas $\varphi, \psi \in \mathbf{WFF}$ are logically equivalent, written

$$\varphi \equiv \psi,$$

if they have the same truth value under every truth assignment.

Remark (Standard quantified statement).

$$\varphi \equiv \psi \iff \varphi, \psi \in \mathbf{WFF} \wedge (\forall v : \mathbf{Prop} \rightarrow \mathbb{B}) [\widehat{v}(\varphi) = \widehat{v}(\psi)].$$

Remark (Definition predicate reading).

$$\text{LogicalEquivalence}(\varphi, \psi)$$

means $\varphi \equiv \psi$.

Remark (Interpretation). Logically equivalent formulas have identical truth-table columns. They are interchangeable for semantic purposes.

Remark (Dependencies).

- [Unique Extension of a Truth Assignment](#)
- [Truth Assignment](#)

Lemma (Coincidence Lemma for Propositional Logic)

Lemma 1.5.14 (Coincidence Lemma for Propositional Logic). *Let $v, w : \text{Prop} \rightarrow \mathbb{B}$ be truth assignments. If v and w agree on every propositional variable occurring in φ , then they assign the same truth value to φ :*

$$v|_{\text{Var}(\varphi)} = w|_{\text{Var}(\varphi)} \implies \widehat{v}(\varphi) = \widehat{w}(\varphi).$$

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi \in \text{WFF})(\forall v, w : \text{Prop} \rightarrow \mathbb{B}) [(\forall P \in \text{Var}(\varphi))(v(P) = w(P)) \rightarrow \widehat{v}(\varphi) = \widehat{w}(\varphi)].$$

Remark (Interpretation). The truth value of a formula depends only on the variables actually occurring in that formula.

Remark (Dependencies).

- [Variable Set of a Formula](#)
- [Unique Extension of a Truth Assignment](#)
- [Structural Induction for Propositional Formulas](#)

Theorem (Finite Truth-Table Theorem)

Theorem 1.5.15 (Finite Truth-Table Theorem). *Every propositional formula has a finite truth table. More precisely, if $\text{Var}(\varphi)$ has n elements, then φ has a truth table with 2^n rows.*

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi \in \text{WFF}) [|\text{Var}(\varphi)| = n \rightarrow \text{TruthTable}(\varphi) \text{ has } 2^n \text{ rows}].$$

Remark (Interpretation). Truth tables are finite because every formula contains only finitely many propositional variables.

Remark (Dependencies).

- [Truth Table](#)
- [Finiteness of Variables in a Formula](#)

Definition (Unary Boolean Function)

Definition 1.5.16 (Unary Boolean Function). A unary Boolean function is a function

$$f : \mathbb{B} \rightarrow \mathbb{B}.$$

Remark (Definition predicate reading).

$$\text{UnaryBooleanFunction}(f)$$

means that f takes one truth value as input and returns one truth value.

Remark (Interpretation). A unary Boolean function has one input row coordinate. It is the semantic shape of a one-place truth operation such as negation.

Remark (Dependencies).

- [Truth Value](#)

Definition (Binary Boolean Function)

Definition 1.5.17 (Binary Boolean Function). A binary Boolean function is a function

$$f : \mathbb{B}^2 \rightarrow \mathbb{B},$$

equivalently a function

$$f : \mathbb{B} \times \mathbb{B} \rightarrow \mathbb{B}.$$

Remark (Definition predicate reading).

$$\text{BinaryBooleanFunction}(f)$$

means that f takes an ordered pair of truth values as input and returns one truth value.

Remark (Interpretation). A binary Boolean function assigns one output truth value to each of the four input rows

$$(T, T), \quad (T, F), \quad (F, T), \quad (F, F).$$

Remark (Dependencies).

- [Truth Value](#)
- [Unary Boolean Function](#)

Definition (Boolean Function)

Definition 1.5.18 (Boolean Function). An n -ary Boolean function is a function

$$f : \mathbb{B}^n \rightarrow \mathbb{B}.$$

Remark (Standard quantified statement).

$$\text{BooleanFunction}_n(f) \iff f : \mathbb{B}^n \rightarrow \mathbb{B}.$$

Remark (Definition predicate reading).

$$\text{BooleanFunction}_n(f)$$

means that f takes n truth values as input and returns one truth value.

Remark (Interpretation). Boolean functions are the abstract truth-functions that formulas may represent.

Remark (Rows versus Boolean functions). An input to an n -ary Boolean function is a tuple $(b_1, \dots, b_n) \in \mathbb{B}^n$. These tuples are the rows of the truth table, and there are 2^n of them. The Boolean function is not a single row; it is the full rule assigning exactly one truth value in $\mathbb{B}$ to every row.

Remark (Dependencies).

- [Truth Value](#)
- [Unary Boolean Function](#)
- [Binary Boolean Function](#)

Definition (Truth Function Induced by a Formula)

Definition 1.5.19 (Truth Function Induced by a Formula). Let $\varphi \in \text{WFF}$, and suppose $\text{Var}(\varphi) \subseteq \{P_1, \dots, P_n\}$. The truth function induced by φ is the Boolean function

$$f_\varphi : \mathbb{B}^n \rightarrow \mathbb{B}$$

defined by

$$f_\varphi(b_1, \dots, b_n) = \widehat{v}(\varphi),$$

where $v(P_i) = b_i$ for each i .

Remark (Standard quantified statement).

$$f_\varphi(b_1, \dots, b_n) = \widehat{v}(\varphi) \quad \text{whenever } v(P_i) = b_i \text{ for } 1 \leq i \leq n.$$

Remark (Definition predicate reading).

$$\text{TruthFunctionOfFormula}(f_\varphi, \varphi)$$

means that f_φ records the truth behavior of φ .

Remark (Interpretation). The induced truth function packages the truth table of a formula as a Boolean function.

Remark (Dependencies).

- [Boolean Function](#)
- [Variable Set of a Formula](#)
- [Unique Extension of a Truth Assignment](#)
- [Coincidence Lemma for Propositional Logic](#)

Theorem (Counting Boolean Functions)

Theorem 1.5.20 (Counting Boolean Functions). *There are exactly*

$$2^{2^n}$$

Boolean functions $\mathbb{B}^n \rightarrow \mathbb{B}$.

Go to proof.

Remark (Standard quantified statement).

$$|\{f : \mathbb{B}^n \rightarrow \mathbb{B}\}| = 2^{2^n}.$$

Remark (Interpretation). The domain $\mathbb{B}^n$ has 2^n elements. A Boolean function chooses one of two truth values for each input row.

Remark (Dependencies).

- [Boolean Function](#)

1.6 Algebra of Propositions

Toolkit

This section uses logical equivalence, equivalence laws, formula contexts, and replacement of logical equivalents to treat propositions algebraically.

Remark (Exposition). The algebra section treats logically equivalent formulas as interchangeable for calculation. Equivalence laws and replacement principles turn semantic agreement under every truth assignment into a controlled rewrite calculus for propositional formulas.

Definition (Propositional Equivalence Law)

Definition 1.6.1 (Propositional Equivalence Law). A propositional equivalence law is a valid schema of the form

$$\varphi \equiv \psi,$$

with $\varphi, \psi \in \text{WFF}$. Each instance of the schema says that the two formulas have the same truth value under every truth assignment.

Remark (Standard quantified statement).

$$\text{PropositionalEquivalenceLaw}(\varphi, \psi) \iff \varphi, \psi \in \text{WFF} \wedge \varphi \equiv \psi.$$

Remark (Definition predicate reading).

$$\text{PropositionalEquivalenceLaw}(\varphi, \psi)$$

means that $\varphi \equiv \psi$ is a valid logical-equivalence schema.

Remark (Interpretation). Equivalence laws are the algebraic rewrite rules of propositional logic. They allow formulas to be transformed without changing truth conditions.

Remark (Dependencies).

- [Logical Equivalence](#)
- [Well-Formed Formula](#)

Theorem (Logical Equivalence is an Equivalence Relation)

Theorem 1.6.2 (Logical Equivalence is an Equivalence Relation). *Logical equivalence is reflexive, symmetric, and transitive on WFF:*

$$\varphi \equiv \varphi, \quad \varphi \equiv \psi \implies \psi \equiv \varphi,$$

and

$$(\varphi \equiv \psi \wedge \psi \equiv \chi) \implies \varphi \equiv \chi.$$

Go to proof.

Remark (Standard quantified statement).

$$\begin{aligned} &(\forall \varphi \in \text{WFF})(\varphi \equiv \varphi), \\ &(\forall \varphi, \psi \in \text{WFF})(\varphi \equiv \psi \rightarrow \psi \equiv \varphi), \\ &(\forall \varphi, \psi, \chi \in \text{WFF})((\varphi \equiv \psi \wedge \psi \equiv \chi) \rightarrow \varphi \equiv \chi). \end{aligned}$$

Remark (Interpretation). Logical equivalence behaves like equality at the semantic level.

Remark (Dependencies).

- [Logical Equivalence](#)

Theorem (Basic Boolean Equivalence Laws)

Theorem 1.6.3 (Basic Boolean Equivalence Laws). *For all formulas $\varphi, \psi, \chi \in \text{WFF}$, the following logical equivalences hold.*

Commutativity:

$$\varphi \wedge \psi \equiv \psi \wedge \varphi, \quad \varphi \vee \psi \equiv \psi \vee \varphi.$$

Associativity:

$$(\varphi \wedge \psi) \wedge \chi \equiv \varphi \wedge (\psi \wedge \chi), \quad (\varphi \vee \psi) \vee \chi \equiv \varphi \vee (\psi \vee \chi).$$

Idempotence:

$$\varphi \wedge \varphi \equiv \varphi, \quad \varphi \vee \varphi \equiv \varphi.$$

Double negation:

$$\neg \neg \varphi \equiv \varphi.$$

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi, \psi, \chi \in \mathbf{WFF}) \left[\begin{array}{l} \varphi \wedge \psi \equiv \psi \wedge \varphi, \quad \varphi \vee \psi \equiv \psi \vee \varphi, \\ (\varphi \wedge \psi) \wedge \chi \equiv \varphi \wedge (\psi \wedge \chi), \quad (\varphi \vee \psi) \vee \chi \equiv \varphi \vee (\psi \vee \chi), \\ \varphi \wedge \varphi \equiv \varphi, \quad \varphi \vee \varphi \equiv \varphi, \quad \neg \neg \varphi \equiv \varphi \end{array} \right].$$

Remark (Interpretation). These are the basic algebraic laws for conjunction, disjunction, and negation.

Remark (Dependencies).

- [Logical Equivalence](#)
- [Unique Extension of a Truth Assignment](#)

Theorem (Identity and Domination Laws)

Theorem 1.6.4 (Identity and Domination Laws). *Let $\top$ be any tautological formula and let $\perp$ be any contradictory formula. Then for every $\varphi \in \mathbf{WFF}$:*

$$\varphi \wedge \top \equiv \varphi, \quad \varphi \vee \perp \equiv \varphi,$$

and

$$\varphi \wedge \perp \equiv \perp, \quad \varphi \vee \top \equiv \top.$$

Go to proof.

Remark (Standard quantified statement).

$$\begin{aligned} &(\text{Tautology}(\top) \wedge \text{Contradiction}(\perp)) \rightarrow (\forall \varphi \in \mathbf{WFF}) \\ &[(\varphi \wedge \top \equiv \varphi) \wedge (\varphi \vee \perp \equiv \varphi) \wedge (\varphi \wedge \perp \equiv \perp) \wedge (\varphi \vee \top \equiv \top)]. \end{aligned}$$

Remark (Interpretation). Tautologies act like true constants; contradictions act like false constants.

Remark (Dependencies).

- [Tautology](#)
- [Contradiction](#)
- [Logical Equivalence](#)

Theorem (De Morgan Laws for Propositional Logic)

Theorem 1.6.5 (De Morgan Laws for Propositional Logic). *For all $\varphi, \psi \in \mathbf{WFF}$,*

$$\neg(\varphi \wedge \psi) \equiv \neg\varphi \vee \neg\psi,$$

and

$$\neg(\varphi \vee \psi) \equiv \neg\varphi \wedge \neg\psi.$$

Go to proof.

Remark (Standard quantified statement).

$$\begin{aligned} (\forall \varphi, \psi \in \mathbf{WFF}) [\neg(\varphi \wedge \psi) &\equiv \neg\varphi \vee \neg\psi], \\ (\forall \varphi, \psi \in \mathbf{WFF}) [\neg(\varphi \vee \psi) &\equiv \neg\varphi \wedge \neg\psi]. \end{aligned}$$

Remark (Predicate reading).

$$\underbrace{\neg(\varphi \wedge \psi)}_{\text{not both}} \equiv \underbrace{\neg\varphi \vee \neg\psi}_{\text{at least one fails}}, \quad \underbrace{\neg(\varphi \vee \psi)}_{\text{neither}} \equiv \underbrace{\neg\varphi \wedge \neg\psi}_{\text{both fail}}.$$

Remark (Interpretation). Negation swaps conjunction with disjunction when pushed inward.

Remark (Dependencies).

- [Logical Equivalence](#)
- [Unique Extension of a Truth Assignment](#)

Theorem (Distributive Laws for Propositional Logic)

Theorem 1.6.6 (Distributive Laws for Propositional Logic). *For all $\varphi, \psi, \chi \in \mathbf{WFF}$,*

$$\varphi \wedge (\psi \vee \chi) \equiv (\varphi \wedge \psi) \vee (\varphi \wedge \chi),$$

and

$$\varphi \vee (\psi \wedge \chi) \equiv (\varphi \vee \psi) \wedge (\varphi \vee \chi).$$

Go to proof.

Remark (Standard quantified statement).

$$\begin{aligned} (\forall \varphi, \psi, \chi \in \mathbf{WFF}) [\varphi \wedge (\psi \vee \chi) &\equiv (\varphi \wedge \psi) \vee (\varphi \wedge \chi)], \\ (\forall \varphi, \psi, \chi \in \mathbf{WFF}) [\varphi \vee (\psi \wedge \chi) &\equiv (\varphi \vee \psi) \wedge (\varphi \vee \chi)]. \end{aligned}$$

Remark (Interpretation). Conjunction distributes over disjunction, and disjunction distributes over conjunction. These laws are the main algebraic engine behind CNF and DNF conversion.

Remark (Dependencies).

- [Logical Equivalence](#)
- [Unique Extension of a Truth Assignment](#)

Theorem (Absorption Laws)

Theorem 1.6.7 (Absorption Laws). *For all $\varphi, \psi \in \mathbf{WFF}$,*

$$\varphi \wedge (\varphi \vee \psi) \equiv \varphi,$$

and

$$\varphi \vee (\varphi \wedge \psi) \equiv \varphi.$$

Go to proof.

Remark (Standard quantified statement).

$$\begin{aligned} (\forall \varphi, \psi \in \mathbf{WFF}) [\varphi \wedge (\varphi \vee \psi) &\equiv \varphi], \\ (\forall \varphi, \psi \in \mathbf{WFF}) [\varphi \vee (\varphi \wedge \psi) &\equiv \varphi]. \end{aligned}$$

Remark (Interpretation). Once φ is already required, adding $\varphi \vee \psi$ contributes no additional constraint. Dually, once φ is already allowed, adding $\varphi \wedge \psi$ contributes no additional possibility.

Remark (Dependencies).

- [Logical Equivalence](#)
- [Basic Boolean Equivalence Laws](#)
- [Distributive Laws for Propositional Logic](#)

Theorem (Conditional Equivalence Laws)

Theorem 1.6.8 (Conditional Equivalence Laws). *For all $\varphi, \psi \in \mathbf{WFF}$,*

$$\varphi \rightarrow \psi \equiv \neg\varphi \vee \psi,$$

and

$$\neg(\varphi \rightarrow \psi) \equiv \varphi \wedge \neg\psi.$$

Go to proof.

Remark (Standard quantified statement).

$$\begin{aligned} (\forall \varphi, \psi \in \mathbf{WFF}) [\varphi \rightarrow \psi &\equiv \neg\varphi \vee \psi], \\ (\forall \varphi, \psi \in \mathbf{WFF}) [\neg(\varphi \rightarrow \psi) &\equiv \varphi \wedge \neg\psi]. \end{aligned}$$

Remark (Predicate reading).

$$\underbrace{\varphi \rightarrow \psi}_{\text{no counterexample}} \equiv \underbrace{\neg\varphi \vee \psi}_{\text{premise false or conclusion true}}, \quad \underbrace{\neg(\varphi \rightarrow \psi)}_{\text{counterexample}} \equiv \underbrace{\varphi \wedge \neg\psi}_{\text{premise true and conclusion false}}.$$

Remark (Interpretation). A conditional fails exactly when the antecedent is true and the consequent is false.

Remark (Dependencies).

- [Logical Equivalence](#)
- [Unique Extension of a Truth Assignment](#)

Theorem (Biconditional Equivalence Laws)

Theorem 1.6.9 (Biconditional Equivalence Laws). *For all $\varphi, \psi \in \text{WFF}$,*

$$\varphi \leftrightarrow \psi \equiv (\varphi \rightarrow \psi) \wedge (\psi \rightarrow \varphi),$$

and

$$\varphi \leftrightarrow \psi \equiv (\varphi \wedge \psi) \vee (\neg\varphi \wedge \neg\psi).$$

Go to proof.

Remark (Standard quantified statement).

$$\begin{aligned} &(\forall \varphi, \psi \in \text{WFF}) [\varphi \leftrightarrow \psi \equiv (\varphi \rightarrow \psi) \wedge (\psi \rightarrow \varphi)], \\ &(\forall \varphi, \psi \in \text{WFF}) [\varphi \leftrightarrow \psi \equiv (\varphi \wedge \psi) \vee (\neg\varphi \wedge \neg\psi)]. \end{aligned}$$

Remark (Predicate reading).

$$\varphi \leftrightarrow \psi \iff (\varphi \rightarrow \psi) \wedge (\psi \rightarrow \varphi)$$

and also

$$\varphi \leftrightarrow \psi \iff (\varphi \wedge \psi) \vee (\neg\varphi \wedge \neg\psi).$$

The biconditional says that the two formulas have the same truth value.

Remark (Interpretation). A biconditional says that two formulas have the same truth value: both true or both false.

Remark (Dependencies).

- [Logical Equivalence](#)
- [Conditional Equivalence Laws](#)

Definition (Formula Context)

Definition 1.6.10 (Formula Context). A formula context $C[-]$ is a well-formed formula with one distinguished placeholder occurrence $[-]$. For each well-formed formula φ , the expression $C[\varphi]$ denotes the formula obtained by replacing the placeholder with φ .

Remark (Standard quantified statement).

$$\text{FormulaContext}(C[-]) \implies (\forall \varphi \in \text{WFF})(C[\varphi] \in \text{WFF}).$$

Remark (Definition predicate reading).

$$\text{FormulaContext}(C[-])$$

means that $C[-]$ is a one-hole propositional formula context.

Remark (Interpretation). A formula context is a syntactic environment into which a formula can be inserted. Contexts make precise the idea of replacing a subformula inside a larger formula.

Remark (Examples). If

$$C[-] := ([-] \wedge R),$$

then

$$C[P] = (P \wedge R)$$

and

$$C[Q \vee S] = ((Q \vee S) \wedge R).$$

Remark (Dependencies).

- [Well-Formed Formula](#)
- [Subformula](#)

Theorem (Replacement of Logical Equivalents)

Theorem 1.6.11 (Replacement of Logical Equivalents). *Replacing logically equivalent formulas inside a formula context preserves logical equivalence:*

$$\varphi \equiv \psi \implies C[\varphi] \equiv C[\psi].$$

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi, \psi \in \mathbf{WFF})(\forall C[-]) [\text{FormulaContext}(C[-]) \wedge \varphi \equiv \psi \rightarrow C[\varphi] \equiv C[\psi]].$$

Remark (Predicate reading).

$$\varphi \equiv \psi \implies C[\varphi] \equiv C[\psi].$$

A logical equivalence may be used inside a larger formula context without changing the truth behavior of the whole formula.

Remark (Interpretation). Logical equivalence is substitutive: equivalent formulas may be replaced inside larger formulas without changing truth behavior.

Remark (Exposition). The equivalence laws in this section may be used as replacement rules inside larger formulas. If a formula contains a subformula matching one side of a listed equivalence law, that subformula may be replaced by the other side.

Common replacement rules include:

$$\neg\neg\varphi \equiv \varphi,$$

$$\neg(\varphi \wedge \psi) \equiv \neg\varphi \vee \neg\psi, \quad \neg(\varphi \vee \psi) \equiv \neg\varphi \wedge \neg\psi,$$

$$\varphi \rightarrow \psi \equiv \neg\varphi \vee \psi,$$

$$\varphi \leftrightarrow \psi \equiv (\varphi \rightarrow \psi) \wedge (\psi \rightarrow \varphi),$$

and

$$\varphi \leftrightarrow \psi \equiv (\varphi \wedge \psi) \vee (\neg\varphi \wedge \neg\psi).$$

Remark (Exposition). The laws in this section allow formulas to be manipulated algebraically. They are not new formation rules; they are semantic equivalences. Later sections use these laws to transform formulas into normal forms and to reduce the set of primitive connectives.

Remark (Exposition). The algebra of propositions is the bridge from semantics to computation: once formulas are equivalent, they can be rewritten without changing truth behavior.

Remark (Dependencies).

- [Formula Context](#)
- [Logical Equivalence](#)
- [Coincidence Lemma for Propositional Logic](#)
- [Unique Extension of a Truth Assignment](#)

1.7 Normal Forms

Toolkit

This section uses logical equivalence, truth assignments, literals, clauses, and algebraic rewrite laws to motivate normal forms.

Remark (Exposition). The normal-forms section studies controlled syntactic shapes for formulas. Logical equivalence is the governing invariant: conversion may change a formula's shape, but it must preserve truth under every assignment while exposing regular literal, clause, DNF, CNF, and canonical structure.

Orientation

Remark (Section map). This section uses the preceding syntax, semantics, and algebra of propositions as its starting point. It first isolates literals and clauses, then introduces negation normal form, disjunctive normal form, conjunctive normal form, canonical normal forms, and semantic classification uses. Later arguments will use these forms to organize satisfiability questions, proof search, functional completeness, and resolution-style methods.

Remark (Why normal forms matter). Normal forms trade arbitrary expression for regular structure. In a truth-table argument, regular structure helps classify formulas as tautologies, contradictions, satisfiable formulas, or contingencies. In proof search and satisfiability procedures, regular structure turns a formula into a finite combinatorial object that can be searched. In functional completeness, normal forms show how complicated truth functions can be built from simpler connectives.

Definition (Literal)

Definition 1.7.1 (Literal). A *literal* is either a propositional variable P or the negation $\neg P$ of a propositional variable P .

Remark (Standard quantified statement). For a formula $\ell \in \text{WFF}$,

$$\ell \text{ is a literal} \iff (\exists P \in \text{Prop})(\ell = P \text{ or } \ell = \neg P).$$

Remark (Definition predicate reading).

$$\text{Literal}(\ell)$$

means that ℓ is an atomic propositional variable or the negation of one atomic propositional variable.

Remark (Interpretation). Literals are the indivisible signed atoms used in normal forms. They remember which propositional variable is being tested and whether it appears positively or negatively.

Remark (Examples). The formulas P , Q , $\neg P$, and $\neg Q$ are literals.

Remark (Non-Examples). The formulas $P \wedge Q$, $P \vee Q$, and $\neg(P \wedge Q)$ are not literals. The last formula is negated, but it negates a compound formula rather than a single propositional variable.

Remark (Dependencies).

- [Propositional Variable](#)
- [Logical Connective](#)

Definition (Positive Literal)

Definition 1.7.2 (Positive Literal). A *positive literal* is a literal of the form P , where P is a propositional variable.

Remark (Standard quantified statement). For a formula $\ell \in \text{WFF}$,

$$\ell \text{ is a positive literal} \iff (\exists P \in \text{Prop})(\ell = P).$$

Remark (Definition predicate reading).

$$\text{PositiveLiteral}(\ell)$$

means that ℓ is a literal whose sign is positive.

Remark (Interpretation). A positive literal asserts the propositional variable directly. It carries no leading negation.

Remark (Examples). The formulas P and Q are positive literals.

Remark (Dependencies).

- [Literal](#)

Definition (Negative Literal)

Definition 1.7.3 (Negative Literal). A *negative literal* is a literal of the form $\neg P$, where P is a propositional variable.

Remark (Standard quantified statement). For a formula $\ell \in \text{WFF}$,

$$\ell \text{ is a negative literal} \iff (\exists P \in \text{Prop})(\ell = \neg P).$$

Remark (Definition predicate reading).

$$\text{NegativeLiteral}(\ell)$$

means that ℓ is a literal whose sign is negative.

Remark (Interpretation). A negative literal denies exactly one propositional variable. In normal forms, negation is pushed down until it appears only at this literal level.

Remark (Examples). The formulas $\neg P$ and $\neg Q$ are negative literals.

Remark (Dependencies).

- [Literal](#)

Definition (Complementary Literals)

Definition 1.7.4 (Complementary Literals). Two literals are *complementary* if they are P and $\neg P$ for the same propositional variable P .

Remark (Standard quantified statement). For literals ℓ_1, ℓ_2 ,

$$\ell_1 \text{ and } \ell_2 \text{ are complementary} \iff (\exists P \in \text{Prop})((\ell_1 = P \wedge \ell_2 = \neg P) \vee (\ell_1 = \neg P \wedge \ell_2 = P)).$$

Remark (Definition predicate reading).

$$\text{ComplementaryLiterals}(\ell_1, \ell_2)$$

means that the two literals name the same propositional variable with opposite signs.

Remark (Interpretation). Complementary literals are the local contradiction pattern inside clauses and terms. The pair $P, \neg P$ is tied to one variable, not merely to two formulas with opposite truth values under a particular assignment.

Remark (Examples). The literals P and $\neg P$ are complementary, as are Q and $\neg Q$.

Remark (Non-Examples). The literals P and $\neg Q$ are not complementary unless $P = Q$.

Remark (Dependencies).

- [Literal](#)

Definition (Clause)

Definition 1.7.5 (Clause). A *clause* is a finite disjunction of literals.

Remark (Standard quantified statement). A formula $C \in \text{WFF}$ is a clause if there are literals $\ell_1, \dots, \ell_n$ with $n \geq 1$ such that

$$C = \ell_1 \vee \dots \vee \ell_n.$$

The empty clause is treated separately as the empty disjunction.

Remark (Definition predicate reading).

$$\text{Clause}(C)$$

means that C is built by finitely disjoining literals.

Remark (Interpretation). A clause records a finite list of literal-level alternatives. It is satisfied when at least one of its literals is true.

Remark (Examples). The formulas $P \vee Q$, $P \vee \neg Q$, and $\neg P \vee Q \vee R$ are clauses.

Remark (Non-Examples). The formula $P \wedge Q$ is not a clause, because its top-level connective is a conjunction. The formula $\neg(P \vee Q)$ is not a clause, because its negation has not been pushed down to the literal level.

Remark (Dependencies).

- [Literal](#)

Definition (Term)

Definition 1.7.6 (Term). A *term*, also called a *conjunction term*, is a finite conjunction of literals.

Remark (Standard quantified statement). A formula $T \in \text{WFF}$ is a term if there are literals $\ell_1, \dots, \ell_n$ with $n \geq 1$ such that

$$T = \ell_1 \wedge \dots \wedge \ell_n.$$

The empty conjunction is treated separately.

Remark (Definition predicate reading).

$$\text{Term}(T)$$

means that T is built by finitely conjoining literals.

Remark (Interpretation). A term records a finite list of literal-level requirements. It is satisfied when all of its literals are true.

Remark (Examples). The formulas $P \wedge Q$, $P \wedge \neg Q$, and $\neg P \wedge Q \wedge R$ are terms.

Remark (Non-Examples). The formula $P \vee Q$ is not a term, because its top-level connective is a disjunction. The formula $\neg(P \wedge Q)$ is not a term, because its negation has not been pushed down to the literal level.

Remark (Dependencies).

- [Literal](#)

Definition (Empty Clause)

Definition 1.7.7 (Empty Clause). The *empty clause* is the empty disjunction of literals. Semantically, it is read as false.

Remark (Standard quantified statement). The empty clause is the disjunction over no literals and has truth value **False** under every truth assignment.

Remark (Definition predicate reading).

$$\text{EmptyClause}(C)$$

means that C is the clause with no disjuncts.

Remark (Interpretation). A nonempty clause is true when at least one listed literal is true. With no literals listed, there is no witness to make the disjunction true, so the empty clause is the false boundary case for clauses.

Remark (Dependencies).

- [Clause](#)
- [Contradiction](#)

Definition (Empty Conjunction)

Definition 1.7.8 (Empty Conjunction). The *empty conjunction* is the conjunction over no literals. Semantically, it is read as true.

Remark (Standard quantified statement). The empty conjunction is the conjunction over no literals and has truth value **True** under every truth assignment.

Remark (Definition predicate reading).

$$\text{EmptyConjunction}(T)$$

means that T is the term with no conjuncts.

Remark (Interpretation). A nonempty term is true when every listed literal is true. With no literals listed, there is no requirement to violate, so the empty conjunction is the true boundary case for terms.

Remark (Dependencies).

- [Term](#)
- [Tautology](#)

Definition (Negation Normal Form)

Definition 1.7.9 (Negation Normal Form). A formula is in *negation normal form* if it uses only propositional variables, negated propositional variables, conjunction, and disjunction, and every negation occurs immediately before a propositional variable.

Remark (Standard quantified statement). For a formula $\varphi \in \text{WFF}$,

$$\varphi \text{ is in negation normal form} \iff \varphi \text{ is built from literals using only } \wedge \text{ and } \vee.$$

Remark (Definition predicate reading).

$\text{NegationNormalForm}(\varphi)$

means that φ has no conditionals, no biconditionals, and no negation outside the literal level.

Remark (Interpretation). Negation normal form is the first cleanup stage for propositional formulas. It does not force a formula to be a disjunction of terms or a conjunction of clauses; it only restricts the available connectives and pushes negation down to propositional variables.

Remark (Examples). The formulas P , $\neg P$, $P \wedge (\neg Q \vee R)$, and $(P \vee Q) \wedge (\neg R \vee S)$ are in negation normal form.

Remark (Non-Examples). The formulas $P \rightarrow Q$, $P \leftrightarrow Q$, and $\neg(P \wedge Q)$ are not in negation normal form. The first two use connectives not allowed in negation normal form, and the last has a negation applied to a compound formula.

Remark (Dependencies).

- [Literal](#)
- [Well-Formed Formula](#)
- [Logical Equivalence](#)

Definition (NNF Conversion Procedure)

Definition 1.7.10 (NNF Conversion Procedure). The *NNF conversion procedure* converts a propositional formula to an equivalent formula in negation normal form by the following steps:

1. eliminate biconditionals using biconditional equivalence laws;
2. eliminate conditionals using conditional equivalence laws;
3. eliminate double negations;
4. push negations inward using De Morgan laws.

Remark (Standard quantified statement). Starting with $\varphi \in \mathbf{WFF}$, repeatedly apply the equivalences

$$\begin{aligned}\alpha \leftrightarrow \beta &\equiv (\alpha \rightarrow \beta) \wedge (\beta \rightarrow \alpha), & \alpha \rightarrow \beta &\equiv \neg\alpha \vee \beta, \\ \neg\neg\alpha &\equiv \alpha, & \neg(\alpha \wedge \beta) &\equiv \neg\alpha \vee \neg\beta, & \neg(\alpha \vee \beta) &\equiv \neg\alpha \wedge \neg\beta,\end{aligned}$$

until negations occur only immediately before propositional variables and only $\wedge$ and $\vee$ remain above the literal level.

Remark (Definition predicate reading).

$\text{NNFConversionProcedure}(\varphi, \psi)$

means that ψ is obtained from φ by eliminating biconditionals and conditionals, removing double negations, and pushing remaining negations inward.

Remark (Interpretation). The procedure is a syntax-directed rewrite process. Each rewrite step is backed by a logical equivalence law, so the output formula has the same truth conditions as the input formula.

Remark (Dependencies).

- [Conditional Equivalence Laws](#)
- [Biconditional Equivalence Laws](#)
- [De Morgan Laws for Propositional Logic](#)
- [Basic Boolean Equivalence Laws](#)

Theorem (Existence of Negation Normal Form)

Theorem 1.7.11 (Existence of Negation Normal Form). *For every formula $\varphi \in \text{WFF}$, there exists a formula $\psi \in \text{WFF}$ such that ψ is in negation normal form and*

$$\varphi \equiv \psi.$$

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi \in \text{WFF})(\exists \psi \in \text{WFF})(\psi \text{ is in negation normal form} \wedge \varphi \equiv \psi).$$

Remark (Predicate reading).

$$\text{NegationNormalFormExists}(\varphi)$$

means that φ has a logically equivalent representative in negation normal form.

Remark (Interpretation). Every propositional formula can be rewritten so that negation appears only at the variable level. This makes negation normal form a semantic normalizing step: it changes shape while preserving logical equivalence.

Remark (Dependencies).

- [Negation Normal Form](#)
- [NNF Conversion Procedure](#)
- [Replacement of Logical Equivalents](#)
- [Structural Induction for Propositional Formulas](#)

Definition (Elementary Conjunction)

Definition 1.7.12 (Elementary Conjunction). An *elementary conjunction* is a finite conjunction of [literals](#). That is, it is a [term](#) of the form

$$L_1 \wedge L_2 \wedge \cdots \wedge L_k,$$

where each L_i is a literal. The one-literal case is permitted.

Remark (Standard quantified statement).

$$L_1 \wedge L_2 \wedge \cdots \wedge L_k, \quad L_i \text{ is a literal for each } i.$$

Remark (Interpretation). Elementary conjunctions are the conjunction terms used as the building blocks of disjunctive normal form. They record simultaneous literal requirements.

Example (Examples).

$$P, \quad P \wedge \neg Q, \quad \neg P \wedge R \wedge S$$

are elementary conjunctions.

Remark (Dependencies).

- [Term](#)
- [Literal](#)

Definition (Minterm)

Definition 1.7.13 (Minterm). Let $P_1, \dots, P_n$ be a finite list of propositional variables. A *minterm* relative to $P_1, \dots, P_n$ is an [elementary conjunction](#) that contains exactly one of P_i or $\neg P_i$ for each $i = 1, \dots, n$.

Remark (Standard quantified statement).

$$M = L_1 \wedge \cdots \wedge L_n,$$

where for each i , exactly one of P_i or $\neg P_i$ appears among the literals $L_1, \dots, L_n$.

Remark (Interpretation). A minterm relative to $P_1, \dots, P_n$ specifies one complete truth-value assignment on those variables: choosing P_i requires P_i to be true, and choosing $\neg P_i$ requires P_i to be false.

Example (Examples). Relative to P, Q, R , the formula

$$P \wedge \neg Q \wedge R$$

is a minterm. Relative to P, Q, R , the formula $P \wedge R$ is not a minterm because it omits both Q and $\neg Q$.

Remark (Dependencies).

- [Elementary Conjunction](#)
- [Variable Set of a Formula](#)

Definition (Disjunctive Normal Form)

Definition 1.7.14 (Disjunctive Normal Form). A formula is in *disjunctive normal form*, abbreviated *DNF*, when it is a finite disjunction of [elementary conjunctions](#). Equivalently, it has the shape

$$C_1 \vee C_2 \vee \cdots \vee C_m,$$

where each C_j is an elementary conjunction.

Remark (Standard quantified statement).

$$\varphi \text{ is in DNF} \iff \varphi = C_1 \vee C_2 \vee \cdots \vee C_m$$

for finitely many elementary conjunctions $C_1, \dots, C_m$.

Remark (Interpretation). DNF is a syntactic form, but it is used semantically: once a formula is replaced by a [logically equivalent](#) DNF formula, questions about [satisfiability](#) reduce to questions about individual conjunction terms.

Example (Examples).

$$(P \wedge Q) \vee (\neg P \wedge R) \vee S$$

is in DNF because each disjunct is an elementary conjunction.

Remark (Dependencies).

- [Elementary Conjunction](#)
- [Logical Equivalence](#)
- [Satisfiable Formula](#)

Theorem (Existence of Disjunctive Normal Form)

Theorem 1.7.15 (Existence of Disjunctive Normal Form). *For every formula $\varphi \in \text{WFF}$, there exists a formula $\psi \in \text{WFF}$ such that ψ is in [disjunctive normal form](#) and*

$$\varphi \equiv \psi.$$

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi \in \text{WFF})(\exists \psi \in \text{WFF})(\psi \text{ is in DNF} \wedge \varphi \equiv \psi).$$

Remark (Interpretation). One first converts φ to negation normal form, then repeatedly applies distributive laws to move conjunctions below disjunctions. Replacement of logical equivalents preserves equivalence at each step.

Remark (Dependencies).

- [Disjunctive Normal Form](#)
- [Negation Normal Form](#)
- [Distributive Laws for Propositional Logic](#)
- [Replacement of Logical Equivalents](#)
- [Existence of Negation Normal Form](#)

Theorem (DNF Satisfiability Criterion)

Theorem 1.7.16 (DNF Satisfiability Criterion). *Let ψ be a formula in **disjunctive normal form**. Then ψ is **satisfiable** if and only if at least one conjunction term in ψ contains no **complementary pair of literals**. Go to proof.*

Remark (Standard quantified statement).

ψ is a satisfiable DNF formula $\iff$ some conjunction term of ψ has no complementary literals. ■

Remark (Interpretation). A DNF formula is true exactly when at least one of its conjunction terms is true. A conjunction term can be made true exactly when it does not require both a variable and its negation.

Remark (Dependencies).

- [Disjunctive Normal Form](#)
- [Complementary Literals](#)
- [Satisfiable Formula](#)

Definition (Elementary Disjunction)

Definition 1.7.17 (Elementary Disjunction). An *elementary disjunction* is a finite disjunction of **literals**. That is, it is a **clause** of the form

$$L_1 \vee L_2 \vee \cdots \vee L_k,$$

where each L_i is a literal. The one-literal case is permitted.

Remark (Standard quantified statement).

$$L_1 \vee L_2 \vee \cdots \vee L_k, \quad L_i \text{ is a literal for each } i.$$

Remark (Interpretation). Elementary disjunctions are the clause-level building blocks used in conjunctive normal form. They record finite literal alternatives.

Example (Examples).

$$P, \quad P \vee \neg Q, \quad \neg P \vee R \vee S$$

are elementary disjunctions.

Remark (Dependencies).

- [Clause](#)
- [Literal](#)

Definition (Maxterm)

Definition 1.7.18 (Maxterm). Let $P_1, \dots, P_n$ be a finite list of propositional variables. A *maxterm* relative to $P_1, \dots, P_n$ is an **elementary disjunction** that contains exactly one of P_i or $\neg P_i$ for each $i = 1, \dots, n$.

Remark (Standard quantified statement).

$$M = L_1 \vee \cdots \vee L_n,$$

where for each i , exactly one of P_i or $\neg P_i$ appears among the literals $L_1, \dots, L_n$.

Remark (Interpretation). A maxterm relative to $P_1, \dots, P_n$ specifies one literal-level alternative for every variable in the list. It is false under exactly the assignment that makes each listed literal false.

Example (Examples). Relative to P, Q, R , the formula

$$P \vee \neg Q \vee R$$

is a maxterm. Relative to P, Q, R , the formula $P \vee R$ is not a maxterm because it omits both Q and $\neg Q$.

Remark (Dependencies).

- [Elementary Disjunction](#)
- [Variable Set of a Formula](#)

Definition (Conjunctive Normal Form)

Definition 1.7.19 (Conjunctive Normal Form). A formula is in *conjunctive normal form*, abbreviated *CNF*, when it is a finite conjunction of [clauses](#). Equivalently, it has the shape

$$C_1 \wedge C_2 \wedge \cdots \wedge C_m,$$

where each C_j is a clause.

Remark (Standard quantified statement).

$$\varphi \text{ is in CNF} \iff \varphi = C_1 \wedge C_2 \wedge \cdots \wedge C_m$$

for finitely many clauses $C_1, \dots, C_m$.

Remark (Interpretation). CNF is a syntactic form used to expose clause-by-clause constraints. Replacing a formula by a [logically equivalent](#) CNF formula turns global validity and [contradiction](#) questions into questions about the interaction of its clauses.

Example (Examples).

$$(P \vee Q) \wedge (\neg P \vee R) \wedge S$$

is in CNF because each conjunct is a clause.

Remark (Dependencies).

- [Clause](#)
- [Logical Equivalence](#)

- [Contradiction](#)

Theorem (Existence of Conjunctive Normal Form)

Theorem 1.7.20 (Existence of Conjunctive Normal Form). *For every formula $\varphi \in \text{WFF}$, there exists a formula $\psi \in \text{WFF}$ such that ψ is in [conjunctive normal form](#) and*

$$\varphi \equiv \psi.$$

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi \in \text{WFF})(\exists \psi \in \text{WFF})(\psi \text{ is in CNF} \wedge \varphi \equiv \psi).$$

Remark (Interpretation). One first converts φ to negation normal form, then repeatedly applies distributive laws to move disjunctions below conjunctions. Replacement of logical equivalents preserves equivalence through every local rewrite.

Remark (Dependencies).

- [Conjunctive Normal Form](#)
- [Negation Normal Form](#)
- [Distributive Laws for Propositional Logic](#)
- [Replacement of Logical Equivalents](#)
- [Existence of Negation Normal Form](#)

Remark (Equivalence-preserving conversion). Normal form conversion is a rewrite process that replaces a formula by a logically equivalent formula with a prescribed syntactic shape. Each local rewrite is justified by an equivalence law, and each replacement inside a larger formula is justified by [Replacement of Logical Equivalents](#).

The goal is not to preserve the exact parse tree of the original formula. The goal is to preserve truth value under every valuation while moving the formula toward [negation normal form](#), [disjunctive normal form](#), or [conjunctive normal form](#).

Remark (Step 1: eliminate biconditionals). Replace every biconditional by an equivalent formula using the [Biconditional Equivalence Laws](#). A standard replacement is

$$\alpha \leftrightarrow \beta \quad \rightsquigarrow \quad (\alpha \rightarrow \beta) \wedge (\beta \rightarrow \alpha).$$

After this step, no occurrence of $\leftrightarrow$ remains.

Remark (Step 2: eliminate conditionals). Replace every conditional by an equivalent disjunction using the [Conditional Equivalence Laws](#). The standard replacement is

$$\alpha \rightarrow \beta \quad \rightsquigarrow \quad \neg \alpha \vee \beta.$$

After this step, the only connectives still present are $\neg$, $\wedge$, and $\vee$.

Remark (Step 3: push negations inward). Use [De Morgan Laws for Propositional Logic](#) and double-negation simplification to move every negation down to the literal level:

$$\neg(\alpha \wedge \beta) \rightsquigarrow \neg\alpha \vee \neg\beta, \quad \neg(\alpha \vee \beta) \rightsquigarrow \neg\alpha \wedge \neg\beta, \quad \neg\neg\alpha \rightsquigarrow \alpha.$$

When this step terminates, the result is in negation normal form: negations occur only immediately in front of propositional variables.

Remark (Step 4A: distribute for DNF). To obtain DNF from negation normal form, distribute conjunction over disjunction using the [Distributive Laws for Propositional Logic](#):

$$\alpha \wedge (\beta \vee \gamma) \rightsquigarrow (\alpha \wedge \beta) \vee (\alpha \wedge \gamma),$$

and similarly

$$(\beta \vee \gamma) \wedge \alpha \rightsquigarrow (\beta \wedge \alpha) \vee (\gamma \wedge \alpha).$$

Repeatedly applying these rewrites moves disjunctions outward until the formula is a finite disjunction of conjunction terms.

Remark (Step 4B: distribute for CNF). To obtain CNF from negation normal form, distribute disjunction over conjunction:

$$\alpha \vee (\beta \wedge \gamma) \rightsquigarrow (\alpha \vee \beta) \wedge (\alpha \vee \gamma),$$

and similarly

$$(\beta \wedge \gamma) \vee \alpha \rightsquigarrow (\beta \vee \alpha) \wedge (\gamma \vee \alpha).$$

Repeatedly applying these rewrites moves conjunctions outward until the formula is a finite conjunction of clauses.

Remark (DNF conversion workflow). To convert a formula to DNF:

1. eliminate biconditionals;
2. eliminate conditionals;
3. push negations inward to obtain negation normal form;
4. distribute conjunction over disjunction until the result is a finite disjunction of elementary conjunctions.

The resulting formula is logically equivalent to the original formula, but it need not be the shortest or only possible DNF representation.

Remark (CNF conversion workflow). To convert a formula to CNF:

1. eliminate biconditionals;
2. eliminate conditionals;
3. push negations inward to obtain negation normal form;
4. distribute disjunction over conjunction until the result is a finite conjunction of clauses.

The resulting formula is logically equivalent to the original formula, but it need not be the shortest or only possible CNF representation.

Remark (Termination). The first three stages terminate because they remove or strictly simplify specific connective patterns. Eliminating biconditionals reduces the number of $\leftrightarrow$ symbols. Eliminating conditionals reduces the number of $\rightarrow$ symbols. Pushing negations inward reduces the distance between each negation and the propositional variables it governs, while double negations shorten the formula.

The distribution stages terminate when guided by a target measure: for DNF, keep applying distributions that move $\vee$ outward past $\wedge$; for CNF, keep applying distributions that move $\wedge$ outward past $\vee$. Each application lowers the number of target-obstructing connective patterns, even though it may duplicate subformulas.

Remark (Correctness). Correctness is by equivalence preservation. Each rewrite instance is one of the listed logical equivalence laws: [biconditional laws](#), [conditional laws](#), [De Morgan laws](#), or [distributive laws](#). Replacement of logical equivalents permits the rewrite to occur inside an arbitrary larger formula. Therefore the final normal form is logically equivalent to the original formula.

Remark (Non-uniqueness). Ordinary DNF and CNF are not unique. Reordering terms or clauses, reordering literals inside a term or clause, duplicating a term, deleting a redundant term, or applying absorption-style simplifications can produce different formulas with the same truth table and the same normal-form type.

Canonical normal forms impose extra conventions to remove this freedom. The ordinary conversion procedures in this topic do not impose those conventions.

Remark (Size blowup). Distribution can duplicate subformulas. As a result, converting a compact formula to DNF or CNF can produce a formula whose size is much larger than the original. This size blowup is a syntactic cost of forcing a formula into a flat disjunction-of-terms or conjunction-of-clauses shape.

Remark (Comparison).

Target form	Final outer connective pattern	Distribution used
NNF	negations only at literals	none
DNF	disjunction of conjunction terms	$\wedge$ over $\vee$
CNF	conjunction of clauses	$\vee$ over $\wedge$

All three workflows use equivalence-preserving rewrites. DNF and CNF share the same first three stages and differ only in the final distribution direction.

Definition (Truth-Table Row Conjunction)

Definition 1.7.21 (Truth-Table Row Conjunction). Fix a finite variable list $P_1, \dots, P_n$ and a row r of the [truth table](#) for those variables. The *truth-table row conjunction* for r is the conjunction

$$L_1 \wedge L_2 \wedge \cdots \wedge L_n,$$

where

$$L_i = \begin{cases} P_i, & \text{if row } r \text{ makes } P_i \text{ true,} \\ \neg P_i, & \text{if row } r \text{ makes } P_i \text{ false.} \end{cases}$$

Remark (Standard quantified statement). For a fixed row r on variables $P_1, \dots, P_n$,

$$\bigwedge_{i=1}^n L_i, \quad L_i = \begin{cases} P_i, & r(P_i) = \text{True}, \\ \neg P_i, & r(P_i) = \text{False}. \end{cases}$$

Remark (Interpretation). The row conjunction names exactly one truth-table row. It is true on that row and false on every row that disagrees with it on at least one listed variable.

Remark (Dependencies).

- [Truth Table](#)
- [Literal](#)
- [Variable Set of a Formula](#)

Definition (Truth-Table Row Clause)

Definition 1.7.22 (Truth-Table Row Clause). Fix a finite variable list $P_1, \dots, P_n$ and a row r of the [truth table](#) for those variables. The *truth-table row clause* for r is the clause

$$M_1 \vee M_2 \vee \cdots \vee M_n,$$

where

$$M_i = \begin{cases} P_i, & \text{if row } r \text{ makes } P_i \text{ false,} \\ \neg P_i, & \text{if row } r \text{ makes } P_i \text{ true.} \end{cases}$$

Remark (Standard quantified statement). For a fixed row r on variables $P_1, \dots, P_n$,

$$\bigvee_{i=1}^n M_i, \quad M_i = \begin{cases} P_i, & r(P_i) = \text{False}, \\ \neg P_i, & r(P_i) = \text{True}. \end{cases}$$

Remark (Interpretation). The row clause excludes exactly one truth-table row. It is false on row r and true on every row that disagrees with r on at least one listed variable.

Remark (Dependencies).

- [Truth Table](#)

- [Clause](#)
- [Variable Set of a Formula](#)

Definition (Canonical Disjunctive Normal Form)

Definition 1.7.23 (Canonical Disjunctive Normal Form). Let $\varphi \in \text{WFF}$, and fix an ordering $P_1, \dots, P_n$ of the finite variable set of φ . The *canonical disjunctive normal form* of φ relative to this ordering is the disjunction of the [truth-table row conjunctions](#) corresponding to the rows on which φ is true.

Remark (Standard quantified statement). For $\text{Var}(\varphi) = \{P_1, \dots, P_n\}$, the canonical disjunctive normal form is

$$\bigvee_{\hat{v}(\varphi)=\text{True}} (L_1 \wedge \dots \wedge L_n),$$

where each row conjunction is determined by the truth-table row of v .

Remark (Interpretation). Canonical DNF records exactly the truth-table rows on which the formula is true, using one row conjunction for each true row.

Remark (Examples). If φ is a contradiction, then it has no true rows. Its canonical DNF is the empty disjunction, represented by the [empty clause](#).

Remark (Dependencies).

- [Disjunctive Normal Form](#)
- [Truth-Table Row Conjunction](#)
- [Truth Table](#)
- [Empty Clause](#)

Definition (Canonical Conjunctive Normal Form)

Definition 1.7.24 (Canonical Conjunctive Normal Form). Let $\varphi \in \text{WFF}$, and fix an ordering $P_1, \dots, P_n$ of the finite variable set of φ . The *canonical conjunctive normal form* of φ relative to this ordering is the conjunction of the [truth-table row clauses](#) corresponding to the rows on which φ is false.

Remark (Standard quantified statement). For $\text{Var}(\varphi) = \{P_1, \dots, P_n\}$, the canonical conjunctive normal form is

$$\bigwedge_{\hat{v}(\varphi)=\text{False}} (M_1 \vee \dots \vee M_n),$$

where each row clause is determined by the truth-table row of v .

Remark (Interpretation). Canonical CNF records exactly the truth-table rows on which the formula is false, using one row clause for each false row.

Remark (Examples). If φ is a tautology, then it has no false rows. Its canonical CNF is the empty conjunction, represented by the [empty conjunction](#).

Remark (Dependencies).

- [Conjunctive Normal Form](#)
- [Truth-Table Row Clause](#)
- [Truth Table](#)
- [Empty Conjunction](#)

Theorem (Canonical Disjunctive Normal Form)

Theorem 1.7.25 (Canonical Disjunctive Normal Form Theorem). *Every formula $\varphi \in \text{WFF}$ is [logically equivalent](#) to its [canonical disjunctive normal form](#) relative to a fixed ordering of its finitely many variables.*

$$\varphi \equiv \text{CDNF}_{P_1, \dots, P_n}(\varphi).$$

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi \in \text{WFF}) \varphi \equiv \text{CDNF}_{P_1, \dots, P_n}(\varphi),$$

where $P_1, \dots, P_n$ is a fixed ordering of the finitely many variables of φ .

Remark (Interpretation). Canonical DNF rebuilds the formula from exactly the truth-table rows where the formula is true. Each row conjunction recognizes one true row, and the final disjunction accepts any of those rows.

Remark (Dependencies).

- [Canonical Disjunctive Normal Form](#)
- [Finite Truth-Table Theorem](#)
- [Coincidence Lemma for Propositional Logic](#)
- [Logical Equivalence](#)

Theorem (Canonical Conjunctive Normal Form)

Theorem 1.7.26 (Canonical Conjunctive Normal Form Theorem). *Every formula $\varphi \in \text{WFF}$ is [logically equivalent](#) to its [canonical conjunctive normal form](#) relative to a fixed ordering of its finitely many variables.*

$$\varphi \equiv \text{CCNF}_{P_1, \dots, P_n}(\varphi).$$

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi \in \mathbf{WFF}) \varphi \equiv \text{CCNF}_{P_1, \dots, P_n}(\varphi),$$

where $P_1, \dots, P_n$ is a fixed ordering of the finitely many variables of φ .

Remark (Interpretation). Canonical CNF rebuilds the formula by excluding exactly the truth-table rows where the formula is false. Each row clause rejects one false row, and the final conjunction requires all false rows to be rejected.

Remark (Dependencies).

- Canonical Conjunctive Normal Form
- Finite Truth-Table Theorem
- Coincidence Lemma for Propositional Logic
- Logical Equivalence

Theorem (Semantic Classification by Normal Forms)

Theorem 1.7.27 (Semantic Classification by Normal Forms). *Normal forms give finite certificates for semantic classification. For any formula $\varphi \in \mathbf{WFF}$:*

1. φ is *satisfiable* if and only if some *DNF* formula equivalent to φ has a satisfiable conjunction term.
2. φ is a *contradiction* if and only if some DNF formula equivalent to φ has no satisfiable conjunction term.
3. φ is a *tautology* if and only if some *CNF* formula equivalent to φ has no falsifiable clause.
4. φ is a *contingency* if and only if it is satisfiable and not a tautology.

Go to proof.

Remark (Standard quantified statement).

$$\begin{aligned} \varphi \text{ is satisfiable} &\iff \text{some equivalent DNF term is satisfiable,} \\ \varphi \text{ is a contradiction} &\iff \text{no equivalent DNF term is satisfiable,} \\ \varphi \text{ is a tautology} &\iff \text{no equivalent CNF clause is falsifiable.} \end{aligned}$$

Remark (Interpretation). The truth-table method classifies φ by evaluating every row directly. The normal-form method first rewrites φ into an equivalent DNF or CNF formula and then checks the finite terms or clauses. Canonical normal forms connect the two approaches: canonical DNF is built from true rows, and canonical CNF is built from false rows.

Remark (Exposition). DNF is naturally witness-producing. A satisfiable term gives an explicit way to make the whole disjunction true. CNF is naturally suited for clause checking: each clause is a local obstruction to falsifying the whole conjunction. This clause-centered view is also the starting point for later resolution methods.

Remark (Dependencies).

- [Disjunctive Normal Form](#)
- [Conjunctive Normal Form](#)
- [Satisfiable Formula](#)
- [Contradiction](#)
- [Tautology](#)
- [Contingency](#)
- [DNF Satisfiability Criterion](#)
- [Canonical Disjunctive Normal Form Theorem](#)
- [Canonical Conjunctive Normal Form Theorem](#)

Summary

Remark (Literal vocabulary). The vocabulary begins at the literal level. A literal is a propositional variable or the negation of one propositional variable. Positive literals and negative literals record the sign of that occurrence. Complementary literals are the opposed pair P and $\neg P$. Clauses are finite disjunctions of literals, while terms are finite conjunctions of literals. The empty clause is the false boundary case, and the empty conjunction is the true boundary case.

Remark (Negation normal form). Negation normal form is the first normalization target. It permits only $\neg$, $\wedge$, and $\vee$, with every negation pushed down to the literal level. Its purpose is to remove conditionals and biconditionals and to make negation local. Once a formula is in NNF, the remaining work is a matter of distributing $\wedge$ and $\vee$ in the direction required by the target normal form.

Remark (Disjunctive normal form). Disjunctive normal form writes a formula as a finite disjunction of terms. It is naturally witness-oriented: to make a DNF formula true, it is enough to make one term true, and a satisfiable term directly records compatible literal requirements. This is why DNF is especially useful for satisfiability checks and for building formulas from true rows of a truth table.

Remark (Conjunctive normal form). Conjunctive normal form writes a formula as a finite conjunction of clauses. It is naturally obstruction-oriented: to falsify a CNF formula, one must falsify a clause, and each clause gives a local disjunctive condition. This clause structure makes CNF the preferred shape for later clause checking, resolution-style arguments, and systematic proof search.

Remark (Canonical normal forms). Ordinary DNF and CNF are not unique. Reordering, duplicating, or simplifying terms and clauses can produce different formulas with the same truth table. Canonical normal forms add truth-table discipline. Canonical DNF records the true rows by minterm-style conjunctions, and canonical CNF records the false rows by maxterm-style clauses. These canonical forms show that every finite truth behavior can be represented syntactically.

Remark (Conversion workflow). The conversion workflow has a common beginning. First eliminate biconditionals. Then eliminate conditionals. Then push negations inward until the formula is in negation normal form. From there, distribute conjunction over disjunction to obtain DNF, or distribute disjunction over conjunction to obtain CNF. Each stage is justified by logical equivalence laws and by replacement of logical equivalents inside larger formulas.

Remark (Back to the algebra of propositions). Normal forms are an application of the algebra of propositions. The conversion rules are not new semantic principles; they are uses of earlier equivalence laws such as De Morgan laws, conditional and biconditional equivalences, distributivity, and replacement of equivalents. The normal-form viewpoint therefore shows how algebraic manipulation and semantic preservation work together.

Remark (Forward to functional completeness). Canonical normal forms point forward to functional completeness. If a truth function can be described by its true rows, then it can be represented by a disjunction of conjunctions of literals. If it can be described by its false rows, then it can be represented by a conjunction of disjunctions of literals. This is the bridge from truth tables to the question of which connectives are enough to express every truth function.

Remark (Forward to arguments and proof systems). Normal forms also prepare the transition from single formulas to arguments and proof systems. DNF gives explicit witnesses for satisfiability, while CNF turns formulas into collections of clauses that can be checked and transformed. The same syntactic regularity used here to classify formulas will later support validity tests, derivation systems, resolution methods, and comparisons between semantic consequence and formal proof.

1.8 Functional Completeness

Toolkit

This section uses Boolean functions, truth functions induced by formulas, logical equivalence, canonical normal forms, and connective rewrite laws.

Remark (Exposition). The functional-completeness section studies expressive power. A connective set is complete exactly when formulas built from it can represent every Boolean function, so normal forms and connective reductions become tests for the adequacy of propositional vocabularies.

Orientation

Remark (Section map). This section begins with Boolean functions and formula representation, then introduces connective bases and definability of connectives. It uses normal forms to prove completeness of the standard connectives and then reduces the primitive vocabulary further. The final topics show that NAND and NOR each form a one-connective complete basis, while some familiar connective sets fail to be functionally complete.

Remark (Connection with normal forms). Normal forms explain why functional completeness is possible. Canonical DNF and canonical CNF show that the truth behavior of a formula can be reconstructed from finitely many truth-table rows. Once those canonical constructions are available, proving that a connective set is functionally complete amounts to showing that the connectives needed for those normal forms can be defined from that set.

Boolean Functions and Representation

Definition (Representation of a Boolean Function)

Definition 1.8.1 (Representation of a Boolean Function). Let $f : \mathbb{B}^n \rightarrow \mathbb{B}$ be a Boolean function. A formula $\varphi \in \text{WFF}$ with variables among $P_1, \dots, P_n$ *represents* f if for every truth assignment v ,

$$\widehat{v}(\varphi) = f(v(P_1), \dots, v(P_n)).$$

Remark (Standard quantified statement).

$$\text{Represents}(\varphi, f; P_1, \dots, P_n) \iff (\forall v : \text{Prop} \rightarrow \mathbb{B}) (\widehat{v}(\varphi) = f(v(P_1), \dots, v(P_n))).$$

Remark (Definition predicate reading). $\text{Represents}(\varphi, f; P_1, \dots, P_n)$ means that φ has exactly the same truth table as the Boolean function f on the listed variables.

Remark (Interpretation). A formula represents a Boolean function when the formula computes the same truth value as the function for every input row. This turns a semantic table of values into a syntactic object inside the propositional language.

Remark (Dependencies).

- [Boolean Function](#)
- [Truth Function Induced by a Formula](#)
- [Truth Assignment](#)

Theorem (DNF Representation of Boolean Functions)

Theorem 1.8.2 (DNF Representation of Boolean Functions). *Every Boolean function $f : \mathbb{B}^n \rightarrow \mathbb{B}$ is represented by a formula in disjunctive normal form using variables $P_1, \dots, P_n$.*

Go to proof.

Remark (Standard quantified statement).

$$(\forall f : \mathbb{B}^n \rightarrow \mathbb{B})(\exists \varphi \in \mathbf{WFF})(\varphi \text{ is in DNF} \wedge \text{Represents}(\varphi, f; P_1, \dots, P_n)).$$

Remark (Predicate reading). Every n -ary Boolean function has a propositional formula representative, and one may choose the representative to be in disjunctive normal form.

Remark (Interpretation). A truth table specifies which input rows make the Boolean function true. For each true row, form the corresponding conjunction of literals; the disjunction of those row conjunctions represents the entire function.

Remark (Dependencies).

- [Representation of a Boolean Function](#)
- [Canonical Disjunctive Normal Form](#)
- [Canonical Disjunctive Normal Form Theorem](#)
- [Disjunctive Normal Form](#)

Connective Bases

Definition (Connective Basis)

Definition 1.8.3 (Connective Basis). A *connective basis* is a specified set B of propositional connectives allowed as primitive connectives for building formulas.

Remark (Standard quantified statement).

$$B \text{ is a connective basis} \iff B \subseteq \{\neg, \wedge, \vee, \rightarrow, \leftrightarrow, \dots\}$$

and formulas over B are built using only connectives from B .

Remark (Definition predicate reading). $\text{ConnectiveBasis}(B)$ means that B is the chosen primitive connective vocabulary for a propositional language or fragment.

Remark (Interpretation). A connective basis controls which connectives are primitive, not which truth functions are expressible. Different bases may have the same expressive power.

Remark (Dependencies).

- [Logical Connective](#)
- [Propositional Language](#)

Definition (Definable Connective)

Definition 1.8.4 (Definable Connective). Let B be a connective basis. A connective $\circ$ is *definable from* B if there is a formula built using only connectives from B whose truth function agrees with the truth function of $\circ$.

Remark (Standard quantified statement).

$$\circ \text{ is definable from } B \iff \exists \theta \in \mathbf{WFF}_B (f_\theta = f_\circ),$$

where $\mathbf{WFF}_B$ denotes the formulas built using only connectives from B .

Remark (Definition predicate reading). $\text{DefinableFrom}(\circ, B)$ means that $\circ$ can be treated as an abbreviation for a formula pattern using only connectives from B .

Remark (Interpretation). Definability separates notation from expressive power. A connective can be omitted from the primitive syntax without losing its use, provided it is recoverable as a defined connective.

Remark (Dependencies).

- [Connective Basis](#)
- [Truth Function Induced by a Formula](#)
- [Boolean Function](#)

Definition (Functionally Complete Connective Set)

Definition 1.8.5 (Functionally Complete Connective Set). A connective basis B is *functionally complete* if every Boolean function $f : \mathbb{B}^n \rightarrow \mathbb{B}$, for every $n \geq 1$, is represented by some formula using only connectives from B .

Remark (Standard quantified statement).

$$\text{FunctionallyComplete}(B) \iff (\forall n \geq 1)(\forall f : \mathbb{B}^n \rightarrow \mathbb{B})(\exists \varphi \in \mathbf{WFF}_B) \text{Represents}(\varphi, f; P_1, \dots, P_n). \blacksquare$$

Remark (Definition predicate reading). $\text{FunctionallyComplete}(B)$ means that formulas over B can represent every finite Boolean input-output table.

Remark (Interpretation). Functional completeness is the central expressive-power property for connective sets. It says that the basis can express all possible truth-functional behavior, not merely the connectives originally listed in the language.

Remark (Dependencies).

- [Connective Basis](#)
- [Representation of a Boolean Function](#)
- [Boolean Function](#)

Definition (Expressively Equivalent Connective Sets)

Definition 1.8.6 (Expressively Equivalent Connective Sets). Two connective bases B and C are *expressively equivalent* if every connective in B is definable from C , and every connective in C is definable from B .

Remark (Standard quantified statement).

$$B \equiv_{\text{expr}} C \iff ((\forall o \in B) \text{DefinableFrom}(o, C)) \wedge ((\forall \star \in C) \text{DefinableFrom}(\star, B)).$$

Remark (Definition predicate reading). $B \equiv_{\text{expr}} C$ means that the two bases can define exactly the same connective behavior.

Remark (Interpretation). Expressive equivalence lets a proof replace one primitive vocabulary with another. This is why a language can be simplified syntactically without changing which truth functions can be expressed.

Remark (Dependencies).

- [Connective Basis](#)
- [Definable Connective](#)

Completeness of the Standard Connectives

Theorem (Standard Connectives are Functionally Complete)

Theorem 1.8.7 (Standard Connectives are Functionally Complete). *The connective basis*

$$\{\neg, \wedge, \vee\}$$

is functionally complete.

Go to proof.

Remark (Standard quantified statement).

$$\text{FunctionallyComplete}(\{\neg, \wedge, \vee\}).$$

Remark (Predicate reading). Every Boolean function is represented by a formula using only negation, conjunction, and disjunction.

Remark (Interpretation). Canonical disjunctive normal form uses only literals, conjunctions, and disjunctions. Since literals use variables and negation, the normal-form theorem shows that $\{\neg, \wedge, \vee\}$ can express every Boolean function.

Remark (Dependencies).

- [Functionally Complete Connective Set](#)
- [DNF Representation of Boolean Functions](#)
- [Disjunctive Normal Form](#)

Theorem (Negation and Conjunction are Functionally Complete)

Theorem 1.8.8 (Negation and Conjunction are Functionally Complete). *The connective basis*

$$\{\neg, \wedge\}$$

is functionally complete.

Go to proof.

Remark (Standard quantified statement).

$$\text{FunctionallyComplete}(\{\neg, \wedge\}).$$

Remark (Predicate reading). Every Boolean function is represented by a formula using only negation and conjunction.

Remark (Interpretation). Disjunction is definable from negation and conjunction by De Morgan's law:

$$\varphi \vee \psi \equiv \neg(\neg\varphi \wedge \neg\psi).$$

Thus every formula using $\neg, \wedge, \vee$ can be translated into an equivalent formula using only $\neg$ and $\wedge$.

Remark (Dependencies).

- [Standard Connectives are Functionally Complete](#)
- [De Morgan Laws for Propositional Logic](#)
- [Replacement of Logical Equivalents](#)

Theorem (Negation and Disjunction are Functionally Complete)

Theorem 1.8.9 (Negation and Disjunction are Functionally Complete). *The connective basis*

$$\{\neg, \vee\}$$

is functionally complete.

Go to proof.

Remark (Standard quantified statement).

$$\text{FunctionallyComplete}(\{\neg, \vee\}).$$

Remark (Predicate reading). Every Boolean function is represented by a formula using only negation and disjunction.

Remark (Interpretation). Conjunction is definable from negation and disjunction by De Morgan's law:

$$\varphi \wedge \psi \equiv \neg(\neg\varphi \vee \neg\psi).$$

Thus every formula using $\neg, \wedge, \vee$ can be translated into an equivalent formula using only $\neg$ and $\vee$.

Remark (Dependencies).

- [Standard Connectives are Functionally Complete](#)
- [De Morgan Laws for Propositional Logic](#)
- [Replacement of Logical Equivalents](#)

Reduction of Connective Sets

Definition (Connective Reduction)

Definition 1.8.10 (Connective Reduction). A *connective reduction* replaces a formula using one connective basis by a logically equivalent formula using a smaller or different connective basis.

Remark (Standard quantified statement).

$$\text{Reduction}_{B \rightarrow C}(\varphi, \psi) \iff \varphi \equiv \psi \quad \text{and} \quad \psi \in \text{WFF}_C.$$

Remark (Definition predicate reading). $\text{Reduction}_{B \rightarrow C}(\varphi, \psi)$ means that ψ is a translation of φ into the connective basis C without changing truth behavior.

Remark (Interpretation). Reduction shows that some primitive connectives are optional. If a connective is definable from the remaining basis, then formulas using that connective can be translated away.

Remark (Exposition). The standard reductions are:

$$\varphi \rightarrow \psi \equiv \neg\varphi \vee \psi,$$

$$\varphi \leftrightarrow \psi \equiv (\varphi \rightarrow \psi) \wedge (\psi \rightarrow \varphi),$$

$$\varphi \vee \psi \equiv \neg(\neg\varphi \wedge \neg\psi), \quad \varphi \wedge \psi \equiv \neg(\neg\varphi \vee \neg\psi).$$

Each replacement preserves logical equivalence and may be performed inside a larger formula context.

Remark (Exposition). Reducing a connective basis can make formulas longer. Functional completeness is therefore an expressive result, not an efficiency result. A small basis may be powerful enough to express every Boolean function while still being awkward for ordinary human-readable formulas.

Remark (Dependencies).

- [Connective Basis](#)
- [Definable Connective](#)
- [Logical Equivalence](#)

NAND

Definition (NAND Connective)

Definition 1.8.11 (NAND Connective). The *NAND* connective, written $\uparrow$, is the binary connective defined by

$$\varphi \uparrow \psi \equiv \neg(\varphi \wedge \psi).$$

Remark (Standard quantified statement).

$$\widehat{v}(\varphi \uparrow \psi) = \text{True} \iff \text{not both } \widehat{v}(\varphi) = \text{True} \text{ and } \widehat{v}(\psi) = \text{True}.$$

Remark (Definition predicate reading). $\text{NAND}(\varphi, \psi)$ means the negation of the conjunction of φ and ψ .

Remark (Interpretation). NAND is read as "not both." It is false only when both inputs are true. This single connective can reproduce negation, conjunction, and disjunction.

Remark (Dependencies).

- [Logical Connective](#)
- [Truth Assignment](#)

Definition (NAND-Only Formula)

Definition 1.8.12 (NAND-Only Formula). A *NAND-only formula* is a well-formed formula whose only connective is $\uparrow$.

Remark (Standard quantified statement).

$$\varphi \text{ is NAND-only} \iff \varphi \in \text{WFF}_{\{\uparrow\}}.$$

Remark (Definition predicate reading). $\text{NANDOnly}(\varphi)$ means that every connective occurrence in φ is an occurrence of $\uparrow$.

Remark (Interpretation). A NAND-only formula is syntactically restricted but expressively powerful. The restriction is severe at the level of primitive symbols, but not at the level of truth functions.

Remark (Dependencies).

- [NAND Connective](#)
- [Well-Formed Formula](#)

Theorem (NAND is Functionally Complete)

Theorem 1.8.13 (NAND is Functionally Complete). *The one-connective basis $\{\uparrow\}$ is functionally complete.*

Go to proof.

Remark (Standard quantified statement).

$$\text{FunctionallyComplete}(\{\uparrow\}).$$

Remark (Predicate reading). Every Boolean function can be represented by a NAND-only formula.

Remark (Interpretation). NAND can define negation and conjunction:

$$\neg\varphi \equiv \varphi \uparrow \varphi, \quad \varphi \wedge \psi \equiv (\varphi \uparrow \psi) \uparrow (\varphi \uparrow \psi).$$

Once negation and conjunction are definable, functional completeness follows from the functional completeness of $\{\neg, \wedge\}$.

Remark (Dependencies).

- [NAND Connective](#)
- [NAND-Only Formula](#)
- [Functionally Complete Connective Set](#)
- [Negation and Conjunction are Functionally Complete](#)

NOR

Definition (NOR Connective)

Definition 1.8.14 (NOR Connective). The *NOR* connective, written $\downarrow$, is the binary connective defined by

$$\varphi \downarrow \psi \equiv \neg(\varphi \vee \psi).$$

Remark (Standard quantified statement).

$$\widehat{v}(\varphi \downarrow \psi) = \text{True} \iff \widehat{v}(\varphi) = \text{False} \text{ and } \widehat{v}(\psi) = \text{False}.$$

Remark (Definition predicate reading). $\text{NOR}(\varphi, \psi)$ means the negation of the disjunction of φ and ψ .

Remark (Interpretation). NOR is read as "neither." It is true exactly when both inputs are false. Like NAND, this single connective can reproduce the standard functionally complete basis.

Remark (Dependencies).

- [Logical Connective](#)
- [Truth Assignment](#)

Definition (NOR-Only Formula)

Definition 1.8.15 (NOR-Only Formula). A *NOR-only formula* is a well-formed formula whose only connective is $\downarrow$.

Remark (Standard quantified statement).

$$\varphi \text{ is NOR-only} \iff \varphi \in \text{WFF}_{\{\downarrow\}}.$$

Remark (Definition predicate reading). $\text{NOROnly}(\varphi)$ means that every connective occurrence in φ is an occurrence of $\downarrow$.

Remark (Interpretation). A NOR-only formula uses a single primitive connective. Its expressive power comes from the ability of NOR to define negation and disjunction.

Remark (Dependencies).

- [NOR Connective](#)
- [Well-Formed Formula](#)

Theorem (NOR is Functionally Complete)

Theorem 1.8.16 (NOR is Functionally Complete). *The one-connective basis $\{\downarrow\}$ is functionally complete.*
Go to proof.

Remark (Standard quantified statement).

$$\text{FunctionallyComplete}(\{\downarrow\}).$$

Remark (Predicate reading). Every Boolean function can be represented by a NOR-only formula.

Remark (Interpretation). NOR can define negation and disjunction:

$$\neg\varphi \equiv \varphi \downarrow \varphi, \quad \varphi \vee \psi \equiv (\varphi \downarrow \psi) \downarrow (\varphi \downarrow \psi).$$

Once negation and disjunction are definable, functional completeness follows from the functional completeness of $\{\neg, \vee\}$.

Remark (Dependencies).

- [NOR Connective](#)
- [NOR-Only Formula](#)
- [Functionally Complete Connective Set](#)
- [Negation and Disjunction are Functionally Complete](#)

Incomplete Connective Sets

Definition (Incomplete Connective Set)

Definition 1.8.17 (Incomplete Connective Set). A connective basis is *incomplete* if it is not functionally complete.

Remark (Standard quantified statement).

$$\text{Incomplete}(B) \iff \neg \text{FunctionallyComplete}(B).$$

Remark (Definition predicate reading). $\text{Incomplete}(B)$ means that some Boolean function cannot be represented by any formula using only connectives from B .

Remark (Interpretation). Incompleteness is usually proved by finding an invariant preserved by every formula built from the basis. If some Boolean function fails that invariant, the basis cannot express every Boolean function.

Remark (Dependencies).

- [Functionally Complete Connective Set](#)
- [Boolean Function](#)

Definition (Monotone Boolean Function)

Definition 1.8.18 (Monotone Boolean Function). A Boolean function $f : \mathbb{B}^n \rightarrow \mathbb{B}$ is *monotone* if changing input coordinates from **False** to **True** never changes the output from **True** to **False**.

Remark (Standard quantified statement).

$$\text{Monotone}(f) \iff (\forall a, b \in \mathbb{B}^n)(a \leq b \rightarrow f(a) \leq f(b)),$$

where **False** $\leq$ **True** coordinatewise.

Remark (Definition predicate reading). $\text{Monotone}(f)$ means that increasing truth inputs cannot make the output decrease from true to false.

Remark (Interpretation). The connectives $\wedge$ and $\vee$ are monotone. Negation is not monotone, because changing P from false to true changes $\neg P$ from true to false.

Remark (Dependencies).

- [Boolean Function](#)

Theorem (Conjunction and Disjunction are not Functionally Complete)

Theorem 1.8.19 (Conjunction and Disjunction are not Functionally Complete). *The connective basis $\{\wedge, \vee\}$ is not functionally complete. Go to proof.*

Remark (Standard quantified statement).

$$\neg \text{FunctionallyComplete}(\{\wedge, \vee\}).$$

Remark (Predicate reading). There is at least one Boolean function that cannot be represented by any formula using only conjunction and disjunction.

Remark (Interpretation). Every formula built using only $\wedge$ and $\vee$ represents a monotone Boolean function. Since negation is not monotone, no formula using only $\wedge$ and $\vee$ can represent negation. Hence the basis is incomplete.

Remark (Dependencies).

- [Incomplete Connective Set](#)
- [Monotone Boolean Function](#)
- [Functionally Complete Connective Set](#)

Minimal Complete Bases

Definition (Minimal Complete Basis)

Definition 1.8.20 (Minimal Complete Basis). A functionally complete connective basis B is *minimal complete* if no proper subset of B is functionally complete.

Remark (Standard quantified statement).

$\text{MinimalComplete}(B) \iff \text{FunctionallyComplete}(B) \wedge (\forall C \subsetneq B) \neg \text{FunctionallyComplete}(C).$

Remark (Definition predicate reading). $\text{MinimalComplete}(B)$ means that B is complete, but removing any primitive connective from B destroys completeness.

Remark (Interpretation). Minimality is relative to deletion of connectives from the basis. It does not mean that formulas using the basis are short or easy to read.

Remark (Dependencies).

- [Functionally Complete Connective Set](#)
- [Connective Basis](#)

Theorem (NAND and NOR are Minimal Complete Bases)

Theorem 1.8.21 (NAND and NOR are Minimal Complete Bases). *The one-connective bases $\{\uparrow\}$ and $\{\downarrow\}$ are minimal complete bases. Go to proof.*

Remark (Standard quantified statement).

$$\text{MinimalComplete}(\{\uparrow\}) \wedge \text{MinimalComplete}(\{\downarrow\}).$$

Remark (Predicate reading). NAND alone is complete, NOR alone is complete, and deleting the sole connective from either basis leaves no functionally complete basis.

Remark (Interpretation). A one-connective complete basis is automatically minimal once the empty basis is not functionally complete. NAND and NOR are therefore minimal complete bases because each is complete by itself.

Remark (Dependencies).

- [Minimal Complete Basis](#)
- [NAND is Functionally Complete](#)
- [NOR is Functionally Complete](#)
- [Incomplete Connective Set](#)

Summary and Transition

Remark (Summary). Functional completeness measures the expressive strength of a connective basis. A functionally complete basis can represent every finite Boolean function by a formula using only the connectives in that basis. Normal forms supply the main construction: truth-table rows can be converted into formulas, and those formulas can be assembled into a representative for the desired Boolean function.

Remark (Section map). The standard connectives $\neg$, $\wedge$, and $\vee$ are functionally complete because they can build disjunctive normal forms. De Morgan laws reduce this basis to $\{\neg, \wedge\}$ or $\{\neg, \vee\}$. NAND and NOR go further: each single connective can define a complete two-connective basis and therefore can represent every Boolean function.

Remark (Transition). The next section moves from expressive power to argument forms and informal proof patterns. Functional completeness explains which truth functions can be expressed. Argument forms ask which patterns of inference preserve truth from premises to conclusions.

1.9 Argument Forms and Informal Proof Patterns

Toolkit

This section uses argument forms, inference patterns, replacement rules, informal proof patterns, and semantic validity.

Remark (Exposition). Argument forms organize recurring patterns of propositional reasoning. This section connects valid and invalid forms with replacement principles, informal proof practice, and the semantic tests that justify those patterns.

Orientation

Remark (Working vocabulary). This orientation uses truth assignments, logical consequence, tautology, logical equivalence, formula contexts, and counterexample assignments to justify common argument forms.

Remark (Exposition). Argument forms are reusable patterns of inference. The purpose of this section is to justify the basic inference tools that will be used throughout the rest of the project. The justification is semantic: an argument form is valid when no truth assignment makes every premise true while making the conclusion false.

Remark (Section map). This section introduces argument forms, substitution instances, validity, invalidity, and counterexample assignments. It then proves the validity of the standard inference patterns, isolates common invalid forms, explains rules of replacement, and connects informal proof patterns with semantic consequence. Formal proof systems are previewed but not developed here.

Remark (Boundary). The goal is rigorous senior-undergraduate propositional logic. The section proves that the proof moves used later are truth-preserving, but it does not build Hilbert systems, sequent calculi, cut elimination, or metatheoretic completeness machinery.

Argument Forms

Remark (Working vocabulary). This topic introduces argument forms, premises, conclusions, substitution instances, valid argument forms, invalid argument forms, and counterexample truth assignments.

Definition (Argument Form)

Definition 1.9.1 (Argument Form). An *argument form* is a schematic pattern consisting of a finite list of premise formulas and one conclusion formula.

Remark (Standard quantified statement).

$$\text{ArgumentForm}(\Gamma, \varphi) \iff \Gamma \subseteq \text{WFF is finite and } \varphi \in \text{WFF}.$$

Remark (Definition predicate reading). $\text{ArgumentForm}(\Gamma, \varphi)$ means that Γ is the set or finite list of premises and φ is the conclusion.

Remark (Interpretation). An argument form records a pattern of inference independently of the particular subject matter. Its validity depends on truth preservation, not on whether the individual formulas happen to be true in one example.

Remark (Dependencies).

- [Well-Formed Formula](#)

Definition (Valid Argument Form)

Definition 1.9.2 (Valid Argument Form). An argument form with premises Γ and conclusion φ is *valid* if every truth assignment that satisfies every formula in Γ also satisfies φ .

Remark (Standard quantified statement).

$$\text{ValidArgumentForm}(\Gamma, \varphi) \iff (\forall v)((\forall \gamma \in \Gamma)(v \models \gamma) \rightarrow v \models \varphi).$$

Remark (Definition predicate reading). $\text{ValidArgumentForm}(\Gamma, \varphi)$ means that truth of all premises forces truth of the conclusion under every truth assignment.

Remark (Interpretation). Validity is a preservation property. A valid argument form may have false premises in a particular instance, but it cannot have true premises and a false conclusion under the same truth assignment.

Remark (Dependencies).

- [Argument Form](#)
- [Satisfaction of a Formula](#)
- [Logical Consequence](#)

Definition (Counterexample Assignment)

Definition 1.9.3 (Counterexample Assignment). A *counterexample assignment* to an argument form (Γ, φ) is a truth assignment that satisfies every premise in Γ and does not satisfy φ .

Remark (Standard quantified statement).

$$\text{CounterexampleAssignment}(v, \Gamma, \varphi) \iff (\forall \gamma \in \Gamma)(v \models \gamma) \wedge v \not\models \varphi.$$

Remark (Definition predicate reading). $\text{CounterexampleAssignment}(v, \Gamma, \varphi)$ means that v shows the premises can all be true while the conclusion is false.

Remark (Interpretation). A counterexample assignment is the semantic witness to invalidity. Finding one settles the question: the argument form is not truth-preserving.

Remark (Dependencies).

- [Valid Argument Form](#)
- [Truth Assignment](#)

Theorem (Truth-Table Validity Test for Argument Forms)

Theorem 1.9.4 (Truth-Table Validity Test for Argument Forms). An argument form (Γ, φ) is valid if and only if no row of its truth table makes every formula in Γ true and φ false.
Go to proof.

Remark (Standard quantified statement).

$$\text{ValidArgumentForm}(\Gamma, \varphi) \iff \neg(\exists v) \text{CounterexampleAssignment}(v, \Gamma, \varphi).$$

Remark (Predicate reading). The argument form is valid exactly when there is no counterexample truth assignment.

Remark (Interpretation). The truth-table test is finite because only the variables appearing in the premises and conclusion matter. A failed row is a counterexample assignment; the absence of such rows proves semantic validity.

Remark (Dependencies).

- [Valid Argument Form](#)
- [Counterexample Assignment](#)
- [Truth Table](#)
- [Finite Truth-Table Theorem](#)

Core Valid Inference Patterns

Remark (Working vocabulary). This topic proves the semantic validity of the standard propositional inference patterns used throughout later proof work.

Theorem (Modus Ponens is Valid)

Theorem 1.9.5 (Modus Ponens is Valid). *The argument form*

$$\varphi, \quad \varphi \rightarrow \psi \quad \therefore \quad \psi$$

is valid.

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi, \psi \in \text{WFF}) \text{ValidArgumentForm}(\{\varphi, \varphi \rightarrow \psi\}, \psi).$$

Remark (Predicate reading). Whenever φ and $\varphi \rightarrow \psi$ are both true, ψ must be true.

Remark (Interpretation). Modus ponens is the basic rule for using a conditional whose antecedent has already been established.

Remark (Dependencies).

- [Valid Argument Form](#)
- [Truth Assignment](#)

Theorem (Modus Tollens is Valid)

Theorem 1.9.6 (Modus Tollens is Valid). *The argument form*

$$\varphi \rightarrow \psi, \quad \neg \psi \quad \therefore \quad \neg \varphi$$

is valid.

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi, \psi \in \text{WFF}) \text{ValidArgumentForm}(\{\varphi \rightarrow \psi, \neg \psi\}, \neg \varphi).$$

Remark (Predicate reading). If φ would imply ψ , and ψ is false, then φ must be false.

Remark (Interpretation). Modus tollens is the semantic form of denying an antecedent by denying the conditional's consequent.

Remark (Dependencies).

- [Valid Argument Form](#)
- [Conditional Equivalence Laws](#)

Theorem (Hypothetical Syllogism is Valid)

Theorem 1.9.7 (Hypothetical Syllogism is Valid). *The argument form*

$$\varphi \rightarrow \psi, \quad \psi \rightarrow \chi \quad \therefore \quad \varphi \rightarrow \chi$$

is valid.

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi, \psi, \chi \in \text{WFF}) \text{ValidArgumentForm}(\{\varphi \rightarrow \psi, \psi \rightarrow \chi\}, \varphi \rightarrow \chi).$$

Remark (Predicate reading). Conditional implication composes: if φ forces ψ , and ψ forces χ , then φ forces χ .

Remark (Interpretation). Hypothetical syllogism justifies chaining conditional statements.

Remark (Dependencies).

- [Valid Argument Form](#)
- [Logical Consequence](#)

Theorem (Disjunctive Syllogism is Valid)

Theorem 1.9.8 (Disjunctive Syllogism is Valid). *The argument form*

$$\varphi \vee \psi, \quad \neg \varphi \quad \therefore \quad \psi$$

is valid.

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi, \psi \in \text{WFF}) \text{ValidArgumentForm}(\{\varphi \vee \psi, \neg \varphi\}, \psi).$$

Remark (Predicate reading). If at least one of φ or ψ is true and φ is false, then ψ must be true.

Remark (Interpretation). Disjunctive syllogism eliminates one disjunct after its negation is known.

Remark (Dependencies).

- [Valid Argument Form](#)
- [Truth Assignment](#)

Theorem (Addition is Valid)

Theorem 1.9.9 (Addition is Valid). *The argument form*

$$\varphi \quad \therefore \quad \varphi \vee \psi$$

is valid.

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi, \psi \in \text{WFF}) \text{ValidArgumentForm}(\{\varphi\}, \varphi \vee \psi).$$

Remark (Predicate reading). A true formula remains true when placed as one disjunct of a disjunction.

Remark (Interpretation). Addition introduces a disjunction from a known true disjunct.

Remark (Dependencies).

- [Valid Argument Form](#)

Theorem (Simplification is Valid)

Theorem 1.9.10 (Simplification is Valid). *The argument form*

$$\varphi \wedge \psi \quad \therefore \quad \varphi$$

is valid.

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi, \psi \in \text{WFF}) \text{ValidArgumentForm}(\{\varphi \wedge \psi\}, \varphi).$$

Remark (Predicate reading). If a conjunction is true, then each conjunct is true.

Remark (Interpretation). Simplification extracts a component from a true conjunction.

Remark (Dependencies).

- [Valid Argument Form](#)

Theorem (Conjunction Introduction is Valid)

Theorem 1.9.11 (Conjunction Introduction is Valid). *The argument form*

$$\varphi, \quad \psi \quad \therefore \quad \varphi \wedge \psi$$

is valid.

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi, \psi \in \text{WFF}) \text{ValidArgumentForm}(\{\varphi, \psi\}, \varphi \wedge \psi).$$

Remark (Predicate reading). If two formulas are both true, then their conjunction is true.

Remark (Interpretation). Conjunction introduction combines independently established formulas into one conjunction.

Remark (Dependencies).

- [Valid Argument Form](#)

Theorem (Constructive Dilemma is Valid)

Theorem 1.9.12 (Constructive Dilemma is Valid). *The argument form*

$$\varphi \rightarrow \chi, \quad \psi \rightarrow \theta, \quad \varphi \vee \psi \quad \therefore \quad \chi \vee \theta$$

is valid.

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi, \psi, \chi, \theta \in \text{WFF}) \text{ValidArgumentForm}(\{\varphi \rightarrow \chi, \psi \rightarrow \theta, \varphi \vee \psi\}, \chi \vee \theta).$$

Remark (Predicate reading). If one of two antecedents holds, and each antecedent has its own consequence, then at least one corresponding consequence holds.

Remark (Interpretation). Constructive dilemma is a proof-by-cases pattern with conditionals attached to both cases.

Remark (Dependencies).

- [Valid Argument Form](#)
- [Disjunctive Syllogism is Valid](#)

Theorem (Destructive Dilemma is Valid)

Theorem 1.9.13 (Destructive Dilemma is Valid). *The argument form*

$$\varphi \rightarrow \chi, \quad \psi \rightarrow \theta, \quad \neg \chi \vee \neg \theta \quad \therefore \quad \neg \varphi \vee \neg \psi$$

is valid.

Go to proof.

Remark (Standard quantified statement).

$$(\forall \varphi, \psi, \chi, \theta \in \text{WFF}) \text{ValidArgumentForm}(\{\varphi \rightarrow \chi, \psi \rightarrow \theta, \neg \chi \vee \neg \theta\}, \neg \varphi \vee \neg \psi).$$

Remark (Predicate reading). If at least one consequent fails, and each antecedent would force its consequent, then at least one antecedent must fail.

Remark (Interpretation). Destructive dilemma combines disjunction with modus tollens-style reasoning.

Remark (Dependencies).

- [Valid Argument Form](#)
- [Modus Tollens is Valid](#)

Invalid Argument Forms

Remark (Working vocabulary). This topic uses counterexample assignments to show why familiar invalid forms fail to preserve truth from premises to conclusion.

Theorem (Affirming the Consequent is Invalid)

Theorem 1.9.14 (Affirming the Consequent is Invalid). *The argument form*

$$\varphi \rightarrow \psi, \quad \psi \quad \therefore \quad \varphi$$

is invalid.

Go to proof.

Remark (Standard quantified statement).

$$\neg \text{ValidArgumentForm}(\{\varphi \rightarrow \psi, \psi\}, \varphi).$$

Remark (Predicate reading). There is a truth assignment under which $\varphi \rightarrow \psi$ and ψ are true while φ is false.

Remark (Interpretation). A conditional can be true because both antecedent and consequent are true, but also because the antecedent is false. Therefore truth of the consequent does not force truth of the antecedent.

Remark (Dependencies).

- [Counterexample Assignment](#)
- [Valid Argument Form](#)

Theorem (Denying the Antecedent is Invalid)

Theorem 1.9.15 (Denying the Antecedent is Invalid). *The argument form*

$$\varphi \rightarrow \psi, \quad \neg\varphi \quad \therefore \quad \neg\psi$$

is invalid.

Go to proof.

Remark (Standard quantified statement).

$$\neg \text{ValidArgumentForm}(\{\varphi \rightarrow \psi, \neg\varphi\}, \neg\psi).$$

Remark (Predicate reading). There is a truth assignment under which $\varphi \rightarrow \psi$ and $\neg\varphi$ are true while $\neg\psi$ is false.

Remark (Interpretation). A conditional does not say that the antecedent is the only way for the consequent to be true. The antecedent can fail while the consequent still holds.

Remark (Exposition). For both invalid forms, take a truth assignment with φ false and ψ true. Then $\varphi \rightarrow \psi$ is true. This assignment makes the premises true and the conclusion false in each corresponding invalid pattern.

Remark (Dependencies).

- [Counterexample Assignment](#)
- [Valid Argument Form](#)

Rules of Replacement

Remark (Working vocabulary). This topic uses formula contexts, logical equivalence, and replacement of logical equivalents to justify algebraic rewriting inside larger formulas.

Definition (Rule of Replacement)

Definition 1.9.16 (Rule of Replacement). A *rule of replacement* is a schematic permission to replace a subformula by a logically equivalent formula inside a formula context.

Remark (Standard quantified statement).

$$\varphi \equiv \psi \implies C[\varphi] \equiv C[\psi].$$

Remark (Definition predicate reading). A rule of replacement licenses an equivalence-preserving rewrite inside a larger formula.

Remark (Interpretation). Rules of replacement differ from rules of inference. Inference rules move from premises to conclusions; replacement rules rewrite formulas without changing truth behavior.

Remark (Dependencies).

- [Formula Context](#)
- [Logical Equivalence](#)
- [Replacement of Logical Equivalents](#)

Theorem (Replacement Rules Preserve Logical Equivalence)

Theorem 1.9.17 (Replacement Rules Preserve Logical Equivalence). *If a formula ψ is obtained from a formula φ by finitely many applications of rules of replacement, then $\varphi \equiv \psi$.
Go to proof.*

Remark (Standard quantified statement).

$$\text{ReplacementSequence}(\varphi, \psi) \implies \varphi \equiv \psi.$$

Remark (Predicate reading). A finite sequence of equivalence-preserving replacements preserves logical equivalence from the first formula to the last formula.

Remark (Interpretation). Replacement rules justify algebraic manipulation of formulas. Each local rewrite preserves truth under every truth assignment, so the entire finite rewrite chain also preserves truth under every truth assignment.

Remark (Exposition). Common replacement rules include double negation, De Morgan replacement, commutativity, associativity, distributivity, material implication, biconditional replacement, and contraposition. Each is justified by a logical equivalence law.

Remark (Dependencies).

- [Rule of Replacement](#)
- [Replacement of Logical Equivalents](#)
- [Logical Equivalence is an Equivalence Relation](#)

Informal Proof Patterns

Remark (Working vocabulary). This topic connects valid argument forms with common informal proof patterns: direct proof, conditional proof, proof by cases, contradiction, contrapositive proof, implication chains, and equivalence chains.

Definition (Informal Proof Pattern)

Definition 1.9.18 (Informal Proof Pattern). An *informal proof pattern* is a reusable truth-preserving strategy for organizing an ordinary mathematical proof.

Remark (Standard quantified statement).

$$\text{InformalProofPattern}(\Pi) \implies \Pi \text{ is justified by one or more valid argument forms.}$$

Remark (Definition predicate reading). $\text{InformalProofPattern}(\Pi)$ means that Π is a recognized proof strategy whose correctness rests on truth-preserving inference.

Remark (Interpretation). Informal proof patterns are not a substitute for proof. They are templates for arranging valid moves so that a proof has a clear logical shape.

Remark (Exposition). A direct proof establishes a target conclusion by moving forward from known premises through valid argument forms. A conditional proof establishes $\varphi \rightarrow \psi$ by assuming φ and deriving ψ inside that assumption context. Proof by cases uses a disjunction of cases and shows that each case leads to the desired conclusion.

Remark (Exposition). Proof by contradiction assumes the negation of the desired conclusion and derives an impossible situation. Contrapositive proof establishes $\varphi \rightarrow \psi$ by showing $\neg\psi \rightarrow \neg\varphi$. Chain-of-implications proofs use repeated hypothetical syllogism, while equivalence-chain proofs use replacement and logical equivalence.

Remark (Exposition). If the target has the form $\varphi \rightarrow \psi$, a conditional proof is often natural. If the available information has the form $\varphi \vee \psi$, proof by cases is often natural. If the target is an equivalence, an equivalence chain or two conditional proofs are often natural.

Remark (Dependencies).

- [Valid Argument Form](#)
- [Logical Consequence](#)

Semantic Justification of Argument Forms

Remark (Working vocabulary). This topic uses tautology, logical consequence, valid argument forms, and truth-table reasoning to connect named inference patterns with semantic validity.

Theorem (Tautological Implication Gives a Valid Argument Form)

Theorem 1.9.19 (Tautological Implication Gives a Valid Argument Form). *Let $\Gamma = \{\gamma_1, \dots, \gamma_n\} \subseteq \text{WFF}$ and let $\varphi \in \text{WFF}$. If*

$$(\gamma_1 \wedge \dots \wedge \gamma_n) \rightarrow \varphi$$

*is a tautology, then the argument form (Γ, φ) is valid.
Go to proof.*

Remark (Standard quantified statement).

$$\text{Tautology}((\gamma_1 \wedge \dots \wedge \gamma_n) \rightarrow \varphi) \implies \text{ValidArgumentForm}(\{\gamma_1, \dots, \gamma_n\}, \varphi).$$

Remark (Predicate reading). If the conditional from the conjunction of all premises to the conclusion is always true, then the premises cannot all be true while the conclusion is false.

Remark (Interpretation). This theorem converts a tautology check into an argument-validity check. It is one reason truth tables can validate named inference rules.

Remark (Exposition). The semantic proof pattern for a valid argument form is always the same. Let an arbitrary truth assignment make all premises true. Then show that the conclusion must also be true. If this can be done for an arbitrary truth assignment, then no counterexample assignment exists.

Remark (Exposition). This section does not yet define formal derivations. The word valid here is semantic: it refers to preservation of truth under truth assignments. Later proof systems will turn selected valid patterns into formal rules of inference.

Remark (Dependencies).

- [Tautology](#)
- [Valid Argument Form](#)
- [Logical Consequence](#)

Argument Forms as a Bridge to Formal Systems

Remark (Working vocabulary). This topic distinguishes semantic validity from formal derivability and explains why formal proof systems are deferred.

Remark (Exposition). A valid argument form is a semantic object: it preserves truth under all truth assignments. A formal rule of inference is a syntactic object inside a specified proof system. Many formal systems include rules corresponding to the valid argument forms justified in this section, but the semantic justification and the formal rule are not the same object.

Remark (Exposition). Formal proof systems require additional machinery: formal derivations, assumptions, discharge rules, axioms or axiom schemata, and precise proof-checking rules. Natural deduction, Hilbert systems, tableaux, resolution, and sequent calculus are examples of such systems. They are important, but they are not the main purpose of this introductory propositional-logic section.

Remark (Transition). The tools validated here are safe to use in ordinary mathematical proof because they are truth-preserving. Later, when formal proof systems are introduced, these same patterns can be represented as formal rules, derived rules, or admissible moves depending on the chosen system.

Summary and Transition

Remark (Working vocabulary). This summary reviews argument forms, validity, counterexamples, valid inference patterns, invalid forms, replacement rules, and the semantic foundation for ordinary proof moves.

Remark (Exposition). An argument form is valid when truth of the premises under an arbitrary truth assignment forces truth of the conclusion. The central negative test is a counterexample assignment: one assignment making all premises true and the conclusion false. This section justified both sides of the method by proving standard valid forms and refuting common invalid forms.

Remark (Exposition). The valid tools now grounded include modus ponens, modus tollens, hypothetical syllogism, disjunctive syllogism, addition, simplification, conjunction introduction, constructive dilemma, and destructive dilemma. The invalid forms of affirming the consequent and denying the antecedent were separated by explicit counterexample assignments.

Remark (Transition). The next section turns to compactness and finite satisfiability as a preview of a deeper theme: infinite-looking logical questions can sometimes be controlled by finite witnesses. The semantic foundation developed here remains the base for that discussion.

1.10 Compactness and Finite Satisfiability Preview

Toolkit

This section uses truth assignments, satisfiable sets of formulas, finite subsets, finite satisfiability, finite unsatisfiability witnesses, and propositional compactness.

Remark (Exposition). The compactness preview records the finite-information pattern behind propositional satisfiability: when a set of formulas is unsatisfiable, the failure is already visible in some finite subset.

Orientation

Remark (Orientation). The semantic ideas developed earlier apply formula by formula and also to sets of formulas. This section isolates the finite-information pattern behind that viewpoint. A truth assignment can satisfy a whole set Γ only when it makes every member of Γ true. Failure will be shown, in propositional logic, to be detected by some finite part of Γ . This is the content of compactness.

The goal is not to replace truth tables. It is to record the principle that truth tables suggest: whenever inconsistency appears in propositional logic, it appears after only finitely many formulas have been selected.

Remark (Guiding question). For a possibly infinite set $\Gamma \subseteq \text{WFF}$, when can the existence of truth assignments for every finite piece of Γ be promoted to a truth assignment for all of Γ ?

Remark (Scope of the preview). Everything in this section is stated using truth assignments, formulas, sets of formulas, and finite subsets. Later chapters will need richer semantic data, but this preview stays inside propositional logic.

Satisfiable Sets

Remark (Recall). By [satisfaction of a set of formulas](#), $v \models \Gamma$ means that $v \models \gamma$ for every $\gamma \in \Gamma$.

Definition (Satisfiable Set of Propositional Formulas)

Definition 1.10.1 (Satisfiable Set of Propositional Formulas). Let $\Gamma \subseteq \text{WFF}$. The set Γ is satisfiable if there exists a truth assignment v such that $v \models \Gamma$.

$$\text{SatisfiableSet}(\Gamma) \iff \exists v (\text{TruthAssignment}(v) \wedge v \models \Gamma).$$

Remark (Standard quantified statement).

$$\Gamma \text{ is satisfiable} \iff \exists v \left(\text{TruthAssignment}(v) \wedge \forall \varphi \in \Gamma (\widehat{v}(\varphi) = \text{True}) \right).$$

Remark (Definition predicate reading).

$$\text{SatisfiableSet}(\Gamma)$$

means that one truth assignment makes every formula in Γ true.

Remark (Negated quantified statement).

$$\neg \text{SatisfiableSet}(\Gamma) \iff \forall v \left(\text{TruthAssignment}(v) \Rightarrow \exists \varphi \in \Gamma (\widehat{v}(\varphi) = \text{False}) \right).$$

Remark (Negation predicate reading).

$$\neg \text{SatisfiableSet}(\Gamma)$$

means that every truth assignment fails on at least one formula in Γ .

Remark (Failure modes). Satisfiability fails when every truth assignment is blocked by at least one formula in Γ .

Remark (Failure mode decomposition).

$$\neg \text{SatisfiableSet}(\Gamma) \iff \forall v \left(\text{TruthAssignment}(v) \Rightarrow \exists \varphi \in \Gamma (\widehat{v}(\varphi) = \text{False}) \right).$$

Thus Γ is not satisfiable exactly when every truth assignment is defeated by at least one formula from Γ .

Remark (Interpretation). Satisfiability of a set is a compatibility condition. The formulas in Γ may impose many requirements, but they are satisfiable only if all of those requirements can be met by the same truth assignment.

Remark (Dependencies).

- [Truth Assignment](#)
- [Satisfaction of a Set of Formulas](#)

Definition (Unsatisfiable Set of Propositional Formulas)

Definition 1.10.2 (Unsatisfiable Set of Propositional Formulas). Let $\Gamma \subseteq \text{WFF}$. The set Γ is unsatisfiable if it is not satisfiable.

$$\text{UnsatisfiableSet}(\Gamma) \iff \neg \text{SatisfiableSet}(\Gamma).$$

Remark (Standard quantified statement).

$$\Gamma \text{ is unsatisfiable} \iff \forall v \left(\text{TruthAssignment}(v) \Rightarrow \exists \varphi \in \Gamma (\widehat{v}(\varphi) = \text{False}) \right).$$

Remark (Definition predicate reading).

$$\text{UnsatisfiableSet}(\Gamma)$$

means that no truth assignment makes every formula in Γ true.

Remark (Negated quantified statement).

$$\neg \text{UnsatisfiableSet}(\Gamma) \iff \text{SatisfiableSet}(\Gamma).$$

Remark (Negation predicate reading).

$$\neg \text{UnsatisfiableSet}(\Gamma)$$

means that Γ has at least one satisfying truth assignment.

Remark (Interpretation). Unsatisfiability is global failure of joint truth. It does not say that every formula in Γ is false; it says that no single truth assignment can make them all true at once.

Remark (Dependencies).

- [Satisfiable Set of Propositional Formulas](#)

Finitely Satisfiable Sets

Definition (Finite Subset of a Set of Propositional Formulas)

Definition 1.10.3 (Finite Subset of a Set of Propositional Formulas). Let $\Gamma \subseteq \text{WFF}$. A finite subset of Γ is a set $\Delta \subseteq \Gamma$ containing only finitely many formulas.

Remark (Standard quantified statement).

$$\Delta \text{ is a finite subset of } \Gamma \iff \Delta \subseteq \Gamma \wedge \Delta \text{ is finite.}$$

Remark (Definition predicate reading).

$$\Delta \subseteq \Gamma \quad \text{and} \quad \Delta \text{ is finite}$$

means that Δ is a finite selection of formulas from Γ .

Remark (Interpretation). A finite subset is a finite test case cut out of a larger set of formulas.

Remark (Dependencies).

- [Well-Formed Formula](#)

Definition (Finitely Satisfiable Set of Propositional Formulas)

Definition 1.10.4 (Finitely Satisfiable Set of Propositional Formulas). Let $\Gamma \subseteq \text{WFF}$. The set Γ is finitely satisfiable if every finite subset of Γ is satisfiable.

$$\text{FinitelySatisfiableSet}(\Gamma) \iff \forall \Delta \left((\Delta \subseteq \Gamma \wedge \Delta \text{ is finite}) \Rightarrow \text{SatisfiableSet}(\Delta) \right).$$

Remark (Standard quantified statement).

Γ is finitely satisfiable $\iff \forall \Delta \left((\Delta \subseteq \Gamma \wedge \Delta \text{ is finite}) \Rightarrow \exists v (\text{TruthAssignment}(v) \wedge v \models \Delta) \right)$.

Remark (Definition predicate reading).

FinitelySatisfiableSet(Γ)

means that every finite portion of Γ has at least one satisfying truth assignment.

Remark (Negated quantified statement).

$\neg \text{FinitelySatisfiableSet}(\Gamma) \iff \exists \Delta \left(\Delta \subseteq \Gamma \wedge \Delta \text{ is finite} \wedge \text{UnsatisfiableSet}(\Delta) \right)$.

Remark (Negation predicate reading).

$\neg \text{FinitelySatisfiableSet}(\Gamma)$

means that some finite subset of Γ already has no satisfying truth assignment.

Remark (Failure modes). Finite satisfiability fails when at least one finite subset of Γ is unsatisfiable.

Remark (Failure mode decomposition).

$\neg \text{FinitelySatisfiableSet}(\Gamma) \iff \exists \Delta \left(\Delta \subseteq \Gamma \wedge \Delta \text{ is finite} \wedge \text{UnsatisfiableSet}(\Delta) \right)$.

So failure of finite satisfiability is witnessed by one finite unsatisfiable subcollection.

Remark (Interpretation). Finite satisfiability says that no finite test has found a contradiction. It does not name a single truth assignment for all of Γ ; it only says that each finite test can be passed.

Remark (Dependencies).

- [Finite Subset of a Set of Propositional Formulas](#)
- [Satisfiable Set of Propositional Formulas](#)
- [Truth Assignment](#)

Finite Unsatisfiability Witnesses

Definition (Unsatisfiability Witness)

Definition 1.10.5 (Unsatisfiability Witness). Let $\Gamma \subseteq \text{WFF}$. A subset $\Delta \subseteq \Gamma$ is an unsatisfiability witness for Γ if Δ is unsatisfiable.

Remark (Standard quantified statement).

Δ is an unsatisfiability witness for $\Gamma \iff \Delta \subseteq \Gamma \wedge \text{UnsatisfiableSet}(\Delta)$.

Remark (Definition predicate reading).

$$\Delta \subseteq \Gamma \quad \text{and} \quad \text{UnsatisfiableSet}(\Delta)$$

means that the formulas in Δ already explain why the larger set Γ cannot be satisfied, provided Δ is part of Γ .

Remark (Interpretation). An unsatisfiability witness is a subcollection that already fails to be jointly satisfiable.

Remark (Dependencies).

- [Unsatisfiable Set of Propositional Formulas](#)

Definition (Finite Unsatisfiability Witness)

Definition 1.10.6 (Finite Unsatisfiability Witness). Let $\Gamma \subseteq \text{WFF}$. A subset $\Delta \subseteq \Gamma$ is a finite unsatisfiability witness for Γ if Δ is finite and unsatisfiable.

$$\text{FiniteUnsatisfiabilityWitness}(\Delta, \Gamma) \iff \Delta \subseteq \Gamma \wedge \Delta \text{ is finite} \wedge \text{UnsatisfiableSet}(\Delta).$$

Remark (Standard quantified statement).

$$\Delta \text{ is a finite unsatisfiability witness for } \Gamma \iff \Delta \subseteq \Gamma \wedge \Delta \text{ is finite} \wedge \text{UnsatisfiableSet}(\Delta).$$

Remark (Definition predicate reading).

$$\text{FiniteUnsatisfiabilityWitness}(\Delta, \Gamma)$$

means that Δ is a finite subset of Γ and that Δ already has no satisfying truth assignment.

Remark (Interpretation). A finite unsatisfiability witness is a finite obstruction. It says that the failure of Γ is already visible before one inspects all formulas in Γ .

Remark (Dependencies).

- [Finite Subset of a Set of Propositional Formulas](#)
- [Unsatisfiable Set of Propositional Formulas](#)

Theorem (Finite Unsatisfiability Witness)

Theorem 1.10.7 (Finite Unsatisfiability Witness). *Let $\Gamma \subseteq \text{WFF}$. If there exists a finite $\Delta \subseteq \Gamma$ such that Δ is unsatisfiable, then Γ is unsatisfiable.*

Go to proof.

Remark (Standard quantified statement).

$$\exists \Delta \text{ FiniteUnsatisfiabilityWitness}(\Delta, \Gamma) \Rightarrow \text{UnsatisfiableSet}(\Gamma).$$

Remark (Predicate reading).

$$\exists \Delta \text{ FiniteUnsatisfiabilityWitness}(\Delta, \Gamma) \Rightarrow \text{UnsatisfiableSet}(\Gamma)$$

means that a single finite unsatisfiable subcollection forces the whole set Γ to be unsatisfiable.

Remark (Failure modes). The theorem can fail only if a truth assignment satisfied Γ while a finite subset $\Delta \subseteq \Gamma$ remained unsatisfiable.

Remark (Failure mode decomposition).

$$\text{SatisfiableSet}(\Gamma) \Rightarrow \forall \Delta (\Delta \subseteq \Gamma \Rightarrow \text{SatisfiableSet}(\Delta)).$$

If the whole set has a satisfying truth assignment, each subset inherits one.

Remark (Contrapositive quantified statement).

$$\text{SatisfiableSet}(\Gamma) \Rightarrow \forall \Delta (\Delta \subseteq \Gamma \Rightarrow \text{SatisfiableSet}(\Delta)).$$

Remark (Contrapositive predicate reading).

$$\text{SatisfiableSet}(\Gamma) \Rightarrow \text{FinitelySatisfiableSet}(\Gamma)$$

means that one truth assignment for all of Γ also satisfies every finite subset of Γ .

Remark (Interpretation). An unsatisfiable finite subset is decisive. Adding more formulas cannot restore a truth assignment that already fails on the finite subset.

Remark (Dependencies).

- [Finite Unsatisfiability Witness](#)
- [Unsatisfiable Set of Propositional Formulas](#)
- [Satisfaction of a Set of Formulas](#)

Corollary (Finite Witness Contrapositive)

Corollary 1.10.8 (Finite Witness Contrapositive). *Let $\Gamma \subseteq \text{WFF}$. If Γ is satisfiable, then every finite subset of Γ is satisfiable.*

Go to proof.

Remark (Standard quantified statement).

$$\text{SatisfiableSet}(\Gamma) \Rightarrow \text{FinitelySatisfiableSet}(\Gamma).$$

Remark (Predicate reading).

$$\text{SatisfiableSet}(\Gamma) \Rightarrow \text{FinitelySatisfiableSet}(\Gamma)$$

means that full satisfiability automatically gives finite satisfiability.

Remark (Interpretation). This is the easy direction connecting full satisfiability to finite satisfiability: one truth assignment that works for all formulas also works for any finite selection.

Remark (Dependencies).

- [Finite Unsatisfiability Witness](#)
- [Finitely Satisfiable Set of Propositional Formulas](#)
- [Satisfiable Set of Propositional Formulas](#)

Propositional Compactness

Theorem (Propositional Compactness)

Theorem 1.10.9 (Propositional Compactness). *Let $\Gamma \subseteq \text{WFF}$. If every finite subset of Γ is satisfiable, then Γ is satisfiable.*

Go to proof.

Remark (Standard quantified statement).

$$\text{FinitelySatisfiableSet}(\Gamma) \Rightarrow \text{SatisfiableSet}(\Gamma).$$

Remark (Predicate reading).

$$\text{FinitelySatisfiableSet}(\Gamma) \Rightarrow \text{SatisfiableSet}(\Gamma)$$

means that passing every finite satisfiability test is enough to produce a truth assignment for all of Γ .

Remark (Negated quantified statement).

$$\text{FinitelySatisfiableSet}(\Gamma) \wedge \neg \text{SatisfiableSet}(\Gamma)$$

is the direct negation of the compactness implication.

Remark (Negation predicate reading).

$$\text{FinitelySatisfiableSet}(\Gamma) \wedge \neg \text{SatisfiableSet}(\Gamma)$$

would mean that all finite tests pass but the full set has no satisfying truth assignment.

Remark (Failure modes).

$$\neg \text{SatisfiableSet}(\Gamma) \Rightarrow \exists \Delta \left(\Delta \subseteq \Gamma \wedge \Delta \text{ is finite} \wedge \neg \text{SatisfiableSet}(\Delta) \right).$$

Compactness says that unsatisfiability cannot first appear only at the infinite level.

Remark (Failure mode decomposition).

$$\text{UnsatisfiableSet}(\Gamma) \Rightarrow \exists \Delta \text{ FiniteUnsatisfiabilityWitness}(\Delta, \Gamma).$$

Remark (Contrapositive quantified statement).

$$\text{UnsatisfiableSet}(\Gamma) \Rightarrow \exists \Delta \text{ FiniteUnsatisfiabilityWitness}(\Delta, \Gamma).$$

Remark (Contrapositive predicate reading). The contrapositive says that if Γ is unsatisfiable, then the failure is already witnessed by some finite subset of Γ .

Remark (Interpretation). Propositional compactness turns infinitely many finite checks into one global truth assignment. Equivalently, every propositional inconsistency has a finite witness.

Remark (Dependencies).

- [Finitely Satisfiable Set of Propositional Formulas](#)
- [Satisfiable Set of Propositional Formulas](#)
- [Finite Unsatisfiability Witness](#)

Finite Truth Table Intuition

Remark (Finite truth table intuition). Truth tables are finite because a single formula contains only finitely many variables. More generally, a finite set of formulas contains only finitely many variables, so its satisfiability can be checked by a finite truth table over those variables.

Remark (Finite test). For a finite $\Delta \subseteq \text{WFF}$, the variables appearing in Δ are finite in number. A truth table over those variables decides whether Δ is satisfiable.

Remark (Why compactness is surprising). Finite truth tables directly decide finite sets. Compactness says that, for propositional logic, passing all finite tests is enough to pass the entire possibly infinite test.

Remark (Dependencies).

- [Finiteness of Variables](#)
- [Finite Truth Table](#)
- [Finitely Satisfiable Set of Propositional Formulas](#)

Summary

Remark (Section summary). The compactness preview records three related ideas.

- A set of propositional formulas is satisfiable when one truth assignment makes all of its formulas true.
- A set is finitely satisfiable when every finite subset is satisfiable.
- Propositional compactness says that finite satisfiability is enough for satisfiability.

Remark (Conceptual takeaway). In propositional logic, an inconsistency always has a finite witness. If no finite witness exists, then the whole set of formulas can be satisfied.

Chapter 2

Predicate Logic



2.1 Model-Theoretic View

Toolkit

This section presents first-order logic as a syntax-plus-semantics model, with the language interpreted by structures and formulas evaluated by the shared free-algebra machinery.

Theory (First-Order Logic)

Depends on: Signature; term; formula; sentence; bridge:consequence; $Cn_{\vdash}$

Language

$$\mathcal{L} = (\text{Const}_{\mathcal{L}}, \text{Func}_{\mathcal{L}}, \text{Rel}_{\mathcal{L}}, =).$$

$$\text{Term}_{\mathcal{L}}, \quad \text{Form}_{\mathcal{L}}, \quad \text{Sent}_{\mathcal{L}}.$$

Presented Theory

$$T := Cn_{\vdash}(\Sigma), \quad \Sigma \subseteq \text{Sent}_{\mathcal{L}}.$$

Theorems and Consequences

1.

$$\Gamma \vdash_{\text{FOL}} \varphi \iff \Gamma \models_{\text{FOL}} \varphi \quad (\text{completeness}).$$

Model (First-Order Logic)

Model 2.1.1 (First-Order Logic). **Depends on:** $\mathcal{L}$ -structure; interpretation; assignment; bridge:free-sigma-algebra-evaluation; bridge:consequence

Note

FOL runs the shared machine twice: once for terms, then once for quantifier-free formulas. Quantifiers sit above that fixed-seed machine.

Structure

$$\mathcal{A} = \langle A, I \rangle, \quad A \neq \emptyset.$$

$$I(c) \in A, \quad I(f) : A^n \rightarrow A, \quad I(R) \subseteq A^n, \quad = \text{ is logical equality on } A.$$

Binding

$$s : \text{Var} \rightarrow A \xrightarrow{\text{eval}(\mathcal{A}_{\text{term}})} \llbracket \cdot \rrbracket_{\mathcal{A},s} \xrightarrow{\text{relation/equality tests}} \sigma_s \xrightarrow{\text{eval}(\mathbf{2})} \models_{\text{qf}}.$$

Binding

$$\Sigma_{\text{term}} := \text{Const}_{\mathcal{L}} \cup \text{Func}_{\mathcal{L}}, \quad \mathcal{A}_{\text{term}} := (A, (If)_f \in \text{Func}, (Ic)_{c \in \text{Const}}).$$

$$\llbracket \cdot \rrbracket_{\mathcal{A},s} := \text{eval}(\mathcal{A}_{\text{term}})(s).$$

Binding

$$\sigma_s(R(\bar{t})) := [(\llbracket t_1 \rrbracket_{\mathcal{A},s}, \dots, \llbracket t_n \rrbracket_{\mathcal{A},s}) \in I(R)],$$

$$\sigma_s(t_1 = t_2) := [\llbracket t_1 \rrbracket_{\mathcal{A},s} = \llbracket t_2 \rrbracket_{\mathcal{A},s}].$$

Syntax–Semantics Bridge

$$\mathcal{A}, s \models \varphi \iff \text{eval}(\mathbf{2})(\sigma_s)(\varphi) = 1 \quad (\varphi \text{ quantifier-free}).$$

Syntax–Semantics Bridge

$$\mathcal{A}, s \models \exists x \varphi \iff \exists a \in A, \mathcal{A}, s[x \mapsto a] \models \varphi.$$

$$\mathcal{A}, s \models \forall x \varphi \iff \forall a \in A, \mathcal{A}, s[x \mapsto a] \models \varphi.$$

Syntax–Semantics Bridge

For $\sigma \in \text{Sent}_{\mathcal{L}}$, $\mathcal{A}, s \models \sigma$ is independent of s by the coincidence lemma, so

$$\mathcal{A} \models \sigma := \mathcal{A}, s \models \sigma \quad (s \text{ arbitrary}).$$

Theorems and Consequences

1. Semantic consequence, $\mathcal{A} \models T$, and $\text{Th}(\mathcal{A})$ are inherited from *Consequence*.
2. Prop is the base case: $\text{seed} = v$. FOL is the inductive case: s is evaluated into term values, then relation tests compute σ_s .

Definitional Extension (First-Order Logic)

Depends on: theory:first-order-logic; expansion; reduct; conservativity; unique existence

Note

$$\exists!y \varphi(y) \quad :\equiv \quad \exists y (\varphi(y) \wedge \forall y' (\varphi(y') \rightarrow y' = y)).$$

Admissibility

Relation symbol R , arity n :

$$\forall \bar{x} (R(\bar{x}) \leftrightarrow \varphi(\bar{x})).$$

Function symbol f , arity n :

$$T \vdash \forall \bar{x} \exists!y \psi(\bar{x}, y), \quad \forall \bar{x} \forall y (f(\bar{x}) = y \leftrightarrow \psi(\bar{x}, y)).$$

Constant symbol c :

$$T \vdash \exists!y \psi(y), \quad \forall y (c = y \leftrightarrow \psi(y)).$$

Theorems and Consequences

1.
$$T^+ \vdash \sigma \quad \Longleftrightarrow \quad T \vdash \sigma \quad (\sigma \in \mathcal{L}).$$
2.
$$\mathcal{A} \models T \implies \exists! \mathcal{A}^+ (\mathcal{A}^+ \upharpoonright \mathcal{L} = \mathcal{A}).$$

2.2 Axiom Schemas

Predicate-Logic Axiom Schemas

The predicate-logic development extends propositional logic with the following quantifier axiom schemas. The side conditions prevent variable capture and ensure that the displayed instances are well formed.

Axiom Schema (Predicate Logic 1)

Axiom 2.2.1 (Universal Instantiation). For every formula $\varphi(x)$ and every term t that is free for x in φ ,

$$\forall x \varphi(x) \implies \varphi(t).$$

Remark (Interpretation). A universally quantified statement may be specialized to any admissible instance.

Axiom Schema (Predicate Logic 2)

Axiom 2.2.2 (Quantifier Distribution). For formulas φ and $\psi(x)$, if x is not free in φ , then

$$\forall x (\varphi \implies \psi(x)) \implies (\varphi \implies \forall x \psi(x)).$$

Remark (Interpretation). When a hypothesis does not depend on the quantified variable, a universal conclusion may be drawn uniformly under that hypothesis.

2.3 Syntax

2.3.1 Syntax of First-Order Logic: Terms and Formulas

Remark (Section toolkit: Syntax ledger). This section fixes the syntactic inventory for first-order logic before any semantic machinery is introduced. The ledger proceeds from symbols to terms, from terms to atomic formulas, and from atomic formulas to all well-formed formulas.

Layer	Item	Role
Symbol stock	Variable	reusable placeholder symbol
Symbol stock	Constant Symbol	nullary name-forming symbol
Symbol stock	Function Symbol	operation-forming symbol
Symbol stock	Predicate Symbol	assertion-forming symbol
Object syntax	Term	expression that names an object syntactically
Formula syntax	Atomic Formula	predicate symbol applied to terms
Formula syntax	Well-Formed Formula	formula generated by the grammar
Formula syntax	Molecular Formula	non-atomic well-formed formula

Remark (Exposition). The grammar has two tracks. Terms are built first, because atomic formulas need terms as their inputs. Formulas are built second, because they begin with atomic formulas and then close under logical connectives and quantifiers. Semantic notions such as structures, assignments, satisfaction, truth, and models enter later; the present section records only which strings count as grammatical expressions.

Remark (Grammar boundary). Terms name objects. Formulas make assertions. A term is not a formula, and a formula is not a term. For example, $f(x)$ is a term, while $P(f(x))$ is a formula.

Remark (Predicate-symbol convention). Object-language predicate symbols, such as P, Q, R , are symbols that occur inside first-order formulas. Project metalanguage predicate readings, such as $\text{Term}(t)$ or $\text{AtomicFormula}(\varphi)$, are external classification predicates used by these notes to describe the grammar. The two levels should not be confused.

Definition (Variable)

Definition 2.3.1 (Variable). For a first-order language $\mathcal{L}$, a *variable* is a symbol drawn from the distinguished variable stock of $\mathcal{L}$. Variables are usually written

$$x, y, z, x_0, x_1, x_2, \dots$$

Remark (Standard quantified statement).

$$\text{Var}_{\mathcal{L}} = \{x, y, z, x_0, x_1, x_2, \dots\}.$$

Remark (Definition predicate reading). $\text{Variable}(x)$ means $x \in \text{Var}_{\mathcal{L}}$.

Remark (Negated quantified statement).

$$x \notin \text{Var}_{\mathcal{L}}.$$

Remark (Negation predicate reading). $\neg \text{Variable}(x)$ means that x is not one of the variable symbols of $\mathcal{L}$.

Remark (Failure modes). A symbol fails to be a variable when it belongs to a different syntactic role, such as a constant symbol, function symbol, predicate symbol, connective, or punctuation mark.

Remark (Failure mode decomposition).

$$\underbrace{c}_{\text{constant symbol}} \quad \underbrace{f}_{\text{function symbol}} \quad \underbrace{P}_{\text{predicate symbol}} \quad \underbrace{\forall}_{\text{logical symbol}}.$$

Remark (Interpretation). A variable is a basic syntactic placeholder. It can occur inside terms and formulas, and later sections explain how quantifiers bind occurrences of variables.

Remark (Examples). x, y, z, x_0 , and x_1 are standard variable symbols.

Remark (Non-Examples). $c, 0, f, P, \forall$, and $\wedge$ are not variables merely because they are symbols of the language.

Remark (Dependencies).

- [Language](#)

Definition (Constant Symbol)

Definition 2.3.2 (Constant Symbol). For a first-order language $\mathcal{L}$, a *constant symbol* is a non-logical symbol of arity 0. A constant symbol may occur as a term without taking input terms.

Remark (Standard quantified statement).

$$\text{Const}_{\mathcal{L}} = \{c \in \mathcal{L} : \text{arity}(c) = 0\}.$$

Remark (Definition predicate reading). $\text{ConstantSymbol}(c)$ means $c \in \text{Const}_{\mathcal{L}}$.

Remark (Negated quantified statement).

$$c \notin \text{Const}_{\mathcal{L}}.$$

Remark (Negation predicate reading). $\neg \text{ConstantSymbol}(c)$ means that c is not a nullary non-logical symbol of $\mathcal{L}$.

Remark (Failure modes). A symbol fails to be a constant symbol when it is not in the language, is logical punctuation, or has positive arity.

Remark (Failure mode decomposition).

$$\underbrace{x}_{\text{variable}} \quad \underbrace{f}_{\text{positive arity}} \quad \underbrace{(}_{\text{punctuation}} \quad .$$

Remark (Interpretation). A constant symbol is a simple name-forming symbol in the grammar. It supplies a base case for terms alongside variables.

Remark (Examples). In an arithmetic language, 0 and 1 are commonly used as constant symbols.

Remark (Non-Examples). $f(x)$ is not a constant symbol; it is a term built from a function symbol and an input term.

Remark (Dependencies).

- [Variable](#)

Definition (Function Symbol)

Definition 2.3.3 (Function Symbol). For a first-order language $\mathcal{L}$, an n -ary *function symbol* is a non-logical symbol of arity $n \geq 1$. If f has arity n , then it forms terms by the pattern

$$f(t_1, \dots, t_n).$$

Remark (Standard quantified statement).

$$\text{Func}_{\mathcal{L}}^{(n)} = \{f \in \mathcal{L} : \text{arity}(f) = n \text{ and } n \geq 1\}.$$

Remark (Definition predicate reading). $\text{FunctionSymbol}(f)$ means $f \in \text{Func}_{\mathcal{L}}^{(n)}$ for some $n \geq 1$.

Remark (Negated quantified statement).

$$\forall n \geq 1 (f \notin \text{Func}_{\mathcal{L}}^{(n)}).$$

Remark (Negation predicate reading). $\neg \text{FunctionSymbol}(f)$ means that f is not a positive-arity function symbol of $\mathcal{L}$.

Remark (Failure modes). A symbol fails to be a function symbol when it is a variable, constant symbol, predicate symbol, logical symbol, punctuation mark, or has no assigned positive arity.

Remark (Failure mode decomposition).

$$\underbrace{c}_{\text{arity 0}} \quad \underbrace{P}_{\text{predicate symbol}} \quad \underbrace{\neg}_{\text{logical symbol}}.$$

Remark (Interpretation). A function symbol is an operation-forming symbol in the grammar. It is not a term by itself; it becomes part of a term only after it is supplied the required number of input terms.

Remark (Examples). In an arithmetic language, S may be unary, while $+$ and $\cdot$ may be binary function symbols.

Remark (Non-Examples). $f(x)$ is not a function symbol. It is a term formed by applying the function symbol f to the term x .

Remark (Dependencies).

- [Constant Symbol](#)

Definition (Predicate Symbol)

Definition 2.3.4 (Predicate Symbol). For a first-order language $\mathcal{L}$, an n -ary *predicate symbol* is a non-logical symbol of arity $n \geq 1$. If P has arity n , then it forms atomic formulas by the pattern

$$P(t_1, \dots, t_n).$$

Remark (Standard quantified statement).

$$\text{Pred}_{\mathcal{L}}^{(n)} = \{P \in \mathcal{L} : \text{arity}(P) = n \text{ and } n \geq 1\}.$$

Remark (Definition predicate reading). $\text{PredicateSymbol}(P)$ means $P \in \text{Pred}_{\mathcal{L}}^{(n)}$ for some $n \geq 1$.

Remark (Negated quantified statement).

$$\forall n \geq 1 (P \notin \text{Pred}_{\mathcal{L}}^{(n)}).$$

Remark (Negation predicate reading). $\neg \text{PredicateSymbol}(P)$ means that P is not a positive-arity predicate symbol of $\mathcal{L}$.

Remark (Failure modes). A symbol fails to be a predicate symbol when it belongs to the term-forming side of the language, is a logical symbol, is punctuation, or has no assigned positive arity as a predicate symbol.

Remark (Failure mode decomposition).

$$\underbrace{x}_{\text{variable}} \quad \underbrace{c}_{\text{constant symbol}} \quad \underbrace{f}_{\text{function symbol}} \quad \underbrace{\exists}_{\text{logical symbol}}.$$

Remark (Interpretation). Predicate symbols are the atomic-formula-forming vocabulary of the language. They are syntactic symbols, not project metalanguage predicates.

Remark (Examples). P , Q , R , $<$, and $=$ may be predicate symbols when the language declares the corresponding arities.

Remark (Non-Examples). $P(x)$ is not a predicate symbol. It is an atomic formula formed from the predicate symbol P and the term x .

Remark (Dependencies).

- **Function Symbol**

Definition (Term)

Definition 2.3.5 (Term). The set of *terms* of a first-order language $\mathcal{L}$, denoted $\text{Term}_{\mathcal{L}}$, is the smallest set of strings satisfying:

1. every variable is a term;
2. every constant symbol is a term;
3. if f is an n -ary function symbol and $t_1, \dots, t_n$ are terms, then $f(t_1, \dots, t_n)$ is a term;
4. no string is a term unless it is generated by finitely many applications of the previous clauses.

Remark (Standard quantified statement).

$$\text{Term}_{\mathcal{L}} = \bigcap \left\{ S : \text{Var}_{\mathcal{L}} \subseteq S, \text{Const}_{\mathcal{L}} \subseteq S, (\forall n \geq 1)(\forall f \in \text{Func}_{\mathcal{L}}^{(n)})(\forall t_1, \dots, t_n \in S) f(t_1, \dots, t_n) \in S \right\}.$$

Remark (Definition predicate reading). $\text{Term}(t)$ means $t \in \text{Term}_{\mathcal{L}}$.

Remark (Negated quantified statement).

$$\sigma \notin \text{Term}_{\mathcal{L}} \iff \exists S \left[\begin{array}{l} \text{Var}_{\mathcal{L}} \subseteq S \wedge \text{Const}_{\mathcal{L}} \subseteq S \\ \wedge (\forall n \geq 1)(\forall f \in \text{Func}_{\mathcal{L}}^{(n)})(\forall t_1, \dots, t_n \in S) f(t_1, \dots, t_n) \in S \\ \wedge \sigma \notin S \end{array} \right].$$

Remark (Negation predicate reading). $\neg \text{Term}(\sigma)$ means that σ is not generated by the term-formation clauses.

Remark (Failure modes). A string fails to be a term when it uses a predicate symbol, connective, or quantifier at the outer syntactic level, when a function symbol has too few or too many input terms, or when punctuation does not match the arity pattern.

Remark (Failure mode decomposition).

$$\underbrace{P(x)}_{\text{atomic formula, not a term}} \quad \underbrace{f(x, y)}_{\text{wrong arity if } f \text{ is unary}} \quad \underbrace{\forall x}_{\text{quantifier fragment}} .$$

Remark (Interpretation). Terms are the noun-like expressions of first-order syntax. They provide the inputs to predicate symbols but do not themselves make assertions.

Remark (Examples). x , c , $f(x)$, $g(c, x)$, and $h(f(x), c)$ are terms when the language assigns the displayed arities.

Remark (Non-Examples). $P(x)$, $\neg P(x)$, $\forall x P(x)$, and $x \wedge y$ are not terms.

Remark (Dependencies).

- [Language](#)

Definition (Atomic Formula)

Definition 2.3.6 (Atomic Formula). An *atomic formula* of a first-order language $\mathcal{L}$ is a string of the form

$$P(t_1, \dots, t_n),$$

where P is an n -ary predicate symbol and $t_1, \dots, t_n$ are terms.

Remark (Standard quantified statement).

$$\text{Atom}_{\mathcal{L}} = \left\{ P(t_1, \dots, t_n) : P \in \text{Pred}_{\mathcal{L}}^{(n)} \text{ and } t_1, \dots, t_n \in \text{Term}_{\mathcal{L}} \right\}.$$

Remark (Definition predicate reading). $\text{AtomicFormula}(\varphi)$ means $\varphi \in \text{Atom}_{\mathcal{L}}$.

Remark (Negated quantified statement).

$$\varphi \notin \text{Atom}_{\mathcal{L}}.$$

Remark (Negation predicate reading). $\neg \text{AtomicFormula}(\varphi)$ means that φ is not a predicate symbol applied to the required number of terms.

Remark (Failure modes). A string fails to be atomic when it does not begin with a predicate symbol, when the predicate symbol receives the wrong number of term inputs, when an input is not a term, or when a logical connective or quantifier has already been applied.

Remark (Failure mode decomposition).

$$\underbrace{f(x)}_{\text{term, not formula}} \quad \underbrace{P(x, y)}_{\text{wrong arity if } P \text{ is unary}} \quad \underbrace{\neg P(x)}_{\text{molecular formula}} .$$

Remark (Interpretation). Atomic formulas are the base cases of formula formation. They are the first strings in the grammar that count as formulas.

Remark (Examples). $P(x)$, $R(x, c)$, $x = y$, and $x < y$ are atomic formulas when the language supplies the relevant predicate symbols and arities.

Remark (Non-Examples). x , $f(x)$, $\neg P(x)$, and $(P(x) \wedge Q(x))$ are not atomic formulas.

Remark (Dependencies).

- [Predicate Symbol](#)
- [Term](#)

Definition (Formula)

Definition 2.3.7 (Formula). For a first-order language $\mathcal{L}$, a *formula* is an expression formed by the formula grammar of $\mathcal{L}$. The grammar starts from atomic formulas and closes under the logical connectives and quantifiers of the language.

Remark (Interpretation). The language supplies the symbols and formation rules; the formula notion is the syntactic category governed by those rules.

Remark (Dependencies).

- [Language](#)

Definition (Well-Formed Formula)

Definition 2.3.8 (Well-Formed Formula). The set of *well-formed formulas* of a first-order language $\mathcal{L}$, denoted $\text{Form}_{\mathcal{L}}$, is the smallest set of strings satisfying:

1. every atomic formula is a well-formed formula;
2. if φ is a well-formed formula, then $\neg\varphi$ is a well-formed formula;
3. if φ and ψ are well-formed formulas and $\circ \in \{\wedge, \vee, \rightarrow, \leftrightarrow\}$, then $(\varphi \circ \psi)$ is a well-formed formula;
4. if φ is a well-formed formula and x is a variable, then $\forall x \varphi$ and $\exists x \varphi$ are well-formed formulas;
5. no string is a well-formed formula unless it is generated by finitely many applications of the previous clauses.

Remark (Standard quantified statement).

$$\begin{aligned} \text{Form}_{\mathcal{L}} = \bigcap \{ S : & \text{Atom}_{\mathcal{L}} \subseteq S, \\ & (\forall \varphi \in S)(\neg\varphi \in S), \\ & (\forall \varphi, \psi \in S)(\forall \circ \in \{\wedge, \vee, \rightarrow, \leftrightarrow\})((\varphi \circ \psi) \in S), \\ & (\forall x \in \text{Var}_{\mathcal{L}})(\forall \varphi \in S)(\forall x \varphi \in S \wedge \exists x \varphi \in S) \}. \end{aligned}$$

Remark (Definition predicate reading). $\text{WellFormedFormula}(\varphi)$ means $\varphi \in \text{Form}_{\mathcal{L}}$.

Remark (Negated quantified statement).

$$\varphi \notin \text{Form}_{\mathcal{L}} \iff \exists S \left[\begin{array}{l} \text{Atom}_{\mathcal{L}} \subseteq S \wedge (\forall \psi \in S)(\neg \psi \in S) \\ \wedge (\forall \psi, \chi \in S)(\forall \circ \in \{\wedge, \vee, \rightarrow, \leftrightarrow\})((\psi \circ \chi) \in S) \\ \wedge (\forall x \in \text{Var}_{\mathcal{L}})(\forall \psi \in S)(\forall x \psi \in S \wedge \exists x \psi \in S) \\ \wedge \varphi \notin S \end{array} \right].$$

Remark (Negation predicate reading). $\neg \text{WellFormedFormula}(\varphi)$ means that φ is not generated by the formula-formation clauses.

Remark (Failure modes). A string fails to be well formed when it is a bare term, has a missing operand, uses a quantifier without a variable and formula, has unmatched punctuation, or cannot be produced by finitely many formation steps.

Remark (Failure mode decomposition).

$$\underbrace{f(x)}_{\text{term only}} \quad \underbrace{P(x) \wedge}_{\text{missing right formula}} \quad \underbrace{\forall P(x)}_{\text{missing bound variable}} \quad \underbrace{(P(x))}_{\text{unmatched punctuation}}.$$

Remark (Interpretation). Well-formed formulas are exactly the grammatical formula strings of the language. Later semantic and proof-theoretic sections apply only to these strings.

Remark (Examples). $P(x)$, $\neg P(x)$, $(P(x) \rightarrow Q(x))$, and $\forall x (P(x) \rightarrow Q(x))$ are well-formed formulas.

Remark (Non-Examples). x , $f(x)$, $P(x) \wedge$, and $\forall P(x)$ are not well-formed formulas.

Remark (Dependencies).

- [Formula](#)

Definition (Molecular Formula)

Definition 2.3.9 (Molecular Formula). A *molecular formula* is a well-formed formula that is not atomic. Equivalently, it is a well-formed formula whose final formation step uses a logical connective or a quantifier.

Remark (Standard quantified statement).

$$\varphi \in \text{Form}_{\mathcal{L}} \setminus \text{Atom}_{\mathcal{L}}.$$

Remark (Definition predicate reading).

$$\text{MolecularFormula}(\varphi) := \text{WellFormedFormula}(\varphi) \wedge \neg \text{AtomicFormula}(\varphi).$$

Remark (Negated quantified statement).

$$\varphi \notin \text{Form}_{\mathcal{L}} \quad \text{or} \quad \varphi \in \text{Atom}_{\mathcal{L}}.$$

Remark (Negation predicate reading). $\neg \text{MolecularFormula}(\varphi)$ means that φ is either not a well-formed formula or is atomic.

Remark (Failure modes). A string fails to be a molecular formula either because it is not a well-formed formula at all or because it is an atomic formula with no outer logical construction.

Remark (Failure mode decomposition).

$$\underbrace{x}_{\text{not a formula}} \quad \underbrace{P(x)}_{\text{atomic formula}} \quad \underbrace{P(x) \wedge}_{\text{not well formed}} .$$

Remark (Interpretation). Molecular formulas are the compound formulas of first-order syntax. They are formed after the atomic stage by applying a connective or quantifier.

Remark (Examples). $\neg P(x)$, $(P(x) \wedge Q(x))$, and $\forall x P(x)$ are molecular formulas.

Remark (Non-Examples). $P(x)$ is atomic rather than molecular, and $f(x)$ is a term rather than a formula.

Remark (Dependencies).

- [Well-Formed Formula](#)
- [Atomic Formula](#)

Variables and Substitution — Quick Reference

Concept	Meaning
Scope of a quantifier	Formula governed by the quantifier
Bound/free occurrence	Whether x is captured by a quantifier
Free variable $FV(\varphi)$	Variables whose values affect truth
Sentence	Formula with $FV(\varphi) = \emptyset$
Substitution $\varphi[t/x]$	Replace free x by term t
Free for substitution	No variable in t becomes bound
Alpha-equivalence	Renaming bound variables

2.3.2 Syntax: Free Variables, Bound Variables, and Substitution

Definition (Scope of a Quantifier)

Definition 2.3.10 (Scope of a Quantifier). Let φ be a formula of a first-order language. If φ is of the form $\forall x \psi$ or $\exists x \psi$, then the formula ψ is called the *scope* of the quantifier. The quantifier is said to *bind* all occurrences of the variable x that appear within its scope.

Remark (Dependencies).

- [Formula](#)
- [Variable](#)
- [Universal Quantifier](#)

- [Existential Quantifier](#)

Remark (English reading). The scope is the reach of a quantifier — the subformula it governs. Scope is syntactically determined by the parenthesisation of the formula, not by proximity to the quantifier symbol.

Example. In $(\forall x P(x)) \wedge Q(x)$, the scope of $\forall x$ is $P(x)$ only. The occurrence of x in $Q(x)$ falls outside the scope and is free.

Definition (Bound and Free Occurrences)

Definition 2.3.11 (Bound and Free Occurrences). An occurrence of a variable x in a formula φ is *bound* if it lies within the scope of a quantifier $\forall x$ or $\exists x$. An occurrence of x is *free* if it is not bound by any quantifier in φ .

Remark (Dependencies).

- [Formula](#)
- [Variable](#)
- [Universal Quantifier](#)
- [Existential Quantifier](#)
- [Scope of a Quantifier](#)

Remark (English reading). The same variable can have both bound and free occurrences in a single formula. Each occurrence is classified independently by checking whether it falls within the scope of a binding quantifier.

Definition (Free Variables of a Formula)

Definition 2.3.12 (Free Variables of a Formula). The set $\text{FV}(\varphi)$ of *free variables* of a formula φ is defined recursively:

1. If $\varphi = P(t_1, \dots, t_n)$ is atomic, then $\text{FV}(\varphi) = \bigcup_{i=1}^n \text{Var}(t_i)$, where $\text{Var}(t_i)$ is the set of variables in term t_i .
2. $\text{FV}(\neg\varphi) = \text{FV}(\varphi)$.
3. $\text{FV}(\varphi \circ \psi) = \text{FV}(\varphi) \cup \text{FV}(\psi)$ for any binary connective $\circ$.
4. $\text{FV}(\forall x \varphi) = \text{FV}(\varphi) \setminus \{x\}$.
5. $\text{FV}(\exists x \varphi) = \text{FV}(\varphi) \setminus \{x\}$.

Remark (Dependencies).

- [Bound and Free Occurrences](#)

- [Atomic Formula](#)
- [Formula](#)
- [Term](#)
- [Variable](#)
- [Union](#)
- [Set Difference](#)

Remark (English reading). $FV(\varphi)$ collects exactly those variables whose values can influence the truth of φ under a structure. Quantifying over x removes x from the free set because the quantifier takes responsibility for ranging over all (or some) values of x .

Remark (Common error). Terms themselves contain only free variables. Variables become bound only through quantification in formulas. The interpretation of a formula depends exactly on the values assigned to its free variables.

Definition (Sentence)

Definition 2.3.13 (Sentence). A *sentence* (or *closed formula*) is a formula with no free variables.

Formally, φ is a sentence if and only if $FV(\varphi) = \emptyset$.

Remark (Dependencies).

- [Formula](#)
- [Free Variables of a Formula](#)
- [Empty Set](#)

Remark (Consequence). For sentences, truth depends only on the structure, not on the variable assignment. This makes sentences the natural objects to be called true or false in a model, without qualification.

Definition (Substitution Notation)

Definition 2.3.14 (Substitution Notation). Let φ be a formula, x a variable, and t a term.

The expression $\varphi[t/x]$ denotes the formula obtained from φ by replacing every *free* occurrence of x with the term t , leaving bound occurrences of x unchanged.

Remark (Dependencies).

- [Variable](#)
- [Term](#)

- [Formula](#)
- [Bound and Free Occurrences](#)

Definition (Free for Substitution)

Definition 2.3.15 (Free for Substitution). A term t is *free for* x in φ if no free occurrence of x in φ lies within the scope of a quantifier $\forall y$ or $\exists y$ where y is a variable occurring in t .

Equivalently, t is free for x in φ if the substitution $\varphi[t/x]$ does not result in any variable in t becoming bound.

Remark (Standard quantified statement). The relation “ t is free for x in φ ” is defined by structural induction on formulas. It is automatic for atomic formulas. It is preserved through Boolean connectives:

$$t \text{ free for } x \text{ in } \neg\varphi \iff t \text{ free for } x \text{ in } \varphi,$$

and similarly componentwise for $\varphi \wedge \psi$, $\varphi \vee \psi$, and $\varphi \rightarrow \psi$. For a quantified formula, the capture-avoidance condition is

$$t \text{ free for } x \text{ in } Qy\varphi \iff (x \text{ is not free in } Qy\varphi) \text{ or } (y \notin \text{Var}(t) \text{ and } t \text{ is free for } x \text{ in } \varphi),$$

where Q is either $\forall$ or $\exists$.

Remark (Dependencies).

- [Variable](#)
- [Term](#)
- [Formula](#)
- [Bound and Free Occurrences](#)
- [Free Variables of a Formula](#)

Remark (Capture-avoiding substitution). A substitution $\varphi[t/x]$ is admissible only if t is free for x in φ . If this condition is violated, a variable occurring in t may become bound after substitution, silently changing the meaning of the formula. This is called *variable capture*.

Example (Variable capture). In the formula $\exists y(x < y)$, the term $y + 1$ is not free for x . A blind substitution would produce $\exists y(y + 1 < y)$, where the y introduced by the term has been captured by the existing quantifier.

Remark (Common error). Variable capture is one of the most common syntactic mistakes in predicate logic. Always check that the term being substituted introduces no variables that fall inside a binding quantifier in the target formula.

Definition (Alpha-Equivalence)

Definition 2.3.16 (Alpha-Equivalence). Formulas that differ only by the names of bound variables are logically equivalent. This is called *alpha-equivalence* (or *alphabetic variance*):

$$\forall x \varphi \equiv \forall y \varphi[y/x] \quad (\text{provided } y \text{ is not free in } \varphi).$$

Remark (Dependencies).

- [Formula](#)
- [Bound and Free Occurrences](#)
- [Substitution Notation](#)
- [Free for Substitution](#)

Remark (Consequence). Alpha-equivalence is the formal basis for the bound variable renaming used in prenex normal form conversion and in any proof where variable capture must be avoided. Two alpha-equivalent formulas are interchangeable in all contexts.

Formula Depth — Quick Reference

Concept	Meaning
Formula depth	Max formation steps from atomic formulas

2.3.3 Syntax: Formula Depth

Definition (Formula Depth)

Definition 2.3.17 (Formula Depth). The *depth* (or *complexity*) of a formula φ , denoted $\text{depth}(\varphi)$, is a natural number defined recursively:

1. **Atomic case.** If φ is atomic, then $\text{depth}(\varphi) = 0$.
2. **Negation.** If $\varphi = \neg\psi$, then $\text{depth}(\varphi) = \text{depth}(\psi) + 1$.
3. **Binary connectives.** If $\varphi = (\psi \circ \chi)$ for $\circ \in \{\wedge, \vee, \rightarrow, \leftrightarrow\}$, then
$$\text{depth}(\varphi) = \max\{\text{depth}(\psi), \text{depth}(\chi)\} + 1.$$
4. **Quantifiers.** If $\varphi = \forall x \psi$ or $\varphi = \exists x \psi$, then $\text{depth}(\varphi) = \text{depth}(\psi) + 1$.

Remark (Dependencies).

- [Formula](#)
- Natural Numbers

- [Universal Quantifier](#)
- [Existential Quantifier](#)

Remark (English reading). The depth of a formula measures the maximum number of logical formation steps required to build the formula from atomic formulas. It corresponds to the height of the formula’s syntactic parse tree.

Remark (Proof strategy). Formula depth is the standard measure used in structural induction proofs over first-order formulas. The base case is depth 0 (atomic formulas); the inductive step handles each formation rule and assumes the result holds for sub-formulas of strictly smaller depth.

2.4 Semantics

2.4.1 Semantics: Structures and Variable Assignments

Remark (Section toolkit: Semantic setup). This section is the first point at which semantic objects enter the Predicate Logic chapter. A language supplies the symbols; a structure supplies a domain and interpretations for the non-logical symbols; an assignment supplies values for variables; term evaluation turns terms into domain elements.

Item	Notation	Role
First-order language	$\mathcal{L}$	symbol stock with arities
Structure	$\mathcal{M} = \langle D, I \rangle$	semantic object for $\mathcal{L}$
Variable assignment	$s : \text{Var}_{\mathcal{L}} \rightarrow D$	values for variables
Modified assignment	$s[x \mapsto d]$	update at one variable
Term evaluation	$\llbracket t \rrbracket_{\mathcal{M}, s}$	value of a term

Definition (First-Order Language)

Definition 2.4.1 (First-Order Language). A *first-order language* $\mathcal{L}$ consists of variables, logical symbols, and a signature of non-logical symbols: constant symbols, function symbols with assigned arities, and predicate symbols with assigned arities. Equality may be included as a distinguished logical predicate.

Remark (Standard quantified statement).

A first-order language $\mathcal{L}$ has symbol classes

$$\text{Var}_{\mathcal{L}}, \quad \text{Const}_{\mathcal{L}}, \quad \text{Func}_{\mathcal{L}}^{(n)}, \quad \text{Pred}_{\mathcal{L}}^{(n)} \quad (n \geq 1),$$

together with the usual logical connectives, quantifiers, and punctuation.

Remark (Definition predicate reading). $\text{FirstOrderLanguage}(L)$ means that L supplies the variable stock, logical symbols, and arity-indexed non-logical symbols used to form first-order terms and formulas.

Remark (Negated quantified statement).

L is not equipped with the required first-order symbol classes and arities.

Remark (Negation predicate reading). $\neg \text{FirstOrderLanguage}(L)$ means that L lacks some required syntactic stock or arity assignment.

Remark (Failure modes). A proposed language fails when its non-logical symbols are not sorted by role, when positive-arity symbols have no arity, or when the logical grammar is not specified.

Remark (Failure mode decomposition).

$$\underbrace{\text{Const}_L}_{\text{constant symbols}} \quad \underbrace{\text{Func}_L^{(n)}}_{\text{function symbols with arity}} \quad \underbrace{\text{Pred}_L^{(n)}}_{\text{predicate symbols with arity}} .$$

Remark (Interpretation). A first-order language is still syntax. It says which expressions can be formed, but it does not assign objects, functions, or relations to the non-logical symbols.

Remark (Examples). The arithmetic language with constant symbols 0, 1, function symbols $S, +, \cdot$, and predicate symbol $<$ is a first-order language once the arities are fixed.

Remark (Dependencies).

- [Variable](#)
- [Constant Symbol](#)
- [Function Symbol](#)
- [Predicate Symbol](#)

Definition (Structure)

Definition 2.4.2 (Structure). Let $\mathcal{L}$ be a first-order language. A *structure* for $\mathcal{L}$ is a pair

$$\mathcal{M} = \langle D, I \rangle$$

where D is a nonempty set and I assigns each constant symbol an element of D , each n -ary function symbol a function $D^n \rightarrow D$, and each n -ary predicate symbol a relation on D^n .

Remark (Standard quantified statement).

Let $\mathcal{L}$ be a first-order language.

$\mathcal{M} = \langle D, I \rangle$ is a structure for $\mathcal{L}$

$$\iff D \neq \emptyset, I(c) \in D, I(f) : D^n \rightarrow D, I(P) \subseteq D^n$$

for each $c \in \text{Const}_{\mathcal{L}}$, $f \in \text{Func}_{\mathcal{L}}^{(n)}$, and $P \in \text{Pred}_{\mathcal{L}}^{(n)}$.

Remark (Definition predicate reading). $\text{Structure}(M, L)$ means that M is a structure for the first-order language L .

Remark (Negated quantified statement).

$D = \emptyset$ or I fails to assign the required object, function, or relation to some symbol.

Remark (Negation predicate reading). $\neg \text{Structure}(M, L)$ means that M is not a well-formed structure for L .

Remark (Failure modes). A proposed structure fails if the domain is empty, if a symbol is left uninterpreted, or if the assigned object has the wrong arity or type.

Remark (Failure mode decomposition).

$$\underbrace{D = \emptyset}_{\text{empty domain}} \vee \underbrace{I(c) \notin D}_{\text{bad constant value}} \vee \underbrace{I(f) : D^n \not\rightarrow D}_{\text{bad function arity}} \vee \underbrace{I(P) \not\subseteq D^n}_{\text{bad predicate relation}} .$$

Remark (Interpretation). A structure is the minimal semantic object needed in Volume I. It gives meaning to the non-logical symbols of a language. Systematic topics such as substructures, expansions, reducts, and elementary classes belong to Volume VIII Model Theory.

Remark (Examples). The natural numbers with their usual interpretations of $0, S, +, \cdot, <$ form a structure for the arithmetic language.

Remark (Dependencies).

- [First-Order Language](#)

Definition (Variable Assignment)

Definition 2.4.3 (Variable Assignment). Let $\mathcal{M} = \langle D, I \rangle$ be a structure for $\mathcal{L}$. A *variable assignment* for $\mathcal{M}$ is a function

$$s : \text{Var}_{\mathcal{L}} \rightarrow D.$$

Remark (Standard quantified statement).

s is a variable assignment for $\mathcal{M} \iff \text{dom}(s) = \text{Var}_{\mathcal{L}}$ and $\text{ran}(s) \subseteq D$.

Remark (Definition predicate reading). $\text{VariableAssignment}(s, M)$ means that s assigns a domain element of M to each variable of the language of M .

Remark (Negated quantified statement).

$$\text{dom}(s) \neq \text{Var}_{\mathcal{L}} \quad \text{or} \quad \exists x \in \text{Var}_{\mathcal{L}} (s(x) \notin D).$$

Remark (Negation predicate reading). $\neg \text{VariableAssignment}(s, M)$ means that s is not a function assigning each variable a domain element of M .

Remark (Failure modes). An assignment fails when it leaves a variable without a value or assigns a value outside the domain.

Remark (Failure mode decomposition).

$$\underbrace{\text{dom}(s) \neq \text{Var}_{\mathcal{L}}}_{\text{missing variable value}} \vee \underbrace{s(x) \notin D}_{\text{value outside domain}} .$$

Remark (Interpretation). Structures interpret non-logical symbols; assignments interpret variables. Open formulas require assignments, while sentences eventually do not depend on the particular assignment.

Remark (Dependencies).

- [Structure](#)

Definition (Modified Assignment)

Definition 2.4.4 (Modified Assignment). Let $s : \text{Var}_{\mathcal{L}} \rightarrow D$, let $x \in \text{Var}_{\mathcal{L}}$, and let $d \in D$. The *modified assignment* $s[x \mapsto d]$ is the assignment defined by

$$s[x \mapsto d](y) = \begin{cases} d, & y = x, \\ s(y), & y \neq x. \end{cases}$$

Remark (Standard quantified statement).

$$\forall y \in \text{Var}_{\mathcal{L}} \quad s[x \mapsto d](y) = d \iff y = x, \quad s[x \mapsto d](y) = s(y) \iff y \neq x.$$

Remark (Definition predicate reading). $\text{ModifiedAssignment}(s, x, d, s')$ means that s' is the assignment obtained from s by sending x to d and leaving every other variable unchanged.

Remark (Negated quantified statement).

$$s'(x) \neq d \quad \text{or} \quad \exists y \in \text{Var}_{\mathcal{L}} (y \neq x \wedge s'(y) \neq s(y)).$$

Remark (Negation predicate reading). $\neg \text{ModifiedAssignment}(s, x, d, s')$ means that s' is not the one-variable update of s at x with value d .

Remark (Failure modes). A proposed modified assignment fails if it does not change x to d , or if it changes some variable other than x .

Remark (Failure mode decomposition).

$$\underbrace{s'(x) \neq d}_{\text{target variable not updated}} \quad \vee \quad \underbrace{\exists y \neq x (s'(y) \neq s(y))}_{\text{unwanted extra change}}.$$

Remark (Interpretation). Modified assignments are the semantic mechanism behind quantifiers. To test a quantified formula, the assignment is varied at the bound variable while all other variable values remain fixed.

Remark (Dependencies).

- [Variable Assignment](#)

Definition (Term Evaluation)

Definition 2.4.5 (Term Evaluation). Let $\mathcal{M} = \langle D, I \rangle$ be a structure and s a variable assignment. The *value* of a term t , denoted $\llbracket t \rrbracket_{\mathcal{M}, s}$, is defined recursively by

$$\llbracket x \rrbracket_{\mathcal{M}, s} = s(x), \quad \llbracket c \rrbracket_{\mathcal{M}, s} = I(c),$$

and

$$\llbracket f(t_1, \dots, t_n) \rrbracket_{\mathcal{M}, s} = I(f)(\llbracket t_1 \rrbracket_{\mathcal{M}, s}, \dots, \llbracket t_n \rrbracket_{\mathcal{M}, s}).$$

Remark (Standard quantified statement).

$$\begin{aligned}\llbracket x \rrbracket_{\mathcal{M},s} &= s(x), \\ \llbracket c \rrbracket_{\mathcal{M},s} &= I(c), \\ \llbracket f(t_1, \dots, t_n) \rrbracket_{\mathcal{M},s} &= I(f)(\llbracket t_1 \rrbracket_{\mathcal{M},s}, \dots, \llbracket t_n \rrbracket_{\mathcal{M},s}).\end{aligned}$$

Remark (Definition predicate reading). $\text{TermEvaluation}(M, s, t, a)$ means that the term t evaluates to the domain element a in M under s .

Remark (Negated quantified statement).

$$\llbracket t \rrbracket_{\mathcal{M},s} \neq a.$$

Remark (Negation predicate reading). $\neg \text{TermEvaluation}(M, s, t, a)$ means that a is not the value of t in M under s .

Remark (Failure modes). Term evaluation fails to have the proposed value when a variable, constant, or function-application clause computes a different domain element.

Remark (Failure mode decomposition).

$$\underbrace{s(x) \neq a}_{\text{variable case}} \quad \vee \quad \underbrace{I(c) \neq a}_{\text{constant case}} \quad \vee \quad \underbrace{I(f)(a_1, \dots, a_n) \neq a}_{\text{function case}}.$$

Remark (Interpretation). Term evaluation is the semantic counterpart of term formation. It assigns a domain element to each grammatical term once a structure and assignment are fixed.

Remark (Dependencies).

- [Term](#)
- [Structure](#)
- [Variable Assignment](#)

2.4.2 Semantics: Satisfaction and Truth

Remark (Section toolkit: Satisfaction ledger). Satisfaction is the recursive semantic relation for formulas. Formulas with free variables are evaluated relative to an assignment. Sentences have no free variables, so their truth in a structure does not depend on which assignment is chosen.

Definition (Satisfaction)

Definition 2.4.6 (Satisfaction). Let $\mathcal{M} = \langle D, I \rangle$ be a structure, let s be a variable assignment, and let φ be a well-formed formula. The relation $\mathcal{M}, s \models \varphi$ is defined recursively by the usual clauses: atomic formulas are evaluated by term values and interpreted predicate relations; equality compares term values; connectives follow the truth conditions for $\neg, \wedge, \vee, \rightarrow, \leftrightarrow$; and

$$\begin{aligned}\mathcal{M}, s \models \forall x \psi &\iff \forall d \in D (\mathcal{M}, s[x \mapsto d] \models \psi), \\ \mathcal{M}, s \models \exists x \psi &\iff \exists d \in D (\mathcal{M}, s[x \mapsto d] \models \psi).\end{aligned}$$

Remark (Standard quantified statement).

$$\begin{aligned}\mathcal{M}, s \models P(t_1, \dots, t_n) &\iff (\llbracket t_1 \rrbracket_{\mathcal{M}, s}, \dots, \llbracket t_n \rrbracket_{\mathcal{M}, s}) \in I(P), \\ \mathcal{M}, s \models t_1 = t_2 &\iff \llbracket t_1 \rrbracket_{\mathcal{M}, s} = \llbracket t_2 \rrbracket_{\mathcal{M}, s}, \\ \mathcal{M}, s \models \neg \psi &\iff \mathcal{M}, s \not\models \psi, \\ \mathcal{M}, s \models (\psi \wedge \chi) &\iff \mathcal{M}, s \models \psi \text{ and } \mathcal{M}, s \models \chi, \\ \mathcal{M}, s \models \forall x \psi &\iff \forall d \in D (\mathcal{M}, s[x \mapsto d] \models \psi), \\ \mathcal{M}, s \models \exists x \psi &\iff \exists d \in D (\mathcal{M}, s[x \mapsto d] \models \psi).\end{aligned}$$

Remark (Definition predicate reading). Satisfaction(M, s, φ) means $\mathcal{M}, s \models \varphi$.

Remark (Negated quantified statement).

$$\mathcal{M}, s \not\models \varphi.$$

Remark (Negation predicate reading). $\neg$ Satisfaction(M, s, φ) means that φ is not satisfied in M under s .

Remark (Failure modes). Satisfaction fails according to the outermost formation rule of the formula: an atomic tuple can miss the interpreted relation, equality can compare different term values, a connective can have the wrong truth pattern, or a quantifier can have a counterexample or lack a witness.

Remark (Failure mode decomposition).

$$\underbrace{(\llbracket t_i \rrbracket) \notin I(P)}_{\text{atomic failure}} \quad \underbrace{\llbracket t_1 \rrbracket \neq \llbracket t_2 \rrbracket}_{\text{equality failure}} \quad \underbrace{\exists d \in D (\mathcal{M}, s[x \mapsto d] \not\models \psi)}_{\text{universal failure}}.$$

Remark (Interpretation). Satisfaction is the bridge from grammar to semantic use. It is recursive because formulas are recursive. The assignment is essential exactly where free variables can affect the value of the formula.

Remark (Dependencies).

- [Structure](#)
- [Variable Assignment](#)

- [Modified Assignment](#)
- [Term Evaluation](#)
- [Well-Formed Formula](#)

Definition (Truth in a Structure)

Definition 2.4.7 (Truth in a Structure). Let φ be a sentence and let $\mathcal{M}$ be a structure. The sentence φ is *true in $\mathcal{M}$* , written $\mathcal{M} \models \varphi$, exactly when $\mathcal{M}, s \models \varphi$ for every variable assignment s for $\mathcal{M}$.

Remark (Standard quantified statement).

$$\mathcal{M} \models \varphi \iff \forall s : \text{Var}_{\mathcal{L}} \rightarrow D (\mathcal{M}, s \models \varphi), \quad \varphi \text{ a sentence.}$$

Remark (Definition predicate reading). $\text{TruthInStructure}(\mathcal{M}, \varphi)$ means $\mathcal{M} \models \varphi$.

Remark (Negated quantified statement).

$$\mathcal{M} \not\models \varphi \iff \exists s : \text{Var}_{\mathcal{L}} \rightarrow D (\mathcal{M}, s \not\models \varphi).$$

Remark (Negation predicate reading). $\neg \text{TruthInStructure}(\mathcal{M}, \varphi)$ means that φ is false in $\mathcal{M}$.

Remark (Failure modes). Truth in a structure fails when some assignment makes the sentence fail. For sentences, this is independent of assignment, but the quantified form keeps the definition uniform with satisfaction.

Remark (Failure mode decomposition).

$$\underbrace{\exists s}_{\text{assignment witness}} \quad \underbrace{\mathcal{M}, s \not\models \varphi}_{\text{satisfaction failure}} .$$

Remark (Interpretation). Sentences have no free variables. Therefore the particular assignment does not carry mathematical information once the formula is a sentence; the notation $\mathcal{M} \models \varphi$ suppresses it.

Remark (Dependencies).

- [Satisfaction](#)

Definition (Validity)

Definition 2.4.8 (Validity). A formula φ is *valid*, or *logically true*, exactly when $\mathcal{M}, s \models \varphi$ for every structure $\mathcal{M}$ for the language of φ and every variable assignment s for $\mathcal{M}$.

Remark (Standard quantified statement).

$$\models \varphi \iff \forall \mathcal{M} \forall s (\mathcal{M}, s \models \varphi).$$

Remark (Definition predicate reading). $\text{ValidFormula}(\varphi)$ means that φ is satisfied in every structure under every assignment.

Remark (Negated quantified statement).

$$\not\models \varphi \iff \exists \mathcal{M} \exists s (\mathcal{M}, s \not\models \varphi).$$

Remark (Negation predicate reading). $\neg \text{ValidFormula}(\varphi)$ means that some structure and assignment make φ fail.

Remark (Failure modes). Validity fails by a counterexample structure together with an assignment.

Remark (Failure mode decomposition).

$$\underbrace{\mathcal{M}}_{\text{counterexample structure}} \quad \underbrace{s}_{\text{counterexample assignment}} \quad \underbrace{\mathcal{M}, s \not\models \varphi}_{\text{failure of satisfaction}} .$$

Remark (Interpretation). Validity is truth by logic alone. It does not depend on a special structure, special domain, or special interpretation of the non-logical symbols.

Remark (Dependencies).

- [Satisfaction](#)

Definition (Satisfiability)

Definition 2.4.9 (Satisfiability). A formula φ is *satisfiable* exactly when there are a structure $\mathcal{M}$ and a variable assignment s such that $\mathcal{M}, s \models \varphi$.

Remark (Standard quantified statement).

$$\varphi \text{ is satisfiable} \iff \exists \mathcal{M} \exists s (\mathcal{M}, s \models \varphi).$$

Remark (Definition predicate reading). $\text{SatisfiableFormula}(\varphi)$ means that φ is satisfied in at least one structure under at least one assignment.

Remark (Negated quantified statement).

$$\varphi \text{ is unsatisfiable} \iff \forall \mathcal{M} \forall s (\mathcal{M}, s \not\models \varphi).$$

Remark (Negation predicate reading). $\neg \text{SatisfiableFormula}(\varphi)$ means that no structure and assignment satisfy φ .

Remark (Failure modes). Satisfiability fails only globally: every attempted structure and assignment fails to satisfy the formula.

Remark (Failure mode decomposition).

$$\underbrace{\forall \mathcal{M}}_{\text{all structures}} \quad \underbrace{\forall s}_{\text{all assignments}} \quad \underbrace{\mathcal{M}, s \not\models \varphi}_{\text{no satisfying instance}} .$$

Remark (Interpretation). Satisfiability asks whether a formula can be made true somewhere. This is the semantic counterpart of consistency only in the minimal Volume I sense; deeper proof-theoretic distinctions are deferred.

Remark (Dependencies).

- [Satisfaction](#)

2.4.3 Semantics: Models, Theories, and Logical Consequence

Remark (Section toolkit: Minimal model notation). Volume I uses theories only as sets of sentences so that the notation $\mathcal{M} \models T$ and $T \models \varphi$ can be read. The systematic study of theories and their classes of models belongs to Volume VIII Model Theory.

Definition (Model of a Sentence)

Definition 2.4.10 (Model of a Sentence). Let φ be a sentence and let $\mathcal{M}$ be a structure for the language of φ . The structure $\mathcal{M}$ is a *model of* φ exactly when $\mathcal{M} \models \varphi$.

Remark (Standard quantified statement).

$$\mathcal{M} \text{ is a model of } \varphi \iff \mathcal{M} \models \varphi.$$

Remark (Definition predicate reading). $\text{ModelOfSentence}(M, \varphi)$ means that M is a model of the sentence φ .

Remark (Negated quantified statement).

$$\mathcal{M} \text{ is not a model of } \varphi \iff \mathcal{M} \not\models \varphi.$$

Remark (Negation predicate reading). $\neg \text{ModelOfSentence}(M, \varphi)$ means that φ is not true in M .

Remark (Failure modes). The only failure mode is semantic: the sentence is false in the proposed structure.

Remark (Failure mode decomposition).

$$\underbrace{\mathcal{M}}_{\text{proposed structure}} \quad \underbrace{\mathcal{M} \not\models \varphi}_{\text{sentence false in the structure}} .$$

Remark (Interpretation). A model is not an extra kind of object. It is a structure viewed relative to a sentence that it makes true.

Remark (Dependencies).

- [Truth in a Structure](#)

Definition (Model of a Set of Sentences)

Definition 2.4.11 (Model of a Set of Sentences). Let T be a set of sentences and let $\mathcal{M}$ be a structure for their common language. The structure $\mathcal{M}$ is a *model of* T , written $\mathcal{M} \models T$, exactly when $\mathcal{M} \models \theta$ for every $\theta \in T$.

Remark (Standard quantified statement).

$$\mathcal{M} \models T \iff \forall \theta \in T (\mathcal{M} \models \theta).$$

Remark (Definition predicate reading). $\text{ModelOfTheory}(M, T)$ means that M is a model of every sentence in T .

Remark (Negated quantified statement).

$$\mathcal{M} \not\models T \iff \exists \theta \in T (\mathcal{M} \not\models \theta).$$

Remark (Negation predicate reading). $\neg \text{ModelOfTheory}(M, T)$ means that at least one sentence of T is false in M .

Remark (Failure modes). A proposed model of a set of sentences fails by a single sentence of the set that is false in the structure.

Remark (Failure mode decomposition).

$$\underbrace{\exists \theta \in T}_{\text{failed sentence}} \quad \underbrace{\mathcal{M} \not\models \theta}_{\text{not true in the structure}}.$$

Remark (Interpretation). The notation $\mathcal{M} \models T$ abbreviates a universal check over the sentences in T . No additional theory of model classes is developed here.

Remark (Dependencies).

- [Model of a Sentence](#)

Definition (Theory)

Definition 2.4.12 (Theory). In Volume I, a *theory* is a set of sentences in a fixed first-order language.

Remark (Standard quantified statement).

$$T \text{ is a theory} \iff \forall \theta \in T (\theta \text{ is a sentence of the fixed language}).$$

Remark (Definition predicate reading). $\text{Theory}(T)$ means that T is a set of sentences in a fixed first-order language.

Remark (Negated quantified statement).

$$\exists \theta \in T (\theta \text{ is not a sentence of the fixed language}).$$

Remark (Negation predicate reading). $\neg \text{Theory}(T)$ means that T is not a set consisting only of sentences of the fixed language.

Remark (Failure modes). A proposed theory fails in this minimal sense when it contains something other than a sentence of the fixed language.

Remark (Failure mode decomposition).

$$\underbrace{\theta \in T}_{\text{member of the proposed theory}} \quad \underbrace{\theta \text{ is not a sentence}}_{\text{bad member}}.$$

Remark (Interpretation). This is intentionally modest. Complete theories, elementary classes, categoricity, saturation, types, diagrams, and elementary extensions are Volume VIII topics. A theory may be called satisfiable in Volume I when it has at least one model.

Remark (Dependencies).

- [Truth in a Structure](#)

Definition (Logical Consequence)

Definition 2.4.13 (Logical Consequence). Let T be a set of sentences and let φ be a sentence in the same language. The sentence φ is a *logical consequence* of T , written $T \models \varphi$, exactly when every model of T is a model of φ .

Remark (Standard quantified statement).

$$T \models \varphi \iff \forall \mathcal{M} (\mathcal{M} \models T \Rightarrow \mathcal{M} \models \varphi).$$

Remark (Definition predicate reading). $\text{LogicalConsequence}(T, \varphi)$ means that every model of T is a model of φ .

Remark (Negated quantified statement).

$$T \not\models \varphi \iff \exists \mathcal{M} (\mathcal{M} \models T \wedge \mathcal{M} \not\models \varphi).$$

Remark (Negation predicate reading). $\neg \text{LogicalConsequence}(T, \varphi)$ means that some structure models T while failing φ .

Remark (Failure modes). Logical consequence fails by a countermodel: a structure satisfying all sentences of T but not satisfying φ .

Remark (Failure mode decomposition).

$$\underbrace{\mathcal{M} \models T}_{\text{premises true}} \quad \underbrace{\mathcal{M} \not\models \varphi}_{\text{conclusion false}}.$$

Remark (Interpretation). Logical consequence is semantic entailment. The premises in T leave no room, across structures, for φ to fail.

Remark (Dependencies).

- [Model of a Set of Sentences](#)
- [Theory](#)

Definition (Logical Equivalence)

Definition 2.4.14 (Logical Equivalence). Two sentences φ and ψ are *logically equivalent*, written $\varphi \equiv \psi$, exactly when each is a logical consequence of the other:

$$\varphi \models \psi \quad \text{and} \quad \psi \models \varphi.$$

Remark (Standard quantified statement).

$$\varphi \equiv \psi \iff \forall \mathcal{M} (\mathcal{M} \models \varphi \iff \mathcal{M} \models \psi).$$

Remark (Definition predicate reading). $\text{LogicalEquivalence}(\varphi, \psi)$ means that φ and ψ have the same truth value in every structure.

Remark (Negated quantified statement).

$$\varphi \not\equiv \psi \iff \exists \mathcal{M} ((\mathcal{M} \models \varphi \wedge \mathcal{M} \not\models \psi) \vee (\mathcal{M} \models \psi \wedge \mathcal{M} \not\models \varphi)).$$

Remark (Negation predicate reading). $\neg \text{LogicalEquivalence}(\varphi, \psi)$ means that some structure separates the two sentences.

Remark (Failure modes). Logical equivalence fails when one sentence is true and the other false in the same structure.

Remark (Failure mode decomposition).

$$\underbrace{\mathcal{M} \models \varphi \wedge \mathcal{M} \not\models \psi}_{\text{first separation}} \quad \vee \quad \underbrace{\mathcal{M} \models \psi \wedge \mathcal{M} \not\models \varphi}_{\text{second separation}}.$$

Remark (Interpretation). Logical equivalence is semantic interchangeability at the sentence level. It records sameness of truth across all structures in the relevant language.

Remark (Dependencies).

- [Logical Consequence](#)

2.4.4 Semantics: Open Formulas, Definable Relations, and Substitution Lemmas

Remark (Section toolkit: Predicate vocabulary). This section separates three uses that are often blurred. A predicate symbol is object-language syntax from the signature. An open formula is a formula with free variables. A definable relation is the set-theoretic relation in a structure determined by an open formula.

Definition (Open Formula)

Definition 2.4.15 (Open Formula). An *open formula* is a well-formed formula with at least one free variable. When the free variables are among $x_1, \dots, x_n$, it may be displayed as $\varphi(x_1, \dots, x_n)$.

Remark (Standard quantified statement).

$$\varphi \text{ is open} \iff \varphi \in \text{Form}_{\mathcal{L}} \text{ and } \text{FV}(\varphi) \neq \emptyset.$$

Remark (Definition predicate reading). $\text{OpenFormula}(\varphi)$ means that φ is a well-formed formula with at least one free variable.

Remark (Negated quantified statement).

$$\varphi \notin \text{Form}_{\mathcal{L}} \quad \text{or} \quad \text{FV}(\varphi) = \emptyset.$$

Remark (Negation predicate reading). $\neg \text{OpenFormula}(\varphi)$ means that φ is either not well formed or is a sentence.

Remark (Failure modes). An expression fails to be an open formula when it is not a well-formed formula or when all of its variables are bound.

Remark (Failure mode decomposition).

$$\underbrace{\varphi \notin \text{Form}_{\mathcal{L}}}_{\text{not well formed}} \quad \vee \quad \underbrace{\text{FV}(\varphi) = \emptyset}_{\text{sentence, not open}} .$$

Remark (Interpretation). Open formulas are syntactic predicates in the lightest Volume I sense. They do not by themselves name a set-theoretic relation until a structure is fixed.

Remark (Examples). $P(x)$, $x < y$, and $\exists z R(x, z)$ are open formulas when the displayed variables remain free as indicated.

Remark (Non-Examples). $\forall x P(x)$ is a sentence, not an open formula. A predicate symbol P alone is a symbol, not a formula.

Remark (Dependencies).

- [Predicate Symbol](#)
- [Well-Formed Formula](#)

Definition (Definable Relation)

Definition 2.4.16 (Definable Relation). Let $\mathcal{M} = \langle D, I \rangle$ be a structure and let $\varphi(x_1, \dots, x_n)$ be an open formula. The *relation defined by φ in $\mathcal{M}$* is the set

$$R_{\varphi}^{\mathcal{M}} = \{(a_1, \dots, a_n) \in D^n : \mathcal{M}, s[x_1 \mapsto a_1, \dots, x_n \mapsto a_n] \models \varphi\}.$$

Remark (Standard quantified statement).

$$(a_1, \dots, a_n) \in R_{\varphi}^{\mathcal{M}} \iff \mathcal{M}, s[x_1 \mapsto a_1, \dots, x_n \mapsto a_n] \models \varphi.$$

Remark (Definition predicate reading). $\text{DefinableRelation}(R, \varphi, M)$ means that R is the relation on the domain of M determined by φ in M .

Remark (Negated quantified statement).

$$R \neq R_{\varphi}^{\mathcal{M}}.$$

Remark (Negation predicate reading). $\neg \text{DefinableRelation}(R, \varphi, M)$ means that R is not the relation determined by φ in M .

Remark (Failure modes). A proposed definable relation fails when at least one tuple is included or excluded differently from the satisfaction condition imposed by the formula.

Remark (Failure mode decomposition).

$$\underbrace{(a_1, \dots, a_n) \in R \wedge \mathcal{M}, s[\vec{x} \mapsto \vec{a}] \not\models \varphi}_{\text{extra tuple}} \quad \vee \quad \underbrace{(a_1, \dots, a_n) \notin R \wedge \mathcal{M}, s[\vec{x} \mapsto \vec{a}] \models \varphi}_{\text{missing tuple}} .$$

Remark (Interpretation). A relation is a set-theoretic subset of D^n . A definable relation is one obtained from a formula after a structure supplies semantic meaning.

Remark (Dependencies).

- [Open Formula](#)
- [Structure](#)
- [Satisfaction](#)

Lemma (Substitution Lemma for Terms)

Lemma 2.4.17 (Substitution Lemma for Terms). *Let $\mathcal{M}$ be a structure, let s be a variable assignment, let x be a variable, and let t and u be terms. Then*

$$\llbracket t[u/x] \rrbracket_{\mathcal{M},s} = \llbracket t \rrbracket_{\mathcal{M},s[x \mapsto \llbracket u \rrbracket_{\mathcal{M},s}]}$$

Go to proof.

Remark (Standard quantified statement).

$$\forall t \forall u \quad \llbracket t[u/x] \rrbracket_{\mathcal{M},s} = \llbracket t \rrbracket_{\mathcal{M},s[x \mapsto \llbracket u \rrbracket_{\mathcal{M},s}]}$$

Remark (Predicate reading). Term substitution commutes with semantic term evaluation in a fixed structure and assignment.

Remark (Interpretation). The lemma says that syntactically substituting a term before evaluation gives the same domain element as first evaluating the substituting term and then updating the assignment.

Remark (Dependencies).

- [Substitution Notation](#)
- [Free for Substitution](#)
- [Term Evaluation](#)

Lemma (Substitution Lemma for Formulas)

Lemma 2.4.18 (Substitution Lemma for Formulas). *Let φ be a formula and let u be a term free for x in φ . For every structure $\mathcal{M}$ and assignment s ,*

$$\mathcal{M}, s \models \varphi[u/x] \iff \mathcal{M}, s[x \mapsto \llbracket u \rrbracket_{\mathcal{M},s}] \models \varphi.$$

Go to proof.

Remark (Standard quantified statement).

$$\forall \varphi \forall u \quad u \text{ free for } x \text{ in } \varphi \Rightarrow (\mathcal{M}, s \models \varphi[u/x] \iff \mathcal{M}, s[x \mapsto \llbracket u \rrbracket_{\mathcal{M},s}] \models \varphi).$$

Remark (Predicate reading). Formula substitution commutes with satisfaction when the substituted term is free for the variable being replaced.

Remark (Interpretation). This lemma connects the syntactic substitution operation from the syntax section with the semantic modified-assignment operation from the semantics section. The free-for condition prevents variable capture.

Remark (Dependencies).

- [Substitution Notation](#)
- [Free for Substitution](#)
- [Satisfaction](#)
- [Substitution Lemma for Terms](#)

2.5 Quantifiers

Toolkit: Quantifiers

Topic	Role
Basic syntax and semantics	universal, existential, bounded, sentence, open formula, scope
Quantifier laws	equivalence laws for negation, distribution, commutation, and renaming
Prenex normal form	syntactic normalization with quantifiers pulled forward
Logical strength	quantifier order, dependency, and non-equivalence examples

Remark (Exposition). Quantifiers extend propositional logic by allowing statements about all domain elements or about witnesses. The main new issue is dependency: later witnesses may depend on earlier universal choices.

Basic Syntax and Semantics

Remark (Section toolkit: Quantifier grammar). This section records the basic quantifier forms and the boundary between open formulas and sentences. Quantifiers bind variables. A formula with free variables requires an assignment for semantic evaluation; a sentence has no free variables and has a truth value in a structure.

Remark (Exposition). Sentences have no free variables and have truth values in structures. Open formulas may contain free variables and require assignments. Quantifiers turn some free occurrences into bound occurrences by placing them inside a scope.

Definition (Universal Formula)

Definition 2.5.1 (Universal Formula). A *universal formula* is a well-formed formula of the form

$$\forall x \varphi,$$

where x is a variable and φ is a well-formed formula.

Remark (Standard quantified statement).

$$\forall x \varphi \in \mathbf{Form}_{\mathcal{L}} \iff x \in \mathbf{Var}_{\mathcal{L}} \text{ and } \varphi \in \mathbf{Form}_{\mathcal{L}}.$$

Remark (Definition predicate reading). $\text{UniversalFormula}(\Phi)$ means that Φ is a formula whose outermost constructor is a universal quantifier.

Remark (Negated quantified statement).

Φ is not of the form $\forall x \varphi$.

Remark (Interpretation). A universal formula asserts that its scope holds for every value assigned to the bound variable.

Remark (Examples). $\forall x P(x)$ and $\forall x \exists y R(x, y)$ are universal formulas.

Remark (Non-Examples). $P(x)$ and $\exists x P(x)$ are not universal formulas.

Remark (Dependencies).

- [Variable](#)

Definition (Existential Formula)

Definition 2.5.2 (Existential Formula). An *existential formula* is a well-formed formula of the form

$$\exists x \varphi,$$

where x is a variable and φ is a well-formed formula.

Remark (Standard quantified statement).

$$\exists x \varphi \in \mathbf{Form}_{\mathcal{L}} \iff x \in \mathbf{Var}_{\mathcal{L}} \text{ and } \varphi \in \mathbf{Form}_{\mathcal{L}}.$$

Remark (Definition predicate reading). $\text{ExistentialFormula}(\Phi)$ means that Φ is a formula whose outermost constructor is an existential quantifier.

Remark (Negated quantified statement).

Φ is not of the form $\exists x \varphi$.

Remark (Interpretation). An existential formula asserts that its scope holds for at least one value of the bound variable.

Remark (Examples). $\exists x P(x)$ and $\exists x \forall y R(x, y)$ are existential formulas.

Remark (Non-Examples). $P(x)$ and $\forall x P(x)$ are not existential formulas.

Remark (Dependencies).

- [Variable](#)

Definition (Bounded Quantifier)

Definition 2.5.3 (Bounded Quantifier). A *bounded quantifier* is an abbreviation that restricts a quantifier to a specified set or predicate:

$$\forall x \in A \varphi \quad := \quad \forall x (x \in A \rightarrow \varphi),$$

$$\exists x \in A \varphi \quad := \quad \exists x (x \in A \wedge \varphi).$$

Remark (Standard quantified statement).

$$\forall x \in A \varphi \equiv \forall x (x \in A \rightarrow \varphi), \quad \exists x \in A \varphi \equiv \exists x (x \in A \wedge \varphi).$$

Remark (Definition predicate reading). $\text{BoundedQuantifier}(\Phi)$ means that Φ uses a restricted universal or existential quantifier.

Remark (Negated quantified statement).

Φ is not an abbreviation of either displayed bounded form.

Remark (Interpretation). The restricted universal uses implication, while the restricted existential uses conjunction. This preserves the usual behavior on empty sets.

Remark (Examples). $\forall x \in A P(x)$ and $\exists x \in A P(x)$ are bounded-quantifier formulas.

Remark (Non-Examples). $\forall x (x \in A \wedge P(x))$ is not the correct expansion of the bounded universal form.

Remark (Dependencies).

- [Universal Formula](#)
- [Existential Formula](#)

Definition (Sentence)

Definition 2.5.4 (Sentence). A *sentence* is a well-formed formula with no free variables.

Remark (Standard quantified statement).

$$\varphi \text{ is a sentence} \iff \varphi \in \text{Form}_{\mathcal{L}} \text{ and } \text{FV}(\varphi) = \emptyset.$$

Remark (Definition predicate reading). $\text{Sentence}(\varphi)$ means that φ is a well-formed formula with no free variables.

Remark (Negated quantified statement).

$$\varphi \notin \text{Form}_{\mathcal{L}} \quad \text{or} \quad \text{FV}(\varphi) \neq \emptyset.$$

Remark (Interpretation). Sentences have truth values in structures without reference to a particular assignment.

Remark (Examples). $\forall x P(x)$ and $\exists x \forall y R(x, y)$ are sentences when all variables shown are bound.

Remark (Non-Examples). $P(x)$ and $\exists y R(x, y)$ are not sentences when x remains free.

Remark (Dependencies).

- [Well-Formed Formula](#)

Definition (Open Formula)

Definition 2.5.5 (Open Formula). An *open formula* is a well-formed formula with at least one free variable.

Remark (Standard quantified statement).

$$\varphi \text{ is open} \iff \varphi \in \text{Form}_{\mathcal{L}} \text{ and } \text{FV}(\varphi) \neq \emptyset.$$

Remark (Definition predicate reading). $\text{OpenFormula}(\varphi)$ means that φ has at least one free variable.

Remark (Negated quantified statement).

$$\varphi \notin \text{Form}_{\mathcal{L}} \quad \text{or} \quad \text{FV}(\varphi) = \emptyset.$$

Remark (Interpretation). Open formulas may become true or false only after a structure and assignment are fixed.

Remark (Examples). $P(x)$ and $R(x, y)$ are open formulas.

Remark (Non-Examples). $\forall x P(x)$ is a sentence rather than an open formula.

Remark (Dependencies).

- [Well-Formed Formula](#)
- [Open Formula](#)

Definition (Quantifier Scope)

Definition 2.5.6 (Quantifier Scope). In a formula $Qx\varphi$, where $Q \in \{\forall, \exists\}$, the *scope* of the quantifier Qx is the formula φ . Occurrences of x inside that scope are bound by the quantifier unless an inner quantifier for x intervenes.

Remark (Standard quantified statement).

$$Qx\varphi \text{ has scope } \varphi, \quad Q \in \{\forall, \exists\}.$$

Remark (Definition predicate reading). $\text{Scope}(x, \varphi)$ means that φ is the scope governed by the quantifier binding x .

Remark (Negated quantified statement).

φ is not the formula governed by the displayed quantifier on x .

Remark (Interpretation). Scope determines which variable occurrences a quantifier controls. Scope errors are the main source of incorrect translations from English into predicate logic.

Remark (Examples). In $\forall x (P(x) \rightarrow Q(x))$, the scope is $(P(x) \rightarrow Q(x))$. In $(\forall x P(x)) \rightarrow Q(x)$, the final occurrence of x is outside the scope of the quantifier.

Remark (Dependencies).

- [Universal Formula](#)
- [Existential Formula](#)

Theorem (Negation of Bounded Quantifiers)

Theorem 2.5.7 (Negation of Bounded Quantifiers). *Let A be a set and let φ be a formula. Then*

$$\neg(\forall x \in A \varphi) \equiv \exists x \in A \neg\varphi, \quad \neg(\exists x \in A \varphi) \equiv \forall x \in A \neg\varphi.$$

Go to proof.

Remark (Standard quantified statement).

$$\neg\forall x (x \in A \rightarrow \varphi) \equiv \exists x (x \in A \wedge \neg\varphi), \quad \neg\exists x (x \in A \wedge \varphi) \equiv \forall x (x \in A \rightarrow \neg\varphi).$$

Remark (Predicate reading). The theorem records the negation law for bounded universal and bounded existential formulas.

Remark (Interpretation). The domain restriction is preserved under negation. What changes is the quantifier and the inner assertion.

Remark (Dependencies).

- [Bounded Quantifier](#)
- [Quantifier Negation Laws](#)

Quantifier Laws

Remark (Section toolkit: Quantifier algebra). Quantifier algebra is the predicate-logic analogue of propositional equivalence laws. These laws justify rewriting quantified formulas while preserving logical equivalence.

Theorem (Quantifier Negation Laws)

Theorem 2.5.8 (Quantifier Negation Laws). *For every formula φ ,*

$$\neg\forall x \varphi \equiv \exists x \neg\varphi, \quad \neg\exists x \varphi \equiv \forall x \neg\varphi.$$

Go to proof.

Remark (Standard quantified statement).

$$\forall \varphi \in \text{Form}_{\mathcal{L}} \quad \neg \forall x \varphi \equiv \exists x \neg \varphi \quad \text{and} \quad \neg \exists x \varphi \equiv \forall x \neg \varphi.$$

Remark (Predicate reading). LogicalEquivalence($\neg \forall x \varphi, \exists x \neg \varphi$) and LogicalEquivalence($\neg \exists x \varphi, \forall x \neg \varphi$) ■

Remark (Interpretation). Negating a quantified statement flips the quantifier and pushes the negation into the scope.

Remark (Dependencies).

- [Universal Formula](#)
- [Existential Formula](#)
- [Logical Equivalence](#)

Theorem (Double Negation and Quantifiers)

Theorem 2.5.9 (Double Negation and Quantifiers). *For every formula φ ,*

$$\neg \neg \forall x \varphi \equiv \forall x \varphi, \quad \neg \neg \exists x \varphi \equiv \exists x \varphi.$$

Go to proof.

Remark (Standard quantified statement).

$$\forall \varphi \in \text{Form}_{\mathcal{L}} \quad \neg \neg \forall x \varphi \equiv \forall x \varphi \quad \text{and} \quad \neg \neg \exists x \varphi \equiv \exists x \varphi.$$

Remark (Predicate reading). LogicalEquivalence($\neg \neg Qx \varphi, Qx \varphi$) for $Q \in \{\forall, \exists\}$.

Remark (Interpretation). Double negation does not change the quantifier form once the two negations are cancelled.

Remark (Dependencies).

- [Quantifier Negation Laws](#)

Theorem (Same-Type Quantifier Commutation)

Theorem 2.5.10 (Same-Type Quantifier Commutation). *Quantifiers of the same type commute:*

$$\forall x \forall y \varphi \equiv \forall y \forall x \varphi, \quad \exists x \exists y \varphi \equiv \exists y \exists x \varphi.$$

Go to proof.

Remark (Standard quantified statement).

$$\forall \varphi \in \text{Form}_{\mathcal{L}} \quad \forall x \forall y \varphi \equiv \forall y \forall x \varphi \quad \text{and} \quad \exists x \exists y \varphi \equiv \exists y \exists x \varphi.$$

Remark (Predicate reading). The theorem asserts logical equivalence after swapping adjacent quantifiers of the same type.

Remark (Interpretation). Two universal choices can be checked in either order, and two existential witnesses can be selected in either order.

Remark (Dependencies).

- [Universal Formula](#)
- [Existential Formula](#)
- [Logical Equivalence](#)

Theorem (Mixed Quantifiers Do Not Commute)

Theorem 2.5.11 (Mixed Quantifiers Do Not Commute). *In general,*

$$\forall x \exists y \varphi \not\equiv \exists y \forall x \varphi.$$

Go to proof.

Remark (Standard quantified statement).

$$\exists \varphi \in \mathbf{Form}_{\mathcal{L}} \quad \forall x \exists y \varphi \not\equiv \exists y \forall x \varphi.$$

Remark (Predicate reading). $\neg \text{LogicalEquivalence}(\forall x \exists y \varphi, \exists y \forall x \varphi)$ in general.

Remark (Interpretation). The first form permits the witness y to depend on x . The second form requires one uniform witness y that works for every x .

Remark (Dependencies).

- [Same-Type Quantifier Commutation](#)

Theorem (Valid Quantifier Distribution)

Theorem 2.5.12 (Valid Quantifier Distribution). *For all formulas φ, ψ ,*

$$\forall x (\varphi \wedge \psi) \equiv (\forall x \varphi) \wedge (\forall x \psi),$$

$$\exists x (\varphi \vee \psi) \equiv (\exists x \varphi) \vee (\exists x \psi).$$

Go to proof.

Remark (Standard quantified statement).

$$\forall \varphi, \psi \in \mathbf{Form}_{\mathcal{L}} \quad \forall x (\varphi \wedge \psi) \equiv (\forall x \varphi) \wedge (\forall x \psi)$$

and

$$\exists x (\varphi \vee \psi) \equiv (\exists x \varphi) \vee (\exists x \psi).$$

Remark (Predicate reading). The displayed universal-conjunction and existential-disjunction distributions are logical equivalences.

Remark (Interpretation). Universal quantification distributes over conjunction because both conjuncts must hold for every value. Existential quantification distributes over disjunction because a witness for either disjunct witnesses the disjunction.

Remark (Dependencies).

- [Logical Equivalence](#)
- [Universal Formula](#)
- [Existential Formula](#)

Theorem (Invalid Quantifier Distribution)

Theorem 2.5.13 (Invalid Quantifier Distribution). *In general,*

$$\forall x (\varphi \vee \psi) \not\equiv (\forall x \varphi) \vee (\forall x \psi),$$

$$\exists x (\varphi \wedge \psi) \not\equiv (\exists x \varphi) \wedge (\exists x \psi).$$

Go to proof.

Remark (Standard quantified statement).

$$\exists \varphi, \psi \in \mathbf{Form}_{\mathcal{L}} \quad \forall x (\varphi \vee \psi) \not\equiv (\forall x \varphi) \vee (\forall x \psi),$$

$$\exists \varphi, \psi \in \mathbf{Form}_{\mathcal{L}} \quad \exists x (\varphi \wedge \psi) \not\equiv (\exists x \varphi) \wedge (\exists x \psi).$$

Remark (Predicate reading). The displayed distribution patterns are not logical equivalences in general.

Remark (Interpretation). The invalid laws fail because witnesses or counterexamples may vary between the two subformulas.

Remark (Dependencies).

- [Valid Quantifier Distribution](#)

Theorem (Vacuous Quantification)

Theorem 2.5.14 (Vacuous Quantification). *If x does not occur free in φ , then*

$$\forall x \varphi \equiv \varphi, \quad \exists x \varphi \equiv \varphi.$$

Go to proof.

Remark (Standard quantified statement).

$$x \notin \mathbf{FV}(\varphi) \Rightarrow (\forall x \varphi \equiv \varphi) \wedge (\exists x \varphi \equiv \varphi).$$

Remark (Predicate reading). Vacuous quantification preserves logical equivalence when the quantified variable is not free in the scope.

Remark (Interpretation). A quantifier that binds no occurrence has no semantic effect.

Remark (Dependencies).

- [Quantifier Scope](#)

Theorem (Renaming Bound Variables)

Theorem 2.5.15 (Renaming Bound Variables). *If y is free for x in φ and no capture is introduced, then*

$$\forall x \varphi \equiv \forall y \varphi[y/x], \quad \exists x \varphi \equiv \exists y \varphi[y/x].$$

Go to proof.

Remark (Standard quantified statement).

$$y \text{ free for } x \text{ in } \varphi \Rightarrow (\forall x \varphi \equiv \forall y \varphi[y/x]) \wedge (\exists x \varphi \equiv \exists y \varphi[y/x]).$$

Remark (Predicate reading). Renaming bound variables preserves logical equivalence when the renaming is capture-avoiding.

Remark (Interpretation). Bound variable names are placeholders. Renaming them changes notation, not meaning, provided no free variable becomes captured.

Remark (Dependencies).

- [Free for Substitution](#)
- [Substitution Notation](#)

Prenex Normal Form

Remark (Section toolkit: Prenex normalization). Prenex normal form is the predicate-logic analogue of the normal-form procedures from propositional logic. It is a syntactic normalization procedure, not a semantic change: each step preserves logical equivalence.

Definition (Quantifier Prefix)

Definition 2.5.16 (Quantifier Prefix). A *quantifier prefix* is a finite string

$$Q_1 x_1 Q_2 x_2 \cdots Q_n x_n,$$

where each $Q_i \in \{\forall, \exists\}$ and each x_i is a variable.

Remark (Standard quantified statement).

$$\Pi = Q_1 x_1 \cdots Q_n x_n \iff \forall i \leq n (Q_i \in \{\forall, \exists\} \text{ and } x_i \in \text{Var}_{\mathcal{L}}).$$

Remark (Definition predicate reading). $\text{QuantifierPrefix}(\Pi)$ means that Π is a finite string of quantifier-variable pairs.

Remark (Interpretation). A prefix records all quantifier choices before the quantifier-free part of a prenex formula is read.

Remark (Dependencies).

- [Universal Formula](#)
- [Existential Formula](#)

Definition (Matrix)

Definition 2.5.17 (Matrix). In a formula $Q_1x_1 \cdots Q_nx_n \psi$, the *matrix* is the trailing formula ψ . For prenex normal form, the matrix is required to be quantifier-free.

Remark (Standard quantified statement).

$Q_1x_1 \cdots Q_nx_n \psi$ has matrix ψ , ψ quantifier-free in prenex normal form.

Remark (Definition predicate reading). $\text{Matrix}(\psi, \Phi)$ means that ψ is the matrix of the formula Φ .

Remark (Interpretation). The matrix is the part of a prenex formula where the propositional connectives and atomic formulas remain after all quantifiers have been pulled forward.

Remark (Dependencies).

- [Well-Formed Formula](#)

Definition (Prenex Formula)

Definition 2.5.18 (Prenex Formula). A *prenex formula* is a formula of the form

$$Q_1x_1 Q_2x_2 \cdots Q_nx_n \psi,$$

where $Q_1x_1 \cdots Q_nx_n$ is a quantifier prefix and ψ is quantifier-free.

Remark (Standard quantified statement).

Φ is prenex $\iff \Phi = Q_1x_1 \cdots Q_nx_n \psi$ with ψ quantifier-free.

Remark (Definition predicate reading). $\text{PrenexFormula}(\Phi)$ means that all quantifiers of Φ occur in an initial prefix.

Remark (Negated quantified statement).

Φ is not of the form $Q_1x_1 \cdots Q_nx_n \psi$ with ψ quantifier-free.

Remark (Interpretation). Prenex formulas isolate quantifier order from the quantifier-free matrix.

Remark (Examples). $\forall x \exists y (P(x) \rightarrow R(x, y))$ is prenex.

Remark (Non-Examples). $\forall x (P(x) \rightarrow \exists y R(x, y))$ is not prenex because a quantifier remains inside the matrix.

Remark (Dependencies).

- [Quantifier Prefix](#)
- [Matrix](#)

Definition (Prenex Normal Form)

Definition 2.5.19 (Prenex Normal Form). A formula ψ is a *prenex normal form* of a formula φ exactly when ψ is prenex and $\varphi \equiv \psi$.

Remark (Standard quantified statement).

$$\psi \text{ is a prenex normal form of } \varphi \iff \psi \text{ is prenex and } \varphi \equiv \psi.$$

Remark (Definition predicate reading). $\text{PrenexNormalForm}(\psi, \varphi)$ means that ψ is a prenex formula logically equivalent to φ .

Remark (Interpretation). Prenex normal form changes the placement of quantifiers but not the meaning of the formula.

Remark (Exposition). Prenex normal form is not unique. Different choices of bound-variable names and different orders of valid pulling steps can produce different but logically equivalent prenex formulas.

Remark (Exposition). A prenex conversion proceeds in stages: eliminate implications and biconditionals, push negations inward, rename bound variables as needed, pull quantifiers forward, and leave a quantifier-free matrix.

Remark (Examples). Starting with

$$\neg(\forall x P(x) \rightarrow \exists y Q(y)),$$

eliminate the implication, push the negation inward, and pull the quantifiers forward to obtain

$$\forall x \forall y (P(x) \wedge \neg Q(y)).$$

Remark (Dependencies).

- [Prenex Formula](#)
- [Logical Equivalence](#)

Theorem (Existence of Prenex Normal Form)

Theorem 2.5.20 (Existence of Prenex Normal Form). *Every first-order formula is logically equivalent to some formula in prenex normal form. Go to proof.*

Remark (Standard quantified statement).

$$\forall \varphi \in \text{Form}_{\mathcal{L}} \exists \psi \in \text{Form}_{\mathcal{L}} (\psi \text{ is prenex} \wedge \varphi \equiv \psi).$$

Remark (Predicate reading). $\forall \varphi \exists \psi \text{ PrenexNormalForm}(\psi, \varphi)$.

Remark (Interpretation). Every formula can be converted into an equivalent one with all quantifiers at the front and a quantifier-free matrix at the end.

Remark (Exposition). The proof proceeds by structural rewriting: remove implication and biconditional connectives, push negations inward, rename bound variables to avoid capture, and then pull quantifiers forward using equivalences whose side conditions have been made true by renaming.

Remark (Dependencies).

- [Prenex Normal Form](#)
- [Quantifier Negation Laws](#)
- [Renaming Bound Variables](#)
- [Valid Quantifier Distribution](#)

Logical Strength and Quantifier Order

Remark (Section toolkit: Strength and order). Quantifier order is one of the main new phenomena in predicate logic. In propositional logic, changing the order of independent connectives often keeps truth conditions recognizable. In predicate logic, moving $\forall$ and $\exists$ can change whether witnesses may depend on earlier choices.

Definition (Stronger Formula)

Definition 2.5.21 (Stronger Formula). A sentence A is *stronger than* a sentence B exactly when

$$A \models B \quad \text{and} \quad B \not\models A.$$

Remark (Standard quantified statement).

$$A \text{ is stronger than } B \iff (\forall \mathcal{M} (\mathcal{M} \models A \Rightarrow \mathcal{M} \models B)) \wedge (\exists \mathcal{N} (\mathcal{N} \models B \wedge \mathcal{N} \not\models A)).$$

Remark (Definition predicate reading). $\text{StrongerFormula}(A, B)$ means that A entails B , but B does not entail A .

Remark (Negated quantified statement).

$$A \not\models B \quad \text{or} \quad B \models A.$$

Remark (Interpretation). The stronger sentence has fewer possible structures in which it is true. It rules out more cases.

Remark (Examples). $\forall x P(x)$ is stronger than $\exists x P(x)$ over nonempty domains.

Remark (Dependencies).

- [Logical Consequence](#)

Definition (Weaker Formula)

Definition 2.5.22 (Weaker Formula). A sentence B is *weaker than* a sentence A exactly when A is stronger than B .

Remark (Standard quantified statement).

$$B \text{ is weaker than } A \iff A \models B \text{ and } B \not\models A.$$

Remark (Definition predicate reading). $\text{WeakerFormula}(B, A)$ means that B is entailed by A but does not entail A .

Remark (Interpretation). The weaker sentence permits more structures. It follows from the stronger sentence but carries less information.

Remark (Dependencies).

- [Stronger Formula](#)

Definition (Quantifier Order)

Definition 2.5.23 (Quantifier Order). The *quantifier order* of a prenex formula is the ordered list of quantifiers in its prefix. A change of order is significant when it changes which witnesses may depend on earlier variables.

Remark (Standard quantified statement).

$$Q_1 x_1 \cdots Q_n x_n \psi \text{ has quantifier order } (Q_1, \dots, Q_n).$$

Remark (Definition predicate reading). $\text{QuantifierOrder}(\Phi, (Q_1, \dots, Q_n))$ records the order of the quantifiers in the prefix of Φ .

Remark (Interpretation). Quantifier order controls dependency. In $\forall x \exists y$, the witness y may depend on x . In $\exists y \forall x$, the same y must work for all x .

Remark (Dependencies).

- [Quantifier Prefix](#)
- [Prenex Formula](#)

Theorem (Universal Implies Existential)

Theorem 2.5.24 (Universal Implies Existential). *Over nonempty domains,*

$$\forall x \varphi(x) \models \exists x \varphi(x).$$

Go to proof.

Remark (Standard quantified statement).

$$\forall \mathcal{M} (\mathcal{M} \models \forall x \varphi(x) \Rightarrow \mathcal{M} \models \exists x \varphi(x)).$$

Remark (Predicate reading). $\text{LogicalConsequence}(\{\forall x \varphi(x)\}, \exists x \varphi(x))$.

Remark (Interpretation). If every domain element satisfies φ , then at least one does because first-order structures have nonempty domains.

Remark (Dependencies).

- [Structure](#)
- [Logical Consequence](#)

Theorem (Existential Does Not Imply Universal)

Theorem 2.5.25 (Existential Does Not Imply Universal). *In general,*

$$\exists x \varphi(x) \not\models \forall x \varphi(x).$$

Go to proof.

Remark (Standard quantified statement).

$$\exists \mathcal{M} (\mathcal{M} \models \exists x \varphi(x) \wedge \mathcal{M} \not\models \forall x \varphi(x)).$$

Remark (Predicate reading). $\neg \text{LogicalConsequence}(\{\exists x \varphi(x)\}, \forall x \varphi(x))$.

Remark (Interpretation). One witness does not make a universal claim true.

Remark (Exposition). In the real numbers, $\exists x (x = 0)$ is true, but $\forall x (x = 0)$ is false.

Remark (Dependencies).

- [Universal Implies Existential](#)

Theorem (Uniform Witness Implies Dependent Witness)

Theorem 2.5.26 (Uniform Witness Implies Dependent Witness). *For every formula $\varphi(x, y)$,*

$$\exists y \forall x \varphi(x, y) \models \forall x \exists y \varphi(x, y).$$

Go to proof.

Remark (Standard quantified statement).

$$\forall \mathcal{M} (\mathcal{M} \models \exists y \forall x \varphi(x, y) \Rightarrow \mathcal{M} \models \forall x \exists y \varphi(x, y)).$$

Remark (Predicate reading). $\text{LogicalConsequence}(\{\exists y \forall x \varphi(x, y)\}, \forall x \exists y \varphi(x, y))$.

Remark (Interpretation). A single witness y that works for every x can be reused as the witness required for each separate x .

Remark (Dependencies).

- [Quantifier Order](#)
- [Logical Consequence](#)

Theorem (Dependent Witness Need Not Be Uniform)

Theorem 2.5.27 (Dependent Witness Need Not Be Uniform). *In general,*

$$\forall x \exists y \varphi(x, y) \not\models \exists y \forall x \varphi(x, y).$$

Go to proof.

Remark (Standard quantified statement).

$$\exists \mathcal{M} (\mathcal{M} \models \forall x \exists y \varphi(x, y) \wedge \mathcal{M} \not\models \exists y \forall x \varphi(x, y)).$$

Remark (Predicate reading). $\neg \text{LogicalConsequence}(\{\forall x \exists y \varphi(x, y)\}, \exists y \forall x \varphi(x, y))$.

Remark (Interpretation). The dependent-witness statement allows y to vary with x . The uniform statement requires one y to handle all x at once.

Remark (Exposition). Over $\mathbb{R}$, let $\varphi(x, y)$ be $y > x$. Then $\forall x \exists y (y > x)$ is true, since $y = x + 1$ works for each x . But $\exists y \forall x (y > x)$ is false, since no real number is larger than every real number.

Remark (Exposition). Quantifier order matters because it records dependency. The phrase "for every x , there exists y " permits a different y after x is known. The phrase "there exists y such that for every x " chooses y before seeing x . This is a genuine expressive jump beyond propositional logic.

Remark (Dependencies).

- [Uniform Witness Implies Dependent Witness](#)

2.6 Proof Theory

Quantifier Inference Rules — Quick Reference

Rule	Schema	Key restriction
UI	$\forall x \varphi \Rightarrow \varphi[t/x]$	t free for x
UG	$\varphi \Rightarrow \forall x \varphi$	x not free in assumptions
EI	$\exists x \varphi \Rightarrow \varphi[c/x]$	c fresh witness
EG	$\varphi[t/x] \Rightarrow \exists x \varphi$	none

2.6.1 Proof Theory: Quantifier Inference Rules

Definition (Universal Instantiation — UI)

Definition 2.6.1 (Universal Instantiation — UI). From a universally quantified statement, infer any instance obtained by substituting a term for the bound variable:

$$\forall x \varphi \Rightarrow \varphi[t/x].$$

Remark (Dependencies).

- [Universal Quantifier](#)
- [Substitution Notation](#)
- [Free for Substitution](#)

Remark (Side condition). The term t must be free for x in φ . If t contains a variable that would become bound after substitution, the inference is invalid.

Definition (Universal Generalization — UG)

Definition 2.6.2 (Universal Generalization — UG). From a formula, infer a universally quantified statement:

$$\varphi \Rightarrow \forall x \varphi.$$

Remark (Dependencies).

- [Universal Quantifier](#)
- [Formula](#)
- [Bound and Free Occurrences](#)

Remark (Restriction on UG). The variable x must not occur free in any undischarged assumption on which φ depends. This restriction ensures that x is truly arbitrary — not tied to a specific hypothesis about x .

Remark (Arbitrary element argument). To prove $\forall x \varphi(x)$, fix an arbitrary element x (introducing no special assumptions about x), prove $\varphi(x)$, and then apply UG. The variable x must remain unconstrained throughout.

Definition (Existential Instantiation — EI)

Definition 2.6.3 (Existential Instantiation — EI). From an existentially quantified statement, infer an instance with a fresh constant (witness):

$$\exists x \varphi \Rightarrow \varphi[c/x].$$

Remark (Dependencies).

- [Existential Quantifier](#)
- [Constant Symbol](#)
- [Substitution Notation](#)
- [Free for Substitution](#)

Remark (Witness discipline for EI). The constant c must be fresh: it must not appear in φ , in any undischarged assumption, or in the final conclusion of the proof. The constant represents an arbitrary but fixed witness to the existential claim, not a specific known object.

Remark (Common error). Using a witness constant introduced by EI outside its allowed scope, or choosing c before establishing the existential premise, invalidates the proof.

Definition (Existential Generalization — EG)

Definition 2.6.4 (Existential Generalization — EG). From a formula containing a term, infer an existential statement:

$$\varphi[t/x] \Rightarrow \exists x \varphi.$$

Remark (Dependencies).

- [Existential Quantifier](#)
- [Term](#)
- [Substitution Notation](#)

Remark (No side conditions). EG has no side conditions: any term t may be replaced by an existentially quantified variable. To prove $\exists x \varphi(x)$, explicitly exhibit a term t such that $\varphi(t)$ holds, then apply EG.

Remark (Counterexample argument). To refute $\forall x \varphi(x)$: produce a single term t such that $\neg\varphi(t)$ holds. To refute $\exists x \varphi(x)$: show that $\varphi(t)$ fails for every term t in the domain.

Quantifier Proof Strategies — Quick Reference

Goal shape	Strategy	Key rule
Prove $\forall x \varphi(x)$	Fix an arbitrary x ; prove $\varphi(x)$ for that x	UG
Prove $\exists x \varphi(x)$	Produce a specific witness t ; verify $\varphi(t)$	EG
Use $\forall x \varphi(x)$	Instantiate at any term t to get $\varphi(t)$	UI
Use $\exists x \varphi(x)$	Introduce a fresh name c for the witness	EI
Negate $\forall x \varphi$	Becomes $\exists x \neg\varphi$	Neg. law
Negate $\exists x \varphi$	Becomes $\forall x \neg\varphi$	Neg. law
Prove $\forall x \exists y \varphi(x, y)$	Fix x ; then construct y depending on x	UG then EG
Prove $\exists y \forall x \varphi(x, y)$	Find y first; then verify for all x	EG then UG

2.6.2 Proof Strategies for Quantified Statements

The inference rules UI, UG, EI, and EG were defined in the previous section. That section answered the question: what does each rule license? This section answers a different question: when the goal of a proof has a particular quantifier structure, which rule should be used, and what does the proof skeleton look like? The two questions are distinct, and both must be understood clearly before reading analysis proofs.

The single most important organizing principle is this: *the shape of the goal determines the proof structure, and the shape of the hypothesis determines how to use it.* A $\forall$ goal calls for a different move than a $\exists$ goal; a $\forall$ hypothesis is used differently from a $\exists$ hypothesis. The four combinations are the four strategies below.

Strategy 1: Proving a Universal $\forall x \varphi(x)$

To prove $\forall x \varphi(x)$: introduce a fresh variable x_0 representing an *arbitrary, unspecified* element of the domain. Prove $\varphi(x_0)$ without making any assumption about x_0 beyond its being in the domain. Then apply Universal Generalization (UG) to conclude $\forall x \varphi(x)$.

Let x_0 be an arbitrary element of the domain.

[prove $\varphi(x_0)$ using only the hypotheses, not any special property of x_0]

Therefore: $\forall x \varphi(x)$. (by UG)

The critical constraint is that x_0 must be *arbitrary*: no hypothesis in the proof may impose a condition on x_0 that is not universally true. Violating this constraint is the UG fallacy (“illicit generalization”).

Remark (Why “let x be arbitrary” works). Fixing an arbitrary element x_0 is the formal counterpart of the English phrase “let x be any element.” The word “arbitrary” means: the proof uses no information about x_0 that is specific to x_0 — only information that applies to every element of the domain. If the argument goes through for such an unspecified x_0 , it goes through for every x , so the universal follows.

Remark (The UG side condition in practice). UG requires that x_0 not appear free in any undischarged assumption. In practice: do not use any hypothesis of the form “ x_0 has property P ” unless P holds of every domain element. If a hypothesis says “ $x_0 > 0$,” then x_0 is not arbitrary, and UG to $\forall x$ is invalid.

Strategy 2: Proving an Existential $\exists x \varphi(x)$

To prove $\exists x \varphi(x)$: exhibit a specific term t and verify that $\varphi(t)$ holds. Then apply Existential Generalization (EG) to conclude $\exists x \varphi(x)$.

Claim: $\exists x \varphi(x)$.

Take: $t :=$ [specific term].

Verify: $\varphi(t)$ holds. [proof that t works]

Therefore: $\exists x \varphi(x)$. (by EG)

The entire burden of an existential proof is finding the right t . Once t is identified and verified, the logic is trivial: one application of EG.

Remark (Where the difficulty lies). In existential proofs, the logical structure is simple but the mathematical insight is often deep. Finding the witness requires understanding the problem; verifying the witness is usually straightforward. This is why analysis proofs often say “take $\delta = \min(\varepsilon/2, 1)$ ” without explaining where that choice came from — the work happened before the formal proof was written.

Strategy 3: Using a Universal $\forall x \varphi(x)$

If a hypothesis or established fact has the form $\forall x \varphi(x)$, one can instantiate it at *any* specific term t to obtain $\varphi(t)$, by Universal Instantiation (UI). There are no side conditions beyond t being free for x in φ .

Available: $\forall x \varphi(x)$.

Choose any term t .

Obtain: $\varphi(t)$. (by UI)

UI can be applied repeatedly to the same universal statement with different choices of t .

Strategy 4: Using an Existential $\exists x \varphi(x)$

If a hypothesis has the form $\exists x \varphi(x)$, introduce a *fresh constant* c to name the (unknown, unspecified) witness, obtaining $\varphi(c)$ by Existential Instantiation (EI). The constant c must be new: it must not appear anywhere else in the proof.

Available: $\exists x \varphi(x)$.

Let c be a witness. (fresh constant, not used elsewhere)

Obtain: $\varphi(c)$. (by EI)

The freshness requirement is not a technicality. Using a constant that already appears in the proof — say, a constant a introduced under a different assumption — would illegally identify the witness with that specific element. The witness c is only known to satisfy $\varphi(c)$; nothing else about it is known, and it cannot be equated with anything specific.

Remark (The scope of a witness constant). Once c is introduced by EI, it names the witness for the duration of the argument. Conclusions derived using c depend on the existence of such a witness; they are valid (they do not depend on which specific element c actually names). However, c cannot be exported outside the scope in which the existential was instantiated, and no universal generalization may be applied to c .

The Central Asymmetry: Universal vs. Existential Goals

The following comparison is the most important thing to understand about quantifier proof structure. Universal and existential goals require fundamentally different strategies, and confusing them is one of the most common serious errors in mathematical writing.

Universal vs. Existential Goals: The Fundamental Difference		
	$\forall x \varphi(x)$	$\exists x \varphi(x)$
To prove:	Fix <i>arbitrary</i> x ; prove $\varphi(x)$	<i>Choose</i> specific t ; verify $\varphi(t)$
Who chooses x?	The adversary (you cannot choose)	You (the prover)
Main rule:	UG	EG
Burden:	Works for every x without assumptions	A single witness suffices
Error:	Assuming something special about x	Failing to exhibit a concrete t

Remark (Who controls the variable). For a universal goal, the variable x is chosen by the “opponent” of the proof: the proof must work no matter what x is. The prover has no control over x ; hence the term “arbitrary.” For an existential goal, the prover chooses the witness t ; the power is entirely on the prover’s side, but so is the obligation to produce something concrete and verifiable.

Nested Quantifiers: The $\forall\exists$ Pattern in Analysis

Every ε - δ proof in analysis is a proof of a statement with the pattern $\forall\varepsilon\exists\delta\cdots$. Understanding the proof structure of such statements is the direct payoff of this section.

Proof Skeleton: $\forall x\exists y\varphi(x,y)$

Step 1. Fix arbitrary x . (Opening move for the outer $\forall$; x is unspecified. This is Strategy 1.)

Step 2. Define y in terms of x . (The witness for $\exists y$ may, and typically does, depend on x . This is Strategy 2 — choosing a witness.)

Step 3. Verify $\varphi(x,y)$ using the specific y from Step 2 and the arbitrary x from Step 1.

Step 4. Conclude: $\exists y\varphi(x,y)$. (by EG)

Step 5. Conclude: $\forall x\exists y\varphi(x,y)$. (by UG, since x was arbitrary in Step 1)

The key insight: y is chosen *after* x is fixed, so y is allowed to depend on x .

Proof Skeleton: $\exists y \forall x \varphi(x, y)$

Step 1. Define y . (The witness for $\exists y$ must be chosen *before* x is introduced. This y must work for *every* x .)

Step 2. Fix arbitrary x . (Now x is introduced.)

Step 3. Verify $\varphi(x, y)$ for the specific y from Step 1 and arbitrary x from Step 2.

Step 4. Conclude: $\forall x \varphi(x, y)$. (by UG)

Step 5. Conclude: $\exists y \forall x \varphi(x, y)$. (by EG)

The key contrast: y is chosen *before* x , so y cannot depend on x . This is why $\exists y \forall x$ is a much stronger claim.

Remark (Why order determines strength). In the $\forall \exists$ skeleton, the witness y is constructed from x , so a different y is allowed for each x . In the $\exists \forall$ skeleton, a single y must work for all x simultaneously. The first pattern appears in continuity (δ may depend on ε and x); the second in uniform continuity (δ may depend on ε but not on x). The difference between continuity and uniform continuity is exactly the difference between these two proof skeletons.

A Complete Worked Example

The following proof traces every UI/UG/EI/EG step explicitly. In practice, proofs are written more concisely, but the logical structure is always this one.

Example (A worked multi-quantifier proof). **Claim:** If $\forall x \exists y P(x, y)$ and $\forall x \forall y (P(x, y) \rightarrow Q(x, y))$, then $\forall x \exists y Q(x, y)$.

Proof:

1. Assume $H_1 := \forall x \exists y P(x, y)$ and $H_2 := \forall x \forall y (P(x, y) \rightarrow Q(x, y))$. (hypotheses)
2. Let a be an arbitrary element of the domain. (introduce arbitrary variable for $\forall x$ in the goal)
3. From H_1 , instantiate at a : $\exists y P(a, y)$. (by UI applied to H_1 with $t = a$)
4. Let b be a witness: $P(a, b)$. (by EI applied to Step 3; b is fresh)
5. From H_2 , instantiate at a : $\forall y (P(a, y) \rightarrow Q(a, y))$. (by UI applied to H_2 with $t = a$)
6. From Step 5, instantiate at b : $P(a, b) \rightarrow Q(a, b)$. (by UI applied to Step 5 with $t = b$)
7. From Step 4 ($P(a, b)$) and Step 6 ($P(a, b) \rightarrow Q(a, b)$): obtain $Q(a, b)$. (by Modus Ponens)
8. From Step 7: $\exists y Q(a, y)$. (by EG: b witnesses the existential)

9. Since a was arbitrary: $\forall x \exists y Q(x, y)$. (by UG: a was not constrained)

Annotation: The proof follows the $\forall\exists$ skeleton exactly. Step 2 fixes the arbitrary x . Steps 3–4 use the first hypothesis to obtain a witness b depending on a . Steps 5–7 apply the second hypothesis to get the desired conclusion about b . Steps 8–9 wrap everything back up with EG and UG.

Remark (Reading analysis proofs). A proof that begins “Let $\varepsilon > 0$ ” is executing Step 2 of the $\forall\exists$ skeleton with domain $\mathbb{R}_{>0}$. A sentence like “take $\delta = \varepsilon/2$ ” is executing Step 2 of the inner existential proof: it is the witness for $\exists\delta$. Everything between “let $\varepsilon > 0$ ” and “take $\delta = \dots$ ” is scaffolding to find the right witness. Everything after “take $\delta = \dots$ ” is the verification that the witness works. Once this structure is seen, every ε - δ proof becomes readable by template.

Equality Rules — Quick Reference

Rule	Schema	Status
Reflexivity (EqI)	$t = t$	Primitive rule
Equality Elim (EqE)	$t_1 = t_2, \varphi[t_1/x] \Rightarrow \varphi[t_2/x]$	Primitive rule
Symmetry	$t_1 = t_2 \Rightarrow t_2 = t_1$	Derived
Transitivity	$t_1 = t_2, t_2 = t_3 \Rightarrow t_1 = t_3$	Derived
Term substitution	$f(\dots, t_1, \dots) = f(\dots, t_2, \dots)$	Derived
Predicate substitution	$P(\dots, t_1, \dots) \Leftrightarrow P(\dots, t_2, \dots)$	Derived

2.6.3 Proof Theory: Equality Rules

Definition (Equality Introduction — Reflexivity)

Definition 2.6.5 (Equality Introduction — Reflexivity). For any term t , one may infer $t = t$.

Remark (Dependencies).

- [Term](#)

Remark (English reading). Reflexivity requires no premises and holds for all terms under all interpretations. It is the foundational rule that allows equality reasoning to get off the ground.

Definition (Equality Elimination — Substitution of Equals)

Definition 2.6.6 (Equality Elimination — Substitution of Equals). From $t_1 = t_2$ and $\varphi[t_1/x]$, one may infer $\varphi[t_2/x]$.

Remark (Dependencies).

- [Term](#)

- [Formula](#)
- [Substitution Notation](#)
- [Free for Substitution](#)

Remark (English reading). Equality elimination permits replacing a term by an equal term in any formula position, provided the substitution is capture-avoiding. This formalizes the intuition that equal things can be substituted for each other.

Theorem (Symmetry of Equality)

Theorem 2.6.7 (Symmetry of Equality). *From $t_1 = t_2$, one may infer $t_2 = t_1$. Go to proof.*

Remark (Standard quantified statement). $\forall t_1, t_2 : t_1 = t_2 \rightarrow t_2 = t_1$.

Remark (Dependencies).

- [Equality Introduction](#)
- [Equality Elimination](#)

Theorem (Transitivity of Equality)

Theorem 2.6.8 (Transitivity of Equality). *From $t_1 = t_2$ and $t_2 = t_3$, one may infer $t_1 = t_3$. Go to proof.*

Remark (Standard quantified statement). $\forall t_1, t_2, t_3 : t_1 = t_2 \wedge t_2 = t_3 \rightarrow t_1 = t_3$.

Remark (Dependencies).

- [Equality Introduction](#)
- [Equality Elimination](#)

Theorem (Term Substitution under Equality)

Theorem 2.6.9 (Term Substitution under Equality). *If $t_1 = t_2$, then for any function symbol f , $f(\dots, t_1, \dots) = f(\dots, t_2, \dots)$. Go to proof.*

Remark (Standard quantified statement). $\forall t_1, t_2 : t_1 = t_2 \rightarrow \forall \text{function symbol } f : f(\dots, t_1, \dots) = f(\dots, t_2, \dots)$. ■

Remark (Dependencies).

- [Equality Elimination](#)

Theorem (Predicate Substitution under Equality)

Theorem 2.6.10 (Predicate Substitution under Equality). *If $t_1 = t_2$ and P is an n -ary predicate symbol, then $P(\dots, t_1, \dots) \Leftrightarrow P(\dots, t_2, \dots)$. Go to proof.*

Remark (Standard quantified statement). $\forall t_1, t_2 : t_1 = t_2 \rightarrow \forall \text{ predicate symbol } P: P(\dots, t_1, \dots) \leftrightarrow P(\dots, t_2, \dots)$.

Remark (Dependencies).

- [Equality Elimination](#)

Remark (Summary). These derived rules collectively express that functions and predicates *respect equality*: equal inputs produce equal outputs (for functions) or equivalent propositions (for predicates). Together they amount to the principle of Leibniz substitutivity.

Remark (Common error). When substituting equals for equals, the substitution must be made consistently. Replacing a term by an equal in one occurrence but not another in the same formula can produce incorrect conclusions.

Soundness and Completeness — Quick Reference

Property	Direction
Soundness	$\Gamma \vdash \varphi \Rightarrow \Gamma \models \varphi$
Completeness	$\Gamma \models \varphi \Rightarrow \Gamma \vdash \varphi$
Soundness Theorem (FOL)	All standard FOL rules are sound
Gödel's Completeness Theorem	FOL is complete

2.6.4 Proof Theory: Soundness and Completeness

Definition (Soundness)

Definition 2.6.11 (Soundness). A proof system is *sound* if every provable formula is valid:

$$\Gamma \vdash \varphi \Rightarrow \Gamma \models \varphi.$$

Remark (Dependencies).

- [Satisfaction](#)
- [Logical Consequence](#)

Definition (Completeness)

Definition 2.6.12 (Completeness). A proof system is *complete* if every valid formula is provable:

$$\Gamma \models \varphi \Rightarrow \Gamma \vdash \varphi.$$

Remark (Dependencies).

- [Satisfaction](#)
- [Logical Consequence](#)

Theorem (Soundness of First-Order Logic)

Theorem 2.6.13 (Soundness of First-Order Logic). *The standard inference rules for first-order logic (UI, UG, EI, EG, and the propositional rules) are sound: they preserve truth in all structures.*

If $\Gamma \vdash \varphi$, then $\Gamma \models \varphi$. Go to proof.

Remark (Standard quantified statement). $\forall \Gamma, \varphi : \Gamma \vdash \varphi \rightarrow \forall \mathcal{M}, \forall s, (\forall \gamma \in \Gamma, \mathcal{M}, s \models \gamma) \rightarrow \mathcal{M}, s \models \varphi$.

Remark (Dependencies).

- [Soundness](#)
- [Satisfaction](#)
- [Logical Consequence](#)
- [Universal Instantiation](#)
- [Quantifier Distribution](#)

Theorem (Gödel's Completeness Theorem)

First-order logic is complete: every logically valid formula is provable.

$$\Gamma \models \varphi \implies \Gamma \vdash \varphi.$$

Equivalently: if a set of sentences Γ is consistent (has no proof of contradiction), then Γ has a model.

Remark (English reading). Soundness ensures proofs do not lead us astray: we cannot prove false statements from true premises. Completeness ensures proofs are powerful enough: every statement that is true in all models can be established by a formal proof.

Together, soundness and completeness show that syntactic provability ($\vdash$) and semantic entailment ($\models$) coincide for first-order logic.

Remark (Completeness vs. incompleteness). Gödel's completeness theorem (1930) should not be confused with his incompleteness theorems (1931). The completeness theorem concerns first-order logic as a proof system. The incompleteness theorems concern the limitations of formal theories capable of expressing arithmetic, and are a separate result.

2.7 Translation

Toolkit: Translation

Task	Question	Typical output
Translation key	What does each symbol mean?	Domain, constants, predicates, relations
Universal claim	Which objects are restricted?	$\forall x (A(x) \rightarrow B(x))$
Existential claim	Which properties are jointly required?	$\exists x (A(x) \wedge B(x))$
Scope analysis	Which quantifier is outermost?	$\forall x \exists y R(x, y)$ or $\exists y \forall x R(x, y)$

Remark (Exposition). This section records the applied translation layer for predicate logic: keys, domains, restricted quantifiers, scope ambiguity, and the modern first-order reading of categorical forms.

English to Logic and Scope Ambiguity

Definition (Translation Key)

Definition 2.7.1 (Translation Key). A *translation key* is a finite record that fixes a domain of discourse and assigns intended English readings to the constant symbols, predicate symbols, and relation symbols used in a translation.

Remark (Standard quantified statement).

K is a translation key

$\iff K$ specifies a domain and readings for each non-logical symbol used.

Remark (Definition predicate reading). $\text{TranslationKey}(K)$ means that K supplies the domain and symbol readings needed to parse a translated formula.

Remark (Interpretation). A translation key prevents hidden changes of meaning. The same symbol must keep the same reading throughout one exercise unless the key is explicitly replaced.

Remark (Examples). For the English sentence “Every applicant submitted some form,” a key may take the domain to be people and forms, with $A(x)$ for “ x is an applicant,” $F(y)$ for “ y is a form,” and $S(x, y)$ for “ x submitted y .”

Remark (Non-Examples). Using $P(x)$ once for “ x is a person” and later for “ x passed” is not a translation key; it is a change of vocabulary inside one translation.

Remark (Dependencies).

- [Constant Symbol](#)
- [Predicate Symbol](#)

Definition (Domain of Discourse)

Definition 2.7.2 (Domain of Discourse). The *domain of discourse* for a translation is the collection of objects over which the variables of the translated formulas range.

Remark (Standard quantified statement).

D is the domain of discourse $\iff$ each variable in the translation ranges over objects in D . ■

Remark (Definition predicate reading). $\text{DomainOfDiscourse}(D, K)$ means that D is the domain fixed by the translation key K .

Remark (Interpretation). The domain decides what the quantifiers are allowed to talk about. If the domain is people, then an exam must either be named by a separate sort-like predicate or represented by a different translation choice.

Remark (Examples). For “All cats are mammals,” one may use the domain of all animals and write $\forall x (C(x) \rightarrow M(x))$. With the smaller domain of cats, the same English claim may be written $\forall x M(x)$.

Remark (Non-Examples). Writing $\forall x M(x)$ while silently changing the domain from animals to cats halfway through the exercise is not a valid translation.

Remark (Dependencies).

- [Translation Key](#)
- [Variable](#)

Definition (Predicate and Relation Symbols in a Translation)

Definition 2.7.3 (Predicate and Relation Symbols in a Translation). In a translation key, a unary predicate symbol names a property of one object, and an n -ary relation symbol names a relation among n objects.

Remark (Standard quantified statement).

$$\begin{aligned} P \text{ is unary in } K &\iff P(x) \text{ has one argument,} \\ R \text{ is } n\text{-ary in } K &\iff R(x_1, \dots, x_n) \text{ has } n \text{ arguments.} \end{aligned}$$

Remark (Definition predicate reading). $\text{TranslationPredicateSymbol}(P, K)$ records that P has a fixed predicate reading in K , and $\text{TranslationRelationSymbol}(R, n, K)$ records that R is used with arity n in K .

Remark (Interpretation). Object-language predicate symbols such as P, Q, R are symbols inside the formal language. Metalinguistic predicate readings such as $\text{Term}(t)$ or $\text{AtomicFormula}(\varphi)$ are repository descriptions of syntax; they are not formulas being translated. ■

Remark (Examples). $A(x)$ may mean “ x is an athlete.” $L(x, y)$ may mean “ x likes y .” The arity is part of the key: $L(x)$ and $L(x, y)$ are not the same symbol use.

Remark (Non-Examples). Writing $L(x, y, z)$ after declaring $L(x, y)$ as “ x likes y ” is an arity error.

Remark (Dependencies).

- [Predicate Symbol](#)
- [Atomic Formula](#)

Definition (Constants and Variables in a Translation)

Definition 2.7.4 (Constants and Variables in a Translation). A *constant* names a fixed object from the translation key, while a *variable* marks a place bound by a quantifier or left open for later binding.

Remark (Standard quantified statement).

c is a translation constant $\iff c$ names one fixed object,
 x is a translation variable $\iff x$ is available for quantification or free occurrence.

Remark (Definition predicate reading). $\text{TranslationConstant}(c, K)$ means that c names a fixed object in K , and $\text{TranslationVariable}(x, K)$ means that x is used as a variable in translations governed by K .

Remark (Interpretation). Constants do not range. Variables do range when bound by quantifiers. Confusing the two usually leaves a formula with the wrong dependency pattern.

Remark (Examples). If a names Alice, then $P(a)$ says Alice is a participant. The formula $\forall x (S(x) \rightarrow P(x))$ uses x as a bound variable.

Remark (Non-Examples). The expression $\forall a S(a)$ is not appropriate if a has already been declared as a constant naming Alice.

Remark (Dependencies).

- [Constant Symbol](#)
- [Variable](#)

Proposition (Universal Translation Pattern)

Proposition 2.7.5 (Universal Translation Pattern). In a broad domain, an English statement of the form “All A are B ” is translated by

$$\forall x (A(x) \rightarrow B(x)).$$

Go to proof.

Remark (Standard quantified statement).

$$\forall x (A(x) \rightarrow B(x)).$$

Remark (Predicate reading). $\text{UniversalTranslationPattern}(A, B)$ records the implication schema for translating universal restricted claims.

Remark (Interpretation). The antecedent $A(x)$ restricts attention to the relevant objects. Objects outside A do not have to satisfy B .

Remark (Exposition). “All cats are mammals” becomes $\forall x (C(x) \rightarrow M(x))$. “No cats are reptiles” becomes $\forall x (C(x) \rightarrow \neg R(x))$. The form $\forall x (C(x) \wedge M(x))$ says every object in the domain is both a cat and a mammal, which is far stronger than the intended restricted claim. In a broad domain, “all,” “every,” and “any” usually translate restricted claims with implication.

Remark (Dependencies).

- [Universal Formula](#)
- [Molecular Formula](#)

Proposition (Existential Translation Pattern)

Proposition 2.7.6 (Existential Translation Pattern). *In a broad domain, an English statement of the form “Some A are B ” is translated by*

$$\exists x (A(x) \wedge B(x)).$$

Go to proof.

Remark (Standard quantified statement).

$$\exists x (A(x) \wedge B(x)).$$

Remark (Predicate reading). $\text{ExistentialTranslationPattern}(A, B)$ records the conjunction schema for translating existential restricted claims.

Remark (Interpretation). The witness must satisfy both restrictions at once. Existence is not merely a conditional promise about what would happen if an A existed.

Remark (Exposition). “Some cats are black” becomes $\exists x (C(x) \wedge B(x))$. “Some cats are not black” becomes $\exists x (C(x) \wedge \neg B(x))$. The form $\exists x (C(x) \rightarrow B(x))$ may be true because the witness is not a cat, so it does not translate “Some cats are black.” In a broad domain, “some,” “there is,” and “at least one” usually translate restricted claims with conjunction.

Remark (Dependencies).

- [Existential Formula](#)
- [Molecular Formula](#)

Proposition (Conditional Translation Pattern)

Proposition 2.7.7 (Conditional Translation Pattern). *An English statement of the form “If A , then B ” is translated by*

$$A \rightarrow B,$$

*with quantifiers placed according to the variables occurring in A and B .
Go to proof.*

Remark (Standard quantified statement).

$$A \rightarrow B.$$

Remark (Predicate reading). $\text{ConditionalTranslationPattern}(A, B)$ records the implication schema for conditional English.

Remark (Interpretation). Conditionals are especially common inside universal translations. The conditional separates the restricting condition from the asserted consequence.

Remark (Exposition). “Every applicant who is eligible is interviewed” becomes $\forall x ((A(x) \wedge E(x)) \rightarrow I(x))$. The form $\forall x (A(x) \wedge E(x) \wedge I(x))$ says every object is an eligible applicant who is interviewed, which is not the intended conditional claim.

Remark (Dependencies).

- [Universal Translation Pattern](#)

Proposition (Negation Translation Pattern)

Proposition 2.7.8 (Negation Translation Pattern). *The negation of a translated quantified statement is obtained by negating the whole formula and then pushing the negation inward using the quantifier negation laws.*

Go to proof.

Remark (Standard quantified statement).

$$\neg \forall x \varphi \iff \exists x \neg \varphi,$$

$$\neg \exists x \varphi \iff \forall x \neg \varphi.$$

Remark (Predicate reading). $\text{NegationTranslationPattern}(\varphi, \psi)$ records that ψ is obtained from $\neg \varphi$ by pushing negation through the quantifiers and connectives.

Remark (Interpretation). The negation of “all” is “some not.” The negation of “some” is “none.” For restricted universal claims, negating the implication produces a witness that satisfies the restriction and fails the consequent.

Remark (Exposition). The negation of $\forall x (A(x) \rightarrow B(x))$ is $\exists x (A(x) \wedge \neg B(x))$. The negation of $\exists x (A(x) \wedge B(x))$ is $\forall x (A(x) \rightarrow \neg B(x))$. The negation of $\forall x (A(x) \rightarrow B(x))$ is not $\forall x (A(x) \rightarrow \neg B(x))$; that latter formula says every A fails to be B , not merely that some counterexample exists.

Remark (Dependencies).

- [Quantifier Negation Laws](#)
- [Universal Translation Pattern](#)
- [Existential Translation Pattern](#)

Definition (Scope Ambiguity)

Definition 2.7.9 (Scope Ambiguity). A sentence has *scope ambiguity* when the same ordinary-language wording admits two or more translations with different quantifier nesting or different connective scope.

Remark (Standard quantified statement).

E has scope ambiguity

$\iff$ there exist distinct candidate translations φ, ψ of E with different scope.

Remark (Definition predicate reading). $\text{ScopeAmbiguity}(E, \varphi, \psi)$ means that the English sentence E can reasonably be translated by φ or by ψ , and the difference is a scope difference.

Remark (Interpretation). Scope is not cosmetic. Changing the outer quantifier can change whether a single witness works for everyone or whether each object may have its own witness.

Remark (Exposition). The formulas $\forall x \exists y R(x, y)$ and $\exists y \forall x R(x, y)$ are not interchangeable. The first allows the witness y to depend on x ; the second requires one y that works for every x . Symbols such as $A(x)$, $B(x)$, and $R(x, y)$ are object-language formula patterns used in translations. Expressions such as $\text{TranslationKey}(K)$, $\text{Term}(t)$, and $\text{AtomicFormula}(\varphi)$ are metalinguistic predicate readings used by the project to describe mathematical objects.

All A are B	$\rightsquigarrow \forall x (A(x) \rightarrow B(x))$
Some A are B	$\rightsquigarrow \exists x (A(x) \wedge B(x))$
No A are B	$\rightsquigarrow \forall x (A(x) \rightarrow \neg B(x))$
Some A are not B	$\rightsquigarrow \exists x (A(x) \wedge \neg B(x))$
Every A has a B	$\rightsquigarrow \forall x (A(x) \rightarrow \exists y (B(y) \wedge R(x, y)))$
There is a B for every A	$\rightsquigarrow \exists y (B(y) \wedge \forall x (A(x) \rightarrow R(x, y)))$

Remark (Examples). “Every A has a B ” is usually $\forall x (A(x) \rightarrow \exists y (B(y) \wedge R(x, y)))$. “There is a B for every A ” is usually $\exists y (B(y) \wedge \forall x (A(x) \rightarrow R(x, y)))$. The bare patterns $\forall x \exists y R(x, y)$ and $\exists y \forall x R(x, y)$ make the same scope contrast visible.

Remark (Non-Examples). Renaming a bound variable from x to z does not create a scope ambiguity. The scope is unchanged.

Remark (Dependencies).

- [Quantifier Scope](#)
- [Quantifier Order](#)

Square of Opposition

Definition (Universal Affirmative Form)

Definition 2.7.10 (Universal Affirmative Form). The *universal affirmative* categorical form, called A , translates “All A are B ” as

$$\forall x (A(x) \rightarrow B(x)).$$

Remark (Standard quantified statement).

$$\forall x (A(x) \rightarrow B(x)).$$

Remark (Definition predicate reading). $\text{CategoricalAForm}(A, B, \varphi)$ means that φ is the universal affirmative translation from A to B .

Remark (Interpretation). The A -form says that every object satisfying the subject predicate also satisfies the predicate predicate. It does not by itself say that anything satisfies the subject predicate.

Remark (Dependencies).

- [Universal Translation Pattern](#)

Definition (Universal Negative Form)

Definition 2.7.11 (Universal Negative Form). The *universal negative* categorical form, called E , translates “No A are B ” as

$$\forall x (A(x) \rightarrow \neg B(x)).$$

Remark (Standard quantified statement).

$$\forall x (A(x) \rightarrow \neg B(x)).$$

Remark (Definition predicate reading). $\text{CategoricalEForm}(A, B, \varphi)$ means that φ is the universal negative translation from A to B .

Remark (Interpretation). The E -form says that anything satisfying A fails to satisfy B . It is still universal, so it can be true when no object satisfies A .

Remark (Dependencies).

- [Universal Translation Pattern](#)
- [Negation Translation Pattern](#)

Definition (Particular Affirmative Form)

Definition 2.7.12 (Particular Affirmative Form). The *particular affirmative* categorical form, called I , translates “Some A are B ” as

$$\exists x (A(x) \wedge B(x)).$$

Remark (Standard quantified statement).

$$\exists x (A(x) \wedge B(x)).$$

Remark (Definition predicate reading). $\text{CategoricalIForm}(A, B, \varphi)$ means that φ is the particular affirmative translation from A to B .

Remark (Interpretation). The I -form requires a witness satisfying both predicates. It carries existential force.

Remark (Dependencies).

- [Existential Translation Pattern](#)

Definition (Particular Negative Form)

Definition 2.7.13 (Particular Negative Form). The *particular negative* categorical form, called O , translates “Some A are not B ” as

$$\exists x (A(x) \wedge \neg B(x)).$$

Remark (Standard quantified statement).

$$\exists x (A(x) \wedge \neg B(x)).$$

Remark (Definition predicate reading). $\text{CategoricalOForm}(A, B, \varphi)$ means that φ is the particular negative translation from A to B .

Remark (Interpretation). The O -form requires a witness satisfying the subject predicate and failing the predicate predicate.

Remark (Dependencies).

- [Existential Translation Pattern](#)
- [Negation Translation Pattern](#)

Proposition (A-O Contradiction)

Proposition 2.7.14 (A-O Contradiction). The A -form $\forall x (A(x) \rightarrow B(x))$ and the O -form $\exists x (A(x) \wedge \neg B(x))$ are contradictory: exactly one of them is true. Go to proof.

Remark (Standard quantified statement).

$$\neg \forall x (A(x) \rightarrow B(x)) \iff \exists x (A(x) \wedge \neg B(x)).$$

Remark (Predicate reading). $\text{ContradictoryPair}(\varphi_A, \varphi_O)$ records that the A -form and O -form cannot share a truth value.

Remark (Interpretation). The O -form is exactly the counterexample to the A -form: an object that is A but not B .

Remark (Dependencies).

- [Universal Affirmative Form](#)
- [Particular Negative Form](#)
- [Quantifier Negation Laws](#)

Proposition (E-I Contradiction)

Proposition 2.7.15 (E-I Contradiction). *The E-form $\forall x (A(x) \rightarrow \neg B(x))$ and the I-form $\exists x (A(x) \wedge B(x))$ are contradictory: exactly one of them is true.*
Go to proof.

Remark (Standard quantified statement).

$$\neg \forall x (A(x) \rightarrow \neg B(x)) \iff \exists x (A(x) \wedge B(x)).$$

Remark (Predicate reading). ContradictoryPair(φ_E, φ_I) records that the E-form and I-form cannot share a truth value.

Remark (Interpretation). The I-form is exactly the counterexample to the E-form: an object that is both A and B .

Remark (Dependencies).

- [Universal Negative Form](#)
- [Particular Affirmative Form](#)
- [Quantifier Negation Laws](#)

Proposition (Contrariety Requires Existential Import)

Proposition 2.7.16 (Contrariety Requires Existential Import). *If $\exists x A(x)$, then the A-form and E-form cannot both be true.*
Go to proof.

Remark (Standard quantified statement).

$$\exists x A(x) \rightarrow \neg (\forall x (A(x) \rightarrow B(x)) \wedge \forall x (A(x) \rightarrow \neg B(x))).$$

Remark (Predicate reading). ContraryWithExistentialImport(φ_A, φ_E) records that contrariety holds only after the subject predicate is known to have a witness.

Remark (Interpretation). With an A -object available, the two universal claims force both B and $\neg B$ of that object. Without such an object, both universals may be vacuously true. The traditional square of opposition assumes existential import; modern first-order logic does not automatically assume that a universal statement implies existence.

Remark (Dependencies).

- [Universal Affirmative Form](#)
- [Universal Negative Form](#)
- [Existential Formula](#)

2.8 Reference

Toolkit: Predicate Logic Reference

Reference	Use
Common errors	Diagnose translation, scope, and inference mistakes.
Summary tables	Compare translation patterns, quantifier laws, categorical forms, and proof cues.

Remark (Exposition). This section collects compact reference material for translation, quantifier negation, common inference errors, and proof strategy cues.

Common Errors and Fallacies

	Error	Incorrect form	Corrected form
	Confusing $\forall$ and $\exists$	$\exists x P(x)$ for “every x has P ”	$\forall x P(x)$
	Universal conjunction	$\forall x (A(x) \wedge B(x))$	$\forall x (A(x) \rightarrow B(x))$
	Existential implication	$\exists x (A(x) \rightarrow B(x))$	$\exists x (A(x) \wedge B(x))$
	Scope error	$\exists y \forall x R(x, y)$	$\forall x \exists y R(x, y)$, or “for each x may have its own y ”
<i>Remark</i> (Translation and proof errors).	Variable capture	$\varphi[t/x]$ without checking bound variables	Rename bound variables before substitution
	Free variable left in a sentence	$\forall x R(x, y)$ as a closed claim	$\forall x \forall y R(x, y)$, or “as intended”
	Affirming the consequent	$P \rightarrow Q, Q \therefore P$	Invalid; use a counterexample or structure
	Denying the antecedent	$P \rightarrow Q, \neg P \therefore \neg Q$	Invalid; use a counterexample or structure
	Invalid quantifier reversal	$\forall x \exists y R(x, y) \Rightarrow \exists y \forall x R(x, y)$	Invalid without hypotheses

Remark (Failure-mode checklist). Before accepting a translation or inference, check the domain, each symbol reading, every bound-variable scope, and whether the final formula is intended to be a sentence. Then test suspicious implications by trying to build one case where the premise is true and the conclusion false.

Summary Tables

	English phrase	Logical pattern
<i>Remark</i> (English phrase to logical pattern).	All A are B	$\forall x (A(x) \rightarrow B(x))$
	Some A are B	$\exists x (A(x) \wedge B(x))$
	No A are B	$\forall x (A(x) \rightarrow \neg B(x))$
	Some A are not B	$\exists x (A(x) \wedge \neg B(x))$
	Every A has a B	$\forall x (A(x) \rightarrow \exists y (B(y) \wedge R(x, y)))$
	There is a B for every A	$\exists y (B(y) \wedge \forall x (A(x) \rightarrow R(x, y)))$
<i>Remark</i> (Quantifier negation laws).		

$$\begin{aligned}
 \neg \forall x \varphi &\iff \exists x \neg \varphi, \\
 \neg \exists x \varphi &\iff \forall x \neg \varphi, \\
 \neg \forall x \exists y \varphi &\iff \exists x \forall y \neg \varphi, \\
 \neg \exists x \forall y \varphi &\iff \forall x \exists y \neg \varphi.
 \end{aligned}$$

	Form	Name	Translation
<i>Remark</i> (Categorical form translations).	A	Universal affirmative	$\forall x (A(x) \rightarrow B(x))$
	E	Universal negative	$\forall x (A(x) \rightarrow \neg B(x))$
	I	Particular affirmative	$\exists x (A(x) \wedge B(x))$
	O	Particular negative	$\exists x (A(x) \wedge \neg B(x))$

	Goal or premise	Useful cue
<i>Remark</i> (Proof strategy cues).	Prove $\forall x \varphi(x)$	Let x be arbitrary, then prove $\varphi(x)$.
	Use $\forall x \varphi(x)$	Instantiate it with a term that is free for x .
	Prove $\exists x \varphi(x)$	Produce a witness and verify φ .
	Use $\exists x \varphi(x)$	Introduce a fresh witness and keep its scope controlled.
	Disprove $\forall x \varphi(x)$	Find a counterexample satisfying $\neg \varphi$.
	Disprove an implication	Make the hypothesis true and the conclusion false.

Chapter 3

Many-Sorted Logic



3.1 Introduction

3.1.1 Examples

3.1.2 Reduction to and comparison with first-order logic

3.1.3 Uses of many-sorted logic

3.1.4 Many-sorted logic as a unifier logic

3.2 Structures

3.2.1 Definition (signature)

3.2.2 Definition (structure)

3.3 Formal many-sorted language

3.3.1 Alphabet

3.3.2 Expressions: formulas and terms

3.3.3 Remarks on notation

3.3.4 Abbreviations

3.3.5 Induction

3.3.6 Free and bound variables

3.4 Semantics

3.4.1 Definitions

3.4.2 Satisfiability, validity, consequence and logical equivalence

3.5 Substitution of a term for a variable

3.6 Semantic theorems

3.6.1 Coincidence lemma

3.6.2 Substitution lemma

3.6.3 Equals substitution lemma

3.6.4 Isomorphism theorem

3.7 The completeness of many-sorted logic

From Cantor to Gödel / Logic, Sets, and Proof / Mathematical Logic and Proof / Many-Sorted

Logic > Reduction to one-sorted logic > Conversion of structures

3.7.1 Deductive calculus

Chapter 4

Standard Second-Order Logic



4.1 Introduction

4.1.1 General idea

4.1.2 Expressive power

4.1.3 Model-theoretic counterparts of expressiveness

4.1.4 Incompleteness

4.2 Second-order grammar

4.2.1 Definition (signature and alphabet)

4.2.2 Expressions: terms, predicates and formulas

4.2.3 Remarks on notation

4.2.4 Induction

4.2.5 Free and bound variables

4.2.6 Substitution

4.3 Standard structures

4.3.1 Definition of standard structures

4.3.2 Relations between standard structures established without the formal language

4.4 Standard semantics

4.4.1 Assignment

4.4.2 Interpretation

4.4.3 Consequence and validity

4.4.4 Logical equivalence

4.4.5 Simplifying our language

4.4.6 Alternative presentation of standard semantics

4.4.7 Definable sets and relations in a given structure

4.4.8 More about the expressive power of standard SOL

4.4.9 Negative results

4.5 Semantic theorems

Chapter 5

Languages, Axioms, and Models



Languages, Axioms, and Models Toolkit - Planned

This section is a rebuild placeholder.

This section is planned for the governance rebuild. No mathematical content has been restored here yet.

5.1 Signatures

Signatures Toolkit — Quick Reference	
Concept	Role
Signature	non-logical vocabulary
Language	signature together with logical syntax
Constant symbol	names a distinguished object syntactically
Function symbol	names an operation syntactically
Relation symbol	names a relation syntactically
Arity	records number of input places

Signatures

Signature Presentation: A Simple Arithmetic Language	
Let	$\Sigma_{\text{arith}} = (\mathcal{C}_{\text{arith}}, \mathcal{F}_{\text{arith}}, \mathcal{R}_{\text{arith}}, \text{ar})$

where

$$\mathcal{C}_{\text{arith}} = \{0, 1\}, \quad \mathcal{F}_{\text{arith}} = \{+, \cdot, S\}, \quad \mathcal{R}_{\text{arith}} = \{<\}.$$

The arity function is specified by

$$\text{ar}(0) = \text{ar}(1) = 0, \quad \text{ar}(S) = 1, \quad \text{ar}(+) = \text{ar}(\cdot) = 2, \quad \text{ar}(<) = 2.$$

Symbol	Kind	Arity	Intended Reading
0, 1	constants	0	distinguished constant names
S	function bol	sym- 1	successor operation symbol
$+, \cdot$	function bols	sym- 2	binary operation symbols
$<$	relation bol	sym- 2	binary order relation symbol

The signature records only the available non-logical symbols and their arities. It does not assert any axioms about those symbols.

Definitional Root (First-Order Signature)

Definition 5.1.1 (First-Order Signature). A *first-order signature* is a tuple

$$\Sigma = (\mathcal{C}, \mathcal{F}, \mathcal{R}, \text{ar}),$$

where $\mathcal{C}$ is a set of constant symbols, $\mathcal{F}$ is a set of function symbols, $\mathcal{R}$ is a set of relation symbols, and

$$\text{ar} : \mathcal{F} \cup \mathcal{R} \rightarrow \mathbb{Z}_{>0}$$

assigns to each function or relation symbol its number of arguments.

Remark (Standard quantified statement). For every $s \in \mathcal{F} \cup \mathcal{R}$, the value $\text{ar}(s)$ is a positive integer. If $s \in \mathcal{F}$, then s forms terms with $\text{ar}(s)$ inputs; if $s \in \mathcal{R}$, then s forms atomic formulas with $\text{ar}(s)$ inputs.

Remark (Interpretation). A signature is the formal vocabulary available before any axioms are chosen. It says which symbols may occur and how many input places each symbol has.

Definition (Arity)

Definition 5.1.2 (Arity). Let $\Sigma = (\mathcal{C}, \mathcal{F}, \mathcal{R}, \text{ar})$ be a first-order signature. The *arity function* is the map

$$\text{ar} : \mathcal{F} \cup \mathcal{R} \rightarrow \mathbb{Z}_{>0}$$

that assigns to each function or relation symbol the number of arguments it requires. When constants are treated as 0-ary function symbols, the codomain is enlarged to include 0.

Remark (Standard quantified statement). For every $s \in \mathcal{F} \cup \mathcal{R}$, there is a unique positive integer n such that $\text{ar}(s) = n$. If $f \in \mathcal{F}$ and $\text{ar}(f) = n$, then an expression $f(t_1, \dots, t_k)$ is a well-formed term exactly when $k = n$. If $R \in \mathcal{R}$ and $\text{ar}(R) = n$, then an expression $R(t_1, \dots, t_k)$ is a well-formed atomic formula exactly when $k = n$.

Remark (Interpretation). Arity is bookkeeping for grammatical well-formedness. It determines whether a symbol has been supplied with the correct number of arguments.

Remark (Dependencies).

- [Signature](#)

Definitional Framework (Language)

Definition 5.1.3 (Language). A *first-order language* has two standard readings. As vocabulary, it is the same data as a signature,

$$\mathcal{L} = (\mathcal{C}, \mathcal{F}, \mathcal{R}, \text{ar}).$$

As a formal language generated by a signature Σ , it is the set of well-formed strings built from the alphabet

$$A = \Sigma \cup V \cup \Lambda,$$

where V is a countable set of variables and Λ is the fixed stock of logical symbols and punctuation. In this second reading,

$$\mathcal{L}(\Sigma) = \{s \in A^* \mid s \in \text{Terms}(\Sigma) \text{ or } s \in \text{WFF}(\Sigma)\}.$$

Remark (Standard quantified statement). The vocabulary reading records which non-logical symbols may occur. The generated-language reading then selects from all finite strings over A only the terms and well-formed formulas admitted by the formation rules determined by Σ .

Remark (Interpretation). The signature supplies the subject-specific vocabulary. The language adds the logical grammar needed to form expressions from that vocabulary.

Remark (Dependencies).

- [Signature](#)

Definition (Constant Symbol)

Definition 5.1.4 (Constant Symbol). A *constant symbol* is a non-logical symbol that behaves syntactically as a 0-ary function symbol. In the streamlined convention where constants are included among function symbols, this is the condition

$$\text{Constant}(c) \iff c \in \mathcal{F} \text{ and } \text{ar}(c) = 0.$$

Because its arity is 0, the symbol c is already a well-formed term and takes no input terms.

Remark (Standard quantified statement). If $c \in \mathcal{F}$ and $\text{ar}(c) = 0$, then $c \in \text{Terms}(\mathcal{L})$. In a structure $\mathfrak{M} = (M, \mathcal{I})$, the interpretation of a constant is a unique element of the domain:

$$\forall c \in \mathcal{C}, \exists! e \in M, \mathcal{I}(c) = e.$$

Remark (Interpretation). A constant symbol is syntactically rigid: unlike a variable, it does not vary with an assignment. In the arithmetic signature, the symbol 0 is a piece of formal syntax; under the standard interpretation, it denotes the number zero.

Remark (Dependencies).

- [Signature](#)
- [Arity](#)

Definition (Function Symbol)

Definition 5.1.5 (Function Symbol). A *function symbol* is a non-logical symbol $f \in \mathcal{F}$ whose assigned arity determines how many terms it must take as inputs. If $\text{ar}(f) = n$, then f is an n -ary term constructor: it takes exactly n terms and produces a new term, not a truth value.

Remark (Standard quantified statement). For every $f \in \mathcal{F}$, if $\text{ar}(f) = n$, then

$$f(t_1, \dots, t_k) \in \text{Terms}(\mathcal{L}) \iff k = n$$

whenever $t_1, \dots, t_k$ are terms. In a structure $\mathfrak{M} = (M, \mathcal{I})$, the interpretation of f is a total operation closed on the domain:

$$\mathcal{I}(f) : M^n \rightarrow M.$$

Remark (Interpretation). A function symbol is operation-forming syntax. In the arithmetic signature, S is unary, while $+$ and $\cdot$ are binary. Under an interpretation, each becomes an actual operation on the chosen domain.

Remark (Dependencies).

- [Signature](#)
- [Arity](#)

Definition (Relation Symbol)

Definition 5.1.6 (Relation Symbol). A *relation symbol*, or predicate symbol, is a non-logical symbol $R \in \mathcal{R}$ whose assigned arity determines how many terms it must take as inputs. If $\text{ar}(R) = n$, then R is an n -ary formula constructor: it takes exactly n terms and produces an atomic formula, hence something with a truth value rather than an object.

Remark (Standard quantified statement). For every $R \in \mathcal{R}$, if $\text{ar}(R) = n$, then

$$R(t_1, \dots, t_k) \in \text{WFF}(\mathcal{L}) \iff k = n$$

whenever $t_1, \dots, t_k$ are terms. In a structure $\mathfrak{M} = (M, \mathcal{I})$, the interpretation of R is a subset of n -tuples from the domain:

$$\mathcal{I}(R) \subseteq M^n.$$

Thus $R(t_1, \dots, t_n)$ is true exactly when the tuple of interpreted terms lies in $\mathcal{I}(R)$.

Remark (Interpretation). A relation symbol is a statement-forming symbol once it is applied to the right number of terms. In the arithmetic signature, $+$ is a binary function symbol because it returns a number, while $<$ is a binary relation symbol because it returns a truth value. Under the standard interpretation, $<$ denotes the set of ordered pairs (a, b) such that a is less than b .

Remark (Dependencies).

- [Signature](#)
- [Arity](#)

Remark (Structural remarks). The same signature may support different choices of axioms later in the chapter. For example, a single arithmetic vocabulary can be used with stronger or weaker axiom sets.

The same signature may also be read over different mathematical universes once interpretations are introduced. At this stage, the signature itself records only the common vocabulary; it does not choose any particular interpretation.

5.2 Axioms

Axioms Toolkit — Quick Reference

Concept	Role
Axiom	formal assertion adopted without proof
Axiom system	collection of assertions in one language
Assumption	temporary assertion for reasoning
Consistency	absence of contradiction
Theory	deductive closure of axioms

Axioms

Axiom Presentation: A Simple Arithmetic Axiom System

Reuse the arithmetic signature

$$\Sigma_{\text{arith}} = (\{0, 1\}, \{+, \cdot, S\}, \{<\}, \text{ar})$$

from the Signatures section. The signature supplies the symbols. An axiom system supplies assertions formed with those symbols.

For example, the following are familiar arithmetic-style axioms:

$$\forall x (x + 0 = x),$$

$$\forall x (S(x) \neq 0),$$

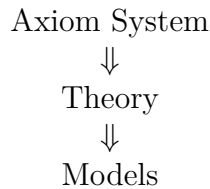
$$\forall x \forall y (S(x) = S(y) \rightarrow x = y).$$

These examples are illustrative. The signature says that 0, +, and S are available symbols. The axioms assert formal relationships involving those symbols.

Remark (Structural remarks). **Vocabulary vs assertion.** A signature is vocabulary: it specifies what may be said. An axiom is an assertion: it specifies what is adopted as true within that vocabulary. The signature supplies symbols; the axioms supply assertions involving those symbols.

Same signature, different axiom systems. A single signature may support multiple collections of axioms. For example, geometric languages can be used for Euclidean or non-Euclidean axiom systems. The usual group signature can be used for group theory, while adding the commutativity axiom gives the stronger axiom system for abelian groups. The vocabulary can remain fixed while the assertions change.

Forward conceptual roadmap. The chapter proceeds from collections of adopted assertions to their deductive and semantic roles:



The present section introduces the assertion layer. Later sections formalize the theory generated by axioms and the models in which those assertions hold.

Definition (Axiom)

Definition 5.2.1 (Axiom). Let $\mathcal{L}$ be a first-order language, and let $\text{Sent}(\mathcal{L})$ denote the set of sentences of $\mathcal{L}$. If $T \subseteq \text{Sent}(\mathcal{L})$ is a theory, then an *axiom* of T is an element $\alpha \in T$.

Remark (Standard quantified statement). For every first-order language $\mathcal{L}$ and every theory $T \subseteq \text{Sent}(\mathcal{L})$, a sentence α is an axiom of T exactly when $\alpha \in T$.

Remark (Interpretation). An axiom is not part of the vocabulary itself. It is an assertion made using the vocabulary of a language. The signature determines which symbols may appear; the axiom states something with those symbols.

Remark (Proof-theoretic reading). In a deductive presentation, an axiom may be used as a derivation with no undischarged premises:

$$\overline{\vdash \alpha}.$$

Thus axioms are the starting assertions of the proof system.

Remark (Dependencies).

- [Language](#)

Definition (Axiom System)

Definition 5.2.2 (Axiom System). An *axiom system* is a collection of axioms expressed in a common language.

Remark (Interpretation). An axiom system records the assertions chosen for a mathematical setting. Two axiom systems may use the same language while adopting different assertions. The shared language fixes the vocabulary; the axiom system fixes the adopted formal claims.

Remark (Dependencies).

- [Axiom](#)
- [Language](#)

Definition (Assumption)

Definition 5.2.3 (Assumption). An *assumption* is a statement temporarily accepted for purposes of reasoning.

Remark (Interpretation). An axiom is adopted as part of the formal background of a system. An assumption is local to an argument: it may be introduced to prove a conditional, to reason within a case, or to derive a contradiction. Axioms belong to the system; assumptions belong to the proof context.

Remark (Dependencies).

- [Axiom](#)

5.3 Theories

Theories Toolkit — Quick Reference

Concept	Role
Theory	all consequences generated by axioms
Theorem	statement derivable from axioms
Consequence	statement following from a theory
Derivation	chain of justified steps
Deductive closure	all derivable statements

Theories

Theory Presentation: Arithmetic Consequences

Start with the arithmetic-style axiom system from the previous section. It uses the arithmetic signature and includes assertions such as

$$\begin{aligned}\forall x (x + 0 &= x), \\ \forall x (S(x) &\neq 0), \\ \forall x \forall y (S(x) = S(y) &\rightarrow x = y).\end{aligned}$$

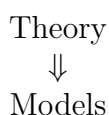
Those axioms are the starting assertions. From them, together with other arithmetic axioms and ordinary logical reasoning, additional statements may be established. For instance, one may later prove new equations, cancellation facts, uniqueness facts, and order facts that were not listed as axioms.

A theory contains the original axioms. A theory also contains the statements derivable from those axioms. Thus the axioms are the generators, while the theory is the full collection of what those generators imply.

Remark (Structural remarks). **Axioms vs theory.** Axioms are starting assertions. A theory includes all logical consequences of those assertions. The distinction is the next step after the vocabulary and assertion distinction: a signature supplies vocabulary, axioms supply adopted assertions, and a theory contains everything derivable from those assertions.

Finite axioms, infinite theory. A small axiom system can generate an enormous collection of consequences. The listed axioms may be finite, but the statements derivable from them need not be. This is why a theory is usually larger than the axiom system that presents it.

Forward roadmap. The next conceptual step is semantic:



A model will later be introduced as a mathematical structure in which the statements of a theory hold.

Definition (Theory)

Definition 5.3.1 (Theory). A *theory* over a first-order language $\mathcal{L}$ is a collection of closed sentences in that language:

$$T \subseteq \text{Sentences}(\mathcal{L}).$$

If Γ is a chosen set of axioms, the theory generated by Γ is the deductive closure

$$T = \{\varphi \in \text{Sentences}(\mathcal{L}) \mid \Gamma \vdash \varphi\}.$$

Remark (Standard quantified statement). Syntactically, a sentence φ belongs to the theory generated by Γ exactly when φ is derivable from Γ . Semantically, for an $\mathcal{L}$ -structure $\mathfrak{M}$, the complete theory of $\mathfrak{M}$ is

$$\text{Th}(\mathfrak{M}) = \{\varphi \in \text{Sentences}(\mathcal{L}) \mid \mathfrak{M} \models \varphi\}.$$

For every sentence φ , either $\mathfrak{M} \models \varphi$ or $\mathfrak{M} \models \neg\varphi$, so $\text{Th}(\mathfrak{M})$ decides every sentence of the language.

Remark (Interpretation). A theory is not merely the list of axioms. It is the closure of those axioms under derivability: every statement that follows from the axioms belongs to the theory. The axioms start the system; the theory records everything the system proves.

Remark (Dependencies).

- [Language](#)
- [Axiom System](#)

Definition (Theorem)

Definition 5.3.2 (Theorem). A *theorem* of a theory T is a sentence φ for which there is a finite formal derivation from T , written

$$T \vdash \varphi.$$

Equivalently, in proof-theoretic terms, φ is a theorem of T exactly when some derivation has leaves drawn from logical axioms or sentences of T , uses valid inference rules, and has φ as its root.

Remark (Standard quantified statement). Syntactically,

$$\text{Theorem}(T, \varphi) \iff \exists d (d : T \vdash \varphi).$$

Semantically, φ is entailed by T when it is true in every model of T :

$$T \models \varphi \iff \forall \mathfrak{M} (\mathfrak{M} \models T \implies \mathfrak{M} \models \varphi).$$

For first-order logic, the completeness theorem connects these readings: $T \vdash \varphi$ exactly when $T \models \varphi$.

Remark (Interpretation). Within an axiom system, a theorem is not an additional starting assumption. It is a statement reached from the adopted axioms by reasoning. Axioms are given; theorems are obtained.

Remark (Dependencies).

- [Theory](#)

Definition (Consequence)

Definition 5.3.3 (Consequence). A sentence φ is a *semantic consequence* of a set of premises Γ , written $\Gamma \models \varphi$, if every model of Γ is also a model of φ . Equivalently, there is no structure in which all sentences of Γ are true while φ is false.

Remark (Standard quantified statement). Semantic consequence is the universal condition

$$\Gamma \models \varphi \iff \forall \mathfrak{M} \left((\forall \gamma \in \Gamma, \mathfrak{M} \models \gamma) \implies \mathfrak{M} \models \varphi \right).$$

Syntactic consequence, written $\Gamma \vdash \varphi$, means that there exists a finite valid deduction of φ from Γ . For first-order logic, the completeness theorem says that $\Gamma \models \varphi$ exactly when $\Gamma \vdash \varphi$.

Remark (Interpretation). Consequence is the relation that connects a theory to what it implies. If the axioms generate a theory, then consequences are the statements carried along by those axioms and by the statements already obtained from them.

Remark (Dependencies).

- [Theory](#)

Definition (Deductive Closure)

Definition 5.3.4 (Deductive Closure). Let $\Gamma \subseteq \text{Sentences}(\mathcal{L})$ be a set of premises. The *deductive closure* of Γ , denoted $\text{Cn}(\Gamma)$, is the set of all sentences derivable from Γ :

$$\text{Cn}(\Gamma) = \{\varphi \in \text{Sentences}(\mathcal{L}) \mid \Gamma \vdash \varphi\}.$$

Remark (Standard quantified statement). A set of sentences T is deductively closed exactly when

$$T = \text{Cn}(T),$$

or equivalently, for every sentence φ ,

$$T \vdash \varphi \implies \varphi \in T.$$

The closure operator satisfies the standard Tarskian closure laws:

$$\Gamma \subseteq \text{Cn}(\Gamma), \quad \Gamma \subseteq \Delta \implies \text{Cn}(\Gamma) \subseteq \text{Cn}(\Delta), \quad \text{Cn}(\text{Cn}(\Gamma)) = \text{Cn}(\Gamma).$$

By completeness for first-order logic, this syntactic closure agrees with the semantic closure

$$\{\varphi \in \text{Sentences}(\mathcal{L}) \mid \Gamma \models \varphi\}.$$

Remark (Interpretation). Deductive closure is the operation that turns an axiom system into the full theory it generates. Starting with the axioms, close under derivability; the result is the theory determined by those starting assertions.

Remark (Dependencies).

- [Axiom System](#)

5.4 Models

Models Toolkit — Quick Reference

Concept	Role
Interpretation	assigns meanings to symbols
Structure	domain with interpreted symbols
Model	structure satisfying the theory
Satisfaction	truth of a statement in a structure
Countermodel	model where a statement fails

Free Σ -Algebra and Evaluation

Depends on: Signature; inductively generated set; unique readability

Parameter

Shared bridge machinery. Parameter: a signature Σ . This card is instantiated by connective evaluation, term evaluation, parsing, and later by initial-algebra presentations such as $\mathbb{N}$.

Structure

Σ is a set of operation symbols, $\text{ar} : \Sigma \rightarrow \mathbb{N}$.

A Σ -algebra is a carrier $|\mathcal{A}|$ with operations

$$f^{\mathcal{A}} : |\mathcal{A}|^{\text{ar}(f)} \rightarrow |\mathcal{A}| \quad (f \in \Sigma).$$

Seeds land in the carrier; homomorphisms are maps of algebras.

Structure

$$T_{\Sigma}(X), \quad f^T(t_1, \dots, t_n) := f(t_1, \dots, t_n), \quad \eta : X \rightarrow T_{\Sigma}(X).$$

Syntax–Semantics Bridge

$$\text{Hom}(h, \mathcal{A}, \mathcal{B}) \quad :\Longleftrightarrow \quad h(f^{\mathcal{A}}(\bar{a})) = f^{\mathcal{B}}(h(a_1), \dots, h(a_n))$$

for every $f \in \Sigma$ and $\bar{a} \in |\mathcal{A}|^n$.

Axioms and Laws

No junk.

$$t \in T_\Sigma(X) \implies t \in X \vee \exists f \in \Sigma \exists \bar{t} (t = f(\bar{t})).$$

No confusion.

$$x \neq f(\bar{t}), \quad f(\bar{s}) = g(\bar{t}) \implies f = g \wedge \bar{s} = \bar{t}.$$

Theorems and Consequences

1. **Universal property.**

$$\forall \mathcal{A} \forall \sigma : X \rightarrow |\mathcal{A}| \exists ! \hat{\sigma} : T_\Sigma(X) \rightarrow |\mathcal{A}|$$

$$\text{Hom}(\hat{\sigma}, T_\Sigma(X), \mathcal{A}) \wedge \hat{\sigma} \circ \eta = \sigma.$$

2.

$$\text{eval}(\mathcal{A})(\sigma) := \hat{\sigma}.$$

Instances

use	Σ	X	target $\mathcal{A}$	seed σ
formula eval	Ω	atoms	2	atom truth values
parse	Ω	X	$T_\Omega(X)$	η
term eval	Σ_{term}	Var	$\mathcal{A}_{\text{term}}$	assignment s
$\mathbb{N}$ preview	$\{0^{(0)}, S^{(1)}\}$	$\emptyset$	(C, c_0, succ)	none

Instances

$$\Omega = \{\neg^{(1)}, \wedge^{(2)}, \vee^{(2)}, \rightarrow^{(2)}, \leftrightarrow^{(2)}\},$$

$$\mathbf{2} = (\{0, 1\}, \neg^{\mathbf{2}}, \wedge^{\mathbf{2}}, \vee^{\mathbf{2}}, \rightarrow^{\mathbf{2}}, \leftrightarrow^{\mathbf{2}}).$$

Boundary

The machine fixes one seed. Connective and term evaluation are exact instances. FOL quantifiers range over the assignment-indexed family $(\sigma_s)_s$:

$$\mathcal{A}, s \models \exists x \varphi \iff \exists a \in A, \mathcal{A}, s[x \mapsto a] \models \varphi.$$

That is the downstream cylindric/polyadic layer. The absolutely free $T_\Sigma(X)$ is also distinct from the Lindenbaum–Tarski quotient by $\vdash$ -equivalence.

Consequence

Depends on: Satisfaction relation $\models \subseteq \text{Mod} \times \text{Form}$

Parameter

A class of models **Mod** and a satisfaction relation

$$m \models \varphi \quad (m \in \mathbf{Mod}).$$

Syntax–Semantics Bridge

$$\Gamma \models \varphi \iff \forall m \in \mathbf{Mod} \left((\forall \gamma \in \Gamma, m \models \gamma) \rightarrow m \models \varphi \right).$$

$$\text{Cn}_{\models}(\Gamma) := \{\varphi : \Gamma \models \varphi\}, \quad \models \varphi \iff \emptyset \models \varphi.$$

$$\text{Th}(m) := \{\varphi : m \models \varphi\}.$$

Binding

logic	Mod	$m \models \varphi$
prop	valuations v	$\widehat{v}(\varphi) = 1$
FOL open	pairs $(\mathcal{A}, s)$	$\mathcal{A}, s \models \varphi$
FOL sentences	structures $\mathcal{A}$	$\mathcal{A} \models \sigma$

Models

Model Presentation: Arithmetic as a Model

Reuse the arithmetic signature and arithmetic-style axioms from the previous sections. A model begins by interpreting the symbols of that signature in an actual mathematical structure.

$$\begin{aligned} \text{Domain} &= \text{the natural numbers,} \\ 0 &\mapsto \text{the number zero,} \\ S &\mapsto \text{the successor operation,} \\ + &\mapsto \text{addition.} \end{aligned}$$

In this structure, the statement

$$\forall x (x + 0 = x)$$

holds because adding zero to any natural number gives back that same natural number. The formal symbol 0 is only a symbol until it is interpreted as the number zero. The formal symbol + is only a symbol until it is interpreted as addition. The symbols acquire meaning only after interpretation.

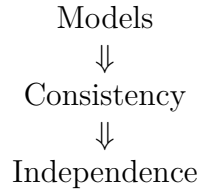
Remark (Structural remarks). **Syntax vs meaning.** Signatures, axioms, and theories are formal objects. They belong to syntax: they specify vocabulary, adopted assertions, and consequences. Models provide semantic meaning by assigning mathematical content to the symbols and by determining which statements hold.

One theory, many models. The same theory may have multiple models. Euclidean

geometry may be realized in different coordinate systems. The group axioms have many models, including different concrete groups. The theory fixes the assertions; a model is one mathematical realization in which those assertions hold.

Countermodels. A statement fails to follow from a theory if there is a model of the theory in which that statement is false. Countermodels are therefore evidence that a claim is not forced by the axioms.

Forward roadmap. The next sections use models to explain consistency and independence:



Definition (Interpretation)

Definition 5.4.1 (Interpretation). Let $\Sigma = (\mathcal{C}, \mathcal{F}, \mathcal{R}, \text{ar})$ be a first-order signature, and let M be a nonempty set. An *interpretation of Σ on M* is an assignment $\mathcal{I}$ that maps each symbol of Σ to concrete data over M : constants are assigned elements of M , function symbols are assigned operations on M , and relation symbols are assigned relations on M .

Remark (Standard quantified statement). For every $c \in \mathcal{C}$, $\mathcal{I}(c) \in M$. For every $f \in \mathcal{F}$, if $\text{ar}(f) = n$, then

$$\mathcal{I}(f) : M^n \rightarrow M.$$

For every $R \in \mathcal{R}$, if $\text{ar}(R) = n$, then

$$\mathcal{I}(R) \subseteq M^n.$$

Remark (Interpretation). An interpretation turns formal vocabulary into mathematical content. It tells what each constant, function symbol, and relation symbol means in a particular setting.

Remark (Dependencies).

- [Signature](#)

Definition ($\mathcal{L}$ -Structure)

Definition 5.4.2 ($\mathcal{L}$ -Structure). Let $\mathcal{L}$ be a first-order signature. An $\mathcal{L}$ -*structure* is a pair

$$\mathfrak{M} = (M, \mathcal{I}),$$

where M is a nonempty domain and $\mathcal{I}$ is an interpretation of the symbols of $\mathcal{L}$ on M .

Remark (Standard quantified statement). For every first-order signature $\mathcal{L}$, an $\mathcal{L}$ -structure consists of a nonempty domain M and an interpretation $\mathcal{I}$ such that every constant symbol is assigned an element of M , every n -ary function symbol is assigned a function $M^n \rightarrow M$, and every n -ary relation symbol is assigned a relation on M^n .

Remark (Interpretation). A structure is the environment in which formal symbols receive meaning. The domain supplies the objects, and the interpretation explains how the symbols refer to objects, operations, and relations on that domain.

Remark (Type-theoretic reading). In dependent type theory, this bundled data can be viewed schematically as

$$\sum_{M:\text{Type}} \left(\prod_{c \in \mathcal{C}} M \right) \times \left(\prod_{f \in \mathcal{F}} (M^{\text{ar}(f)} \rightarrow M) \right) \times \left(\prod_{R \in \mathcal{R}} (M^{\text{ar}(R)} \rightarrow \text{Prop}) \right).$$

This is the same semantic package represented by a Lean structure carrying a carrier type, interpretations of constants and functions, and predicates for relations.

Remark (Dependencies).

- [Signature](#)
- [Interpretation](#)

Definition (Model)

Definition 5.4.3 (Model). Let $\mathcal{L}$ be a first-order signature, let T be a theory over $\mathcal{L}$, and let $\mathfrak{M} = (M, \mathcal{I})$ be an $\mathcal{L}$ -structure. The structure $\mathfrak{M}$ is a *model of* T , written $\mathfrak{M} \models T$, if every sentence in T is true in $\mathfrak{M}$.

Remark (Standard quantified statement). For every theory T over $\mathcal{L}$ and every $\mathcal{L}$ -structure $\mathfrak{M}$,

$$\mathfrak{M} \models T \iff \forall \varphi \in T, \mathfrak{M} \models \varphi.$$

Remark (Interpretation). A model is a structure that makes the theory true. It is not just an interpretation of the vocabulary; it is an interpretation in which the adopted assertions and their consequences hold.

Remark (Dependencies).

- [Structure](#)
- [Theory](#)

Definition (Satisfaction)

Definition 5.4.4 (Satisfaction). Let $\mathfrak{M} = (M, \mathcal{I})$ be an $\mathcal{L}$ -structure. A *variable assignment* is a function $s : \text{Var} \rightarrow M$. The assignment extends recursively to a term-evaluation map $\bar{s}$ by

$$\bar{s}(x) = s(x), \quad \bar{s}(c) = \mathcal{I}(c),$$

and

$$\bar{s}(f(t_1, \dots, t_n)) = \mathcal{I}(f)(\bar{s}(t_1), \dots, \bar{s}(t_n)).$$

The satisfaction relation $\mathfrak{M} \models \varphi[s]$ is then defined by induction on formulas: atomic relations are tested inside the interpreted relation, equality is interpreted as equality in M , Boolean connectives are interpreted by the corresponding truth operations, and quantifiers range over all elements of M .

Remark (Standard quantified statement). For terms $t_1, \dots, t_n$ and an n -ary relation symbol R ,

$$\mathfrak{M} \models R(t_1, \dots, t_n)[s] \iff (\bar{s}(t_1), \dots, \bar{s}(t_n)) \in \mathcal{I}(R).$$

For equality,

$$\mathfrak{M} \models (t_1 = t_2)[s] \iff \bar{s}(t_1) = \bar{s}(t_2).$$

For quantifiers,

$$\mathfrak{M} \models (\forall x \varphi)[s] \iff \forall a \in M, \mathfrak{M} \models \varphi[s[x \mapsto a]],$$

and

$$\mathfrak{M} \models (\exists x \varphi)[s] \iff \exists a \in M, \mathfrak{M} \models \varphi[s[x \mapsto a]].$$

Remark (Interpretation). Satisfaction is the semantic test for a statement. Once the symbols have been interpreted in a structure, the statement can be checked as true or false in that setting.

Remark (Sentences). If φ is a sentence, then it has no free variables, so its truth does not depend on the initial assignment s . In that case the notation is shortened from $\mathfrak{M} \models \varphi[s]$ to $\mathfrak{M} \models \varphi$. This is the bridge from satisfaction of a single sentence to the definition of a model of a theory.

Remark (Dependencies).

- [Structure](#)

Definition (Countermodel)

Definition 5.4.5 (Countermodel). Let $\Gamma \subseteq \text{Sent}(\mathcal{L})$ be a set of premises, let $\varphi \in \text{Sent}(\mathcal{L})$, and let $\mathfrak{M}$ be an $\mathcal{L}$ -structure. A *countermodel to the entailment* $\Gamma \models \varphi$ is a structure $\mathfrak{M}$ that satisfies every sentence in Γ but does not satisfy φ .

Remark (Standard quantified statement). For $\Gamma \subseteq \text{Sent}(\mathcal{L})$ and $\varphi \in \text{Sent}(\mathcal{L})$,

$$\text{CounterModel}(\mathfrak{M}, \Gamma, \varphi) \iff (\forall \gamma \in \Gamma, \mathfrak{M} \models \gamma) \wedge (\mathfrak{M} \not\models \varphi).$$

Equivalently, for classical closed sentences,

$$\text{CounterModel}(\mathfrak{M}, \Gamma, \varphi) \iff (\forall \gamma \in \Gamma, \mathfrak{M} \models \gamma) \wedge (\mathfrak{M} \models \neg \varphi).$$

Remark (Interpretation). A countermodel shows that the theory does not force the statement. It keeps the axioms true while making the proposed statement false.

Remark (Special cases). If $\Gamma = \emptyset$, then a countermodel to validity of φ is a structure $\mathfrak{M}$ such that $\mathfrak{M} \models \neg \varphi$. For a theory T , models of T satisfying φ and models of T satisfying $\neg \varphi$ witness the independence pattern: the theory alone does not settle φ .

Remark (Lean reading). In a mechanized setting, invalidity is witnessed by constructing semantic data:

$$\exists \mathfrak{M} ((\forall \gamma \in \Gamma, \mathfrak{M} \models \gamma) \wedge \mathfrak{M} \models \neg \varphi).$$

The constructed structure and satisfaction proofs are the certificate that the entailment cannot be valid.

Remark (Dependencies).

- [Model](#)
- [Satisfaction](#)

5.5 Consistency

Consistency Toolkit — Quick Reference

Concept	Role
Contradiction	assertion that cannot be true in an interpretation
Consistency	absence of contradiction
Inconsistency	contradiction in or from axioms
Model existence	evidence that axioms fit together
Relative consistency	consistency transferred from another theory

Consistency

Consistency Presentation: A Simple Example

Consider two informal axiom systems.

Example A.

- All objects are red.
- Some object is not red.

These assertions cannot simultaneously hold. The first assertion says every object has the property; the second says at least one object lacks it.

Example B.

- All objects are red.
- Object a is red.

These assertions can hold simultaneously. The second assertion agrees with what the first assertion already requires.

These examples are only for intuition. A consistent axiom system is one whose assertions fit together without contradiction. An inconsistent axiom system is one whose assertions cannot be held together coherently.

Remark (Structural remarks). **Why contradictions matter.** Contradictory systems are unusable because their assertions do not fit together. If a system both requires and forbids the same situation, then it no longer gives a coherent basis for reasoning.

Models as evidence. If a model exists, the axioms hold together coherently in at least one interpretation. A model is therefore evidence that the axioms are mutually compatible.

Absolute vs relative consistency. Most consistency results in mathematics are relative consistency results. They show that one theory is consistent provided some other background theory is consistent. This section records the idea only; the technical development belongs elsewhere.

Forward roadmap. The next section uses models and consistency to introduce independence:

Consistency
 $\Downarrow$
Independence

Definition (Contradiction)

Definition 5.5.1 (Contradiction). A *contradiction* is a statement that cannot be true under a given interpretation.

Remark (Interpretation). A contradiction marks a breakdown in compatibility. It identifies an assertion that cannot be made true in the interpretation under consideration.

Remark (Dependencies).

- [Satisfaction](#)

Definition (Consistent Axiom System)

Definition 5.5.2 (Consistent Axiom System). A *consistent axiom system* is an axiom system that does not lead to contradiction.

Remark (Interpretation). Consistency says that the adopted assertions fit together. A consistent axiom system avoids forcing an impossible situation.

Remark (Dependencies).

- [Axiom System](#)
- [Contradiction](#)

Definition (Inconsistent Axiom System)

Definition 5.5.3 (Inconsistent Axiom System). An *inconsistent axiom system* is an axiom system containing or producing contradictory assertions.

Remark (Interpretation). Inconsistency means that the adopted assertions cannot all be maintained coherently. The system asks for incompatible requirements at once.

Remark (Dependencies).

- [Axiom System](#)

- [Contradiction](#)

Definition (Model Existence Criterion (Informal))

Definition 5.5.4 (Model Existence Criterion (Informal)). Informally, if an axiom system has a model, then the axioms are mutually compatible.

Remark (Interpretation). This is an informal criterion in this section, not a formal theorem. The idea is that a model provides a setting in which all the axioms hold together, so the axioms have at least one coherent realization.

Remark (Dependencies).

- [Model](#)

Definition (Relative Consistency)

Definition 5.5.5 (Relative Consistency). Let T_0 and T_1 be theories. The theory T_1 is *consistent relative to* T_0 if

$$\text{Con}(T_0) \implies \text{Con}(T_1).$$

Equivalently, every contradiction in T_1 would imply a contradiction in T_0 .

Remark (Standard quantified statement). For theories T_0 and T_1 ,

$$\text{RelCon}(T_1, T_0) \iff (\neg \exists d_0 (d_0 : T_0 \vdash \perp)) \implies (\neg \exists d_1 (d_1 : T_1 \vdash \perp)).$$

Equivalently, in contrapositive form,

$$\exists d_1 (d_1 : T_1 \vdash \perp) \implies \exists d_0 (d_0 : T_0 \vdash \perp).$$

Remark (Interpretation). Relative consistency does not establish consistency from nothing. It says that the target theory introduces no new contradiction unless the base theory was already inconsistent. In practice, a relative consistency proof often gives a translation that turns any contradiction derived in T_1 into a contradiction derived in T_0 .

Remark (Dependencies).

- [Consistent Axiom System](#)

5.6 Independence

Independence Toolkit - Planned

This section is a rebuild placeholder.

This section is planned for the governance rebuild. No mathematical content has been restored here yet.

5.7 Comparing Theories

Comparing Theories Toolkit - Planned

This section is a rebuild placeholder.

This section is planned for the governance rebuild. No mathematical content has been restored here yet.

5.8 Examples And Previews

Examples And Previews Toolkit - Planned

This section is a rebuild placeholder.

This section is planned for the governance rebuild. No mathematical content has been restored here yet.

5.9 Summary

Summary Toolkit - Planned

This section is a rebuild placeholder.

This section is planned for the governance rebuild. No mathematical content has been restored here yet.

Chapter 6

Proof Techniques



6.1 Proof Architecture

The Two Questions Before You Write Anything. Before writing a single symbol, ask two questions in order.

Question 1. *What is the logical form of the statement?*

Read the claim and identify its outermost logical structure: universal, conditional, bi-conditional, existential, uniqueness, equality, or structural assertion.

Question 2. *What proof strategy does this form demand?*

The form determines the skeleton of the proof almost mechanically, before any mathematics is done.

Remark 6.1.1 (Why order matters — and why most students get it backwards). Question 2 cannot be answered before Question 1, but almost every beginner tries to answer it first. The default instinct is to reach for a familiar technique — proof by contradiction is the usual reflex — and start writing. This is the wrong order.

Reading the statement *is* the first mathematical act of the proof. The logical form of the statement is not preliminary bookkeeping; it is the information that determines the entire structure of the argument. A proof whose structure mismatches the statement's form is not just inelegant — it is often invalid or at best needlessly circular.

Until you are fluent enough to read the form automatically, slow down deliberately before every proof. Write the statement out. Identify its outermost connective. Then and only then choose a strategy.

Example 6.1.2 (Applying the two questions). **Statement.** *The identity element of a group is unique.*

Q1. Logical form. Strip the natural-language phrasing down to its logical skeleton. “The identity is unique” means: for every group G and for any two elements $e, e' \in G$ that both satisfy the identity axiom, we have $e = e'$. The outermost structure is a universal statement with a uniqueness conclusion: $\forall G, \forall e, e' \in G, [\text{both satisfy identity axiom}] \Rightarrow e = e'$.

Q2. Strategy. A universally quantified uniqueness claim demands the *assume-two-and-compare* pattern: introduce an arbitrary group G , assume two objects e and e' both satisfying the definition, and derive $e = e'$.

The strategy is forced entirely by the form. No creative insight is required to select it. The selection is a consequence of reading.

Remark 6.1.3 (What “logical form” means in practice). You do not need to write out the full first-order formula every time. You need to identify one thing: the *outermost* structure.

Is the statement “for all $x, \dots$ ”? Then introduce an arbitrary x . Is it “ P implies Q ”? Then assume P . Is it “there exists x ”? Then construct a witness. Is it “there exists a unique x ”? Then split into existence and uniqueness. Is it “ $A = B$ as sets”? Then prove double inclusion. Is it “ $a = b$ as elements in an algebraic structure”? Then either do algebra or use satisfy-and-cite.

The statement map in the next section makes this correspondence explicit.

Statement Forms and Their Proof Strategies. The following table maps every common statement form to the proof strategy it demands. This is not a list of suggestions — it is a mechanical lookup. Once you have identified the outermost logical form of the statement you are trying to prove, the table tells you how to open your proof.

Statement Form	Proof Strategy	Opening Move
$\forall x, P(x)$	Introduce arbitrary element	“Let x be arbitrary.”
$P \Rightarrow Q$	Direct proof	“Assume P .”
$P \Rightarrow Q$	Contrapositive	“Assume $\neg Q$.”
$P \Rightarrow Q$ (when negation more useful)	Contradiction	“Assume P and $\neg Q$.”
$P \Leftrightarrow Q$	Two separate directions	Prove $P \Rightarrow Q$, then $Q \Rightarrow P$.
$\exists x, P(x)$	Construct a witness	“Define $x := \dots$ and verify $P(x)$.”
$\exists! x, P(x)$	Existence then uniqueness	Construct witness; then assume-two-and-compare.
$A = B$ (sets)	Double inclusion	Prove $A \subseteq B$ and $B \subseteq A$.
$a = b$ (algebraic elements)	Satisfy-and-cite or algebra	Show a satisfies the defining property of b .
$P(n)$ for all $n \in \mathbb{N}$	Induction	Base case; inductive step.
“ X is unique”	Assume-two-and-compare	“Let x, y both satisfy the definition.”

Remark 6.1.4 (Reading the table: defaults and alternatives). Some statement forms admit more than one strategy, and the table reflects this with multiple rows for the same form.

For $P \Rightarrow Q$, the default is direct proof: assume P and derive Q by forward reasoning. Contrapositive is reached for when $\neg Q$ is more useful than P as an assumption — this happens when the negation of the conclusion is a strong, concrete condition (for example, “ n is odd” is concrete; “ n is not even” is the same thing but sounds weaker). Contradiction is reached for when neither P alone nor $\neg Q$ alone gets you far, but combining them produces a quick absurdity.

The hierarchy is: try direct proof first. If the hypothesis is hard to use forward, try contrapositive. If neither works, try contradiction. Do not reach for contradiction as a default just because the proof feels difficult.

Remark 6.1.5 (Set equality versus element equality are genuinely different). The two equality rows in the table look similar but demand completely different techniques, and conflating them is a common error.

$A = B$ as *sets* is proved by double inclusion because sets are defined extensionally: two sets are equal if and only if they have the same elements. Proving $A = B$ by any other means — saying they are “clearly the same” or “follow from the definition” — is not a proof. The double inclusion must be made explicit.

$a = b$ as *elements in an algebraic structure* is different. Here b is typically a uniquely-determined object: an inverse, an identity, a limit, a fixed point. The strategy is to show a satisfies the same defining property as b , and then invoke the previously proved uniqueness

theorem to conclude $a = b$. This is the satisfy-and-cite pattern (Section 6.1).

The mistake is trying to use double inclusion on algebraic elements, or trying to use algebraic manipulation on sets. The form of the objects determines which technique applies.

Remark 6.1.6 (The biconditional is always two separate proofs). A biconditional $P \Leftrightarrow Q$ is *never* proved in a single argument that runs from P to Q and “loops back.” It is always proved in two distinct proofs:

$(\Rightarrow)$ Assume P . Prove Q .

$(\Leftarrow)$ Assume Q . Prove P .

Label the two directions explicitly. The two directions often require completely different techniques and different amounts of work. In characterisation theorems in analysis, one direction is typically a two-line unraveling of a definition while the other requires a nontrivial inequality argument. Treating them as one argument is how gaps get hidden.

The Five Proof Archetypes. Every mathematical proof, at the highest level of compression, is an instance of one of five archetypes. The statement map tells you the specific opening move. The archetypes tell you the *global shape* of the argument — what kind of reasoning is being done, and why it constitutes a valid proof of the claim.

The Five Proof Archetypes

1. **Construct something.** Exhibit a concrete object and verify it has every required property. This is the proof structure for all existence claims.
2. **Assume two and compare.** Assume two objects both satisfy a definition; derive that they are equal. This is the proof structure for all uniqueness claims.
3. **Take a minimal element.** Apply the well-ordering principle to a nonempty set of counterexamples; exploit the minimality of the least element to derive a contradiction. Used in number theory, descent arguments, and equivalence with induction.
4. **Induct.** Verify a base case; show the property propagates to successors. Used whenever the claim concerns all natural numbers or any recursively defined structure.
5. **Chase an arbitrary element.** Take an arbitrary element satisfying one condition; track it through definitions and established facts until it satisfies the target condition. Used for set inclusions, function properties, and direct proofs where the logic flows in a straight line from hypothesis to conclusion.

Remark 6.1.7 (Why five archetypes, not more?). The five archetypes are not a classification chosen for convenience. They correspond to the five fundamental proof obligations that mathematical statements impose:

To prove something *exists*, you must produce it. To prove something is *unique*, you must show that any two instances are equal. To prove a statement holds *for all natural numbers*, the inductive structure of $\mathbb{N}$ provides the only available tool. To prove a *conditional* claim

$P \Rightarrow Q$, you must trace an arbitrary element or object satisfying P to a situation where Q holds. The minimal element archetype is the well-ordering counterpart to induction, available whenever you need to reason about the smallest counterexample.

Everything else in proof writing is refinement of these five. The specific proof structures in Section 6.3 (direct proof, contrapositive, contradiction, cases, existence-uniqueness) are instantiations of these archetypes adapted to specific logical forms. The tactics in Section 6.5 are the move-level tools used to execute whichever archetype applies.

Remark 6.1.8 (Archetypes can nest, and this is a feature). A single proof often uses more than one archetype. The most common nested pattern in abstract algebra is: construct a witness (archetype 1), then invoke a uniqueness theorem (archetype 2) to conclude that the constructed object equals a previously named one.

This is the satisfy-and-cite tactic (Section 6.1). It appears in almost every proof of the form “ $a = b$ ” where b is an algebraically defined object. The nesting is not a complication — it is the standard pattern for proving equalities in algebraic structures, and once recognized, it becomes mechanical.

Another common nesting: induct (archetype 4), and inside the inductive step, use a direct proof argument (archetype 5) to move from $P(n)$ to $P(n + 1)$. The outer structure is induction; the inner argument is element-chasing.

Remark 6.1.9 (The archetype identifies the proof’s centre of gravity). When you read a finished proof in a textbook, the archetype is usually visible in the first line or two. “Suppose x, y both satisfy the definition” — that is archetype 2. “Define $f : X \rightarrow Y$ by $f(x) = \dots$ ” — that is archetype 1. “Let $n \in \mathbb{N}$ be arbitrary” followed by induction markup — that is archetype 4. “Let $x \in A$ ” followed by a chain of implications — that is archetype 5.

Reading proofs actively means identifying the archetype at the start and then watching how the author executes it. This is far more efficient than trying to absorb a proof line by line without a sense of its overall shape.

The Satisfy-and-Cite Pattern. There is a proof pattern that does not appear in any of the standard proof archetypes by name, yet appears constantly — in every chapter of abstract algebra and analysis. Most undergraduates have never been taught it explicitly, which means they either reinvent it poorly each time or miss it entirely and write longer, less clear proofs.

The pattern is called **satisfy-and-cite**. Here is its logic.

The Satisfy-and-Cite Pattern

Setup. You want to prove $a = b$, where b is a *uniquely determined* object: the unique identity, the unique inverse, the unique limit, the unique fixed point, the unique element satisfying some algebraic condition.

Step 1. Show that a satisfies the defining property of b .

Step 2. Cite the previously proved uniqueness theorem: there is exactly one object satisfying that property.

Step 3. Conclude that $a = b$, since b is the unique such object and a is one too.

Remark 6.1.10 (Why this is not the same as assume-two-and-compare). The confusion between these two patterns is extremely common, so it is worth being precise about what each one does.

The **assume-two-and-compare** pattern *proves* a uniqueness theorem. You use it when the goal is to establish that some object is unique. You say: let x and y both satisfy the definition; then show $x = y$.

The **satisfy-and-cite** pattern *uses* an already proved uniqueness theorem. You use it when you already know some object b is unique and you want to identify a with b . You show a satisfies b 's defining property, then invoke uniqueness to collapse a to b .

Assume-two-and-compare appears in the proof that the identity is unique. Satisfy-and-cite appears in every proof that uses that fact afterward. The first creates the tool; the second uses it. Conflating them means writing a new uniqueness proof every time you want to identify two equal objects, which is circular at best and wrong at worst.

Example 6.1.11 (Socks-shoes property: $(ab)^{-1} = b^{-1}a^{-1}$). We want to show $(ab)^{-1} = b^{-1}a^{-1}$ in a group.

The inverse of ab is, by definition, the unique element x satisfying $(ab)x = e$ and $x(ab) = e$.

We compute directly, using only associativity and the inverse axiom:

$$(ab)(b^{-1}a^{-1}) = a(bb^{-1})a^{-1} = aea^{-1} = aa^{-1} = e.$$

Similarly $(b^{-1}a^{-1})(ab) = e$.

So $b^{-1}a^{-1}$ satisfies the defining property of $(ab)^{-1}$. By the uniqueness of inverses (which we have already proved), we conclude $(ab)^{-1} = b^{-1}a^{-1}$. $\square$

Notice what did not happen: we never assumed two inverses of ab existed and compared them. We showed one specific element satisfies the inverse condition, then cited uniqueness to identify it with the named inverse.

Example 6.1.12 (Ring theorem: $a \cdot (-b) = -(ab)$). We want to show $a(-b) = -(ab)$ in a ring.

The additive inverse $-(ab)$ is, by definition, the unique element x satisfying $ab + x = 0$.

We compute using distributivity:

$$ab + a(-b) = a(b + (-b)) = a \cdot 0 = 0.$$

So $a(-b)$ satisfies the defining property of $-(ab)$. By the uniqueness of additive inverses (previously proved), $a(-b) = -(ab)$. $\square$

Same pattern: compute, satisfy, cite.

Remark 6.1.13 (The pattern across mathematics). Once you see satisfy-and-cite, you will see it everywhere. It appears wherever uniqueness theorems have been proved:

In **algebra**: every proof of the form “this element equals the inverse/identity/zero” uses it. The four examples above are $(ab)^{-1} = b^{-1}a^{-1}$, $a \cdot (-b) = -(ab)$, $-(-a) = a$, and $a \cdot 0 = 0$.

In **analysis**: the uniqueness of limits means that to show $\lim_{n \rightarrow \infty} x_n = L$, you verify x_n converges to L by the definition; uniqueness then implies L equals any other expression you have shown to be the limit of the same sequence.

In **linear algebra**: the zero vector is unique; the additive inverse is unique; a linear transformation is uniquely determined by its values on a basis. Each uniqueness theorem creates a new application of satisfy-and-cite.

Each time a uniqueness theorem is proved, it creates a new proof tool that can be activated by satisfy-and-cite for every subsequent argument in that structure. The theorem is not just a fact — it is a lever.

6.2 The Proof Construction Algorithm

Remark 6.2.1 (What this section is for). Architecture (Section 6.1) tells you what kind of proof to write. This section tells you how to write it, line by line, once the architecture is chosen.

The procedure below is intentionally explicit and more detailed than anything you will find in a standard textbook. That is on purpose. With practice, these steps will become internalized and automatic. But until they are — until you have written enough proofs that the process feels natural — working through them consciously will prevent the most common errors: undefined objects, unjustified steps, and forgotten stop conditions.

A longer proof with every step explicit is always better than a shorter proof with implicit gaps.

Step 0: Classify the Statement. Before anything else, identify the logical form of the claim. Common forms:

- Universal: $\forall x P(x)$
- Conditional: $P \rightarrow Q$
- Biconditional: $P \leftrightarrow Q$
- Existential: $\exists x P(x)$
- Existence-and-uniqueness: $\exists! x P(x)$
- Set equality: $A = B$
- Algebraic equality: $a = b$
- Structural assertion: “ f is injective,” “ R is an equivalence relation”

The logical form determines the skeleton of the proof. Consulting the statement map (Section 6.1) is the direct link from this classification to an opening strategy.

Step 1: Restate the Givens. Write out explicitly what is given. Introduce every set, relation, function, and algebraic structure, and record any properties assumed about them.

Let G be a group. Let $a \in G$.

This is not a formality. By recording the givens, you license their use in the proof — you can now invoke associativity, the existence of inverses, and the identity element without reintroducing them each time. What is not in the givens cannot be used.

Step 2: Identify Objects and Their Types. Before reasoning begins, verify the *type* of each object you will work with: is it an element, a set, a function, or a relation? What is its ambient universe? What operations apply to it?

$$x \in A, \quad f : A \rightarrow B, \quad R \subseteq A \times A.$$

This step prevents the most common logical errors in undergraduate proofs: writing $x \in f$ when you mean $x \in \text{dom}(f)$; writing $A = x$ when you mean $x \in A$; confusing membership ($\in$) with inclusion ($\subseteq$). Every object used in a proof must have a declared type. If you cannot say what type something is, you do not yet understand what you are working with.

Step 3: Introduce Arbitrary Elements. If the claim is universal ($\forall x P(x)$), immediately introduce an arbitrary element:

Let $x \in A$ be arbitrary.

The word “arbitrary” is not decorative. It means the element is not given any properties beyond those of the set A . Whatever you prove about this x will hold for all $x \in A$, precisely because you have assumed nothing about it beyond membership. The moment you specialize — “let $x = 0$ ” or “let x be such that . . . ” — you have lost the ability to conclude the universal statement.

Step 4: Expand the Goal. Rewrite the conclusion using definitions rather than named concepts.

- To prove f is injective: expand injectivity as “ $f(a) = f(b) \Rightarrow a = b$ ” and take that as the new goal.
- To prove $A = B$ (sets): take as new goal “ $A \subseteq B$ and $B \subseteq A$.”
- To prove $x \in A \cup B$: rewrite the goal as “ $x \in A$ or $x \in B$.”
- To prove $\{x_n\}$ converges: expand to “ $\forall \varepsilon > 0 \exists N$ s.t. $n \geq N \Rightarrow d(x_n, L) < \varepsilon$.”

This step is the one most students skip, and skipping it is the single most reliable predictor of a stuck proof. You cannot make progress toward a goal you have not named in operative terms. “Prove f is injective” is not an operative goal. “Prove that $f(a) = f(b)$ implies $a = b$ ” is.

Step 5: Introduce Helper Objects. Introduce any auxiliary objects needed for the argument: witnesses for existential claims, intermediate elements for indirect arguments, bounds for inequality proofs, function values.

$$\text{Let } N_1 \in \mathbb{N} \text{ s.t. } n \geq N_1 \Rightarrow d(x_n, L) < \varepsilon/2.$$

No object should appear in a proof without being explicitly introduced. If you write “let c be the cancellation element” without first establishing that such an element exists, your proof has an undefined object. Every name carries an obligation: the obligation to have said what the name refers to.

Step 6: Apply One Rule at a Time. Each line of a proof follows from exactly one of four sources:

1. A definition (unpacking what a term means),
2. A hypothesis (something assumed in the current proof),
3. A previously proved theorem (cited by name and reference),
4. A basic logical rule (modus ponens, universal instantiation, etc.).

Lines that are justified by “clearly” or “obviously” or no reason at all are gaps. At the learning stage, every step should have an explicit justification. When a proof stalls, the right question is always: which of the four sources has not been consulted? Almost always, the answer is that some definition has not been unpacked.

Step 7: Use Forward and Backward Reasoning. Proofs alternate between two modes of reasoning, and recognizing which mode is productive at each stage is a significant skill.

Forward reasoning: deduce consequences from what you have established so far. If you know $a^2 = e$ in a group, derive that $a(ae) = a \cdot a \cdot e = a^2 \cdot e = e$. You are moving away from the hypotheses, toward the goal.

Backward reasoning: examine the goal and ask what would suffice to establish it. If the goal is “ $a = a^{-1}$,” ask: by what definition would that follow? It follows if $aa = e$, since that is the condition for a to be its own inverse. Now your new goal is to prove $aa = e$, which may be more directly accessible.

In practice, most proofs proceed in both directions simultaneously: you derive forward from the hypotheses while simplifying the goal backward until the two sides meet. When you are stuck, try the direction you have not been working in.

Step 8: Handle Cases Explicitly. If a statement splits into mutually exclusive and exhaustive cases, enumerate all of them.

Either n is even, or n is odd.

Handle each case separately. Close each case before opening the next. Verify, explicitly, that the cases together cover every possibility. A proof with unlabeled cases — where it is unclear which case is being argued or whether all cases have been covered — is not a proof.

Step 9: Close the Argument. Once the desired conclusion has been established, state it explicitly:

- “Thus $x \in B$, and so $A \subseteq B$.”
- “Hence $f(a) = f(b)$ implies $a = b$, so f is injective.”
- “In both cases $P(n + 1)$ holds.”

The explicit closing step serves two functions. First, it tells the reader (and you) that the proof goal has been achieved. Second, it forces you to check that what you have proved actually *is* the goal. Students who reach the end of a proof and write $\square$ without stating a conclusion often discover, on re-reading, that they proved something adjacent to the goal but not the goal itself.

Step 10: Signal Completion. Conclude with $\square$, $\blacksquare$, or “This completes the proof.”

Legal Moves and Stop Conditions

What you may never do:

- Choose an existential witness before proving it exists.
- Introduce an “arbitrary” element and then give it special properties.
- Assume the conclusion at any point in the proof.
- Apply a definition partially (all conditions must be checked).

A proof is complete when:

- A universal claim has been proved for a genuinely arbitrary element.
- An existential claim has produced a verified witness.
- A set equality has established both inclusions explicitly.
- Each direction of a biconditional has been proved separately.
- An existence-uniqueness claim has done both parts.

Line-by-Line Discipline. Before writing each line, ask three silent questions:

- (i) Has every object mentioned in this line been defined or introduced earlier?
- (ii) Which definition, hypothesis, or theorem justifies this step?
- (iii) Does this step move the argument toward the stated goal?

If any answer is no, stop and resolve it before continuing.

6.3 Proof Structures

Direct Proof. A direct proof of $P \Rightarrow Q$ assumes P and derives Q through a chain of valid inferences.

Template: Direct Proof

Given. P .

Goal. Q .

Assume P .

$\therefore$ [expand definitions; apply lemmas; derive consequences]

Therefore Q . □

Remark 6.3.1 (Direct proof is the default — always try it first). Direct proof is not one option among equals. It is the default. Every time you sit down to prove $P \Rightarrow Q$, the first thing you write is “Assume P ” and the first thing you do is unpack what P says. You use what P gives you, apply definitions and previously proved results, and derive Q .

Reach for contrapositive or contradiction only when direct proof has genuinely stalled — when you have unpacked the hypothesis and the goal and there is no visible path between them. Do not reach for contradiction just because the statement feels hard. Most statements that feel hard are hard only because a definition has not been unpacked.

Remark 6.3.2 (The anatomy of a direct proof). Every direct proof has three phases that correspond to the first four steps of the construction algorithm:

Phase 1 — Unpack the hypothesis. P is a named concept or a logical statement. Expand it into its operative form. If P says “ G is a group,” this licenses associativity, the existence of an identity, and the existence of inverses. Write those down as available tools.

Phase 2 — Expand the goal. Q is also a named concept. Expand it into what you need to establish. If Q says “ $a = a^{-1}$,” the operative target is showing a satisfies the defining property of an inverse: $aa = e$.

Phase 3 — Bridge the gap. Apply definitions, lemmas, and axioms to move from the expanded hypothesis to the expanded conclusion.

When Phase 3 stalls, the cause is almost always that Phase 1 or Phase 2 was not completed. The most common error is having a vague goal (“show $a = a^{-1}$ ”) without having written out what that means in terms of the group axioms.

Example 6.3.3 (Direct proof in a group). **Claim.** In a group G , if $a^2 = e$ then $a = a^{-1}$.

Proof. Assume $a^2 = e$, meaning $aa = e$.

We want to show $a = a^{-1}$, which by definition of inverse means we need $aa = e$ and $ea = a$ (or equivalently, a is its own inverse).

Multiply both sides of $aa = e$ on the right by a^{-1} :

$$(aa)a^{-1} = e \cdot a^{-1}.$$

By associativity: $a(aa^{-1}) = a^{-1}$. By the inverse axiom: $ae = a^{-1}$. By the identity axiom: $a = a^{-1}$.

Hence $a^2 = e$ implies $a = a^{-1}$. □

Each step is justified by exactly one of: the hypothesis ($aa = e$), the inverse axiom ($aa^{-1} = e$), the identity axiom ($ae = a$), or associativity. No step is unjustified.

Proof by Contrapositive. The contrapositive of $P \Rightarrow Q$ is $\neg Q \Rightarrow \neg P$. These two statements are logically equivalent: one is true if and only if the other is. A proof by contrapositive establishes $\neg Q \Rightarrow \neg P$ by a direct argument, and since this is equivalent to $P \Rightarrow Q$, the original claim follows.

Template: Proof by Contrapositive

Goal. $P \Rightarrow Q$.

Equivalent goal. $\neg Q \Rightarrow \neg P$.

Assume $\neg Q$.

$\therefore$ [derive $\neg P$ from $\neg Q$]

Therefore $\neg P$.

Hence $P \Rightarrow Q$. □

Remark 6.3.4 (When contrapositive is the right tool). Contrapositive is useful when the negation of the conclusion is more concrete and easier to work with than the hypothesis.

The canonical example is injectivity. To prove $P \Rightarrow Q$ where P says “ $f(a) = f(b)$ ” and Q says “ $a = b$ ”: the contrapositive is $a \neq b \Rightarrow f(a) \neq f(b)$, which may be directly accessible if the function is order-preserving or explicitly defined.

Another signal: when Q is the statement that some object exists or some set is non-empty. The negation $\neg Q$ says no such object exists, which may be a very restrictive and workable condition.

The general diagnostic: after writing $\neg Q$, look at what it says. If it gives you a strong, concrete assumption — something you can actually manipulate — contrapositive is likely the right tool. If $\neg Q$ is vague or complicated, try a different approach.

Remark 6.3.5 (Contrapositive versus contradiction: the essential difference). Students routinely conflate contrapositive and contradiction. They are not the same, and knowing the difference matters for clarity.

In a **contrapositive** proof, you assume *only* $\neg Q$ and derive $\neg P$. The hypothesis P is never mentioned. The logical engine is: you proved $\neg Q \Rightarrow \neg P$, which by equivalence gives $P \Rightarrow Q$.

In a **contradiction** proof, you assume *both* P and $\neg Q$ simultaneously and derive *any* absurdity — not necessarily $\neg P$. The logical engine is: the assumption $P \wedge \neg Q$ leads to a false statement, so $P \wedge \neg Q$ must itself be false, so $P \Rightarrow Q$.

When contrapositive works, it is cleaner: you need one assumption, not two, and the conclusion you derive ($\neg P$) is specific rather than “any contradiction.” Always try contrapositive before reaching for contradiction.

Example 6.3.6 (Contrapositive for divisibility). **Claim.** If n^2 is even then n is even.

Proof (contrapositive). We prove the contrapositive: if n is odd then n^2 is odd.

Assume n is odd. Then $n = 2k + 1$ for some $k \in \mathbb{Z}$.

$$n^2 = (2k + 1)^2 = 4k^2 + 4k + 1 = 2(2k^2 + 2k) + 1,$$

which has the form $2m + 1$ and is therefore odd.

Hence n odd $\Rightarrow n^2$ odd, which is the contrapositive of n^2 even $\Rightarrow n$ even. □

Notice: the hypothesis “ n^2 is even” was never used. We proved the contrapositive directly, and the original implication follows by logical equivalence. There is no need to loop back and mention the original statement; the contrapositive suffices.

Proof by Contradiction. To prove P , assume $\neg P$ and derive a statement that contradicts a known truth: a hypothesis, an axiom, or a previously proved theorem. Since the assumption $\neg P$ led to a false statement, $\neg P$ is false, and therefore P is true.

Template: Proof by Contradiction

Goal. P .

Suppose for contradiction that $\neg P$.

$\therefore$ [derive consequences; identify the absurdity]

This contradicts [name the violated fact].

Therefore P . □

Remark 6.3.7 (The logic: why it works). Contradiction relies on the law of the excluded middle: every statement is either true or false. To prove P , it suffices to show $\neg P$ is false. You show $\neg P$ is false by deriving from it a statement that is known to be false — a contradiction.

The contradiction can be anything: $0 = 1$, a set that is both empty and non-empty, a number that is simultaneously even and odd, an inequality $x < x$. The only requirement is that the contradicted fact was genuinely established before the current proof began.

This is the move that students most often get wrong: they think they have derived a contradiction when they have merely derived something surprising. A statement being counterintuitive is not a contradiction. The contradicted fact must be something you can point to: a hypothesis, an axiom, a definition, or a theorem with a name.

Remark 6.3.8 (When contradiction is the right tool). Contradiction is best reserved for claims whose negation is a strong, productive assumption. The archetypes where contradiction excels are:

Irrationality proofs. Assuming $\sqrt{2} \in \mathbb{Q}$ gives you a fraction in lowest terms; arithmetic then forces the numerator and denominator to be simultaneously even, contradicting the lowest-terms assumption.

Infinitude arguments. Assuming there are finitely many primes lets you list them; constructing a number not divisible by any of them contradicts the assumption that the list was complete.

Nonexistence results. If you want to show no object satisfies some property, assume one exists and derive an absurdity.

Uniqueness (via assume-two-and-compare). Assuming two distinct objects both satisfy a uniqueness condition produces a contradiction from the definitions.

Avoid contradiction as a default. When a direct proof or contrapositive works, prefer it: the argument is shorter and the logical structure is more transparent. Contradiction is a sign that the natural direction of inference does not work, and that you need to reason from an impossible assumption instead.

Remark 6.3.9 (Always name the contradiction explicitly). The closing line of a contradiction proof must name what has been contradicted. “This is a contradiction” with no further specification is a red flag: it suggests the student either does not know what was contradicted, or is hoping the reader will not check.

Write: “This contradicts the assumption that $\gcd(p, q) = 1$.” Write: “This contradicts Theorem X, which asserts ...” Write: “This contradicts $m \in S$ being the least element.”

Naming the violated fact performs two functions: it confirms that the contradiction is genuine, and it locates the logical break in the argument so that anyone reading the proof can verify it.

Example 6.3.10 ($\sqrt{2}$ is irrational). *Proof.* Suppose for contradiction that $\sqrt{2} \in \mathbb{Q}$. Write $\sqrt{2} = p/q$ where $p, q \in \mathbb{Z}$, $q \neq 0$, and $\gcd(p, q) = 1$ (in lowest terms; this is possible by the definition of $\mathbb{Q}$).

Squaring: $2q^2 = p^2$. So p^2 is even, which forces p to be even (by the contrapositive example above). Write $p = 2k$.

Then $2q^2 = (2k)^2 = 4k^2$, so $q^2 = 2k^2$. So q^2 is even, which forces q to be even.

But then both p and q are even, so $\gcd(p, q) \geq 2$. This contradicts $\gcd(p, q) = 1$.

Therefore $\sqrt{2} \notin \mathbb{Q}$. □

The structure is: assume $\neg P$ (i.e., $\sqrt{2} \in \mathbb{Q}$); extract information from this assumption (lowest-terms representation); derive a consequence (both p and q even); identify the contradiction (conflicts with lowest terms); conclude P .

Proof by Cases. If the hypotheses or conclusion naturally divide into a finite number of mutually exclusive and collectively exhaustive alternatives, handle each case separately.

Template: Proof by Cases

Goal: Prove Q , given that exactly one of $P_1, P_2, \dots, P_k$ holds (and one of them always holds).

Verify exhaustiveness: Every possibility falls under some P_i .

Verify mutual exclusivity (when needed).

Case 1: Assume P_1 . [Argue.] Hence Q .

Case 2: Assume P_2 . [Argue.] Hence Q .

⋮

Case k : Assume P_k . [Argue.] Hence Q .

Since $P_1, \dots, P_k$ are exhaustive and Q holds in each case, Q holds unconditionally. □

Remark 6.3.11 (The exhaustiveness obligation is non-negotiable). A case proof is only valid when the cases together cover every possibility. Always verify exhaustiveness explicitly, even when it feels obvious.

The most common exhaustive splits:

- n is even, or n is odd.

- $a > 0$, $a = 0$, or $a < 0$.
- $a \mid b$, or $a \nmid b$.
- $x \in A$, or $x \notin A$.
- $x = y$, or $x \neq y$.

Writing “we consider two cases” without naming what they are, or naming cases that do not cover all possibilities, is a structural error. The proof is not valid until exhaustiveness is established.

Remark 6.3.12 (Each case is a self-contained argument). Inside each case, you have an additional assumption: P_i is true. That assumption is local to the case. You may use it within Case i , but you may not carry it into Case j .

When you close a case — “hence Q , completing Case 1” — you are signalling that the local assumption is no longer in scope. Case 2 starts fresh, with only the original given plus the assumption P_2 .

This discipline prevents the most common case-analysis error: drifting from one case into another by accidentally retaining a local assumption beyond its scope.

Remark 6.3.13 (Symmetric cases). When two cases are exactly symmetric — when the argument for Case 2 is obtained from the argument for Case 1 by renaming variables, with no other change — it is acceptable to write “Case 2 is symmetric to Case 1 by swapping a and b .” But be careful: only write this when the symmetry is complete. A partial symmetry that requires a different step at one point must be written out in full.

Example 6.3.14 (Parity of $n(n+1)$). **Claim.** For every $n \in \mathbb{Z}$, $n(n+1)$ is even.

Proof. Every integer is either even or odd (these cases are exhaustive and mutually exclusive).

Case 1: n is even. Write $n = 2k$ for some $k \in \mathbb{Z}$. Then $n(n+1) = 2k(n+1)$. Since this is 2 times an integer, $n(n+1)$ is even.

Case 2: n is odd. Write $n = 2k+1$ for some $k \in \mathbb{Z}$. Then $n+1 = 2k+2 = 2(k+1)$, so $n(n+1) = (2k+1) \cdot 2(k+1)$. This is 2 times an integer, so $n(n+1)$ is even.

In both cases $n(n+1)$ is even. Since every integer falls into one of these cases, the proof is complete. $\square$

Notice that the cases were named before the argument, the exhaustiveness was verified (“every integer is even or odd”), and each case was closed explicitly before moving to the next.

Example 6.3.15 (Case split induced by a definition). **Claim.** The discrete metric satisfies the triangle inequality.

Proof. Let $x, y, z \in X$. We must show $d(x, z) \leq d(x, y) + d(y, z)$.

Case 1: $x = z$. Then $d(x, z) = 0$. Since $d(x, y) \geq 0$ and $d(y, z) \geq 0$, we have $0 \leq d(x, y) + d(y, z)$.

Case 2: $x \neq z$. Then $d(x, z) = 1$. We cannot have both $x = y$ and $y = z$, for that would force $x = z$. So at least one of $d(x, y), d(y, z)$ equals 1, hence $d(x, y) + d(y, z) \geq 1 = d(x, z)$.

The cases $x = z$ and $x \neq z$ are exhaustive. The triangle inequality holds in both cases. $\square$

The case split here is induced by the two-value structure of the discrete metric: distances are 0 or 1. The right case split to use is the one naturally suggested by the definition you are working with.

Existence and Uniqueness. The statement $\exists! x P(x)$ asserts two things simultaneously: that some object satisfying P exists, and that it is the *only* such object. These two obligations are proved separately, in order.

Template: Existence and Uniqueness ($\exists! x P(x)$)

Step 1 — Existence.

Define $x :=$ [explicit construction].

Verify that $P(x)$ holds. (Check every condition in P .)

Step 2 — Uniqueness (assume-two-and-compare).

Let x and y be arbitrary objects both satisfying P .

Apply the definition of x with y as input; apply the definition of y with x as input. The two applications collapse.

Conclude $x = y$.

Step 3 — Conclude.

By Steps 1 and 2, there exists exactly one object satisfying P . □

Remark 6.3.16 (Existence must be proved before uniqueness). The uniqueness argument has the form: “let x and y both satisfy P .” This assumption is only meaningful if we already know that at least one object satisfying P exists. Without Step 1, the uniqueness argument is vacuous: we are asserting something about objects that might not exist.

This is not pedantry. There are statements where the existence fails and the uniqueness argument would appear to go through — but the whole proof would then be establishing uniqueness for an empty set of objects, which is trivially true and useless. Existence first is not just a convention; it is logically necessary.

Remark 6.3.17 (The key move in assume-two-and-compare). The assume-two-and-compare argument has a characteristic structure that appears across mathematics. The critical move is this: if x and y both satisfy the defining property P , apply x ’s property with y as the argument, and simultaneously apply y ’s property with x as the argument. The two applications collapse to $x = y$.

Here is the pattern in the group identity example: if e satisfies the identity axiom, then $e \cdot e' = e'$ (using e as identity with argument e'). If e' satisfies the identity axiom, then $e \cdot e' = e$ (using e' as identity with argument e). Combining: $e' = e$.

The same move works for: uniqueness of inverses, uniqueness of the zero vector, uniqueness of limits, uniqueness of the supremum. In every case, the two assumed objects are “played against each other”: each one’s defining property is applied with the other as input.

Example 6.3.18 (Uniqueness of the group identity). *Proof.* Let G be a group. Suppose e and e' both satisfy the identity axiom:

$$e \cdot g = g \cdot e = g \quad \text{for all } g \in G,$$

$$e' \cdot g = g \cdot e' = g \quad \text{for all } g \in G.$$

Apply e' 's identity property with $g = e$:

$$e' \cdot e = e.$$

Apply e 's identity property with $g = e'$:

$$e \cdot e' = e'.$$

But $e' \cdot e = e \cdot e'$ (both equal the product $e \cdot e'$ in G by commutativity — actually, both equal $e \cdot e'$ directly). Actually, more directly: $e = e' \cdot e = e'$ where the first step uses e' as identity and the second uses e as identity. Hence $e = e'$. $\square$

The single line is: $e = e \cdot e' = e'$. First equality: e acts as identity on e' . Second equality: e' acts as identity on e . Both identities applied to the same product; both sides collapse to the same expression; $e = e'$.

Remark 6.3.19 (Why proving uniqueness creates a proof tool). Every uniqueness theorem, once proved, creates a new application of the satisfy-and-cite tactic. This is the connection that most students miss.

Proving uniqueness says: there is exactly one object satisfying P . This means: if you can ever show that some element a satisfies P , you automatically get $a = b$, where b is the canonical name for the unique object satisfying P .

The group inverse provides the clearest example. Once you have proved that inverses are unique, the pattern for proving $(ab)^{-1} = b^{-1}a^{-1}$ is immediate: show $b^{-1}a^{-1}$ satisfies the defining property of the inverse of ab , then cite uniqueness. You never need to set up another assume-two-and-compare argument. The uniqueness theorem absorbs all future comparisons.

This is why uniqueness results are not just facts — they are *tools*. Prove them early; use them everywhere afterward.

6.4 Mathematical Induction

Why Induction Works. Mathematical induction is often presented as a technique: base case, inductive step, done. That presentation is useful but incomplete. Induction *feels* like a procedure, but it is actually a theorem, and it is worth understanding why it is true before using it.

Remark 6.4.1 (The Peano axiom behind induction). The natural numbers are characterized by the Peano axioms. The critical one for induction is Axiom P5:

If $A \subseteq \mathbb{N}$ satisfies (i) $0 \in A$ and (ii) $n \in A \Rightarrow S(n) \in A$, then $A = \mathbb{N}$.

This axiom says that $\mathbb{N}$ is the *smallest* set containing 0 and closed under the successor function S . There is no proper subset of $\mathbb{N}$ with these two properties.

The induction principle is a direct translation. Let $P(n)$ be any statement. Define $A = \{n \in \mathbb{N} : P(n)\}$. If $P(0)$ holds, then $0 \in A$. If $P(n) \Rightarrow P(S(n))$ for every n , then A is closed under S . By P5, $A = \mathbb{N}$. Therefore $P(n)$ holds for every $n \in \mathbb{N}$.

The full development of this from the Peano axioms is in the Axiom Systems chapter. Here, the takeaway is: induction is not a trick. It is a theorem derivable from the definition of $\mathbb{N}$.

Remark 6.4.2 (Why the base case is not optional). The inductive step alone says: *if* $P(n)$ holds for some n , then $P(n + 1)$ holds. This is a conditional, not an assertion. Without a base case, there is no n for which $P(n)$ is known to hold. The chain of conditionals has no starting point, and the proof produces nothing.

The most famous illustration is the false theorem “all horses are the same colour.” A flawed argument runs: base case trivial ($n = 1$), inductive step [fatally flawed reasoning]. The lesson is not just that the step is wrong — it is that even a correct base case does not rescue a bad step, and even a correct step does not rescue a missing base case. Each part is independent; both are necessary.

In terms of P5: the base case establishes $0 \in A$. The inductive step establishes closure under S . Both conditions of P5 are required for the conclusion $A = \mathbb{N}$.

Remark 6.4.3 (The domino picture and its limitations). The common picture of induction as an infinite row of falling dominoes is a useful mnemonic but a misleading explanation. It suggests that induction requires each domino to knock over the next, which captures the inductive step. But it does not explain why the argument is valid — the validity comes from P5, not from a physical metaphor.

More importantly, the domino picture suggests that the steps happen *in time*: first domino 0 falls, then 1, then 2, and so on. But induction proves a statement about *all* natural numbers simultaneously, not by a sequential process. The proof is instantaneous: once you have established the base case and the inductive step, the conclusion $P(n)$ for all n follows at once from P5.

Use the domino picture as memory aid. For understanding, use P5.

Remark 6.4.4 (Induction and well-ordering are equivalent). The induction principle and the well-ordering principle for $\mathbb{N}$ (every nonempty subset has a least element) are logically equivalent. This means every induction argument can be rephrased as a minimal-counterexample argument, and vice versa. Both are valid proof tools. The well-ordering approach is sometimes more natural when you want to reason about the *first* failure rather than climbing from a base case. The equivalence is proved in Section 6.4.

Theorem: Weak (Standard) Induction

Let $P(n)$ be a statement depending on $n \in \mathbb{N}$. Suppose:

1. **Base case.** $P(n_0)$ is true for some starting value n_0 .
2. **Inductive step.** For all $n \geq n_0$, $P(n) \Rightarrow P(n + 1)$.

Then $P(n)$ is true for all $n \geq n_0$.

Template: Weak Induction Proof

Let $P(n)$ denote the statement: [*write it out completely as a sentence*].

Base case ($n = [n_0]$). [*Verify $P(n_0)$ directly.*] Hence $P(n_0)$ holds.

Inductive step. Let $n \geq n_0$ and assume $P(n)$ holds. [*State explicitly what $P(n)$ says.*]

We show $P(n+1)$ holds.

[*Rewrite $P(n+1)$ to expose $P(n)$ inside it. Substitute the inductive hypothesis. Simplify.*]

Hence $P(n+1)$ holds.

By induction, $P(n)$ holds for all $n \geq n_0$. □

Remark 6.4.5 (The most important discipline: write $P(n)$ as a sentence). The single most impactful habit in induction proofs is writing the inductive hypothesis as a *complete mathematical sentence* before the inductive step begins.

“Assume $P(n)$ holds” is not sufficient if you have not first written out what $P(n)$ says. Write: “Assume $\sum_{k=1}^n k = \frac{n(n+1)}{2}$.” That sentence is your inductive hypothesis. It is the precise tool you have available in the inductive step.

Without this sentence, students typically make one of two errors: they try to use $P(n)$ without knowing what it says, guessing at it from context; or they prove the inductive step using information beyond what $P(n)$ supplies, making the proof circular or invalid. Write the sentence. It takes five seconds and eliminates the most common induction error.

Remark 6.4.6 (The core skill of the inductive step). Every inductive step involves the same core action: *rewrite $P(n+1)$ to expose $P(n)$ inside it, then substitute the inductive hypothesis.*

For sum formulas, this means: split off the last term.

$$\sum_{k=1}^{n+1} k = \left(\sum_{k=1}^n k \right) + (n+1).$$

Now substitute $P(n)$: replace $\sum_{k=1}^n k$ with $\frac{n(n+1)}{2}$. Simplify to get the $P(n+1)$ form.

For divisibility, this means: write $4^{n+1} - 1 = 4(4^n - 1) + 3$ to expose $4^n - 1$ inside $4^{n+1} - 1$.

In both cases, the step is: factor out the $P(n)$ term, substitute, and verify the arithmetic produces $P(n+1)$.

When the rewriting step is not obvious, it usually means $P(n)$ needs to be strengthened (see Section 6.4).

Example 6.4.7 (Sum formula). **Proposition.** For all $n \geq 1$: $\sum_{k=1}^n k = \frac{n(n+1)}{2}$.

Proof. Let $P(n)$ denote: $\sum_{k=1}^n k = \frac{n(n+1)}{2}$.

Base case ($n = 1$). $\sum_{k=1}^1 k = 1 = \frac{1 \cdot 2}{2}$. Hence $P(1)$ holds.

Inductive step. Let $n \geq 1$ and assume $\sum_{k=1}^n k = \frac{n(n+1)}{2}$. (*Inductive hypothesis.*)

We show $\sum_{k=1}^{n+1} k = \frac{(n+1)(n+2)}{2}$.

Split off the last term:

$$\begin{aligned}\sum_{k=1}^{n+1} k &= \left(\sum_{k=1}^n k \right) + (n+1) \\ &= \frac{n(n+1)}{2} + (n+1) && \text{(inductive hypothesis)} \\ &= (n+1) \left(\frac{n}{2} + 1 \right) = \frac{(n+1)(n+2)}{2}.\end{aligned}$$

Hence $P(n+1)$ holds. By induction, $P(n)$ holds for all $n \geq 1$. □

Example 6.4.8 (Divisibility). **Proposition.** For all $n \geq 0$, $3 \mid (4^n - 1)$.

Proof. Let $P(n)$ denote: $3 \mid (4^n - 1)$.

Base case ($n = 0$). $4^0 - 1 = 0 = 3 \cdot 0$. Hence $3 \mid 0$ and $P(0)$ holds.

Inductive step. Assume $3 \mid (4^n - 1)$, i.e., $4^n - 1 = 3m$ for some $m \in \mathbb{Z}$.

Expose $4^n - 1$ inside $4^{n+1} - 1$:

$$4^{n+1} - 1 = 4 \cdot 4^n - 1 = 4(4^n - 1) + 4 - 1 = 4(4^n - 1) + 3 = 4 \cdot 3m + 3 = 3(4m + 1).$$

Hence $3 \mid (4^{n+1} - 1)$ and $P(n+1)$ holds.

By induction, $P(n)$ holds for all $n \geq 0$. □

The rewriting step $4^{n+1} - 1 = 4(4^n - 1) + 3$ is the key move: it exposes the inductive hypothesis $3 \mid (4^n - 1)$ inside the goal $3 \mid (4^{n+1} - 1)$.

Choosing $P(n)$: The Hard Part. In textbook problems, the statement $P(n)$ is usually handed to you: the claim is explicitly “prove that $\sum_{k=1}^n k = n(n+1)/2$ ”, and $P(n)$ is that equality. In original proofs, you must construct $P(n)$ from the statement of the goal. And sometimes even when $P(n)$ is given, the straightforward choice fails and must be strengthened.

Remark 6.4.9 (Rule 1: $P(n)$ mirrors the logical form of the goal). Every claim to be proved by induction has the form $\forall n, \Phi(n)$ for some predicate Φ . Strip the universal quantifier: what remains is $P(n)$.

Goal form	Shape of $P(n)$
$\forall n, f(n) = g(n)$	Equality: $P(n) :\equiv f(n) = g(n)$
$\forall n, A(n) \Rightarrow B(n)$	Conditional: $P(n) :\equiv A(n) \Rightarrow B(n)$
$\forall n, A(n) \Leftrightarrow B(n)$	Biconditional: $P(n) :\equiv A(n) \Leftrightarrow B(n)$

Do not simplify. Do not add conditions. Transcribe the logical form of $\Phi(n)$ exactly into $P(n)$.

Remark 6.4.10 (Rule 2: induct on the recursive variable). When the goal involves a recursively defined operation, identify which variable the definition recurses on. Induct on that variable and hold all others fixed as arbitrary.

To find the recursive variable: locate the definition of the operation and identify which variable is reduced in the recursive clause.

Operation	Recursive clause	Induct on
Addition (Peano)	$(n++) + m := (n + m)++$	n
Multiplication (Peano)	$(n++) \times m := (n \times m) + m$	n
Exponentiation	$m^{n++} := m^n \times m$	n

Remark 6.4.11 (The strengthening principle: when $P(n)$ is too weak). Sometimes the natural $P(n)$ fails not because the statement is false but because the inductive hypothesis is insufficient to prove the step. The fix is counterintuitive: *add more to $P(n)$* , making the claim stronger.

This works because a stronger $P(n)$ gives you more in the inductive hypothesis, which is what you actually need to prove $P(n+1)$. The stronger $P(n)$ is harder to prove overall but easier to propagate.

The signal that strengthening is needed: you write the inductive step, you need to use some fact about the sequence that $P(n)$ alone does not provide, and you find yourself thinking “if only I also knew $P(n-1)$.” That thought is the signal to switch to strong induction (which implicitly strengthens $P(n)$ to “ $P(k)$ for all $k \leq n$ ”).

Example 6.4.12 (Strengthening: Fibonacci bound requires two prior values). **Goal.** Show that the Fibonacci sequence satisfies $F_n < 2^n$ for all $n \geq 1$.

Naive attempt. Let $P(n)$: “ $F_n < 2^n$.”

Inductive step: assume $F_n < 2^n$; show $F_{n+1} < 2^{n+1}$.

We have $F_{n+1} = F_n + F_{n-1}$. We know $F_n < 2^n$ by the inductive hypothesis. But what do we know about F_{n-1} ? Nothing: $P(n)$ says nothing about F_{n-1} . The step is stuck.

Fix: strong induction gives two terms. Use the strong inductive hypothesis: $F_k < 2^k$ for all $k \leq n$. Then:

$$F_{n+1} = F_n + F_{n-1} < 2^n + 2^{n-1} < 2^n + 2^n = 2^{n+1}.$$

The step completes because strong induction provides $F_{n-1} < 2^{n-1}$. This is the canonical signal that strong induction is needed: the recursive step requires knowing $P(k)$ for some $k < n$ other than $k = n-1$.

Remark 6.4.13 (Vacuous base cases in conditional statements). When $P(n)$ is a conditional and its hypothesis is false at the base value, the base case is vacuously true. This is expected and requires no further argument.

For example, if the claim is “for all $n \geq 1$, if $n > 0$ and $m > 0$ then $nm > 0$ ” and you induct starting at $n = 0$, then $P(0)$: “if $0 > 0$ and $m > 0$ then $0 \times m > 0$ ” has a false hypothesis ($0 > 0$ is false), so $P(0)$ is vacuously true.

This signals that the claim only has content for values of n where the hypothesis holds. The induction then establishes the claim for all n where the hypothesis is satisfied.

- Remark 6.4.14* (Diagnostic questions when stuck choosing $P(n)$). 1. What exactly do I need to prove for $n = 0, 1, 2, 3$? Write it out. The pattern of $P(n)$ will appear.
2. Does the inductive step require knowing $P(n - 1)$ as well as $P(n)$? If yes, switch to strong induction.
3. Does the inductive step produce a bound or equation slightly weaker than $P(n + 1)$? If yes, strengthen $P(n)$.
4. After writing $P(n++)$, can I apply the recursive definition to expose $P(n)$? If not, the wrong variable may have been chosen.

Theorem: Strong Induction

Let $P(n)$ be a statement depending on $n \in \mathbb{N}$. Suppose that for every $n \in \mathbb{N}$:

$$\left(\forall k < n, P(k) \right) \Rightarrow P(n).$$

Then $P(n)$ holds for all $n \in \mathbb{N}$.

Remark 6.4.15 (Strong and weak induction prove the same statements). Strong induction is sometimes described as “more powerful” than weak induction, but this is imprecise. Both principles are logically equivalent (they both follow from P5 and each implies the other). Strong induction is not more powerful in the sense of proving statements that weak induction cannot — every statement provable by one is provable by the other.

What strong induction offers is convenience: when the proof of $P(n+1)$ genuinely requires knowing $P(k)$ for some $k < n$ that is not $n - 1$, strong induction supplies that information directly. Rephrasing the proof as weak induction would require artificially strengthening $P(n)$ to “ $P(k)$ for all $k \leq n$ ”, which is precisely the hypothesis that strong induction builds in automatically.

Use strong induction when the step needs information from more than one earlier case. Use weak induction when $P(n)$ alone suffices.

Remark 6.4.16 (The base case in strong induction). When $n = 0$, the hypothesis $\forall k < 0, P(k)$ is vacuously true: there are no natural numbers less than 0. So the step for $n = 0$ requires proving $P(0)$ with no inductive hypothesis at all — this is the base case, and it must be proved directly.

In practice, always write the base case explicitly, even though it is implicit in the universal statement. Students who skip the base case in strong induction often write “by the inductive hypothesis applied to $n - 1$ ” when $n = 0$, which is an error: $n - 1 = -1$ is not a natural number and has no inductive hypothesis.

Remark 6.4.17 (When to use strong induction: the diagnostic). The diagnostic for strong induction is simple. Write out the inductive step: $P(n + 1) = f(P(?), P(?), \dots)$, and ask what earlier values of P you need. If the answer includes anything other than exactly $P(n)$, use strong induction.

Canonical situations where strong induction is needed:

- Recursive sequences with two prior terms: $a_{n+1} = a_n + a_{n-1}$ requires both $P(n)$ and $P(n-1)$.
- Prime factorization: writing $n = ab$ with $a, b < n$ requires $P(a)$ and $P(b)$ where a and b are arbitrary.
- Any argument that splits $n+1$ into parts: you need P at the part values, which are not predetermined.

Template: Strong Induction Proof

Let $P(n)$ denote: [write it out].

Base case ($n = [\text{start}]$). [Verify directly, with no inductive hypothesis.]

Inductive step. Let $n > [\text{start}]$ and assume $P(k)$ holds for all $[\text{start}] \leq k < n$. (Strong inductive hypothesis.) We show $P(n)$ holds.

[Use $P(k)$ for some specific $k < n$.]

Hence $P(n)$ holds.

By strong induction, $P(n)$ holds for all $n \geq [\text{start}]$. □

Example 6.4.18 (Every integer $n \geq 2$ has a prime factor). *Proof.* Let $P(n)$: “ n has a prime factor.”

Base case ($n = 2$). 2 is prime, so 2 is a prime factor of itself. $P(2)$ holds.

Inductive step. Let $n > 2$ and assume $P(k)$ holds for all $2 \leq k < n$.

Case 1: n is prime. Then n is its own prime factor, and $P(n)$ holds.

Case 2: n is composite. Write $n = ab$ with $2 \leq a, b < n$. By the strong inductive hypothesis applied to a , a has a prime factor p . Then $p \mid a$ and $a \mid n$, so $p \mid n$. Hence $P(n)$ holds.

In both cases $P(n)$ holds. By strong induction, $P(n)$ holds for all $n \geq 2$. □

Why weak induction cannot work directly: in Case 2, the value a satisfies $2 \leq a < n$ but is otherwise arbitrary. It is not necessarily $n-1$. Weak induction supplies $P(n-1)$, which is useless when $a \neq n-1$. Strong induction supplies $P(k)$ for all $k < n$, covering whatever value a takes.

Well-Ordering Principle

Every nonempty subset of $\mathbb{N}$ has a least element.

Remark 6.4.19 (The minimal counterexample argument). Well-ordering drives a proof strategy that is the structural dual of induction: the **minimal counterexample argument**.

Instead of climbing up from a base case, you assume the claim fails somewhere, take the smallest failure, and derive a contradiction from its minimality. The logic:

1. Suppose $P(n)$ fails for some n . Let $S = \{n \in \mathbb{N} : \neg P(n)\}$. By assumption $S \neq \emptyset$.
2. By well-ordering, S has a least element m .
3. Derive a contradiction: either show $P(m)$ holds (contradicting $m \in S$), or find some $k < m$ with $\neg P(k)$ (contradicting the minimality of m).

The key tool is minimality: the fact that m is the *smallest* failure means all smaller values are successes, which is a powerful assumption. It is equivalent to the strong inductive hypothesis.

Proposition (Induction and well-ordering are equivalent)

Proposition 6.4.20 (Induction and well-ordering are equivalent). *Over the axioms of $\mathbb{N}$ without $P5$, the following three principles are logically equivalent:*

1. *The (weak) induction principle.*
2. *The well-ordering principle.*
3. *The strong induction principle.*

Remark 6.4.21 (Proof sketch of the equivalence). **Induction $\Rightarrow$ Well-ordering.** Suppose $S \subseteq \mathbb{N}$ has no least element. Let $P(n)$: “ $n \notin S$.” Then $P(0)$ holds (otherwise $0 \in S$ would be the least element). If $P(k)$ holds for all $k \leq n$, then $n + 1 \notin S$ (otherwise $n + 1$ would be the least element of S). By induction, $P(n)$ holds for all n , so $S = \emptyset$.

Well-ordering $\Rightarrow$ Induction. Suppose the base case $P(n_0)$ holds and the inductive step holds. Let $S = \{n \geq n_0 : \neg P(n)\}$. If $S \neq \emptyset$, let m be its least element. Since $P(n_0)$ holds, $m \neq n_0$, so $m > n_0$ and $m - 1 \geq n_0$. By minimality of m , $P(m - 1)$ holds. But by the inductive step, $P(m - 1) \Rightarrow P(m)$, contradicting $m \in S$. Hence $S = \emptyset$.

Template: Minimal Counterexample

Suppose for contradiction that $P(n)$ fails for some $n \in \mathbb{N}$.

Let $S = \{n \in \mathbb{N} : \neg P(n)\}$. By assumption $S \neq \emptyset$. By well-ordering, S has a least element m .

[Derive a contradiction using minimality of m : either show $P(m)$ holds, or construct a smaller counterexample.]

This contradicts [minimality of m / the base case]. Therefore $P(n)$ holds for all $n \in \mathbb{N}$. □

Example 6.4.22 (Minimal counterexample: exponential bound). **Claim.** For all $n \geq 1$, $2^n > n$.

Proof (minimal counterexample). Suppose the claim fails. Let $S = \{n \geq 1 : 2^n \leq n\}$. By assumption $S \neq \emptyset$; let m be its least element.

Since $2^1 = 2 > 1$, we have $m \neq 1$, so $m \geq 2$. Then $m - 1 \geq 1$ and $m - 1 < m$, so $m - 1 \notin S$ (by minimality of m). Thus $2^{m-1} > m - 1$.

Then:

$$2^m = 2 \cdot 2^{m-1} > 2(m - 1) = 2m - 2 \geq m$$

(using $m \geq 2$ for the last inequality).

This contradicts $m \in S$ (which requires $2^m \leq m$). Therefore $S = \emptyset$ and the claim holds for all $n \geq 1$. □

The minimal counterexample argument proceeds by taking the smallest failure and showing

it cannot be the smallest failure. The contradiction is always of the same form: either the “failure” was not a failure, or there is a smaller one.

Structural Induction (Preview). Induction on $\mathbb{N}$ is a special case of a general principle that applies to any recursively defined structure. You do not need $\mathbb{N}$ to do induction; you need a recursive definition.

Remark 6.4.23 (The general principle). Let X be a set defined by:

1. A collection of *base cases* (atomic elements not built from other elements), and
2. A collection of *constructor rules* (ways of building larger elements from smaller ones).

Structural induction on X : to prove $P(x)$ for all $x \in X$, prove P for all base cases, then prove that if P holds for the parts, it holds for the whole.

This is exactly standard induction where the “less than” ordering on $\mathbb{N}$ is replaced by the “proper sub-element” ordering on X .

Example 6.4.24 (Induction on algebraic terms). Define an *algebraic term* over a ring R by:

- Base case: any element $r \in R$ is a term.
- Constructor: if s, t are terms, so are $s + t$ and $s \cdot t$.

To prove a property P holds for all terms:

1. Prove $P(r)$ for all $r \in R$ (base cases).
2. Prove: if $P(s)$ and $P(t)$ hold, then $P(s + t)$ holds.
3. Prove: if $P(s)$ and $P(t)$ hold, then $P(s \cdot t)$ holds.

The recursive structure of terms determines the structure of the proof.

Remark (Exposition). Structural induction is the proof principle paired with structural recursion. Both follow the constructors of the object being studied. Structural recursion defines a function by giving its value on the base cases and then giving its value on each constructed object in terms of the values already defined on the immediate parts. Structural induction proves a predicate or property by showing that it holds for the base cases and is preserved by every constructor.

Thus structural induction is like ordinary mathematical induction: it proves a statement $P(x)$ for every object x in a recursively generated class. The difference is that the next step is not always the Peano successor $n \mapsto n++$. The next step is whatever constructor builds the object. For natural numbers the constructors are 0 and successor. For formulas the constructors are atomic formulas, negation, binary connectives, and quantifiers. For trees the constructors are leaves and nodes.

A useful way to see this is to attach a syntax tree to each well-formed formula. Define trees for propositional formulas recursively:

- if $P \in \mathbf{Prop}$, then $\text{leaf}(P)$ is a tree;
- if T is the tree for φ , then $\text{node}(\neg, T)$ is the tree for $\neg\varphi$;

- if T_φ and T_ψ are the trees for φ and ψ , then $\text{node}(\circ, T_\varphi, T_\psi)$ is the tree for $(\varphi \circ \psi)$.

Operations on this tree are recursive in the same sense. The depth of a leaf is 0; the depth of a negation node is one plus the depth of its child; the depth of a binary node is one plus the maximum of the depths of its two children. Structural induction then proves statements about all such trees, and hence about all formulas, by checking the leaf case, the negation-node case, and the binary-node case. The proof follows the same recursive blueprint as the definition.

Remark 6.4.25 (Where structural induction appears in this project). Induction on $\mathbb{N}$ is structural induction on the Peano structure $(\mathbb{N}, 0, S)$: the base case is $n = 0$, the constructor is $n \mapsto n++$.

In the Abstract Algebra chapter, polynomial properties are proved by induction on degree, which is structural induction on the recursive definition of polynomials over a ring.

In computer science, correctness of recursive algorithms is proved by structural induction on the input data structure (lists, trees, expressions). The full treatment of structural induction appears in the Abstract Algebra chapter.

Common Induction Mistakes.

Remark 6.4.26 (Mistake 1: Missing base case). A proof that establishes the inductive step but not the base case proves nothing. The step says: *if $P(n)$ holds for some n , then $P(n+1)$ holds*. Without a base case, there is no n for which $P(n)$ is established, so the chain never starts.

The classic illustration: the false theorem “all horses are the same colour.” The base case for $n = 1$ is trivially true (one horse has one colour). The inductive step has a subtle flaw in the overlap argument. The lesson is that both parts are essential and independent. Always verify the base case, even when it seems trivial. A trivial base case takes one line; write the one line.

Remark 6.4.27 (Mistake 2: Circular inductive step). The inductive step proves $P(n+1)$ using only $P(n)$ and previously established facts. It is a fatal error to assume $P(n+1)$ at any point in the proof of $P(n+1)$.

This error most often appears when a student “derives” both sides of an equation and meets in the middle, inadvertently assuming what they are trying to prove. The symptom: the final few lines of the step are not deductions from $P(n)$ but rather manipulations of $P(n+1)$ itself.

The cure: always work in one direction. Start with the left side of $P(n+1)$ (or whatever the goal is), apply the inductive hypothesis once, and algebraically arrive at the right side. Never write the target as an intermediate line and work toward it from both sides.

Remark 6.4.28 (Mistake 3: Unstated inductive hypothesis). The most common error in practice is writing “assume $P(n)$ ” without saying what $P(n)$ actually says. This is a sign that $P(n)$ has not been written out as a full sentence.

The effect is that the inductive step becomes vague at the crucial moment: the student needs to use the inductive hypothesis but cannot cite it precisely, so they either paraphrase it loosely (which may introduce errors) or implicitly use a stronger version than what was stated.

Write $P(n)$ as a complete sentence at the top of the proof. Then write “assume $P(n)$: [full sentence].” This forces precision and prevents the most common errors in the body of the proof.

Remark 6.4.29 (Mistake 4: Wrong base case). Induction proves $P(n)$ for all $n \geq n_0$, where n_0 is whatever value you use as the base case. If the claim is only true for $n \geq 2$, the base case $n = 0$ will fail, and the proof is invalid.

Always check: what is the smallest n for which the claim is true? That is your base case. Do not default to $n = 0$ or $n = 1$ without verifying that the claim holds there. For divisibility statements, $n = 0$ often gives $0 = 0 \cdot k$, which works. For growth bounds, $n = 1$ or $n = 2$ may be needed. For Fibonacci identities, specific small values may require individual checking.

Remark 6.4.30 (Mistake 5: Using strong induction where weak suffices, and vice versa). Strong induction is not automatically better than weak. A proof that invokes the full strong hypothesis “ $P(k)$ for all $k < n$ ” when only $P(n - 1)$ is actually used is unnecessarily complicated and obscures the proof’s logic.

Conversely, a proof that tries to use weak induction on a recursion requiring two prior terms will stall at the step.

The diagnostic: write the step and check explicitly which prior values you use. If only $P(n)$, use weak. If $P(k)$ for some $k < n$ that is not $n - 1$, use strong.

Remark 6.4.31 (Mistake 6: Forgetting vacuity in strong induction). In strong induction, the case $n = 0$ has a vacuously true hypothesis $\forall k < 0, P(k)$, since no natural number is less than 0. This means $P(0)$ must be proved *directly*, not from the inductive hypothesis. Many students write “by the inductive hypothesis applied to $n - 1$ ” for $n = 0$, which refers to $n - 1 = -1$, a non-existent value.

The base case in strong induction is always proved from scratch. The strong hypothesis is only available for $n \geq 1$.

6.5 Algebraic Tactics

The DU/TA/AM Framework. Every line of an algebraic proof is justified by exactly one of three sources. This framework makes that obligation explicit and provides the tagging system used throughout these notes.

The Three Line Tags

- **DU (Definition / Unpack).** The step applies a definition or unpacks a named concept into its explicit conditions. *Example:* expanding “ e is an identity” into “ $ea = ae = a$ for all $a \in G$ ”.
- **TA (Theorem / Axiom).** The step cites a previously proved proposition or a structural axiom (group axiom, ring axiom, field axiom, Peano axiom). *Example:* citing G1 (associativity) to regroup $(ab)c = a(bc)$.
- **TA (Algebraic Manipulation, abbreviated AM).** The step performs a computation whose justification is itself a combination of DU and TA moves, compressed for readability. *Example:* simplifying $a \cdot e = a$ in one step after the identity axiom has already been cited.

Remark 6.5.1 (Why the framework matters: finding where proofs break). The DU/TA/AM system serves a diagnostic purpose. Every unjustified step in a proof is either a DU step that has not been taken (some definition has not been unpacked) or a TA step that has not been cited (some axiom or theorem is being used silently).

When an algebraic proof stalls, the right question is: which definition have I not yet unpacked? Which axiom am I using but not citing? The tags make these omissions visible.

In practice, stalling almost always traces to a definition that has been left in its named form (“ G is a group”) when it needed to be expanded into operative conditions (“associativity holds; an identity exists; every element has an inverse”). Expand the definition and the stuck proof typically becomes unstuck.

Remark 6.5.2 (The hierarchy: DU and TA are atomic; AM is shorthand). DU and TA are the atomic justifications. There is nothing more basic. AM is not a separate category of reasoning — it is a compressed sequence of DU and TA steps that have been verified and labeled as a single unit for concision.

A fully formal proof contains only DU and TA steps, explicitly tagged. The three-column proof format used in this project — tag in the left column, statement in the middle, commentary in the right — enforces explicit justification of every line and is the recommended format while you are learning.

As fluency develops, AM steps can absorb more and more of the lower-level manipulations. But the underlying DU/TA structure is always present, even when not written. The ability to decompose any AM step into its DU and TA components is the hallmark of genuine understanding.

Remark 6.5.3 (A proof is a chain of citations). The deepest way to understand the DU/TA/AM system is this: a mathematical proof is a sequence of statements, each of which is justified by citing something that is already known (a definition, an axiom, or a previously proved theorem). The proof is valid if and only if every step has a valid citation.

When you read a proof and feel uncertain whether it is correct, the right move is to label every step. If you can assign a DU or TA justification to every line, the proof is valid. If any line has no justification, there is a gap.

This is not just a learning technique. Working mathematicians who find a suspicious step

in a proof do exactly this: they ask “what justifies this?” and try to answer. If no answer is available, the step is an error.

Catalog of Algebraic Tactics. The following catalog lists the specific algebraic moves available once a proof is in progress and the architecture has been chosen. Each entry states the tactic, its precondition (what must already be established or assumed), and its effect (what it achieves in the proof).

Tactic	Precondition	Effect
Multiply by inverse	$a \neq 0$ in a field; or $a \in G$ in a group	Multiply both sides by a^{-1} ; use associativity to collapse $a^{-1}a = e$; isolates the target variable.
Cancellation	$ac = bc$ (or $ca = cb$) in a group or integral domain	Conclude $a = b$. In a group: multiply by c^{-1} . In an integral domain: cite no-zero-divisors.
Distributivity bridge	Multiplication by 0, -1 , or a negative element	Rewrite the product as a sum using distributivity, then use additive group structure to collapse. Crosses from $\times$ to $+$.
Add zero	Need to introduce a term without changing the expression	Write $a = a + 0 = a + (b + (-b))$; regroup to expose the needed structure.
Satisfy-and-cite	A uniqueness theorem for some named object b is in scope	Show a satisfies b ’s defining property; cite uniqueness; conclude $a = b$. Avoids assume-two-and-compare entirely.
Double negation	$-(-a)$ appears or is the goal	$-a$ is the unique element x with $a + x = 0$; $-(-a)$ satisfies this; by uniqueness, $-(-a) = a$.
Absorb into axiom	A subexpression matches the LHS of an axiom	Rewrite using the axiom. Tag as TA .

Remark 6.5.4 (The distributivity bridge in detail). The distributivity bridge is the central tactic for ring and field proofs involving zero and negatives. Every instance of “ $a \cdot 0 = 0$ ” and “ $a(-b) = -(ab)$ ” uses it. Once you have seen the pattern, it is immediately recognizable.

The pattern always has four steps:

1. Introduce $a \cdot 0$ or $a \cdot (-b)$ into the argument.
2. Expand: write $0 = x + (-x)$ or $-b = b + (-b) + (-b)$ using the additive inverse definition.

3. Apply distributivity: $a(x + y) = ax + ay$.
4. Use uniqueness of the additive inverse to identify the result with $-(ab)$ or 0.

The reason this requires distributivity as a bridge is that the additive structure (with its identity 0 and its inverses $-a$) and the multiplicative structure (with its products ab) are separate. The only connection between them is distributivity. To get from the multiplicative side ($a \cdot 0$) to the additive side ($0 = 0 + 0$), you must cross the bridge.

Remark 6.5.5 (Cancellation: two theorems that must not be conflated). “Cancellation” names two distinct results with different preconditions.

Group cancellation. In any group, $ac = bc \Rightarrow a = b$. Proved by multiplying both sides on the right by c^{-1} , which always exists in a group. The hypothesis $c \neq 0$ is not needed because inverses always exist in a group.

Domain cancellation (or integral domain cancellation). In an integral domain, $ac = bc$ and $c \neq 0$ imply $a = b$. Proved by rewriting as $(a - b)c = 0$ and invoking the no-zero-divisors property. The hypothesis $c \neq 0$ is essential: without it, cancellation can fail (e.g., in $\mathbb{Z}_6$, $2 \cdot 3 = 4 \cdot 3$ but $2 \neq 4$).

The error to avoid: using group cancellation in an integral domain argument (it works but uses stronger machinery than needed and may not be available), or trying to use domain cancellation without verifying $c \neq 0$.

Remark 6.5.6 (Add zero: the simplest but most overlooked tactic). The add-zero tactic sounds trivial but appears in almost every algebraic proof where an intermediate term needs to be introduced without changing the value of an expression. The template is:

$$a = a + 0 = a + (b + (-b)) = (a + b) + (-b).$$

This exposes $a + b$ as a subexpression of a , which can then be worked with. It is the algebraic equivalent of adding and subtracting the same term in a real analysis inequality argument.

When you need a term to appear in an expression and cannot see how to introduce it, ask: can I write some zero as $x + (-x)$ and distribute?

6.6 Proof Strategies Reference

Lookup Table: Stuck Situations and Strategies.

Remark 6.6.1 (How to use this table). When progress on a proof stalls, locate the row that describes your situation. The middle column names the strategy. The right column gives the specific first move and a cross-reference to the section where that strategy is developed in detail.

This table is not a substitute for understanding the strategies; it is a navigation aid for when you know what you are trying to do but have forgotten the precise machinery. The full explanations and worked examples are in the sections cited.

Stuck because...	Strategy	First move / See
I need to prove P but don't know where to start	Two-question check	Write the logical form of P . Consult the statement map. §6.1
I need to show two things are equal and one is uniquely defined	Satisfy-and-cite	Show the first satisfies the definition of the second. Cite the uniqueness theorem. §6.1
I need to show something is the only object of its kind	Assume-two-and-compare	"Let x, y both satisfy the definition. Show $x = y$." §6.3
I have a conditional $P \Rightarrow Q$ and P is hard to use forward	Contrapositive	"Assume $\neg Q$." Derive $\neg P$. §6.3
I have a conditional and the conclusion seems impossible to reach directly	Contradiction	"Assume P and $\neg Q$." Derive any contradiction. §6.3
The claim splits naturally into a finite number of cases	Case analysis	List all cases explicitly. Verify exhaustiveness first. §6.3
The claim is about all $n \in \mathbb{N}$	Weak induction	Write $P(n)$ as a full sentence. Prove the base case directly. §6.4
The inductive step needs $P(k)$ for some $k < n$ (not just $n - 1$)	Strong induction	Replace "assume $P(n)$ " with "assume $P(k)$ for all $k < n$." §6.4
The claim is about all $n \geq n_0$ and feels easier to negate	Minimal counterexample	"Let $S = \{n : \neg P(n)\} \neq \emptyset$. Let $m = \min S$." §6.4
I need to prove $a \cdot 0 = 0$ or $a(-b) = -(ab)$ in a ring	Distributivity bridge	Write $0 = x + (-x)$; apply distributivity; use uniqueness of additive inverse. §6.5
I have $ac = bc$ and need $a = b$	Cancellation	Group: multiply by c^{-1} . Domain: write $(a - b)c = 0$; cite no zero divisors. §6.5
I need to prove two sets are equal	Double inclusion	Prove $A \subseteq B$: let $x \in A$ arbitrary, derive $x \in B$. Then prove $B \subseteq A$. §6.1
A definition has not been unpacked	DU move	Identify the named concept and write out its definition. Tag the line DU. §6.5
A step uses an axiom or theorem	TA move	Cite the axiom or theorem explicitly

Remark 6.6.2 (Cross-references to primary worked examples). The following table maps each strategy to a worked example in these notes where it is the primary tool.

Strategy	Primary worked examples
Assume-two-and-compare	Uniqueness of group identity (this section)
Satisfy-and-cite	$(ab)^{-1} = b^{-1}a^{-1}$; $a(-b) = -(ab)$ (this section)
Distributivity bridge	$a \cdot 0 = 0$; $a(-b) = -(ab)$ (Tactics section)
Cancellation (group)	Group left-cancellation (Tactics section)
Cancellation (domain)	Integral domain cancellation (Tactics section)
Weak induction	Sum formula $\sum k = n(n+1)/2$; $3 \mid 4^n - 1$
Strong induction	Fibonacci bound; prime factor theorem
Minimal counterexample	$2^n > n$
Contrapositive	$n^2 \text{ even} \Rightarrow n \text{ even}$
Contradiction	$\sqrt{2} \notin \mathbb{Q}$

6.7 Analysis: What Changes and Why

Why Analysis Proofs Feel Different. Everything in the preceding sections applies to mathematics broadly: direct proof, induction, contradiction, and the rest are the universal grammar of mathematical argument. But real analysis — sequences, metric spaces, limits, continuity — introduces a particular *flavour* of proof work that surprises many students when they first encounter it.

This section prepares you for that surprise before you arrive at Volume III. The goal is not to teach you analysis; it is to give you the framework so that when you get there, the proofs do not feel alien.

Remark 6.7.1 (The shift from computation to verification). In calculus, most tasks are *computational*: differentiate this function, evaluate this integral, find this limit by l'Hôpital's rule. You apply known procedures and the answer is a number or a function.

In analysis, most tasks are *verificational*: prove that this sequence converges, show that this space is complete, establish that this function is continuous. The answer is a logical argument. There is no procedure to apply; there is a structure to find.

This shift is not just psychological. It changes what “progress” means. In a computation, you know you are making progress when the expressions simplify. In a proof, you know you are making progress when the definitions are unpacked, the hypotheses are applied, and the conclusion comes into reach. The habits of mind required are different.

Remark 6.7.2 (Definitions are not background; they are the machinery). The single most important habit in analysis is **returning to definitions**. Every concept — convergence, Cauchy, compactness, continuity — has a precise definition in terms of quantifiers and in-

equalities. That definition is not background knowledge. It is the working machinery of every proof involving that concept.

Here is the canonical example. To prove that $1/n \rightarrow 0$, a calculus student says: “obviously it approaches zero.” An analysis student unpacks the definition:

For every $\varepsilon > 0$, there exists $N \in \mathbb{N}$ such that $n \geq N$ implies $|1/n - 0| < \varepsilon$.

Then they find N : choose $N > 1/\varepsilon$ (possible by the Archimedean property). For $n \geq N$: $1/n \leq 1/N < \varepsilon$.

The proof is three lines, and every line is either the definition (unpacked), the Archimedean property (a theorem), or arithmetic. There is no mystery. The entire proof is the definition plus two lines of calculation. This is the prototype of an analysis proof.

The lesson: when a proof stalls, the first question to ask is always: *which definition have I not yet unpacked?*

Remark 6.7.3 (Analysis proofs are structured, not creative). Here is perhaps the most liberating thing to understand about analysis: **proofs are far less creative than they look.**

Experienced analysts do not invent proofs from scratch. They recognize the type of exercise, recall the standard move for that type, and apply it. The vocabulary of standard moves is small — twelve moves cover virtually all of the proofs in a first course in metric spaces. The creativity, if any, is in recognizing which move to apply.

This is the same observation that governs the Architecture section of this chapter (Section 6.1), scaled down to the specific landscape of analysis. The statement map tells you the global structure. In analysis, a more fine-grained map is available: there are specific named moves for specific recurring situations. Learning to name them is the same skill as learning to read a statement and immediately identify its proof type.

The rest of this section makes that vocabulary explicit.

The Six Exercise Types. Every exercise in sequences and metric spaces belongs to one of six structural types. Identifying the type is the first step before writing anything. The type determines the shape of the solution — not the details, but the overall structure.

Quick Reference: The Six Types

Type	Signal words	Shape	First move
Proof	<i>Prove, show, establish</i>	Hypotheses to conclusion	Unpack all definitions
Verification	<i>Verify that ... is a metric, show ... satisfies</i>	Check each condition	List every condition
Prove/Counterexample	<i>True? Prove or find a counterexample</i>	Decide, then justify	Decide first
Construction	<i>Find, construct, exhibit, give an example of</i>	Name the object; verify it	Name the object
Computation	<i>Compute, calculate, find the diameter of</i>	Evaluate from a definition	Write the definition
Characterization	<i>If and only if, characterize all ... that</i>	Two separate directions	Split two directions

Remark 6.7.4 (Type 1 (Proof): you are told the statement is true). In a proof exercise, the statement is true and your job is to trace the logical path from the hypotheses to the conclusion. There is no choice to make about truth value.

The opening move is always the same: unpack every definition in the hypotheses and in the conclusion. The definitions turn abstract concepts into concrete quantifier-and-inequality statements that can be manipulated. Without this step, you have nothing to work with.

Example. *Prove that every convergent sequence is Cauchy.* Unpack “convergent”: $\forall \varepsilon > 0 \exists N$ s.t. $n \geq N \Rightarrow d(x_n, p) < \varepsilon$. Unpack “Cauchy”: $\forall \varepsilon > 0 \exists M$ s.t. $m, n \geq M \Rightarrow d(x_m, x_n) < \varepsilon$. Now the proof goal is visible: use the convergence condition (which controls $d(x_n, p)$) to establish the Cauchy condition (which asks for control over $d(x_m, x_n)$). The triangle inequality bridges them.

Remark 6.7.5 (Type 2 (Verification): check every condition). In a verification exercise, a specific named object is given and you must confirm it satisfies every condition of a definition. The definition has a finite list of conditions. Your job is to work through every one, explicitly and separately.

The error students make: checking four out of five conditions and writing QED. A verification is complete only when every condition is checked. When you begin a verification, write the complete list of conditions first. Then work through them in order.

Example. *Verify that the discrete metric is a metric.* A metric has four conditions: non-negativity, identity of indiscernibles, symmetry, triangle inequality. List all four. Check each one. The triangle inequality requires a case split (on whether $x = z$ or $x \neq z$), which is itself a standard move in analysis.

Remark 6.7.6 (Type 3 (Prove or counterexample): decide before you argue). This is the most dangerous type for students who rush. The exercise gives you a statement and asks you to determine its truth value, then justify. If you skip the determination stage and try to prove a false statement, you will waste effort and grow confused.

Stage 1 is always: decide. Test the statement on simple cases — $\mathbb{R}$ with the standard metric first, then discrete spaces, then other special cases. One counterexample falsifies; failure to find a counterexample after testing several cases suggests the statement is true.

Stage 2: justify. If true, write a proof. If false, exhibit a specific counterexample and verify it fails.

Example. *Is the open ball always equal to the closed ball?* Test in $\mathbb{R}$: $B(0, 1) = (-1, 1)$ and $C(0, 1) = [-1, 1]$. The point -1 is in $C(0, 1)$ but not $B(0, 1)$. False. Exhibit this specific counterexample in the solution, with the witness (-1) and the calculation $(d(-1, 0) = 1, \text{ so } -1 \in C(0, 1) \text{ and } -1 \notin B(0, 1))$.

Remark 6.7.7 (Type 4 (Construction): name first, verify second). A construction exercise asks you to produce an object with given properties. The solution has two parts: stating the object, and verifying it has every required property. Both parts are required.

State the object completely and unambiguously before any verification. If you cannot state it clearly, you do not yet have a solution. Then verify every required property from scratch — do not assume properties that were not established.

Example. *Find a metric on $\mathbb{N}$ such that $\mathbb{N}$ is bounded.* The idea: embed $\mathbb{N}$ into $(0, 1] \subset \mathbb{R}$ via $n \mapsto 1/n$ and pull back the metric. Define $d(m, n) = |1/m - 1/n|$. Then verify: (i) it is a metric (four conditions), and (ii) it is bounded ($d(m, n) \leq 1$ for all m, n).

Remark 6.7.8 (Type 5 (Computation): the definition is the answer). In a computation exercise, a specific numerical or set-theoretic value is asked for. The answer always comes from writing out the relevant definition as a supremum, infimum, or set, and then evaluating it.

Do not skip straight to the answer. Write the definition first. This is not just for clarity — it prevents errors that arise from misremembering what is being computed.

Example. *Compute $\text{diam}([a, b])$.* Write: $\text{diam}([a, b]) = \sup\{|x - y| : x, y \in [a, b]\}$. Upper bound: $|x - y| \leq b - a$ for all $x, y \in [a, b]$. Attained: $|b - a| = b - a$. Therefore $\text{diam}([a, b]) = b - a$.

Remark 6.7.9 (Type 6 (Characterization): two proofs, not one). A characterization exercise asks for an equivalence or an if-and-only-if. The solution is always two separate proofs, labeled $(\Rightarrow)$ and $(\Leftarrow)$. Label them before writing a single line.

The two directions often require completely different techniques and different amounts of work. The $(\Rightarrow)$ direction may be three lines while the $(\Leftarrow)$ direction requires a sequence argument. Writing them separately prevents the error of running arguments in both directions simultaneously and losing track of which assumptions are in play.

Example. *Show that a sequence converges if and only if it is eventually constant (in the discrete metric).* $(\Leftarrow)$: Suppose eventually constant; then convergence holds trivially. $(\Rightarrow)$: Suppose it converges; apply the definition with $\varepsilon = 1/2$ and use the fact that distances in the discrete metric are either 0 or 1.

The Twelve Canonical Proof Moves. A *proof move* is a recurring logical or computational technique that appears across many different proofs in analysis. Once you learn to name these moves, reading and writing proofs becomes a matter of recognizing which move applies — not inventing an argument from scratch.

The moves below are not tricks. Each one encodes a genuine mathematical idea. But once the idea is understood, the move can be deployed almost mechanically in the right

situations.

The Twelve Moves at a Glance

#	Move	Use when ...	Level
1	Unpack a definition	starting any proof	L1
2	Triangle inequality	need to bound $d(x, z)$ via y	L2
3	ε -splitting	triangle ineq. produces a sum	L2
4	Tail argument	combining two N 's	L3
5	Case splitting	discrete or binary structure	L2
6	Choose N explicitly	ε - N arguments	L3
7	Contradiction	uniqueness or non-existence	L4
8	Subsequence extraction	sequence misbehaves globally	L3
9	Squeeze (sandwich)	bound $d(x_n, p)$ from above	L2
10	Inherited metric / embedding	constructing a new metric	L4
11	Open cover / finite subcover	compactness arguments	L4
12	Diagonal argument	simultaneous subsequence conditions	L3/4

Remark 6.7.10 (Move 1: Unpack a definition). This is always the first move. Every analysis concept — convergence, Cauchy, compactness, closure, diameter — is a compressed form of a precise statement with quantifiers and inequalities. Unpacking the definition turns the abstract concept into something you can work with.

Without this step, you have named objects but no machinery. With it, you have the raw material for every subsequent move.

Template. Write out the definition of every term in the hypothesis and in the goal. The hypotheses become your assumptions; the goal becomes your target inequality or quantifier statement.

Remark 6.7.11 (Moves 2 and 3: Triangle inequality and ε -splitting). These two moves almost always appear together. The triangle inequality lets you insert an intermediate point y and split $d(x, z)$ into $d(x, y) + d(y, z)$. ε -splitting means choosing the bounds on each piece so they add up to ε .

The intermediate point y is not arbitrary: it is chosen to be something you already have control over (a limit, a fixed center, a sequence element). The fractions $\varepsilon/2$, $\varepsilon/3$ are chosen *before* picking N , so that the final bound comes out to ε exactly.

Template. When you need $d(x_m, x_n) < \varepsilon$ and you know each term is close to a limit p : insert p via the triangle inequality, budget $\varepsilon/2$ for each piece, and choose N so both pieces are bounded.

Remark 6.7.12 (Moves 4 and 6: Tail argument and choosing N). The ε - N definition of convergence requires you to *produce* an N . This is Move 6: work backwards from the target inequality to determine what N must satisfy, then assert its existence (typically by the Archimedean property: for any $\varepsilon > 0$ there is an integer greater than $1/\varepsilon$).

Move 4 (the tail argument) handles the situation where two separate conditions each give their own N_i . The resolution: take $N = \max(N_1, N_2, \dots)$. For $n \geq N$, all conditions hold simultaneously.

These two moves are the engine of every ε - N argument. They appear in virtually every proof about convergent sequences.

Remark 6.7.13 (Move 5: Case splitting in analysis). Case splitting is the same structural technique described in Section 6.3, now applied to the specific context of metric spaces. The most common case split in metric space proofs is: $x = y$ (distance is 0) versus $x \neq y$ (distance is 1 in the discrete metric, or some positive value in other spaces).

Case splitting is particularly important in verification exercises: the triangle inequality almost always requires splitting on whether the two outer points coincide.

Remark 6.7.14 (Move 7: Contradiction in analysis). In analysis, contradiction is used most characteristically for *uniqueness* results: suppose two limits exist; assume they are different; derive a contradiction from the triangle inequality and the convergence hypothesis.

The template: suppose the goal fails (e.g., two limits exist with $d(x, y) > 0$). Choose $\varepsilon = d(x, y)/2$. Then for large enough n , both $d(x_n, x) < \varepsilon$ and $d(x_n, y) < \varepsilon$. The triangle inequality gives $d(x, y) \leq 2\varepsilon = d(x, y)$, a contradiction.

Notice: contradiction here uses the triangle inequality (Move 2) and the tail argument (Move 4) internally. Proof moves nest and combine.

Remark 6.7.15 (Moves 8 and 12: Subsequence extraction and diagonal argument). Subsequence extraction is the operation of selecting indices $n_1 < n_2 < \dots$ so that the restricted sequence $\{x_{n_k}\}$ has some desired property (being monotone, being close to a given point, etc.). It is a Level 3 operation: you are selecting from an infinite sequence, which requires genuine sequence machinery.

The diagonal argument applies subsequence extraction infinitely many times and diagonalizes the result: the k -th term of the diagonal sequence is $x_k^{(k)}$, where $\{x_n^{(k)}\}$ is the k -th extracted subsequence. The diagonal sequence satisfies all conditions simultaneously. This is the technique behind Arzelà–Ascoli and related compactness results.

Remark 6.7.16 (Move 9: The squeeze (sandwich)). If you can show $0 \leq d(x_n, p) \leq c_n$ where $c_n \rightarrow 0$, then $x_n \rightarrow p$ follows immediately: any ε that works for c_n also works for $d(x_n, p)$.

The squeeze is particularly useful when $d(x_n, p)$ is hard to bound directly but can be dominated by a simpler sequence whose convergence is known.

Remark 6.7.17 (Move 10: Inherited metric / embedding). To construct a metric on a new space X , it is often easiest to find an injective function $f : X \rightarrow Y$ into a space Y whose metric is known, and define $d_X(x, z) := d_Y(f(x), f(z))$. All four metric axioms are then inherited from d_Y via f . The only things to verify are injectivity of f (to ensure the identity axiom) and whatever additional properties are required.

This is a Level 4 move because it operates on the global structure of the space: you are choosing an ambient space and an embedding, not just manipulating inequalities.

Remark 6.7.18 (Move 11: Open cover / finite subcover). Compactness says: every open cover of a compact set K has a finite subcover. The power of this is that it converts infinite problems into finite ones. If you need a single bound δ that works everywhere on K , you find a δ_x at each x (infinitely many), then extract a finite subcover and take the minimum

of finitely many δ_x . A finite minimum exists; a minimum over infinitely many values might not.

This move appears in all compactness arguments: uniform continuity on compact sets, compactness implies boundedness, compactness implies completeness in complete metric spaces.

The Four-Layer Hierarchy. The twelve moves are not all at the same level. They form a four-layer hierarchy that reflects the logical structure of metric space proofs: each layer depends on the one below it, and together the four layers describe a progression from the most basic (unpacking definitions) to the most structural (global topological reasoning).

Understanding the hierarchy is not an organizational convenience. It tells you something deep: *why* metric spaces are defined the way they are, and why nearly every proof in the subject follows the same basic upward shape. Once you see this, metric space theory stops looking like a collection of unrelated tricks and starts looking like a small, elegant machine built from a few core mechanisms.

The Four Levels				
L	Name		Moves	What it does
L1	Definition	Expansion	Move 1	Turns abstract concepts into inequalities
L2	Inequality	Mechanics	Moves 2, 3, 5, 9	Combines and bounds distances
L3	Sequence Control		Moves 4, 6, 8, 12	Controls asymptotic and tail behavior
L4	Structural / logical	Topological	Moves 7, 10, 11	Reasons about the space as a whole

Remark 6.7.19 (Level 1: Definition Expansion — the foundation of everything). Every other move operates on inequalities. Those inequalities do not appear until you unpack a definition. Before Level 1, you have a statement about abstract objects. After Level 1, you have quantifiers and specific inequality conditions that can be manipulated.

This is why Level 1 is always the first step in any analysis proof, regardless of how complex the argument becomes. The inequality $d(x_n, p) < \varepsilon$ does not exist in the proof until you write it down. You write it down by unpacking the definition of convergence.

What Level 1 produces:

- “ $x_n \rightarrow p$ ” $\mapsto \forall \varepsilon > 0 \exists N$ s.t. $n \geq N \Rightarrow d(x_n, p) < \varepsilon$
- “ $\{x_n\}$ is Cauchy” $\mapsto \forall \varepsilon > 0 \exists M$ s.t. $m, n \geq M \Rightarrow d(x_m, x_n) < \varepsilon$
- “ K is compact” $\mapsto$ every open cover of K has a finite subcover
- “ E is closed” $\mapsto$ every convergent sequence in E converges to a point of E

The abstract word is a compressed form of the inequality. Unpacking it is not simplification; it is *activation*.

Remark 6.7.20 (Level 2: Inequality Mechanics — the algebra of metric spaces). Once Level 1 has produced inequalities, Level 2 manipulates them. This is the algebra of metric spaces: you combine, bound, split, and chain inequalities to move from what you know toward what you need.

The triangle inequality is a *metric axiom* for a reason: it was included in the definition of a metric precisely because it is the tool needed to chain distance bounds. Without it, you cannot get from control over $d(x, y)$ and $d(y, z)$ to control over $d(x, z)$. The axiom is there because it enables Level 2.

Notice that Level 2 does not involve sequences, indices, or tails. It is purely about bounding distances between specific points. The sequence machinery comes at the next level.

Remark 6.7.21 (Level 3: Sequence Control — the engine of limit arguments). Analysis is fundamentally about sequences approaching limits, distances becoming arbitrarily small, behavior stabilizing as an index grows large. Level 3 controls this asymptotic behavior.

The key idea: sequence control is about managing the word “eventually.” The definition of convergence says that for large enough n , the distance $d(x_n, p)$ is small. The tail argument combines multiple “large enough n ” conditions by taking the maximum index. Choosing N explicitly produces the specific threshold. Subsequence extraction selects which terms of the sequence to keep.

These moves require genuine sequence machinery — you are reasoning about indexed infinite processes, not just about individual distances between points. That is what places them at Level 3 rather than Level 2.

Remark 6.7.22 (Level 4: Structural / Topological — global reasoning). Level 4 moves operate on the global structure of the metric space, not on individual sequences or inequalities. Contradiction reasons about the logical structure of the whole proof. Embedding reasons about how the space sits inside a larger ambient space. Compactness reasons about the global covering properties of the space.

Compactness is the flagship Level 4 property. It converts infinite problems into finite ones: an infinite family of open sets reduces to a finite subcover, and a finite collection has a minimum, a maximum, a single controlling constant. This is the most powerful tool in metric space theory.

Level 4 and Level 3 interact bidirectionally. Sequential compactness (every sequence has a convergent subsequence) is a Level 3 characterization of a Level 4 property. Many deep results in analysis are precisely about these interactions.

Remark 6.7.23 (Reading a proof using the hierarchy). When you read a proof in the textbook, label each step by level. This is not a mechanical exercise; it reveals the logical architecture and tells you why each step appears where it does.

Here is the proof that convergent implies Cauchy, labeled:

Let $\varepsilon > 0$. *[L1: introduce ε from the Cauchy definition]*

Since $x_n \rightarrow p$, there exists N such that $n \geq N \Rightarrow d(x_n, p) < \varepsilon/2$. *[L3: choose N from the convergence hypothesis]*

For $m, n \geq N$:	<i>[L3: tail condition]</i>
$d(x_m, x_n) \leq d(x_m, p) + d(p, x_n)$	<i>[L2: triangle inequality]</i>
$< \varepsilon/2 + \varepsilon/2 = \varepsilon.$	<i>[L2: ε-splitting]</i>
Hence $\{x_n\}$ is Cauchy.	<i>[L1: conclusion matches Cauchy definition]</i>

This proof uses Levels 1, 2, and 3, but not Level 4. It is a local argument about sequences, not a global structural argument. To reach Level 4, you would need compactness or an embedding argument.

The levels a proof reaches tell you its depth and what tools it uses. A verification exercise stays at Levels 1–2. A limit argument reaches Level 3. A compactness argument reaches Level 4.

Remark 6.7.24 (Why the metric axioms are the axioms they are). The metric axioms — non-negativity, identity of indiscernibles, symmetry, triangle inequality — are not arbitrary. They are exactly the conditions needed to make Level 2 possible.

Without the triangle inequality, you cannot chain distance bounds. Without identity of indiscernibles, the notion of a limit point degenerates. Without symmetry, the definitions of balls and convergence become asymmetric and unworkable.

The axioms are the *minimal conditions on a distance function that support the full four-level proof hierarchy*. When you study a new property of metric spaces, ask: which level does it live at? The answer tells you what tools to expect and what kind of proofs will be required.

How to Read an Analysis Exercise Before Writing Anything. The preceding sections have given you the vocabulary: six exercise types, twelve proof moves, four levels. This section shows how to put them together in the five minutes before you write your first symbol.

Pre-Writing Checklist

1. **Identify the type.** Which of the six types is this: Proof, Verification, Prove-or-Counterexample, Construction, Computation, Characterization? Write the type at the top of your scratch paper before anything else.
2. **State the hypotheses and the goal.** In plain English, what are you assuming? What are you trying to show? Do not proceed until both are stated explicitly.
3. **Unpack every definition in the goal.** If the goal involves a named concept — convergence, compactness, closed, bounded — write out its definition in full. The unpacked goal is your actual target.
4. **Identify the key move.** Which of the twelve moves is most likely to bridge the hypothesis to the unpacked goal? Which level does that move belong to? Knowing the level tells you roughly how deep the proof will go and what other moves will likely appear alongside it.
5. **Sketch the shape in words.** Write in plain language: “I will choose $\varepsilon > 0$. I will use convergence of $\{x_n\}$ to find N_1 . I will use convergence of $\{y_n\}$ to find N_2 . I will take $N = \max(N_1, N_2)$. Then I will apply the triangle inequality.” Only after this sketch do you fill in the symbols.

Remark 6.7.25 (The shape tells you what you need before you need it). Step 5 is the most important and the most skipped. Students who jump directly to symbols frequently paint themselves into corners: they choose N for one condition and then need it to satisfy a second condition that their choice does not support.

If you write the shape of the proof in words first, you see *how many conditions N must satisfy* before you commit to a specific value. Then you choose N to satisfy all of them simultaneously. The tail argument tells you how: take the maximum of however many separate N_i ’s the conditions require.

The shape also tells you how many terms the triangle inequality will produce and therefore what the right ε -splitting fractions are. These are not things you discover mid-proof; they are visible from the sketch.

Remark 6.7.26 (Common proofs and their shapes). The following table summarizes the shape of the most common proof patterns in a first analysis course, listed by the move sequence and the levels they traverse.

Goal	Move sequence	Levels
Convergent $\Rightarrow$ Cauchy	Unpack both definitions; triangle inequality; ε -splitting; tail argument (max of two N 's).	L1–L3
Uniqueness of limits	Contradiction (assume two limits); triangle inequality; choose $\varepsilon = d(x, y)/2$; tail argument.	L2–L4
Convergent $\Rightarrow$ bounded	Unpack convergence; choose $\varepsilon = 1$; tail argument (bound $n \geq N$); take max of first N terms.	L1–L3
Verify d is a metric	List four conditions; check non-negativity, identity, symmetry directly; case split for triangle inequality.	L1–L2
Compute diameter of $[a, b]$	Write diameter as supremum; give upper bound; exhibit achieving pair.	L1–L2

Remark 6.7.27 (A note on templates). For each of the six exercise types, there is a standard template structure. These templates live in Appendix A of the companion *Primer for Real Analysis Exercises*, which you will receive at the start of Volume III. The templates are starting-point structures, not scripts to be memorized. Their purpose is to ensure that every solution has all required parts and that no structural element is accidentally omitted.

What matters at this stage is the vocabulary: six types, twelve moves, four levels. You now have the map. Volume III will fill in the terrain.

Chapter 7

Set Theory



7.1 Model of Set Theory

Theory (ZF, ZFC)
Depends on: <code>theory:first-order-logic</code>
Language $\mathcal{L}_\in = \{\in\}$ First-order language with equality. The symbol $\in$ is binary. Equality is logical.
Presented Theory $\text{ZF} := \text{Cn}_\vdash(\Sigma_{\text{ZF}}), \quad \text{ZFC} := \text{Cn}_\vdash(\Sigma_{\text{ZF}} \cup \{\text{AC}\}).$

Axioms and Laws

1. 1. Extensionality

$$\forall x \forall y [\forall z (z \in x \leftrightarrow z \in y) \rightarrow x = y].$$

2. 2. Empty Set

$$\exists x \forall y \neg(y \in x).$$

3. 3. Pairing

$$\forall x \forall y \exists z \forall w (w \in z \leftrightarrow (w = x \vee w = y)).$$

4. 4. Union

$$\forall x \exists y \forall z [z \in y \leftrightarrow \exists w (w \in x \wedge z \in w)].$$

5. 5. Power Set

$$\forall x \exists y \forall z [z \in y \leftrightarrow \forall w (w \in z \rightarrow w \in x)].$$

6. 6. Infinity

$$\begin{aligned} \exists x \left[\exists y (y \in x \wedge \forall w \neg(w \in y)) \right. \\ \left. \wedge \forall y (y \in x \rightarrow \exists z (z \in x \wedge \forall w (w \in z \leftrightarrow (w \in y \vee w = y)))) \right]. \end{aligned}$$

7. 7. Separation Schema For each formula $\varphi(z, \bar{w})$ in which y is not free:

$$\forall \bar{w} \forall x \exists y \forall z [z \in y \leftrightarrow (z \in x \wedge \varphi(z, \bar{w}))].$$

8. 8. Replacement Schema For each formula $\varphi(x, y, \bar{w})$ in which B is not free:

$$\begin{aligned} \forall \bar{w} \forall A \left[\forall x (x \in A \rightarrow \exists! y \varphi(x, y, \bar{w})) \right. \\ \left. \rightarrow \exists B \forall y (y \in B \leftrightarrow \exists x (x \in A \wedge \varphi(x, y, \bar{w}))) \right]. \end{aligned}$$

9. 9. Foundation

$$\forall x \left[\exists y (y \in x) \rightarrow \exists y (y \in x \wedge \neg \exists z (z \in x \wedge z \in y)) \right].$$

Layer Delta

AC

$$\begin{aligned} \forall x \left[(\forall y (y \in x \rightarrow \exists z z \in y)) \right. \\ \wedge \forall y \forall y' ((y \in x \wedge y' \in x \wedge y \neq y') \rightarrow \neg \exists z (z \in y \wedge z \in y')) \\ \left. \rightarrow \exists c \forall y (y \in x \rightarrow \exists! z (z \in y \wedge z \in c)) \right]. \end{aligned}$$

Model (Set Theory)

Model 7.1.1 (Set Theory Model). **Depends on:** `model:first-order-logic`; `bridge:free-sigma-algebra-evaluation`; `theory:set-theory`; `bridge:consequence`

Structure

$$\begin{aligned}\mathcal{A} &= (A, E), & A &\neq \emptyset, & E &\subseteq A \times A. \\ a \ E \ b & :\iff (a, b) \in E. \\ \mathcal{A} &= \langle A, I \rangle, & I(\in) &= E.\end{aligned}$$

Sorts and Domains

A universe

Interpretation

Constants. None.

Interpretation

Functions. None.

Relations

E $E \subseteq A \times A$
Interpretation of $\in$.

Layer Delta

No constant or function symbols occur in $\mathcal{L}_\in$, so every term is a variable and FOL term evaluation reduces to $\llbracket x \rrbracket_{\mathcal{A}, s} = s(x)$.

Syntax–Semantics Bridge

$$\mathcal{A}, s \models x \in y \iff (s(x), s(y)) \in E.$$

Models of the Theory

$$\begin{aligned}\mathcal{A} \models \text{ZF} & :\iff \forall \varphi (\varphi \in \text{ZF} \rightarrow \mathcal{A} \models \varphi). \\ \mathcal{A} \models \text{ZFC} & :\iff \mathcal{A} \models \text{ZF} \wedge \mathcal{A} \models \text{AC}.\end{aligned}$$

Theorems and Consequences

1. One nonlogical symbol is interpreted: $I(\in) = E$.
2. $\text{Th}(\mathcal{A})$ is inherited from *Consequence*.

Definitional Extension (Set Theory)

Depends on: `defext:first-order-logic; theory:set-theory; model:set-theory`

Layer Delta

existence $\leftarrow$ Empty Set, Pairing, Union, Power Set,
Infinity, Separation, Replacement;
uniqueness $\leftarrow$ Extensionality;
default value $\leftarrow \emptyset$.

Representative-independence is delegated to quotient-specific extensions.

Instances

$\subseteq$ relation

$$\forall x \forall y (x \subseteq y \leftrightarrow \forall w (w \in x \rightarrow w \in y)).$$

No unique-existence obligation.

$\emptyset$ constant

$$\psi_{\emptyset}(y) := \forall w \neg(w \in y), \quad \exists! \text{ receipt: Empty Set + Extensionality.}$$

$$\forall y (\emptyset = y \leftrightarrow \forall w \neg(w \in y)).$$

$\{\cdot, \cdot\}$ binary function

$$\psi(x_1, x_2, y) := \forall w (w \in y \leftrightarrow (w = x_1 \vee w = x_2)).$$

$\exists!$ receipt: Pairing + Extensionality.

$$\forall x_1 \forall x_2 \forall y (\{x_1, x_2\} = y \leftrightarrow \psi(x_1, x_2, y)).$$

$\bigcup$ unary function

$$\psi(x, y) := \forall z (z \in y \leftrightarrow \exists w (w \in x \wedge z \in w)).$$

$\exists!$ receipt: Union + Extensionality.

$$\forall x \forall y (\bigcup x = y \leftrightarrow \psi(x, y)).$$

$\mathcal{P}$ unary function

$$\psi(x, y) := \forall z (z \in y \leftrightarrow \forall w (w \in z \rightarrow w \in x)).$$

$\exists!$ receipt: Power Set + Extensionality.

$$\forall x \forall y (\mathcal{P}(x) = y \leftrightarrow \psi(x, y)).$$

7.2 Sets

7.2.1 Sets, Membership, and ZFC Axioms

Axiom (Extensionality)

Axiom 7.2.1 (Axiom of Extensionality). Two sets are equal iff they have the same elements.

$$\forall A \forall B \left(A = B \iff \forall x (x \in A \leftrightarrow x \in B) \right).$$

Axiom (Empty Set)

Axiom 7.2.2 (Axiom of Empty Set). There exists a set with no elements.

$$\exists A \forall x (x \notin A).$$

Axiom (Pairing)

Axiom 7.2.3 (Axiom of Pairing). For any two sets, there exists a set containing exactly those two sets.

$$\forall A \forall B \exists C \forall x (x \in C \leftrightarrow (x = A \vee x = B)).$$

Axiom (Union)

Axiom 7.2.4 (Axiom of Union). For any family of sets, there exists a set containing exactly the elements of those sets.

$$\forall A \exists U \forall x (x \in U \leftrightarrow \exists B (B \in A \wedge x \in B)).$$

Axiom (Power Set)

Axiom 7.2.5 (Axiom of Power Set). For any set, there exists a set of all its subsets.

$$\forall A \exists P \forall x (x \in P \leftrightarrow x \subseteq A).$$

Axiom (Infinity)

Axiom 7.2.6 (Axiom of Infinity). There exists an infinite set.

$$\exists A \left(\emptyset \in A \wedge \forall x (x \in A \rightarrow x \cup \{x\} \in A) \right).$$

Axiom Schema (Separation)

Axiom 7.2.7 (Axiom Schema of Separation). Given a set and a property, there exists a subset containing exactly the elements satisfying that property. For any formula $\varphi(x)$,

$$\forall A \exists B \forall x (x \in B \leftrightarrow (x \in A \wedge \varphi(x))).$$

Axiom Schema (Replacement)

Axiom 7.2.8 (Axiom Schema of Replacement). If $\varphi(x, y)$ defines a functional relation on a set A , then the image of A under φ is a set. For any formula $\varphi(x, y)$,

$$\forall A \left(\left(\forall x \in A \exists! y \varphi(x, y) \right) \rightarrow \exists B \forall y \left(y \in B \leftrightarrow \exists x \in A \varphi(x, y) \right) \right).$$

Axiom (Foundation)

Axiom 7.2.9 (Axiom of Foundation). Every nonempty set has an $\in$ -minimal element.

$$\forall A \left(A \neq \emptyset \rightarrow \exists x \in A (x \cap A = \emptyset) \right).$$

Axiom (Choice)

Axiom 7.2.10 (Axiom of Choice). For any family of nonempty sets, there exists a selection function.

$$\forall A \left(\left(\forall B \in A (B \neq \emptyset) \right) \rightarrow \exists f \forall B \in A (f(B) \in B) \right).$$

Definition (Set and Membership)

Definition 7.2.11 (Set and Membership). In axiomatic set theory, the notions of *set* and *membership* are *primitive*.

- A *set* is an object.
- *Membership* is a binary relation, denoted by $\in$, between objects.

If x is an object and A is a set, the statement $x \in A$ is read as “ x is an element of A .” No definition of “set” or “ $\in$ ” is given in more basic terms. Their meaning is determined entirely by the axioms governing them.

Remark (English reading). Primitive means we do not define these in terms of simpler concepts. Rather, we fix their meaning implicitly by stating the axioms they must obey. This is the standard approach in formal mathematics: rules of behaviour replace informal definitions.

Remark (Consequence). All subsequent notions—subsets, ordered pairs, relations, functions, number systems—are *definitions* introduced within this axiomatic framework. Every proved

result is a *theorem* derived logically from the axioms via the rules of inference.

Remark (Role of the axioms). The axioms are not statements to be proved, but rules specifying how $\in$ behaves and which sets are permitted to exist. From this point onward, set theory functions as the underlying language of mathematics: all reasoning about mathematical objects is ultimately grounded in these axioms, even when they are not cited explicitly.

Remark (Separation vs. Replacement). Separation is a schema: one axiom instance per definable property φ . It carves out subsets of an already-existing set. Replacement is strictly stronger: it can produce sets whose elements are not already contained in any pre-existing set, allowing the construction of large stages of the cumulative hierarchy.

Remark (Axiom of Choice and right inverses). The Axiom of Choice is equivalent to many statements used later in analysis, including Zorn's Lemma, the well-ordering theorem, and the statement that every surjective function has a right inverse. It is independent of the other ZFC axioms.

Foundations — Quick Reference

Concept	Meaning
Set, membership	Primitive notions; meaning fixed by axioms
Extensionality	Sets equal iff same elements
Empty Set	Exists a set with no elements
Pairing	Any two sets can be collected
Union	Elements of sets in a family
Power Set	All subsets form a set
Infinity	An infinite set exists
Separation	Subsets by definable property
Replacement	Functional image of a set is a set
Foundation	No infinite descending $\in$ -chains
Choice	Selection function exists

Set Operations — Quick Reference

Concept	Notation	Membership condition
Empty set	$\emptyset$	$x \in \emptyset$ never
Subset	$A \subseteq B$	$x \in A \Rightarrow x \in B$
Proper subset	$A \subsetneq B$	$A \subseteq B$ and $A \neq B$
Set equality	$A = B$	same elements both ways
Union	$A \cup B$	in A or in B
Intersection	$A \cap B$	in A and in B
Set difference	$A \setminus B$	in A but not B
Symmetric diff.	$A \Delta B$	in exactly one of A, B
Complement	$A^c = U \setminus A$	in U but not A
Cartesian product	$A \times B$	all ordered pairs (a, b)
Power set	$\mathcal{P}(A)$	all subsets of A
De Morgan	$(A \cup B)^c = A^c \cap B^c$	complement of union

7.2.2 Set Constructions and Operations

Definition (Empty Set)

Definition 7.2.12 (Empty Set). The unique set with no elements is called the *empty set* and is denoted $\emptyset$.

Remark (Dependencies).

- [Axiom of Empty Set](#)

Remark (Vacuous truth). Statements of the form $\forall x \in \emptyset, P(x)$ are *vacuously true*: there is no element $x \in \emptyset$ for which $P(x)$ could fail. This is not a special convention but a consequence of how universal quantification is defined.

Definition (Subset)

Definition 7.2.13 (Subset). Let A and B be sets. We say A is a *subset* of B , written $A \subseteq B$, if every element of A is also an element of B :

$$A \subseteq B \iff \forall x (x \in A \rightarrow x \in B).$$

Remark (Dependencies).

- [Axiom Schema of Separation](#)
- [Universal Formula](#)

Definition (Proper Subset)

Definition 7.2.14 (Proper Subset). A is a *proper subset* of B , written $A \subsetneq B$, if $A \subseteq B$ and $A \neq B$.

Remark (Dependencies).

- [Subset](#)

Remark (Notation convention). Some authors write $A \subset B$ for a proper subset; others use it to mean $A \subseteq B$. In these notes, $\subseteq$ always denotes subset (possibly equal) and $\subsetneq$ always denotes proper subset.

Definition (Set Equality)

Definition 7.2.15 (Set Equality). Two sets A and B are *equal*, written $A = B$, if they have the same elements:

$$A = B \iff \forall x (x \in A \leftrightarrow x \in B).$$

Equivalently, $A = B \iff (A \subseteq B \wedge B \subseteq A)$.

Remark (Dependencies).

- [Subset](#)
- [Axiom of Extensionality](#)

Remark (Proof strategy). In practice, to prove $A = B$ one shows mutual inclusion: first $A \subseteq B$ (take arbitrary $x \in A$ and deduce $x \in B$), then $B \subseteq A$. This two-step structure appears throughout set-theoretic proofs.

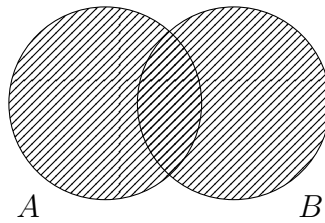
Definition (Union)

Definition 7.2.16 (Union). The *union* of A and B , denoted $A \cup B$, is the set of all elements belonging to at least one of the two sets:

$$A \cup B = \{x \mid x \in A \vee x \in B\}.$$

Remark (Dependencies).

- [Set and Membership](#)
- [Axiom Schema of Separation](#)



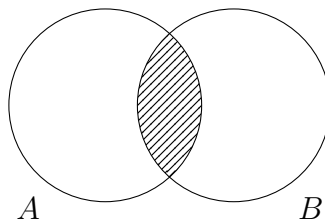
Definition (Intersection)

Definition 7.2.17 (Intersection). The *intersection* of A and B , denoted $A \cap B$, is the set of all elements common to both:

$$A \cap B = \{x \mid x \in A \wedge x \in B\}.$$

Remark (Dependencies).

- [Set and Membership](#)
- [Axiom Schema of Separation](#)



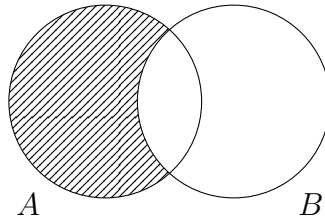
Definition (Set Difference)

Definition 7.2.18 (Set Difference). The *set difference* $A \setminus B$ is the set of elements in A but not in B :

$$A \setminus B = \{x \mid x \in A \wedge x \notin B\}.$$

Remark (Dependencies).

- [Set and Membership](#)
- [Axiom Schema of Separation](#)



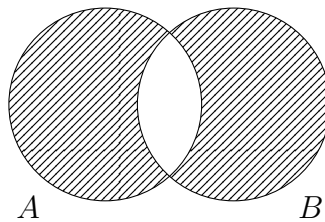
Definition (Symmetric Difference)

Definition 7.2.19 (Symmetric Difference). The *symmetric difference* $A \triangle B$ is the set of elements belonging to exactly one of A and B :

$$A \triangle B = (A \setminus B) \cup (B \setminus A) = \{x \mid (x \in A \vee x \in B) \wedge x \notin A \cap B\}.$$

Remark (Dependencies).

- [Set Difference](#)
- [Union](#)
- [Intersection](#)



Remark (Algebraic structure of $\triangle$). The symmetric difference is commutative and associative. Under $\triangle$, the power set $\mathcal{P}(U)$ forms an abelian group with identity $\emptyset$ and every element its own inverse ($A \triangle A = \emptyset$).

Definition (Complement)

Definition 7.2.20 (Complement). Let U be a fixed universe and $A \subseteq U$. The *complement* of A relative to U , denoted A^c , is:

$$A^c = U \setminus A.$$

Remark (Dependencies).

- [Set and Membership](#)
- [Axiom Schema of Separation](#)

Definition (Cartesian Product)

Definition 7.2.21 (Cartesian Product). The *Cartesian product* of sets A and B is:

$$A \times B = \{ (a, b) \mid a \in A \text{ and } b \in B \}.$$

Remark (Dependencies).

- [Ordered Pair](#)
- [Axiom of Pairing](#)
- [Axiom of Power Set](#)
- [Axiom Schema of Separation](#)

Remark (Ordered pairs). The ordered pair (a, b) is formally defined as $\{\{a\}, \{a, b\}\}$. This Kuratowski encoding ensures the key property: $(a, b) = (c, d)$ iff $a = c$ and $b = d$.

Remark (Non-commutativity). In general, $A \times B \neq B \times A$. The product is however associative up to canonical isomorphism: $(A \times B) \times C \cong A \times (B \times C)$.

Definition (Power Set)

Definition 7.2.22 (Power Set). The *power set* of A , denoted $\mathcal{P}(A)$, is the set of all subsets of A :

$$\mathcal{P}(A) := \{ S \mid S \subseteq A \}.$$

Remark (Dependencies).

- [Axiom of Power Set](#)

Remark (Size). If A has n elements, then $|\mathcal{P}(A)| = 2^n$. For infinite A , Cantor's theorem shows $|\mathcal{P}(A)| > |A|$ strictly.

Theorem (De Morgan's Laws)

Theorem 7.2.23 (De Morgan's Laws). *Let U be a universe and $A, B \subseteq U$. Then*

$$(A \cup B)^c = A^c \cap B^c \quad \text{and} \quad (A \cap B)^c = A^c \cup B^c.$$

Go to proof.

Remark (Standard quantified statement). $\forall A, B \subseteq U : \forall x (x \notin A \cup B \leftrightarrow (x \notin A \wedge x \notin B))$, and dually $\forall x (x \notin A \cap B \leftrightarrow (x \notin A \vee x \notin B))$.

Remark (Dependencies).

- [Union](#)
- [Intersection](#)
- [Complement](#)

Remark (Connection to propositional logic). De Morgan's laws mirror the propositional logic identities $\neg(P \vee Q) \equiv \neg P \wedge \neg Q$ and $\neg(P \wedge Q) \equiv \neg P \vee \neg Q$ under the correspondence $\cap \leftrightarrow \wedge$, $\cup \leftrightarrow \vee$, $A^c \leftrightarrow \neg A$.

Definition (Set Duality)

Definition 7.2.24 (Set Duality). Two set-theoretic expressions over a fixed universe U are *dual* if one is obtained from the other by simultaneously replacing $\cup \leftrightarrow \cap$ and $\emptyset \leftrightarrow U$, with complements unchanged.

Remark (Standard quantified statement). $\forall$ expressions E, E' over U : E and E' are dual iff E' is obtained from E by the substitution $\cup \mapsto \cap$, $\cap \mapsto \cup$, $\emptyset \mapsto U$, $U \mapsto \emptyset$.

Remark (Dependencies).

- [Union](#)
- [Intersection](#)
- [Empty Set](#)
- [Complement](#)

Corollary

Corollary 7.2.25 (Principle of Set Duality). *Any identity involving $\cup$, $\cap$, $\emptyset$, and U that holds for all subsets of a universe remains valid when each operation and constant is replaced by its dual.*

Remark (Standard quantified statement). For any identity $\forall A_1, \dots, A_n \subseteq U : E(A_i, \cup, \cap, \emptyset, U) = E'(A_i, \cup, \cap, \emptyset, U)$, the dual identity $\forall A_1, \dots, A_n \subseteq U : E^d = E'^d$ holds, where E^d replaces $\cup \leftrightarrow \cap$ and $\emptyset \leftrightarrow U$. ■

Remark (Dependencies).

- [Set Duality](#)
- [Axiom of Extensionality](#)

Remark (Using duality in proofs). To prove a statement involving unions and intersections, it often suffices to prove one version; the dual statement follows immediately.

Example (Distributive law via duality). The two distributive laws

$$A \cap (B \cup C) = (A \cap B) \cup (A \cap C) \quad \text{and} \quad A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$$

are duals of each other. Proving either one and applying the Principle of Set Duality yields the other immediately.

Remark (Transition to relations). The Cartesian product provides a way to encode ordered information. Relations and functions will be defined as special subsets of Cartesian products, so they require no new foundational objects.

Set Algebra Laws — Quick Reference

Law	Union form	Intersection form
Commutativity	$A \cup B = B \cup A$	$A \cap B = B \cap A$
Associativity	$(A \cup B) \cup C = A \cup (B \cup C)$	analogous
Distributivity	$A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$	dual
Identity	$A \cup \emptyset = A$	$A \cap U = A$
Absorption	$A \cup (A \cap B) = A$	$A \cap (A \cup B) = A$
Involution	$(A^c)^c = A$	—

7.2.3 Algebraic Laws of Set Operations

Remark (Algebraic structure of set operations). The operations $\cup$, $\cap$, and complement satisfy algebraic laws analogous to those of logical connectives. Together they endow $\mathcal{P}(U)$ with the structure of a *Boolean algebra*. These laws justify manipulation of set expressions in later proofs, particularly those involving equivalence classes, partitions, and functions.

Theorem (Commutativity of Union and Intersection)

Theorem 7.2.26 (Commutativity of Union and Intersection). *Let A, B be sets. Then*

$$A \cup B = B \cup A \quad \text{and} \quad A \cap B = B \cap A.$$

Go to proof.

Remark (Standard quantified statement). $\forall A, B : \forall x (x \in A \cup B \leftrightarrow x \in B \cup A)$ and $\forall x (x \in A \cap B \leftrightarrow x \in B \cap A)$.

Remark (Dependencies).

- [Axiom of Extensionality](#)
- [Union](#)
- [Intersection](#)

Theorem (Associativity of Union and Intersection)

Theorem 7.2.27 (Associativity of Union and Intersection). *Let A, B, C be sets. Then*

$$(A \cup B) \cup C = A \cup (B \cup C) \quad \text{and} \quad (A \cap B) \cap C = A \cap (B \cap C).$$

Go to proof.

Remark (Standard quantified statement). $\forall A, B, C : \forall x ((x \in (A \cup B) \cup C) \leftrightarrow (x \in A \cup (B \cup C)))$, and analogously for $\cap$.

Remark (Dependencies).

- [Axiom of Extensionality](#)
- [Union](#)
- [Intersection](#)

Theorem (Distributive Laws)

Theorem 7.2.28 (Distributive Laws). *Let A, B, C be sets. Then*

$$A \cap (B \cup C) = (A \cap B) \cup (A \cap C),$$

$$A \cup (B \cap C) = (A \cup B) \cap (A \cup C).$$

Go to proof.

Remark (Standard quantified statement). $\forall A, B, C : \forall x (x \in A \cap (B \cup C) \leftrightarrow x \in (A \cap B) \cup (A \cap C))$, and dually for $\cup$ over $\cap$.

Remark (Dependencies).

- [Axiom of Extensionality](#)
- [Union](#)
- [Intersection](#)

Theorem (Identity and Absorption Laws)

Theorem 7.2.29 (Identity and Absorption Laws). *Let A, B be sets and U a universe with $A \subseteq U$. Then*

$$A \cup \emptyset = A, \quad A \cap U = A,$$

$$A \cup (A \cap B) = A, \quad A \cap (A \cup B) = A.$$

Go to proof.

Remark (Standard quantified statement). $\forall A \subseteq U : \forall x (x \in A \cup \emptyset \leftrightarrow x \in A)$ and $\forall x (x \in A \cap U \leftrightarrow x \in A)$. Absorption: $\forall A, B : \forall x (x \in A \cup (A \cap B) \leftrightarrow x \in A)$, and dually.

Remark (Dependencies).

- [Axiom of Extensionality](#)
- [Empty Set](#)
- [Union](#)
- [Intersection](#)
- [Subset](#)

Theorem (Involution of Complement)

Theorem 7.2.30 (Involution of Complement). *For any $A \subseteq U$,*

$$(A^c)^c = A.$$

Go to proof.

Remark (Standard quantified statement). $\forall A \subseteq U : \forall x (x \in (A^c)^c \leftrightarrow x \in A)$.

Remark (Dependencies).

- [Axiom of Extensionality](#)
- [Complement](#)

Remark (Non-commutative and non-associative operations). Set difference $\setminus$ is neither commutative nor associative:

$$A \setminus B \neq B \setminus A, \quad (A \setminus B) \setminus C \neq A \setminus (B \setminus C) \quad \text{in general.}$$

It interacts with union and intersection via:

$$A \setminus (B \cup C) = (A \setminus B) \cap (A \setminus C), \quad A \setminus (B \cap C) = (A \setminus B) \cup (A \setminus C).$$

These follow from $A \setminus B = A \cap B^c$ together with De Morgan's laws.

Remark (Cartesian product). The Cartesian product $\times$ is not commutative, is associative only up to canonical isomorphism, and distributes over union in each coordinate:

$$A \times (B \cup C) = (A \times B) \cup (A \times C).$$

7.3 Families

Indexed Families — Quick Reference

Concept	Meaning
Indexed family	Function $F : I \rightarrow \mathcal{P}(U)$; written $\{A_i\}_{i \in I}$
Indexed union	$\bigcup_{i \in I} A_i$: in at least one A_i
Indexed intersection	$\bigcap_{i \in I} A_i$: in every A_i
Pairwise disjoint	$i \neq j \Rightarrow A_i \cap A_j = \emptyset$
Cover	$\bigcup_{C \in \mathcal{C}} C = A$
Indexed De Morgan	Complements exchange indexed unions and intersections
Arbitrary product	$\prod_{i \in I} A_i$: choice functions $f : I \rightarrow \bigcup A_i$

7.3.1 Indexed Families of Sets

Remark (Motivation). Many set-theoretic constructions require not just pairs of sets but infinite families. Indexed families provide the formal language for partitions, equivalence classes, unions over countably or uncountably many sets, and products indexed by arbitrary index sets.

Definition (Indexed Family of Sets)

Definition 7.3.1 (Indexed Family of Sets). Let I be a set (the *index set*) and U a universe. An *indexed family of sets* is a function

$$F : I \rightarrow \mathcal{P}(U),$$

with $F(i)$ typically written A_i . The family is denoted $\{A_i\}_{i \in I}$.

Remark (Dependencies).

- [Function](#)
- [Power Set](#)

Remark (Family vs. set of sets). An indexed family is formally a *function*, not a set of sets. Different indices may correspond to the same set: $i \neq j$ does not imply $A_i \neq A_j$. This distinction matters for equivalence class constructions.

Definition (Indexed Union)

Definition 7.3.2 (Indexed Union). The *union* of the indexed family $\{A_i\}_{i \in I}$ is:

$$\bigcup_{i \in I} A_i := \{x \mid \exists i \in I \text{ such that } x \in A_i\}.$$

Remark (Dependencies).

- [Indexed Family of Sets](#)
- [Union](#)
- [Existential Formula](#)

Definition (Indexed Intersection)

Definition 7.3.3 (Indexed Intersection). For $I \neq \emptyset$, the *intersection* of $\{A_i\}_{i \in I}$ is:

$$\bigcap_{i \in I} A_i := \{x \mid \forall i \in I, x \in A_i\}.$$

Remark (Dependencies).

- [Indexed Family of Sets](#)
- [Intersection](#)
- [Universal Formula](#)

Remark (Why $I \neq \emptyset$ is required). If $I = \emptyset$, then $\forall i \in I, x \in A_i$ holds vacuously for every x , making the intersection the entire universe U . This is normally left undefined or requires specifying a background universe explicitly.

Theorem (De Morgan's Laws for Indexed Families)

Theorem 7.3.4 (De Morgan's Laws for Indexed Families). *Let $\{A_i\}_{i \in I}$ be an indexed family of subsets of a universe U . Then*

$$U \setminus \bigcup_{i \in I} A_i = \bigcap_{i \in I} (U \setminus A_i).$$

If $I \neq \emptyset$, then

$$U \setminus \bigcap_{i \in I} A_i = \bigcup_{i \in I} (U \setminus A_i).$$

Go to proof.

Remark (Dependencies).

- [Indexed Family of Sets](#)
- [Indexed Union](#)
- [Indexed Intersection](#)
- [Complement](#)

Definition (Pairwise Disjoint Family)

Definition 7.3.5 (Pairwise Disjoint Family). The family $\{A_i\}_{i \in I}$ is *pairwise disjoint* if

$$\forall i, j \in I, i \neq j \rightarrow A_i \cap A_j = \emptyset.$$

Remark (Dependencies).

- [Indexed Family of Sets](#)
- [Intersection](#)
- [Empty Set](#)
- [Set Equality](#)

Definition (Cover)

Definition 7.3.6 (Cover). A collection $\mathcal{C} \subseteq \mathcal{P}(A)$ is a *cover* of A if

$$\bigcup_{C \in \mathcal{C}} C = A.$$

Remark (Standard quantified statement). $\mathcal{C}$ covers $A \leftrightarrow \forall x \in A, \exists C \in \mathcal{C}, x \in C$.

Arbitrary Cartesian Products

Definition (Arbitrary Cartesian Product)

Definition 7.3.7 (Arbitrary Cartesian Product). Let $\{A_i\}_{i \in I}$ be an indexed family of sets. The *Cartesian product* is:

$$\prod_{i \in I} A_i := \left\{ f : I \rightarrow \bigcup_{i \in I} A_i \mid \forall i \in I, f(i) \in A_i \right\}.$$

Remark (Dependencies).

- [Indexed Family of Sets](#)
- [Function](#)
- [Indexed Union](#)

Remark (Elements as choice functions). An element of $\prod_{i \in I} A_i$ is a *choice function*: a function that assigns to each index i an element of the corresponding set A_i . For finite $I = \{1, \dots, n\}$, this reduces to the familiar n -tuple $(a_1, \dots, a_n)$.

Remark (Axiom of Choice connection). For infinite I , the product $\prod_{i \in I} A_i$ is nonempty iff a choice function exists. This existence is not guaranteed by the other ZFC axioms: it is equivalent to the Axiom of Choice. Thus the Axiom of Choice is precisely the assertion that arbitrary products of nonempty sets are nonempty.

Covers and FIP — Quick Reference	
Concept	Meaning
Cover of A	$\bigcup_{C \in \mathcal{C}} C \supseteq A$
Subcover	Sub-collection of a cover that is still a cover
Finite cover	Cover with finitely many members
Finite subcover	Finite subcollection that is still a cover
Open cover	Cover whose members are all open sets
Finite Intersection Property	Every finite subcollection has nonempty intersection
FIP–cover duality	$\mathcal{C}$ covers $A \iff$ closed complements fail FIP

7.3.2 Covers, Subcovers, and the Finite Intersection Property

Compactness is one of the central concepts of analysis, and its definition is stated in terms of covers: a space is compact if every open cover has a finite subcover. This is a purely set-theoretic condition — it makes no reference to distance or continuity. The work of understanding compactness therefore begins here, with the vocabulary of covers. Once these definitions are in place, the definition of compactness will be a single sentence, and the difficulty will lie entirely in the geometric and analytic content, not the set-theoretic framework.

Definition (Cover)

Definition 7.3.8 (Cover). Let A be a set and $\mathcal{C}$ a collection of sets. $\mathcal{C}$ is a *cover* of A if every element of A belongs to at least one member of $\mathcal{C}$:

$$A \subseteq \bigcup_{C \in \mathcal{C}} C.$$

Remark (Dependencies).

- [Subset](#)
- [Union](#)

Definition (Subcover)

Definition 7.3.9 (Subcover). $\mathcal{C}'$ is a *subcover* of $\mathcal{C}$ if $\mathcal{C}' \subseteq \mathcal{C}$ and $\mathcal{C}'$ is still a cover of A .

Remark (Dependencies).

- [Cover](#)

- [Subset](#)

Definition (Finite Cover)

Definition 7.3.10 (Finite Cover). A cover is *finite* if it has finitely many members. A *finite subcover* is a subcover $\mathcal{C}' \subseteq \mathcal{C}$ with $|\mathcal{C}'| < \infty$.

Remark (Dependencies).

- [Cover](#)
- [Finite and Infinite Sets](#)
- [Subset](#)

Remark (The key question). The condition of being a cover is easy to satisfy: take $\mathcal{C} = \mathcal{P}(A)$ and it is trivially a cover. The interesting question is whether large covers can be replaced by smaller ones — specifically, finite ones. An infinite cover has infinitely many sets, which is hard to work with. A finite subcover is just as good for covering purposes, but is finite and therefore tractable. When a finite subcover always exists, no matter which cover one starts with, the space is called compact.

Definition (Open Cover)

Definition 7.3.11 (Open Cover). In a topological space (X, τ) , an *open cover* of a subset $A \subseteq X$ is a cover $\mathcal{C}$ of A all of whose members are open sets: $\mathcal{C} \subseteq \tau$.

Remark (Standard quantified statement). $\mathcal{C}$ is an open cover of $A \iff (\forall C \in \mathcal{C}, C \in \tau) \wedge (\forall x \in A, \exists C \in \mathcal{C}, x \in C)$.

Remark (Dependencies).

- [Cover](#)
- [Finite Intersection Property](#)
- [Complement](#)
- [Intersection](#)
- [Subset](#)

Remark (Why open covers). The emphasis on open covers in compactness is because open sets are the sets one controls in a topological space. An open cover $\{U_\alpha\}$ of A means: every point of A has at least one U_α containing it. A finite subcover then gives a list of finitely many open sets whose union already contains all of A . Compactness says this reduction is always possible, which is a very strong structural property.

Definition (Finite Intersection Property)

Definition 7.3.12 (Finite Intersection Property). A collection $\mathcal{F}$ of sets has the *finite intersection property* (FIP) if every finite subcollection has nonempty intersection: for all finite $\mathcal{F}' \subseteq \mathcal{F}$,

$$\bigcap_{F \in \mathcal{F}'} F \neq \emptyset.$$

Remark (Dependencies).

- [Intersection](#)
- [Empty Set](#)
- [Subset](#)
- [Finite and Infinite Sets](#)

Remark (How to read the FIP). The FIP is a consistency condition: no finite list of sets from $\mathcal{F}$ is mutually exclusive. It does *not* say the entire intersection $\bigcap_{F \in \mathcal{F}} F$ is nonempty — only that every *finite* sub-intersection is. The difference matters when $\mathcal{F}$ is infinite.

Example: In $\mathbb{R}$, the family $\mathcal{F} = \{(0, 1/n) : n \in \mathbb{N}\}$ has the FIP (any finite subcollection has nonempty intersection, since $(0, 1/n) \supseteq (0, 1/N)$ for the largest N in the subcollection). But $\bigcap_{n=1}^{\infty} (0, 1/n) = \emptyset$. Compact spaces are characterized by the property that the FIP implies nonempty total intersection.

Proposition (FIP–Cover Duality)

Proposition 7.3.13 (FIP–Cover Duality). Let X be a set, $A \subseteq X$, and $\{U_\alpha\}_{\alpha \in I}$ a family of subsets of X . Let $F_\alpha = U_\alpha^c = X \setminus U_\alpha$. Then:

$$\{U_\alpha\} \text{ covers } A \iff \{F_\alpha \cap A\}_{\alpha \in I} \text{ does not have the FIP.}$$

More precisely: the family $\{U_\alpha\}$ does not cover A if and only if the family of closed complements $\{F_\alpha\}$ has the FIP relative to A . Go to proof.

Remark (Standard quantified statement). $A \subseteq \bigcup_{\alpha \in I} U_\alpha \iff \neg(\forall \text{ finite } I_0 \subseteq I, \bigcap_{\alpha \in I_0} F_\alpha \cap A \neq \emptyset)$. Reading pointwise: $\forall x \in A, \exists \alpha \in I, x \in U_\alpha \iff \{F_\alpha \cap A\}$ fails the FIP.

Remark (Dependencies).

- [Cover](#)
- [Finite Intersection Property](#)
- [Complement](#)
- [Intersection](#)
- [Subset](#)

Remark (Proof by De Morgan). The duality is a direct consequence of De Morgan's law for indexed families (Theorem 1.3.4). The family $\{U_\alpha\}$ fails to cover A means:

$$A \not\subseteq \bigcup_{\alpha} U_{\alpha} \iff A \cap \left(\bigcup_{\alpha} U_{\alpha} \right)^c \neq \emptyset \iff A \cap \bigcap_{\alpha} U_{\alpha}^c \neq \emptyset \iff A \cap \bigcap_{\alpha} F_{\alpha} \neq \emptyset.$$

Applying this to every finite subcollection gives the equivalence with the FIP.

Remark (Why this duality matters for compactness). Compactness has two equivalent formulations:

- (i) Every open cover has a finite subcover.
- (ii) Every family of closed sets with the FIP has nonempty intersection.

These formulations are equivalent by FIP–Cover Duality: take complements and apply De Morgan. Formulation (i) is the standard definition; formulation (ii) is often easier to apply when working with closed sets (e.g., nested compact sets). Both are purely set-theoretic conditions, and the equivalence is entirely a matter of set algebra. When compactness is encountered in topology and metric spaces, the only new content is the geometric meaning — the set-theoretic equivalence is established here.

Example (Nested closed sets and FIP). In $\mathbb{R}$, the family $\{[n, \infty) : n \in \mathbb{N}\}$ has the FIP: any finite subcollection $\{[n_1, \infty), \dots, [n_k, \infty)\}$ has intersection $[\max(n_i), \infty) \neq \emptyset$. But $\bigcap_{n=1}^{\infty} [n, \infty) = \emptyset$. This shows $\mathbb{R}$ is not compact (the family of closed sets has the FIP but empty total intersection). A compact space would not allow this failure.

Chapter 8

Relations



Remark (Source orientation). This chapter follows Tarski’s treatment of relations as objects that can be formed, compared, reversed, and specialized into functional and one-one relations Tarski [1941].

8.1 Relations — Quick Reference

Relations — Quick Reference	
Concept	Meaning
Ordered pair	$(a, b) := \{\{a\}, \{a, b\}\}$; identity iff both coords equal
Cartesian product	$A \times B$: all ordered pairs from A and B
Relation	Any subset $R \subseteq A \times B$
Domain of R	Inputs that occur in R
Range of R	Outputs that occur in R
Converse	Reverse every ordered pair
Identity relation	Diagonal relation Δ_A
Relation composition	Compose through an intermediate witness

Concept	Meaning
Null relation	$\emptyset$
Universal relation	$A \times B$
Relation inclusion	$R \subseteq S$ as set inclusion
Relation equality	Mutual relation inclusion
Union of relations	Pair belongs to at least one relation
Intersection	Pair belongs to both relations
Difference	Pair belongs to first relation but not second
Complement	Pairs in $A \times B$ not in R
Converse laws	Converse reverses composition and preserves Boolean operations
Composition laws	Associative; distributes over unions, not all Boolean operations
One-to-many	One input has two distinct outputs
Many-to-one	Two distinct inputs share one output
Many-to-many	One-to-many and many-to-one phenomena both occur
Ternary relation	Subset of $A \times B \times C$

8.1.1 Ordered Pairs and Relations

Remark (Geometric picture). A binary relation from A to B can be pictured as directed edges from the A -side to the B -side of a bipartite diagram. The converse reverses every edge, and composition follows two edges in series through an intermediate set.

Definition (Ordered Pair)

Definition 8.1.1 (Ordered Pair). Let a and b be sets. The *ordered pair* (a, b) is defined (Kuratowski) as

$$(a, b) := \{\{a\}, \{a, b\}\}.$$

Remark (Dependencies).

- [Axiom of Pairing](#)

Theorem (Uniqueness of Ordered Pairs)

Theorem 8.1.2 (Uniqueness of Ordered Pairs). For any sets a, b, c, d ,

$$(a, b) = (c, d) \iff (a = c \wedge b = d).$$

Go to proof.

Remark (Standard quantified statement). $\forall a, b, c, d : (a, b) = (c, d) \leftrightarrow (a = c \wedge b = d)$.

Remark (Dependencies).

- [Ordered Pair](#)

- [Axiom of Pairing](#)
- [Axiom of Extensionality](#)

Remark (Kuratowski encoding). The purpose of the Kuratowski definition is purely to guarantee the uniqueness theorem above. Once $(a, b) = (c, d) \iff a = c \wedge b = d$ is established, we may treat ordered pairs as a primitive with this property and forget the encoding.

Definition (Cartesian Product — as foundation for relations)

Definition 8.1.3 (Cartesian Product — as foundation for relations). The *Cartesian product* of sets A and B is:

$$A \times B := \{ (a, b) \mid a \in A \text{ and } b \in B \}.$$

Remark (Dependencies).

- [Ordered Pair](#)
- [Axiom of Pairing](#)
- [Axiom of Power Set](#)
- [Axiom Schema of Separation](#)

Definition (Relation)

Definition 8.1.4 (Relation). A *relation* from A to B is any subset $R \subseteq A \times B$. If $(a, b) \in R$ we write $a R b$ and say “ a is related to b .” When $A = B$ we say R is a *relation on* A .

Remark (Dependencies).

- [Cartesian Product](#)
- [Subset](#)

Remark (Relations as structured sets). Relations introduce no new foundational objects: they are simply sets of ordered pairs, constructed using operations already available. Properties of relations can therefore be studied with ordinary set-theoretic reasoning.

Definition (Domain of a Relation)

Definition 8.1.5 (Domain of a Relation). Let $R \subseteq A \times B$ be a relation. The *domain* of R is

$$\text{Dom}(R) := \{ a \in A \mid \exists b \in B, (a, b) \in R \}.$$

Remark (Dependencies).

- [Relation](#)

- [Axiom Schema of Separation](#)

Definition (Range of a Relation)

Definition 8.1.6 (Range of a Relation). Let $R \subseteq A \times B$ be a relation. The *range* of R is

$$\text{Ran}(R) := \{ b \in B \mid \exists a \in A, (a, b) \in R \}.$$

Tarski also calls this class the *counter-domain* or *converse domain*.

Remark (Dependencies).

- [Relation](#)
- [Axiom Schema of Separation](#)

Remark (Domain and range). The domain records the first coordinates that occur in a relation; the range records the second coordinates that occur.

Definition (Converse Relation)

Definition 8.1.7 (Converse Relation). If $R \subseteq A \times B$, the *converse* of R is the relation $R^{-1} \subseteq B \times A$ defined by

$$(b, a) \in R^{-1} \iff (a, b) \in R.$$

Equivalently, $b R^{-1} a$ iff $a R b$.

Remark (Dependencies).

- [Relation](#)
- [Ordered Pair](#)

Definition (Identity Relation)

Definition 8.1.8 (Identity Relation). For a set A , the *identity relation* on A is

$$\Delta_A := \{ (a, a) \mid a \in A \}.$$

Remark (Dependencies).

- [Relation](#)
- [Cartesian Product](#)

Definition (Composition of Relations)

Definition 8.1.9 (Composition of Relations). Let $R \subseteq A \times B$ and $S \subseteq B \times C$. The *composition* $S \circ R \subseteq A \times C$ is defined by

$$(a, c) \in S \circ R \iff \exists b \in B, (a, b) \in R \text{ and } (b, c) \in S.$$

Remark (Dependencies).

- [Relation](#)
- [Cartesian Product](#)

Theorem (Associativity of Relation Composition)

Theorem 8.1.10 (Associativity of Relation Composition). *Let $R \subseteq A \times B$, $S \subseteq B \times C$, and $T \subseteq C \times D$. Then*

$$T \circ (S \circ R) = (T \circ S) \circ R.$$

Go to proof.

Remark (Standard quantified statement). For all $a \in A$ and $d \in D$, $(a, d) \in T \circ (S \circ R)$ iff there are $b \in B$ and $c \in C$ with $(a, b) \in R$, $(b, c) \in S$, and $(c, d) \in T$, which is exactly the membership condition for $(a, d) \in (T \circ S) \circ R$.

Remark (Dependencies).

- [Composition of Relations](#)
- [Relation Equality](#)

Theorem (Identity Laws for Relation Composition)

Theorem 8.1.11 (Identity Laws for Relation Composition). *Let $R \subseteq A \times B$. Then*

$$R \circ \Delta_A = R \quad \text{and} \quad \Delta_B \circ R = R.$$

Go to proof.

Remark (Dependencies).

- [Identity Relation](#)
- [Composition of Relations](#)

Definition (Null Relation)

Definition 8.1.12 (Null Relation). For sets A and B , the *null relation* from A to B is $\emptyset \subseteq A \times B$. No element of A is related to any element of B .

Remark (Dependencies).

- [Relation](#)
- [Empty Set](#)
- [Cartesian Product](#)

Definition (Universal Relation)

Definition 8.1.13 (Universal Relation). For sets A and B , the *universal relation* from A to B is $A \times B$ itself. Every element of A is related to every element of B .

Remark (Dependencies).

- [Relation](#)
- [Cartesian Product](#)

Remark (Extreme relations). The null relation and universal relation are the two extreme binary relations from A to B .

Definition (Relation Inclusion)

Definition 8.1.14 (Relation Inclusion). Let $R, S \subseteq A \times B$. Relation inclusion is ordinary set inclusion:

$$R \subseteq S \iff \forall a \in A \forall b \in B, ((a, b) \in R \Rightarrow (a, b) \in S),$$

Remark (Dependencies).

- [Relation](#)
- [Subset](#)

Definition (Relation Equality)

Definition 8.1.15 (Relation Equality). Let $R, S \subseteq A \times B$. Relation equality is ordinary set equality:

$$R = S \iff R \subseteq S \text{ and } S \subseteq R.$$

Remark (Dependencies).

- [Relation](#)
- [Relation Inclusion](#)
- [Set Equality](#)

Definition (Union of Relations)

Definition 8.1.16 (Union of Relations). Let $R, S \subseteq A \times B$. The *union* $R \cup S$ is the relation with

$$(a, b) \in R \cup S \iff (a, b) \in R \text{ or } (a, b) \in S.$$

Remark (Dependencies).

- [Relation](#)

- [Union](#)

Definition (Intersection of Relations)

Definition 8.1.17 (Intersection of Relations). Let $R, S \subseteq A \times B$. The *intersection* $R \cap S$ is the relation with

$$(a, b) \in R \cap S \iff (a, b) \in R \text{ and } (a, b) \in S.$$

Remark (Dependencies).

- [Relation](#)
- [Intersection](#)

Definition (Difference of Relations)

Definition 8.1.18 (Difference of Relations). Let $R, S \subseteq A \times B$. The *difference* $R \setminus S$ is the relation with

$$(a, b) \in R \setminus S \iff (a, b) \in R \text{ and } (a, b) \notin S.$$

Remark (Dependencies).

- [Relation](#)
- [Set Difference](#)

Definition (Complement of a Relation)

Definition 8.1.19 (Complement of a Relation). Let $R \subseteq A \times B$. The *complement* of R relative to $A \times B$ is

$$(A \times B) \setminus R.$$

Remark (Dependencies).

- [Relation](#)
- [Set Difference](#)

Remark (Relation algebra operations). Relation inclusion, equality, union, intersection, difference, and complement are inherited from the corresponding set operations on subsets of $A \times B$.

Theorem (Converse Laws for Relation Operations)

Theorem 8.1.20 (Converse Laws for Relation Operations). *Let $R, S \subseteq A \times B$, let $T \subseteq B \times C$, and take complements relative to the indicated Cartesian products. Then*

$$\begin{aligned}(R^{-1})^{-1} &= R, & (T \circ R)^{-1} &= R^{-1} \circ T^{-1}, \\ (R \cup S)^{-1} &= R^{-1} \cup S^{-1}, & (R \cap S)^{-1} &= R^{-1} \cap S^{-1}, \\ (R \setminus S)^{-1} &= R^{-1} \setminus S^{-1}, & ((A \times B) \setminus R)^{-1} &= (B \times A) \setminus R^{-1}.\end{aligned}$$

Go to proof.

Remark (Dependencies).

- [Converse Relation](#)
- [Composition of Relations](#)
- [Union of Relations](#)
- [Intersection of Relations](#)
- [Difference of Relations](#)
- [Complement of a Relation](#)

Theorem (Relation Composition and Boolean Operations)

Theorem 8.1.21 (Relation Composition and Boolean Operations). *Let $P, Q \subseteq A \times B$, $R \subseteq B \times C$, and $S, T \subseteq B \times C$. Then*

$$R \circ (P \cup Q) = (R \circ P) \cup (R \circ Q), \quad (S \cup T) \circ P = (S \circ P) \cup (T \circ P).$$

Also,

$$R \circ (P \cap Q) \subseteq (R \circ P) \cap (R \circ Q), \quad (S \cap T) \circ P \subseteq (S \circ P) \cap (T \circ P).$$

The analogous formulas for difference and complement hold only as inclusions or under extra hypotheses; they are not general distributive laws. Go to proof.

Remark (Dependencies).

- [Composition of Relations](#)
- [Union of Relations](#)
- [Intersection of Relations](#)

Remark (Warning on relation composition). Relation composition distributes over unions on both sides. It does not generally distribute over intersections, differences, or complements. The reason is existential: a pair can lie in both composed relations using two different middle witnesses, even when no single witness satisfies both conditions at once.

Remark (Relation algebra interaction table).

Operation	Converse	Composition
Union	$(R \cup S)^{-1} = R^{-1} \cup S^{-1}$	Distributes exactly on both sides
Intersection	$(R \cap S)^{-1} = R^{-1} \cap S^{-1}$	Gives $\subseteq$ in general; equality needs extra hypotheses
Difference	$(R \setminus S)^{-1} = R^{-1} \setminus S^{-1}$	No general distributive equality
Complement	$((A \times B) \setminus R)^{-1} = (B \times A) \setminus R^{-1}$	No general distributive equality
Composition	$(T \circ R)^{-1} = R^{-1} \circ T^{-1}$	Associative with identity relation Δ_A

Definition (Transitive Closure)

Definition 8.1.22 (Transitive Closure). Let R be a relation on A . The *transitive closure* of R is the smallest transitive relation on A containing R .

Remark (Dependencies).

- [Relation](#)
- [Transitive Relation](#)
- [Relation Inclusion](#)

Definition (Equivalence Closure)

Definition 8.1.23 (Equivalence Closure). Let R be a relation on A . The *equivalence closure* of R is the smallest equivalence relation on A containing R .

Remark (Dependencies).

- [Relation](#)
- [Equivalence Relation](#)
- [Relation Inclusion](#)

Definition (One-to-Many Relation)

Definition 8.1.24 (One-to-Many Relation). Let $R \subseteq A \times B$. R is *one-to-many* if some input is related to two distinct outputs:

$$\exists a \in A \exists b_1, b_2 \in B, \quad (a, b_1) \in R \wedge (a, b_2) \in R \wedge b_1 \neq b_2.$$

Remark (Dependencies).

- [Relation](#)

Definition (Many-to-One Relation)

Definition 8.1.25 (Many-to-One Relation). Let $R \subseteq A \times B$. R is *many-to-one* if two distinct inputs are related to a common output:

$$\exists a_1, a_2 \in A \exists b \in B, \quad (a_1, b) \in R \wedge (a_2, b) \in R \wedge a_1 \neq a_2.$$

Remark (Dependencies).

- [Relation](#)

Definition (Many-to-Many Relation)

Definition 8.1.26 (Many-to-Many Relation). Let $R \subseteq A \times B$. R is *many-to-many* if it is one-to-many and many-to-one.

Remark (Dependencies).

- [One-to-Many Relation](#)
- [Many-to-One Relation](#)

Remark (Relation cardinality patterns). The one-to-many, many-to-one, and many-to-many labels classify how many inputs and outputs can be connected by a relation.

Remark (Why one-to-many is not functional). A one-to-many relation fails the determinacy condition needed for a function: one input admits at least two possible outputs. A many-to-one relation may still be a function, because different inputs are allowed to share an output.

Definition (Many-Place Relation)

Definition 8.1.27 (Many-Place Relation). A *ternary relation* among sets A , B , and C is a subset of $A \times B \times C$. More generally, an n -place relation on $A_1, \dots, A_n$ is a subset of

$$A_1 \times \cdots \times A_n.$$

For example, betweenness in geometry is naturally a three-place relation, and the arithmetic statement $x + y = z$ is naturally a three-place relation among x , y , and z .

Remark (Dependencies).

- [Relation](#)
- [Cartesian Product](#)

Remark (Tarski-style hierarchy). The hierarchy used in the next chapter is:

relation $\longrightarrow$ functional relation $\longrightarrow$ function
 $\longrightarrow$ one-one or onto function $\longrightarrow$ bijection.

The first step adds determinacy; the second adds totality; the later steps control whether outputs are shared or missed.

Relational Properties — Quick Reference		
Property	Formal definition	Canonical example
Reflexive	$\forall a, (a, a) \in R$	$=$ on any set
Irreflexive	$\forall a, (a, a) \notin R$	$<$ on $\mathbb{N}$
Symmetric	$(a, b) \in R \Rightarrow (b, a) \in R$	“same age as”
Antisymmetric	$(a, b), (b, a) \in R \Rightarrow a = b$	$\leq$ on $\mathbb{N}$
Asymmetric	$(a, b) \in R \Rightarrow (b, a) \notin R$	$<$ on $\mathbb{N}$
Transitive	$(a, b), (b, c) \in R \Rightarrow (a, c) \in R$	$\leq$ on $\mathbb{N}$
Total (Connex)	$(a, b) \in R \vee (b, a) \in R$	$\leq$ on $\mathbb{R}$
<i>Structural classes formed by combining properties:</i>		
Equivalence	Reflexive + Symmetric + Transitive	$=$ on any set
Preorder	Reflexive + Transitive	$\leq$ on $\mathbb{N}$
Partial order	Reflexive + Antisymmetric + Transitive	$\subseteq$ on $\mathcal{P}(A)$
Strict partial order	Irreflexive + Transitive ($\Rightarrow$ Asymmetric)	$<$ on $\mathbb{R}$
Total order	Partial order + Total	$\leq$ on $\mathbb{R}$

8.1.2 Properties of Relations

Definition (Reflexive Relation)	
Definition 8.1.28 (Reflexive Relation).	A binary relation $R \subseteq A \times A$ is <i>reflexive</i> on A if every element of the domain is related to itself:
$\forall a \in A, (a, a) \in R.$	
In infix notation, this is the condition $\forall a \in A, aRa$.	

Remark (Standard quantified statement). Reflexivity is a universal requirement over the whole domain:

$$\text{Reflexive}_A(R) \iff \forall a \in A, (a, a) \in R.$$

For first-order logic with equality, the corresponding logical baseline is the identity axiom

$$\forall x (x = x),$$

which forces equality to behave reflexively in every structure.

Remark (Dependencies).

- [Relation](#)
- [Universal Formula](#)

Definition (Irreflexive Relation)

Definition 8.1.29 (Irreflexive Relation). A binary relation $R \subseteq A \times A$ is *irreflexive* on A if no element of the domain is related to itself:

$$\forall a \in A, (a, a) \notin R.$$

In infix notation, this is the condition $\forall a \in A, \neg(aRa)$.

Remark (Standard quantified statement). Irreflexivity is not the same as merely failing to be reflexive:

$$\text{Irreflexive}_A(R) \iff \forall a \in A, \neg(aRa),$$

whereas

$$\neg \text{Reflexive}_A(R) \iff \exists a \in A, \neg(aRa).$$

Thus a relation may be neither reflexive nor irreflexive. In first-order axiomatizations of strict order, irreflexivity is commonly imposed by the axiom

$$\forall x \neg(x < x).$$

Remark (Dependencies).

- [Relation](#)
- [Universal Formula](#)

Remark (Independence of reflexive and irreflexive). A relation may be neither reflexive nor irreflexive (if some but not all elements satisfy $(a, a) \in R$). However, a relation cannot be both reflexive and irreflexive unless $A = \emptyset$.

Definition (Symmetric Relation)

Definition 8.1.30 (Symmetric Relation). A binary relation $R \subseteq A \times A$ is *symmetric* on A if every related pair also appears in the reverse order:

$$\forall a, b \in A, (a, b) \in R \rightarrow (b, a) \in R.$$

In infix notation, this is $\forall a, b \in A, aRb \rightarrow bRa$.

Remark (Standard quantified statement). Symmetry is a conditional property:

$$\text{Symmetric}_A(R) \iff \forall a, b \in A, (aRb \rightarrow bRa).$$

It does not require every pair to be related; it requires only that any existing directed relation can be reversed. For first-order logic with equality, symmetry is expressed by the identity axiom

$$\forall x \forall y (x = y \rightarrow y = x).$$

Remark (Dependencies).

- [Relation](#)
- [Universal Formula](#)

Definition (Antisymmetric Relation)

Definition 8.1.31 (Antisymmetric Relation). A binary relation $R \subseteq A \times A$ is *antisymmetric* on A if any two elements related in both directions must be equal:

$$\forall a, b \in A, ((a, b) \in R \wedge (b, a) \in R) \rightarrow a = b.$$

In infix notation, this is $\forall a, b \in A, ((aRb \wedge bRa) \rightarrow a = b)$.

Remark (Standard quantified statement). Antisymmetry forbids two-way relation cycles between distinct elements:

$$\text{Antisymmetric}_A(R) \iff \forall a, b \in A, ((aRb \wedge bRa) \rightarrow a = b).$$

It is not the same as asymmetry. Antisymmetry permits aRa ; asymmetry forbids it. Thus $\leq$ is antisymmetric, while $<$ is asymmetric. In first-order axiomatizations of partial orders, antisymmetry is usually written

$$\forall x \forall y ((x \leq y \wedge y \leq x) \rightarrow x = y).$$

Remark (Dependencies).

- [Relation](#)
- [Universal Formula](#)

Definition (Asymmetric Relation)

Definition 8.1.32 (Asymmetric Relation). A binary relation $R \subseteq A \times A$ is *asymmetric* on A if every related pair excludes the reverse pair:

$$\forall a, b \in A, (a, b) \in R \rightarrow (b, a) \notin R.$$

In infix notation, this is $\forall a, b \in A, (aRb \rightarrow \neg(bRa))$.

Remark (Standard quantified statement). Asymmetry is the strict one-way condition

$$\text{Asymmetric}_A(R) \iff \forall a, b \in A, (aRb \rightarrow \neg(bRa)).$$

It implies irreflexivity by taking $a = b$, and it implies antisymmetry because two-way relation between distinct elements is impossible. Conversely, a relation that is both irreflexive and antisymmetric is asymmetric. For strict orders, the typical first-order axiom is

$$\forall x \forall y (x < y \rightarrow \neg(y < x)).$$

Remark (Dependencies).

- [Relation](#)
- [Universal Formula](#)

Remark (Asymmetric vs. antisymmetric). Every asymmetric relation is antisymmetric, but not conversely. $\leq$ on $\mathbb{R}$ is antisymmetric but not asymmetric (since $1 \leq 1$). Strict order $<$ is both asymmetric and irreflexive.

Definition (Transitive Relation)

Definition 8.1.33 (Transitive Relation). A binary relation $R \subseteq A \times A$ is *transitive* on A if any two composable related pairs collapse to a direct related pair:

$$\forall a, b, c \in A, ((a, b) \in R \wedge (b, c) \in R) \rightarrow (a, c) \in R.$$

In infix notation, this is $\forall a, b, c \in A, ((aRb \wedge bRc) \rightarrow aRc)$.

Remark (Standard quantified statement). Transitivity is the formal chaining law:

$$\text{Transitive}_A(R) \iff \forall a, b, c \in A, ((aRb \wedge bRc) \rightarrow aRc).$$

For equality, the corresponding first-order axiom is

$$\forall x \forall y \forall z ((x = y \wedge y = z) \rightarrow x = z).$$

Together with reflexivity and symmetry, transitivity gives equivalence relations; together with reflexivity and antisymmetry, it gives partial orders.

Remark (Dependencies).

- [Relation](#)
- [Universal Formula](#)

Definition (Total (Connex) Relation)

Definition 8.1.34 (Total (Connex) Relation). R is *total* (or *connex*) if every pair is comparable:

$$\forall a, b \in A, (a, b) \in R \vee (b, a) \in R.$$

Remark (Dependencies).

- [Relation](#)
- [Universal Formula](#)

Definition (Equivalence Relation)

Definition 8.1.35 (Equivalence Relation). R is an *equivalence relation* on A if it is reflexive, symmetric, and transitive.

Remark (Standard quantified statement). R is an equivalence relation on A iff:

$$(\forall a \in A, (a, a) \in R) \wedge (\forall a, b \in A, (a, b) \in R \rightarrow (b, a) \in R) \wedge (\forall a, b, c \in A, (a, b) \in R \wedge (b, c) \in R \rightarrow (a, c) \in R)$$

Remark (Role in mathematics). Equivalence relations axiomatize the idea of “sameness up to a chosen criterion.” They partition sets into equivalence classes and are the basis for quotient constructions throughout algebra, topology, and analysis.

Definition (Preorder)

Definition 8.1.36 (Preorder). R is a *preorder* on A if it is reflexive and transitive.

Remark (Standard quantified statement). R is a preorder on A iff:

$$(\forall a \in A, (a, a) \in R) \wedge (\forall a, b, c \in A, (a, b) \in R \wedge (b, c) \in R \rightarrow (a, c) \in R).$$

Remark (Dependencies).

- [Reflexive Relation](#)
- [Transitive Relation](#)

Definition (Partial Order)

Definition 8.1.37 (Partial Order). R is a *partial order* on A if it is reflexive, antisymmetric, and transitive.

Remark (Standard quantified statement). R is a partial order on A iff:

$$(\forall a \in A, (a, a) \in R) \wedge (\forall a, b \in A, (a, b) \in R \wedge (b, a) \in R \rightarrow a = b) \wedge (\forall a, b, c \in A, (a, b) \in R \wedge (b, c) \in R \rightarrow (a, c) \in R)$$

Remark (Dependencies).

- [Reflexive Relation](#)
- [Antisymmetric Relation](#)
- [Transitive Relation](#)

Definition (Strict Partial Order)

Definition 8.1.38 (Strict Partial Order). R is a *strict partial order* on A if it is irreflexive and transitive.

Remark (Standard quantified statement). R is a strict partial order on A iff:

$$(\forall a \in A, (a, a) \notin R) \wedge (\forall a, b, c \in A, (a, b) \in R \wedge (b, c) \in R \rightarrow (a, c) \in R).$$

Remark (Dependencies).

- [Irreflexive Relation](#)

- [Transitive Relation](#)

Remark (Asymmetry is automatic). Irreflexivity and transitivity together force asymmetry: if $(a, b) \in R$ and $(b, a) \in R$, transitivity gives $(a, a) \in R$, contradicting irreflexivity. Hence every strict partial order is asymmetric, and a strict partial order can equivalently be characterised as an asymmetric, transitive relation.

Remark (Correspondence with partial orders). Strict and non-strict orders carry the same information. Given a [partial order](#) $\leq$, the relation $a < b \iff (a \leq b \wedge a \neq b)$ is a strict partial order; conversely, given a strict partial order $<$, the relation $a \leq b \iff (a < b \vee a = b)$ is a partial order. The two presentations are interchangeable, which is why an ordered structure may be specified by either.

Definition (Total Order)

Definition 8.1.39 (Total Order). R is a *total order* on A if it is a partial order and is total.

Remark (Standard quantified statement). R is a total order on A iff R is a partial order on A and additionally:

$$\forall a, b \in A, (a, b) \in R \vee (b, a) \in R.$$

Remark (Dependencies).

- [Partial Order](#)
- [Total Relation](#)

Remark (Using structural properties in proofs). When R is asserted to be an equivalence relation or partial order, this is shorthand for the conjunction of its defining properties. In proofs, cite only the specific component needed: “since R is a partial order, antisymmetry implies ...” rather than restating all properties.

Example (Using a structural property in a proof). Let R be an equivalence relation on A . For any $a, b \in A$, $(a, b) \in R \Rightarrow [a] = [b]$, where $[a] = \{x \in A \mid (a, x) \in R\}$.

Remark (Properties are logically independent). No basic property implies another in general. A relation may be transitive without being reflexive, or symmetric without being transitive. The structural classes are distinguished precisely by requiring specific combinations.

8.2 Equivalence

Equivalence and Partitions — Quick Reference

Concept	Meaning
Equivalence class	$[a]_R = \{x \in A \mid (a, x) \in R\}$
Quotient set	A/R : all equivalence classes
Index of R	Cardinality of A/R
Canonical surjection	$\pi : A \rightarrow A/R, \pi(a) = [a]$
Partition	Nonempty, disjoint, covering collection of subsets
Rep. independence	$[a] = [b] \iff (a, b) \in R$
Equiv.–partition correspondence	Bijection between equiv. relations and partitions

8.2.1 Equivalence Classes and Partitions

Definition (Equivalence Class)

Definition 8.2.1 (Equivalence Class). Let R be an equivalence relation on A . For $a \in A$, the *equivalence class* of a is:

$$[a]_R := \{x \in A \mid (a, x) \in R\}.$$

When R is clear from context, we write $[a]$.

Remark (Dependencies).

- [Equivalence Relation](#)
- [Relation](#)

Remark (English reading). $[a]_R$ is the set of all elements that R declares “the same as a .” Two elements lie in the same class iff they are related: this is precisely the Representative Independence Lemma below.

Definition (Quotient Set)

Definition 8.2.2 (Quotient Set). The *quotient set* of A by R is the set of all equivalence classes:

$$A/R := \{[a]_R \mid a \in A\}.$$

Remark (Dependencies).

- [Equivalence Relation](#)
- [Equivalence Class](#)

Definition (Index of an Equivalence Relation)

Definition 8.2.3 (Index of an Equivalence Relation). The *index* of R on A is the cardinality $|A/R|$, i.e. the number of equivalence classes.

Remark (Standard quantified statement). $\text{index}(R) = |A/R| = |\{[a]_R \mid a \in A\}|$.

Remark (Dependencies).

- [Quotient Set](#)
- [Same Cardinality](#)

Definition (Canonical Surjection)

Definition 8.2.4 (Canonical Surjection). The *canonical surjection* (quotient map) is the function

$$\pi : A \rightarrow A/R, \quad \pi(a) := [a].$$

Remark (Forward reference). This definition uses the function vocabulary developed in the next chapter. It is included here because quotient sets are naturally introduced with their canonical map; proofs using its function-theoretic properties may be revisited after the function chapter.

Remark (Dependencies).

- [Function](#)
- [Quotient Set](#)
- [Equivalence Class](#)
- [Surjective Function](#)

Remark (Properties of π). π is surjective by construction. Elements $a, b \in A$ satisfy $\pi(a) = \pi(b)$ iff $(a, b) \in R$. The canonical surjection is the prototype for all quotient constructions: it reappears as the quotient homomorphism in algebra and the quotient map in topology.

Definition (Partition)

Definition 8.2.5 (Partition). A *partition* of A is a collection $\mathcal{P}$ of subsets of A such that:

- (i) every block is nonempty: $\forall P \in \mathcal{P}, P \neq \emptyset$;
- (ii) distinct blocks are disjoint: $\forall P, Q \in \mathcal{P}, P \neq Q \rightarrow P \cap Q = \emptyset$;
- (iii) the blocks cover A : $\bigcup_{P \in \mathcal{P}} P = A$.

The sets $P \in \mathcal{P}$ are called the *blocks* of the partition.

Remark (Dependencies).

- [Subset](#)
- [Empty Set](#)
- [Intersection](#)
- [Union](#)
- [Cover](#)

Remark (Partition vs. cover). Every partition of A is a cover of A whose members are nonempty and pairwise disjoint. Partitions are precisely the covers satisfying the disjointness condition.

Lemma (Representative Independence Lemma)

Lemma 8.2.6 (Representative Independence Lemma). *Let R be an equivalence relation on A . For any $a, b \in A$,*

$$[a] = [b] \iff (a, b) \in R.$$

Go to proof.

Remark (Standard quantified statement). $\forall a, b \in A : [a]_R = [b]_R \leftrightarrow (a, b) \in R$.

Remark (Dependencies).

- [Equivalence Relation](#)
- [Equivalence Class](#)
- [Partition](#)
- [Quotient Set](#)

Remark (Well-definedness on quotient sets). This lemma is the key fact underlying well-definedness of functions on quotient sets: a function $f : A \rightarrow B$ defined by $f([a]) = \dots$ is well-defined iff the formula gives the same output for all representatives of $[a]$.

Theorem (Equivalence Relations and Partitions)

Theorem 8.2.7 (Equivalence Relations and Partitions). *Let A be a set.*

- (i) *If R is an equivalence relation on A , then A/R is a partition of A .*
- (ii) *If $\mathcal{P}$ is a partition of A , then the relation $R_{\mathcal{P}}$ defined by*

$$(a, b) \in R_{\mathcal{P}} \iff \exists P \in \mathcal{P} \text{ with } a \in P \text{ and } b \in P$$

is an equivalence relation on A .

- (iii) *These constructions are inverse: $R_{A/R} = R$ and $A/R_{\mathcal{P}} = \mathcal{P}$.*

Go to proof.

Remark (Standard quantified statement). (i) $\forall a \in A, [a]_R \neq \emptyset; \forall a, b \in A, [a]_R \neq [b]_R \rightarrow [a]_R \cap [b]_R = \emptyset; \bigcup_{a \in A} [a]_R = A$.
(ii) $\forall a, b \in A : (a, b) \in R_P \leftrightarrow \exists P \in \mathcal{P}, a \in P \wedge b \in P$.

Remark (Dependencies).

- [Equivalence Relation](#)
- [Equivalence Class](#)
- [Quotient Set](#)
- [Canonical Surjection](#)
- [Function](#)
- [Composition](#)

Remark (Duality of perspectives). This theorem establishes a bijection between equivalence relations on A and partitions of A . The two perspectives—“same block” (partition) and “related” (equivalence relation)—are interchangeable and each is more natural in different contexts.

Example (Extremal equivalence relations). On any set A :

1. The *equality relation* $(a, b) \in R \iff a = b$ gives singleton classes $[a] = \{a\}$ and the finest partition of A .
2. The *universal relation* $R = A \times A$ gives $[a] = A$ for all a , and the coarsest partition (one block).

All other equivalence relations on A lie strictly between these extremes.

Theorem (Universal Property of the Quotient Map)

Theorem 8.2.8 (Universal Property of the Quotient Map). *Let R be an equivalence relation on A , let $\pi : A \rightarrow A/R$ be the canonical surjection, and let $f : A \rightarrow B$ be any function. Then f is constant on equivalence classes — meaning $(a, b) \in R$ implies $f(a) = f(b)$ — if and only if there exists a unique function $\bar{f} : A/R \rightarrow B$ such that*

$$f = \bar{f} \circ \pi.$$

When it exists, $\bar{f}$ is defined by $\bar{f}([a]) = f(a)$. Go to proof.

Remark (Forward reference). This universal property depends on functions, surjectivity, and composition. It is stated here to complete the quotient-set picture, but its dependency ceiling is intentionally forward-looking.

Remark (Standard quantified statement). $(\forall a, b \in A, (a, b) \in R \rightarrow f(a) = f(b)) \leftrightarrow (\exists! \bar{f} : A/R \rightarrow B, \forall a \in A, \bar{f}([a]) = f(a))$.

Remark (Dependencies).

- [Equivalence Relation](#)
- [Canonical Surjection](#)
- [Quotient Set](#)
- [Function](#)
- [Composition](#)

Remark (Why this is called the universal property). The condition $f = \bar{f} \circ \pi$ says the diagram commutes: going from A to B directly via f gives the same result as going first to A/R via π and then to B via $\bar{f}$. The word “universal” refers to the fact that $\pi : A \rightarrow A/R$ is the *unique* quotient with this property: any other map that “respects the equivalence” factors through π .

The uniqueness of $\bar{f}$ is forced by π being surjective: every element of A/R is $[a]$ for some a , so $\bar{f}([a]) = f(a)$ is the only possible definition.

Remark (Well-definedness is the key verification). The heart of the theorem is well-definedness: the formula $\bar{f}([a]) = f(a)$ could be ambiguous if $[a] = [b]$ but $f(a) \neq f(b)$. The hypothesis that f is constant on equivalence classes is precisely what prevents this: $(a, b) \in R$ (i.e., $[a] = [b]$) implies $f(a) = f(b)$, so the output does not depend on which representative of the class is chosen.

This “well-definedness check” recurs throughout mathematics. Whenever a function is defined on equivalence classes by a formula involving representatives, the first obligation is to verify that the formula gives the same answer for all representatives. The universal property tells exactly what condition on f makes this work.

Remark (Where this appears downstream). This theorem is not an abstract curiosity. It is the engine behind:

- *Integer construction*: the integers $\mathbb{Z}$ are defined as equivalence classes of pairs of natural numbers, and arithmetic operations are defined by formulas on representatives. The universal property verifies the operations are well-defined.
- *Rational construction*: $\mathbb{Q}$ is defined as equivalence classes of pairs (p, q) with $q \neq 0$, and addition/multiplication on representatives must be checked.
- *Quotient groups in algebra*: the quotient group G/N is defined by the same factorization, and the quotient homomorphism is π .
- *Quotient topology*: the quotient topological space is defined as the finest topology on A/R making π continuous — a condition stated precisely via the universal property.

Chapter 9

Functions



Remark (Source orientation). This chapter follows Tarski’s treatment of functions as special relations: first many-one relations, then one-one relations and bijective correspondences [Tarski \[1941\]](#).

9.1 Functions — Quick Reference

Functions — Quick Reference

Concept	Meaning
Functional relation	Each input has at most one output
Total relation on A	Each input has at least one output
Function	Left-total, right-unique relation
Partial function	Functional, not necessarily total
Function equality	Same domain, codomain, and pointwise values
Many-place function	Functional relation in the last coordinate
Domain	$\text{dom}(f) = A$ for $f : A \rightarrow B$
Codomain	$\text{cod}(f) = B$ for $f : A \rightarrow B$
Image of function	$\text{im}(f) = \{f(a) \mid a \in A\}$
Image of set	$f(S) = \{f(a) \mid a \in S\}$ for $S \subseteq A$
Preimage	$f^{-1}(T) = \{a \in A \mid f(a) \in T\}$ for $T \subseteq B$
Fiber	$f^{-1}(\{b\})$: preimage of a singleton
Graph	$\{(a, b) \in A \times B \mid b = f(a)\}$
Injective	Distinct inputs $\Rightarrow$ distinct outputs
Surjective	Every codomain element is achieved
Bijjective	Injective and surjective
Function test	Function iff total and functional
One-one test	Injective iff converse graph is functional
Bijjective test	Unique correspondence in both directions
Identity	$\text{id}_A(a) = a$
Inclusion map	$\iota : A \hookrightarrow B$, $\iota(a) = a$ for $A \subseteq B$
Composition	$(g \circ f)(a) = g(f(a))$
Inverse	Defined for bijections; $f^{-1} \circ f = \text{id}_A$
Left inverse	$g \circ f = \text{id}_A$
Right inverse	$f \circ h = \text{id}_B$
Restriction	$f _C : C \rightarrow B$
Extension	$g : A' \rightarrow B$ with $g _A = f$
Empty function	Unique function $\emptyset \rightarrow B$

9.1.1 Functions

Remark (Geometric picture). A function is a deterministic machine: every input in the domain is accepted, and no input can fork into two different outputs. Fibers are the level sets sitting over points of the codomain; they collect all inputs that the machine sends to the same output.

Definition (Functional Relation)

Definition 9.1.1 (Functional Relation). Let $R \subseteq A \times B$ be a relation. We say that R is *functional from A to B* if each input has at most one output:

$$\forall a \in A \forall b_1, b_2 \in B, \quad ((a, b_1) \in R \wedge (a, b_2) \in R) \Rightarrow b_1 = b_2.$$

Remark (Dependencies).

- [Relation](#)
- [Cartesian Product](#)

Definition (Total Relation on a Domain)

Definition 9.1.2 (Total Relation on a Domain). Let $R \subseteq A \times B$ be a relation. We say that R is *total on A* if each input has at least one output:

$$\forall a \in A \exists b \in B, \quad (a, b) \in R.$$

Remark (Dependencies).

- [Relation](#)

Definition (Function)

Definition 9.1.3 (Function). Let A and B be sets. A *function* from A to B is a relation $f \subseteq A \times B$, together with the declared domain A and codomain B , such that:

- (i) (*Existence / left-total*) for every $a \in A$, there exists $b \in B$ with $(a, b) \in f$;
- (ii) (*Uniqueness / right-unique*) if $(a, b_1) \in f$ and $(a, b_2) \in f$ then $b_1 = b_2$.

If $(a, b) \in f$ we write $f(a) = b$.

Remark (Dependencies).

- [Relation](#)
- [Cartesian Product](#)

Remark (English reading). A function is a rule assigning to each input exactly one output. The two conditions formalize “defined everywhere” (existence) and “single-valued” (uniqueness).

Remark (Function as relation). Every function is a relation, but not every relation is a function. The two conditions that distinguish functions are left-totality and right-uniqueness. The codomain is part of the function data, not something recoverable from the graph alone.

Theorem (Functional Relation Characterization of Functions)

Theorem 9.1.4 (Functional Relation Characterization of Functions). *Let A and B be sets, and let $R \subseteq A \times B$. Then R is a function from A to B if and only if*

$$\forall a \in A \exists! b \in B, \quad (a, b) \in R.$$

Equivalently, R is a function from A to B if and only if R is total on A and functional from A to B . Go to proof.

Remark (Dependencies).

- [Function](#)
- [Functional Relation](#)
- [Total Relation on a Domain](#)

Definition (Partial Function)

Definition 9.1.5 (Partial Function). Let A and B be sets. A *partial function* from A to B is a functional relation $R \subseteq A \times B$. It need not be total on A : some inputs may have no output.

Remark (Dependencies).

- [Functional Relation](#)

Proposition (Partial Functions Are Functional Relations)

Proposition 9.1.6 (Partial Functions Are Functional Relations). *For sets A and B and a relation $R \subseteq A \times B$, R is a partial function from A to B if and only if R is functional from A to B . Go to proof.*

Remark (Dependencies).

- [Partial Function](#)
- [Functional Relation](#)

Definition (Function Equality)

Definition 9.1.7 (Function Equality). For functions $f : A \rightarrow B$ and $g : C \rightarrow D$, we write $f = g$ if

$$A = C, \quad B = D, \quad \text{and} \quad \forall a \in A, f(a) = g(a).$$

Remark (Dependencies).

- [Function](#)

- [Set Equality](#)

Proposition (One-to-Many Relations Are Not Functional)

Proposition 9.1.8 (One-to-Many Relations Are Not Functional). *If $R \subseteq A \times B$ is one-to-many, then R is not functional from A to B . Consequently, R is not a function from A to B . Go to proof.*

Remark (Dependencies).

- [One-to-Many Relation](#)
- [Functional Relation](#)
- [Function](#)

Remark (Many-to-one is allowed). A function may send different inputs to the same output. That is the ordinary many-one case in Tarski's terminology. What is forbidden for a function is not shared output, but one input having two distinct outputs.

Definition (Domain of a Function)

Definition 9.1.9 (Domain of a Function). If f is a function from A to B , we write $f : A \rightarrow B$, where A is the *domain* $\text{dom}(f)$.

Remark (Dependencies).

- [Function](#)

Definition (Codomain of a Function)

Definition 9.1.10 (Codomain of a Function). If f is a function from A to B , we write $f : A \rightarrow B$, where B is the *codomain* $\text{cod}(f)$.

Remark (Dependencies).

- [Function](#)

Remark (Domain and codomain). The notation $f : A \rightarrow B$ records both the domain A and the codomain B . The same graph may define maps into larger codomains, but surjectivity and bijectivity depend on the declared codomain.

Definition (Image of a Function)

Definition 9.1.11 (Image of a Function). The *image* (or *range*) of $f : A \rightarrow B$ is:

$$\text{im}(f) := \{b \in B \mid \exists a \in A, f(a) = b\}.$$

The image may be a proper subset of the codomain.

Remark (Dependencies).

- [Function](#)
- [Set and Membership](#)
- [Axiom Schema of Separation](#)

Remark (Range and image). For a function viewed as its graph, the relation range and the function image coincide. The codomain may be larger than this range/image.

Definition (Image of a Set)

Definition 9.1.12 (Image of a Set). For $S \subseteq A$, the *image* of S under f is:

$$f(S) := \{ f(a) \mid a \in S \}.$$

Note $f(A) = \text{im}(f)$.

Remark (Dependencies).

- [Function](#)
- [Subset](#)
- [Axiom Schema of Separation](#)

Definition (Preimage)

Definition 9.1.13 (Preimage). For $T \subseteq B$, the *preimage* (inverse image) of T under f is:

$$f^{-1}(T) := \{ a \in A \mid f(a) \in T \}.$$

Remark (Dependencies).

- [Function](#)
- [Subset](#)
- [Axiom Schema of Separation](#)

Remark (Preimage notation warning). $f^{-1}(T)$ does *not* denote an inverse function. It is defined for any function f , regardless of whether f is injective or bijective.

Definition (Fiber)

Definition 9.1.14 (Fiber). The *fiber* of f over $b \in B$ is the preimage of the singleton:

$$f^{-1}(\{b\}) = \{ a \in A \mid f(a) = b \}.$$

Remark (Dependencies).

- [Function](#)

- [Preimage](#)

Remark (Fibers partition the domain). The collection $\{f^{-1}(\{b\}) \mid b \in \text{im}(f)\}$ is a partition of A . Every function thus induces an equivalence relation $a_1 \sim_f a_2 \iff f(a_1) = f(a_2)$, whose equivalence classes are precisely the fibers.

Definition (Graph of a Function)

Definition 9.1.15 (Graph of a Function). The *graph* of $f : A \rightarrow B$ is:

$$\text{Graph}(f) := \{ (a, b) \in A \times B \mid b = f(a) \}.$$

Remark (Dependencies).

- [Function](#)
- [Ordered Pair](#)
- [Cartesian Product](#)

Remark (Function as graph). A relation $G \subseteq A \times B$ is the graph of a function $f : A \rightarrow B$ iff G is left-total and right-unique. Functions may therefore be identified with their graphs only after the domain and codomain have been fixed. As bare sets of ordered pairs, graphs do not encode the codomain.

Definition (Injective Function)

Definition 9.1.16 (Injective Function). $f : A \rightarrow B$ is *injective* (one-to-one) if distinct elements of A have distinct images:

$$\forall a_1, a_2 \in A, f(a_1) = f(a_2) \implies a_1 = a_2.$$

Remark (Dependencies).

- [Function](#)

Definition (Empty Function)

Definition 9.1.17 (Empty Function). For every set B , the *empty function* from $\emptyset$ to B is the empty relation

$$\emptyset : \emptyset \rightarrow B.$$

It is the unique function with domain $\emptyset$ and codomain B .

Remark (Dependencies).

- [Function](#)
- [Empty Set](#)

- [Codomain of a Function](#)

Definition (Surjective Function)

Definition 9.1.18 (Surjective Function). $f : A \rightarrow B$ is *surjective* (onto) if every element of B is achieved:

$$\forall b \in B, \exists a \in A \text{ such that } f(a) = b.$$

Equivalently, $\text{im}(f) = B$.

Remark (Dependencies).

- [Function](#)

Definition (Bijective Function)

Definition 9.1.19 (Bijective Function). $f : A \rightarrow B$ is *bijective* if it is both injective and surjective.

Remark (Dependencies).

- [Function](#)
- [Injective Function](#)
- [Surjective Function](#)

Remark (Fiber interpretation). Injectivity: each fiber has at most one element. Surjectivity: every fiber is nonempty. Bijectivity: every fiber has exactly one element.

Proposition (One-One Relations and Converse Functionality)

Proposition 9.1.20 (One-One Relations and Converse Functionality). *Let $f : A \rightarrow B$ be a function, identified with its graph $G_f \subseteq A \times B$. Then f is injective if and only if the converse relation $G_f^{-1} \subseteq B \times A$ is functional from B to A . Go to proof.*

Remark (Dependencies).

- [Injective Function](#)
- [Converse Relation](#)
- [Functional Relation](#)
- [Graph of a Function](#)

Proposition (Bijections Have Unique Correspondence Both Ways)

Proposition 9.1.21 (Bijections Have Unique Correspondence Both Ways). *Let $f : A \rightarrow B$ be a function. Then f is bijective if and only if*

$$\forall a \in A \exists! b \in B, \quad f(a) = b,$$

and

$$\forall b \in B \exists! a \in A, \quad f(a) = b.$$

Go to proof.

Remark (Dependencies).

- [Bijective Function](#)
- [Injective Function](#)
- [Surjective Function](#)

Definition (Many-Place Functional Relation)

Definition 9.1.22 (Many-Place Functional Relation). A relation $R \subseteq A_1 \times \cdots \times A_n \times B$ is *functional in the last coordinate* if

$$\forall a_1 \in A_1 \cdots \forall a_n \in A_n \forall b_1, b_2 \in B,$$

$$((a_1, \dots, a_n, b_1) \in R \wedge (a_1, \dots, a_n, b_2) \in R) \Rightarrow b_1 = b_2.$$

When such a relation is also total in the first n coordinates, it determines a function of n variables, written $f(a_1, \dots, a_n) = b$.

Remark (Dependencies).

- [Many-Place Relation](#)
- [Functional Relation](#)

Definition (Identity Function)

Definition 9.1.23 (Identity Function). The *identity function* on A is $\text{id}_A : A \rightarrow A$, $\text{id}_A(a) = a$.

Remark (Dependencies).

- [Function](#)

Definition (Inclusion Map)

Definition 9.1.24 (Inclusion Map). If $A \subseteq B$, the *inclusion map* is $\iota : A \hookrightarrow B$, $\iota(a) = a$.

Remark (Dependencies).

- [Function](#)
- [Subset](#)

Remark (Identity vs. inclusion). Both send each element to itself, but the inclusion map has a strictly larger codomain when $A \subsetneq B$. The inclusion map is always injective.

Definition (Constant Function)

Definition 9.1.25 (Constant Function). $f : A \rightarrow B$ is *constant* if there exists $b_0 \in B$ with $f(a) = b_0$ for all $a \in A$.

Remark (Standard quantified statement). f is constant $\leftrightarrow \exists b_0 \in B, \forall a \in A, f(a) = b_0$.

Remark (Dependencies).

- [Function](#)

Composition & Inverses — Quick Reference

Result	Statement	Proof method
Associativity of $\circ$	$h \circ (g \circ f) = (h \circ g) \circ f$	element-wise
Identity and $\circ$	$f \circ \text{id}_A = \text{id}_B \circ f = f$	element-wise
Inj/Surj under $\circ$	Closed under composition; partial converses	element-wise
Inverse characterization	$f^{-1} \circ f = \text{id}_A, f \circ f^{-1} = \text{id}_B$	bijectivity
Inverse of composition	$(g \circ f)^{-1} = f^{-1} \circ g^{-1}$	verify both sides
One-sided inverses	Left inverse $\iff$ injective; right $\iff$ surjective	
Preimage preserves ops	f^{-1} commutes with $\cup, \cap, \setminus, c$	element-wise
Image and set ops	Images preserve $\cup$; contain $\subseteq$ for $\cap$	counterexample
Image/preimage adjunction	$S \subseteq f^{-1}(f(S)), f(f^{-1}(T)) \subseteq T$	element-wise

9.1.2 Composition, Inverses, and Set-Image Laws

Definition (Composition)

Definition 9.1.26 (Composition). Let $f : A \rightarrow B$ and $g : B \rightarrow C$. The *composition* $g \circ f : A \rightarrow C$ is:

$$(g \circ f)(a) := g(f(a)) \quad \text{for all } a \in A.$$

Remark (Dependencies).

- [Function](#)
- [Cartesian Product](#)

Remark (Non-commutativity). Composition is generally not commutative: $g \circ f \neq f \circ g$ even when both are defined.

Theorem (Associativity of Composition)

Theorem 9.1.27 (Associativity of Composition). For $f : A \rightarrow B$, $g : B \rightarrow C$, $h : C \rightarrow D$:

$$h \circ (g \circ f) = (h \circ g) \circ f.$$

Go to proof.

Remark (Standard quantified statement). $\forall f : A \rightarrow B$, $g : B \rightarrow C$, $h : C \rightarrow D$, $\forall a \in A$:
 $[h \circ (g \circ f)](a) = [(h \circ g) \circ f](a)$.

Remark (Dependencies).

- [Function](#)
- [Composition](#)

Theorem (Identity and Composition)

Theorem 9.1.28 (Identity and Composition). For any $f : A \rightarrow B$:

$$f \circ \text{id}_A = f \quad \text{and} \quad \text{id}_B \circ f = f.$$

Go to proof.

Remark (Standard quantified statement). $\forall f : A \rightarrow B$, $\forall a \in A$: $(f \circ \text{id}_A)(a) = f(a)$ and $(\text{id}_B \circ f)(a) = f(a)$.

Remark (Dependencies).

- [Function](#)
- [Composition](#)
- [Identity Function](#)

Theorem (Injectivity and Surjectivity Under Composition)

Theorem 9.1.29 (Injectivity and Surjectivity Under Composition). Let $f : A \rightarrow B$ and $g : B \rightarrow C$.

- (i) If f and g are injective, then $g \circ f$ is injective.
- (ii) If f and g are surjective, then $g \circ f$ is surjective.
- (iii) If $g \circ f$ is injective, then f is injective.
- (iv) If $g \circ f$ is surjective, then g is surjective.

Go to proof.

Remark (Standard quantified statement). (i) $(\forall a_1, a_2 \in A, f(a_1) = f(a_2) \rightarrow a_1 = a_2) \wedge (\forall b_1, b_2 \in B, g(b_1) = g(b_2) \rightarrow b_1 = b_2) \rightarrow \forall a_1, a_2 \in A, g(f(a_1)) = g(f(a_2)) \rightarrow a_1 = a_2$.
 (iii) $(\forall a_1, a_2 \in A, g(f(a_1)) = g(f(a_2)) \rightarrow a_1 = a_2) \rightarrow \forall a_1, a_2 \in A, f(a_1) = f(a_2) \rightarrow a_1 = a_2$.

Remark (Dependencies).

- [Function](#)
- [Composition](#)
- [Injective Function](#)
- [Surjective Function](#)

Remark (Sharpness of the implications). In (iii), g need not be injective. In (iv), f need not be surjective. Bijectivity of $g \circ f$ does not imply bijectivity of either f or g individually.

Definition (Inverse Function)

Definition 9.1.30 (Inverse Function). Let $f : A \rightarrow B$ be bijective. The *inverse function* is $f^{-1} : B \rightarrow A$ defined by: $f^{-1}(b) = a$ iff $f(a) = b$.

Remark (Dependencies).

- [Function](#)
- [Bijective Function](#)
- [Identity Function](#)

Theorem (Characterization of Inverse Functions)

Theorem 9.1.31 (Characterization of Inverse Functions). Let $f : A \rightarrow B$ be bijective. Then

$$f^{-1} \circ f = \text{id}_A \quad \text{and} \quad f \circ f^{-1} = \text{id}_B.$$

Conversely, a function admits an inverse iff it is bijective. Go to proof.

Remark (Standard quantified statement). $\forall a \in A : f^{-1}(f(a)) = a$ and $\forall b \in B : f(f^{-1}(b)) = b$.

Converse: $(\exists g : B \rightarrow A, g \circ f = \text{id}_A \wedge f \circ g = \text{id}_B) \leftrightarrow f$ is bijective.

Remark (Dependencies).

- [Function](#)
- [Composition](#)
- [Inverse Function](#)

- [Bijective Function](#)
- [Identity Function](#)

Theorem (Inverse of a Composition)

Theorem 9.1.32 (Inverse of a Composition). *Let $f : A \rightarrow B$ and $g : B \rightarrow C$ be bijective. Then $g \circ f$ is bijective and*

$$(g \circ f)^{-1} = f^{-1} \circ g^{-1}.$$

Go to proof.

Remark (Standard quantified statement). $\forall c \in C : (g \circ f)^{-1}(c) = f^{-1}(g^{-1}(c))$.

Remark (Dependencies).

- [Function](#)
- [Composition](#)
- [Inverse Function](#)
- [Bijective Function](#)

Definition (Left Inverse)

Definition 9.1.33 (Left Inverse). Let $f : A \rightarrow B$. A *left inverse* of f is a function $g : B \rightarrow A$ with $g \circ f = \text{id}_A$.

Remark (Dependencies).

- [Function](#)
- [Composition](#)
- [Identity Function](#)

Definition (Right Inverse)

Definition 9.1.34 (Right Inverse). Let $f : A \rightarrow B$. A *right inverse* (or *section*) of f is a function $h : B \rightarrow A$ with $f \circ h = \text{id}_B$.

Remark (Dependencies).

- [Function](#)
- [Composition](#)
- [Identity Function](#)

Remark (One-sided inverses). Left inverses and right inverses are the two one-sided inverse notions.

Theorem (One-Sided Inverses and Function Properties)

Theorem 9.1.35 (One-Sided Inverses and Function Properties). Let $f : A \rightarrow B$.

- (i) f has a left inverse $\iff f$ is injective (and $A \neq \emptyset$ or $A = B = \emptyset$).
- (ii) f has a right inverse $\iff f$ is surjective.
- (iii) If f has both a left inverse g and a right inverse h , then $g = h$ and f is bijective.

Go to proof.

Remark (Standard quantified statement). (i) $(\exists g : B \rightarrow A, \forall a \in A, g(f(a)) = a) \leftrightarrow \forall a_1, a_2 \in A, f(a_1) = f(a_2) \rightarrow a_1 = a_2$.

(ii) $(\exists h : B \rightarrow A, \forall b \in B, f(h(b)) = b) \leftrightarrow \forall b \in B, \exists a \in A, f(a) = b$.

Remark (Dependencies).

- [Function](#)
- [Composition](#)
- [Left Inverse](#)
- [Right Inverse](#)
- [Injective Function](#)
- [Surjective Function](#)
- [Bijective Function](#)

Remark (Axiom of Choice). The existence of a right inverse for every surjective function is equivalent to the Axiom of Choice.

Definition (Restriction)

Definition 9.1.36 (Restriction). Let $f : A \rightarrow B$ and $C \subseteq A$. The *restriction* of f to C is $f|_C : C \rightarrow B$, $f|_C(a) = f(a)$ for all $a \in C$.

Remark (Dependencies).

- [Function](#)
- [Subset](#)
- [Domain](#)

Definition (Extension)

Definition 9.1.37 (Extension). Let $A \subseteq A'$, and let $f : A \rightarrow B$. A function $g : A' \rightarrow B$ is an *extension* of f if $g|_A = f$.

Remark (Dependencies).

- [Function](#)
- [Subset](#)
- [Restriction](#)
- [Domain](#)
- [Codomain](#)

Remark (Extensions not unique). An extension agrees with f on all of A but may be defined on a larger domain A' . Extensions are generally not unique; uniqueness requires additional constraints (such as continuity in analysis or linearity in algebra), leading to important results like the Tietze Extension Theorem and the Hahn–Banach Theorem.

Theorem (Preimages Preserve Set Operations)

Theorem 9.1.38 (Preimages Preserve Set Operations). *Let $f : A \rightarrow B$ and $S, T \subseteq B$. Then*

- (i) $f^{-1}(S \cup T) = f^{-1}(S) \cup f^{-1}(T)$,
- (ii) $f^{-1}(S \cap T) = f^{-1}(S) \cap f^{-1}(T)$,
- (iii) $f^{-1}(S \setminus T) = f^{-1}(S) \setminus f^{-1}(T)$,
- (iv) $f^{-1}(S^c) = (f^{-1}(S))^c$.

Go to proof.

Remark (Standard quantified statement). (i) $\forall a \in A : a \in f^{-1}(S \cup T) \leftrightarrow (f(a) \in S \vee f(a) \in T)$.

(ii) $\forall a \in A : a \in f^{-1}(S \cap T) \leftrightarrow (f(a) \in S \wedge f(a) \in T)$.

(iv) $\forall a \in A : a \in f^{-1}(S^c) \leftrightarrow f(a) \notin S$.

Remark (Dependencies).

- [Function](#)
- [Preimage](#)
- [Union](#)
- [Intersection](#)
- [Set Difference](#)
- [Complement](#)

Theorem (Images and Set Operations)

Theorem 9.1.39 (Images and Set Operations). *Let $f : A \rightarrow B$ and $S, T \subseteq A$. Then*

- (i) $f(S \cup T) = f(S) \cup f(T)$,
- (ii) $f(S \cap T) \subseteq f(S) \cap f(T)$,
- (iii) $f(S \setminus T) \supseteq f(S) \setminus f(T)$.

Equality holds in (ii) and (iii) for all S, T iff f is injective. Go to proof.

Remark (Standard quantified statement). (i) $\forall b \in B : b \in f(S \cup T) \leftrightarrow (\exists a \in S, f(a) = b) \vee (\exists a \in T, f(a) = b)$.
(ii) $\forall b \in B : (\exists a \in S \cap T, f(a) = b) \rightarrow (\exists a \in S, f(a) = b) \wedge (\exists a \in T, f(a) = b)$; equality iff f injective.

Remark (Dependencies).

- [Function](#)
- [Image of a Set](#)
- [Union](#)
- [Intersection](#)
- [Set Difference](#)
- [Injective Function](#)

Theorem (Image–Preimage Adjunction)

Theorem 9.1.40 (Image–Preimage Adjunction). *Let $f : A \rightarrow B$, let $S \subseteq A$, and let $T \subseteq B$. Then*

$$S \subseteq f^{-1}(f(S)), \quad f(f^{-1}(T)) \subseteq T.$$

Moreover, $S = f^{-1}(f(S))$ for every $S \subseteq A$ iff f is injective, and $f(f^{-1}(T)) = T$ for every $T \subseteq B$ iff f is surjective. Go to proof.

Remark (Dependencies).

- [Function](#)
- [Image of a Set](#)
- [Preimage](#)
- [Injective Function](#)
- [Surjective Function](#)

Remark (Images, preimages, and Boolean operations).

Operation	Preimage law, for $S, T \subseteq B$	Image law, for $C, D \subseteq A$
Union	$f^{-1}(S \cup T) = f^{-1}(S) \cup f^{-1}(T)$	$f(C \cup D) = f(C) \cup f(D)$
Intersection	$f^{-1}(S \cap T) = f^{-1}(S) \cap f^{-1}(T)$	$f(C \cap D) \subseteq f(C) \cap f(D)$; equality for all C, D iff f is injective
Difference	$f^{-1}(S \setminus T) = f^{-1}(S) \setminus f^{-1}(T)$	$f(C \setminus D) \supseteq f(C) \setminus f(D)$; equality for all C, D iff f is injective
Complement	$f^{-1}(B \setminus S) = A \setminus f^{-1}(S)$	In general, $f(A \setminus C)$ need not equal $B \setminus f(C)$

Remark (Composition and inverse interaction).

Construction	Set-operation interaction	Reference
Composition	If $f : A \rightarrow B$, $g : B \rightarrow C$, and $U \subseteq C$, then $(g \circ f)^{-1}(U) = f^{-1}(g^{-1}(U))$.	Composition , Preimage
Inverse function	If $f : A \rightarrow B$ is bijective, inverse images along f^{-1} are ordinary preimages for the function $f^{-1} : B \rightarrow A$.	Inverse Function , Inverse Characterization
Inverse of composition	If f and g are bijective, then $(g \circ f)^{-1} = f^{-1} \circ g^{-1}$, so inverse order reverses composition.	Inverse of a Composition
One-sided inverse	A left inverse detects injectivity; a right inverse detects surjectivity. These properties control when image inclusions become equalities.	One-Sided Inverses

Remark (Asymmetry between images and preimages). Preimages preserve union, intersection, difference, and complement exactly. Forward images preserve unions exactly, but generally do not preserve intersections, differences, or complements. The inclusions for intersections and differences become equalities uniformly when the function is injective. This asymmetry is fundamental and explains why preimages appear more naturally in topology and measure theory.

Chapter 10

Orderings



10.1 Order

Ordered Sets — Quick Reference	
Concept	Meaning
Partially ordered set (poset)	$(A, \leq)$: set with a partial order
Strict order	$a < b \iff a \leq b \wedge a \neq b$
Comparable / incomparable	$a \leq b$ or $b \leq a$ / neither
Totally ordered set (toset/loset)	Partial order with all pairs comparable
Upper / lower bound	$u \geq s$ / $\ell \leq s$ for all $s \in S$
Minimal element	No smaller element in S
Maximal element	No larger element in S
Least element	Smaller than all elements of S
Greatest element	Larger than all elements of S
Order-preserving map	Function f with $a \leq b \Rightarrow f(a) \leq' f(b)$
Order isomorphism	Bijjective map preserving and reflecting order
Well-ordered set (woset)	Every nonempty subset has a least element
Chain / antichain	All comparable / none comparable
Initial segment	Downward-closed subset

10.1.1 Ordered Sets

Definition (Ordered Set)

Definition 10.1.1 (Ordered Set). An *ordered set* is a set equipped with an order relation. Unless explicitly qualified, “ordered set” means a **partially ordered set**, and its relation is written $\leq$.

Remark (Dependencies).

- [Relation](#)
- [Partial Order](#)

Remark (Genus and species). “Ordered set” is a genus term: a species is fixed by specifying which order relation the carrier carries — a [partial order](#) gives a [poset](#), a [total order](#) gives a [toset](#), and so on. The order relations themselves (preorder, partial order, total order) are defined once, in the Relations chapter; this chapter studies the *structures* built on them and never restates a relation’s axioms, so the two chapters cannot drift apart.

Definition (Partially Ordered Set)

Definition 10.1.2 (Partially Ordered Set). A *partially ordered set* is a pair $(A, \leq)$ where A is a set and $\leq$ is a [partial order](#) on A (a relation that is reflexive, antisymmetric, and transitive).

Remark (Dependencies).

- [Partial Order](#)

Remark (English reading). A partially ordered set is a set with a specified way to compare elements that need not make every pair comparable.

Definition (Order-Structure Synonyms)

Definition 10.1.3 (Order-Structure Synonyms). The following short forms abbreviate their full names and carry no independent mathematical content; each resolves to the canonical structure it names:

- $\textit{poset} :=$ [partially ordered set](#).
- $\textit{toset}$, $\textit{loset}$, $\textit{linearly ordered set} :=$ [totally ordered set](#) (total order and linear order coincide, so these name one structure).
- $\textit{woset} :=$ [well-ordered set](#).

Definition (Strict Order)

Definition 10.1.4 (Strict Order). Let $(A, \leq)$ be an ordered set. The *strict order* $<$ is defined by:

$$a < b \iff (a \leq b \wedge a \neq b).$$

Remark (Dependencies).

- [Ordered Set](#)
- [Partial Order](#)

Remark (Strict and non-strict are equivalent). The relation $<$ is irreflexive and transitive. Conversely, given any strict partial order $<$, one recovers a non-strict order by setting $a \leq b \iff (a < b \vee a = b)$. The two formulations carry exactly the same information.

Definition (Comparable and Incomparable Elements)

Definition 10.1.5 (Comparable and Incomparable Elements). In an ordered set $(A, \leq)$, elements $a, b \in A$ are *comparable* if $a \leq b$ or $b \leq a$; they are *incomparable* if neither holds.

Remark (Standard quantified statement). a, b comparable $\leftrightarrow a \leq b \vee b \leq a$. a, b incomparable $\leftrightarrow \neg(a \leq b) \wedge \neg(b \leq a)$.

Remark (Incomparable elements). Incomparable elements can only occur in partial orders. In a total order, every pair is comparable by definition.

Remark (Dependencies).

- [Partially Ordered Set](#)
- [Total Order](#)
- [Comparable and Incomparable Elements](#)

Definition (Totally Ordered Set)

Definition 10.1.6 (Totally Ordered Set). A *totally ordered set* (or *linearly ordered set*) is a pair $(A, \leq)$ where $\leq$ is a [total order](#) on A — equivalently, a partially ordered set in which every pair of elements is [comparable](#):

$$\forall a, b \in A, a \leq b \vee b \leq a.$$

Remark (Examples). $(\mathbb{R}, \leq)$ is a total order. $(\mathcal{P}(A), \subseteq)$ is a partial order that is generally not total.

Definition (Upper and Lower Bounds)

Definition 10.1.7 (Upper and Lower Bounds). Let $(A, \leq)$ be an ordered set and $S \subseteq A$. An element $u \in A$ is an *upper bound* of S if $\forall s \in S, s \leq u$. An element $\ell \in A$ is a *lower bound* of S if $\forall s \in S, \ell \leq s$.

Remark (Dependencies).

- [Ordered Set](#)
- [Subset](#)

Remark (Consequence). Bounds need not exist and need not be unique. They need not lie in S itself. Bounds are central to the definition of completeness for ordered fields.

Definition (Minimal Element)

Definition 10.1.8 (Minimal Element). Let $S \subseteq A$ in an ordered set $(A, \leq)$. An element $m \in S$ is a *minimal element* of S if there is no $s \in S$ with $s < m$.

Remark (Dependencies).

- [Ordered Set](#)
- [Subset](#)
- [Strict Order](#)

Definition (Maximal Element)

Definition 10.1.9 (Maximal Element). Let $S \subseteq A$ in an ordered set $(A, \leq)$. An element $M \in S$ is a *maximal element* of S if there is no $s \in S$ with $M < s$.

Remark (Dependencies).

- [Ordered Set](#)
- [Subset](#)
- [Strict Order](#)

Definition (Least Element)

Definition 10.1.10 (Least Element). Let $S \subseteq A$ in an ordered set $(A, \leq)$. An element $\ell \in S$ is the *least element* of S if

$$\forall s \in S, \ell \leq s.$$

Remark (Dependencies).

- [Ordered Set](#)
- [Subset](#)

Definition (Greatest Element)

Definition 10.1.11 (Greatest Element). Let $S \subseteq A$ in an ordered set $(A, \leq)$. An element $g \in S$ is the *greatest element* of S if

$$\forall s \in S, s \leq g.$$

Remark (Dependencies).

- [Ordered Set](#)
- [Subset](#)

Remark (Minimal vs. least). A *least* element is below every element of S ; it must be unique if it exists. A *minimal* element merely has nothing below it within S ; there may be many. Every least element is minimal, but not conversely.

Definition (Order-Preserving Map)

Definition 10.1.12 (Order-Preserving Map). Let $(M, \leq)$ and $(M', \leq')$ be partially ordered sets. A function $f : M \rightarrow M'$ is *order-preserving* (or *monotone*) if

$$\forall a, b \in M, \quad a \leq b \implies f(a) \leq' f(b).$$

Remark (Dependencies).

- [Partially Ordered Set](#)
- [Function](#)

Definition (Order Isomorphism)

Definition 10.1.13 (Order Isomorphism). Let $(M, \leq)$ and $(M', \leq')$ be partially ordered sets. A function $f : M \rightarrow M'$ is an *order isomorphism* if it is bijective and

$$\forall a, b \in M, \quad a \leq b \iff f(a) \leq' f(b).$$

Remark (Dependencies).

- [Partially Ordered Set](#)
- [Function](#)
- [Bijective Function](#)

Remark (Order-isomorphic posets). An order isomorphism preserves and reflects the order structure exactly, including comparability, minimal and maximal elements, and bounds. Two posets are *order-isomorphic* if such a map exists; they are structurally identical from the order-theoretic viewpoint.

Definition (Well-Ordered Set)

Definition 10.1.14 (Well-Ordered Set). A [totally ordered set](#) $(A, \leq)$ is *well-ordered* if every nonempty subset $S \subseteq A$ has a least element:

$$\forall S \subseteq A, \quad (S \neq \emptyset \implies \exists m \in S \text{ s.t. } m \leq s \forall s \in S).$$

Remark (Dependencies).

- [Totally Ordered Set](#)
- [Subset](#)

Remark (Well-order implies total order). Every well-ordered set is totally ordered. The well-ordering condition is strictly stronger: it requires a least element in every nonempty subset, not just in A as a whole.

Example (Well-order examples). $(\mathbb{N}, \leq)$ is well-ordered. $(\mathbb{Z}, \leq)$ is not: the set $\{\dots, -3, -2, -1\}$ has no least element. $(\mathbb{R}, \leq)$ is not: $(0, 1)$ has no least element.

Remark (Connection to ordinal numbers). An *ordinal number* is defined as an equivalence class of well-ordered sets under order isomorphism: it measures the *order type* of a well-ordered set, not merely its cardinality.

Definition (Chain and Antichain)

Definition 10.1.15 (Chain and Antichain). A subset $C \subseteq A$ of a poset $(A, \leq)$ is a *chain* if every pair of elements in C is comparable. It is an *antichain* if no two distinct elements of C are comparable.

Remark (Standard quantified statement). C is a chain $\leftrightarrow \forall a, b \in C, a \leq b \vee b \leq a$.
 C is an antichain $\leftrightarrow \forall a, b \in C, a \neq b \rightarrow \neg(a \leq b) \wedge \neg(b \leq a)$.

Definition (Initial Segment)

Definition 10.1.16 (Initial Segment). A subset $I \subseteq A$ is an *initial segment* of $(A, \leq)$ if

$$a \in I \text{ and } b \leq a \Rightarrow b \in I.$$

Remark (Standard quantified statement). I is an initial segment $\leftrightarrow \forall a \in I, \forall b \in A, b \leq a \rightarrow b \in I$.

Remark (Transition). Ordered sets provide the abstract framework for order structures on the real numbers, function spaces, and metric spaces. In later sections, order interacts with topology and analysis through intervals, monotone functions, and the completeness axiom for $\mathbb{R}$.

Definition (Directed Set)

Definition 10.1.17 (Directed Set). A preorder $(I, \leq)$ is *directed* (or a *directed set*) if every finite subset of I has an upper bound in I : for all $i, j \in I$ there exists $k \in I$ with $i \leq k$ and $j \leq k$.

Remark (Dependencies).

- [Partial Order](#)
- [Upper and Lower Bounds](#)
- [Finite and Infinite Sets](#)

Remark (What directed means). In a directed set, any two elements can always be “dominated” by a third. The condition does not require a maximum element to exist, only that no two elements are ever incomparable in a final way: there is always a common upper bound

available. Every totally ordered set is directed (take $k = \max(i, j)$), but directed sets can be far more general.

Example (Canonical directed sets). (i) $(\mathbb{N}, \leq)$ is directed: given $m, n \in \mathbb{N}$, take $k = \max(m, n)$.

(ii) The collection of all finite subsets of a set X , ordered by inclusion, is directed: given finite sets $F_1, F_2 \subseteq X$, take $k = F_1 \cup F_2$.

(iii) The set of all open neighborhoods of a point x in a topological space, ordered by reverse inclusion ($U \leq V$ iff $U \supseteq V$), is directed: given neighborhoods U and V of x , take $k = U \cap V$, which is also a neighborhood of x .

(iv) A partially ordered set is directed if and only if every finite subset has an upper bound. Partially ordered sets that are *not* directed have “incomparable maximal elements” — elements with no common upper bound.

Definition (Net)

Definition 10.1.18 (Net). Let $(I, \leq)$ be a directed set and X a set. A *net* in X is a function $x : I \rightarrow X$. The value at $i \in I$ is written x_i , and the net is written $(x_i)_{i \in I}$.

Remark (Dependencies).

- [Directed Set](#)
- [Function](#)

Remark (Nets generalize sequences). A sequence is a net with index set $(\mathbb{N}, \leq)$. Nets are the correct generalization of sequences to general topological spaces: in a metric space, sequences suffice to detect all topological properties (closure, continuity, compactness). In a general topological space, sequences can fail to detect these properties — one needs nets. Specifically, a subset A of a topological space is closed if and only if every convergent net in A has its limit in A ; and a function is continuous if and only if it preserves limits of nets.

The definition of net is placed here because it is purely order-theoretic: a directed index set and a function. The analytic content — what it means for a net to converge — belongs to topology and functional analysis. By establishing the set-theoretic definition now, those chapters can immediately state convergence conditions without set-theoretic digression.

Supremum, Infimum, and Bounds — Quick Reference

Concept	Meaning
Upper bound	$u \geq s$ for all $s \in S$
Lower bound	$\ell \leq s$ for all $s \in S$
Supremum	Least upper bound of S
Infimum	Greatest lower bound of S
Uniqueness	Suprema and infima are unique when they exist
Max/min vs. sup/inf	Maximum $\Rightarrow$ supremum; not conversely
Duality	$\sup$ in $(A, \leq) = \inf$ in $(A, \geq)$

10.1.2 Supremum and Infimum

Every student of analysis encounters the least upper bound property of $\mathbb{R}$ early and prominently. But this property is not a peculiarity of real numbers: it is an order-theoretic concept that can be formulated in any partially ordered set. Before we reach $\mathbb{R}$, it is essential to understand supremum and infimum in their natural habitat — abstract ordered sets — so that when the completeness axiom appears in Volume II, you will recognise exactly what it is asserting and why $\mathbb{Q}$ fails to satisfy it.

The definitions in this section are also the backbone of measure theory. The extended real line $\overline{\mathbb{R}} = \mathbb{R} \cup \{-\infty, +\infty\}$, equipped with the usual order, is the ambient poset in which measures take their values. Countable additivity, the monotone convergence theorem, and Fatou's lemma are all statements about suprema of sequences. Understanding supremum and infimum at the abstract level here means those later theorems will feel inevitable rather than arbitrary.

The definition of upper and lower bound was given in [the Ordered Sets section](#). For convenience we recall: $u \in A$ is an *upper bound* of $S \subseteq A$ if $\forall s \in S, s \leq u$; a *lower bound* satisfies the reverse. We say S is *bounded above* (resp. *bounded below*, *bounded*) if it has at least one upper (resp. lower, both) bound.

Remark (Bounds depend on the ambient poset). Whether S is bounded above depends on the poset $(A, \leq)$, not on S alone. The set $\{r \in \mathbb{Q} : r^2 < 2\}$ is bounded above in $(\mathbb{Q}, \leq)$ — the rational number 2 is an upper bound — but it has *no* supremum inside $\mathbb{Q}$. The same set in $(\mathbb{R}, \leq)$ has supremum $\sqrt{2} \in \mathbb{R}$. Being bounded above is necessary but not sufficient for a supremum to exist; the completeness axiom for $\mathbb{R}$ is precisely the assertion that sufficiency holds in $\mathbb{R}$.

Definition (Supremum and Infimum)

Definition 10.1.19 (Supremum and Infimum). Let $(A, \leq)$ be a poset and $S \subseteq A$. The *supremum* (or *least upper bound*) of S , written $\sup S$, is an element $u^* \in A$ satisfying:

- (i) u^* is an upper bound of S : $\forall s \in S, s \leq u^*$;
- (ii) u^* is the *least* upper bound: if u is *any* upper bound of S , then $u^* \leq u$.

The *infimum* (or *greatest lower bound*) of S , written $\inf S$, is an element $\ell^* \in A$ satisfying:

- (i) ℓ^* is a lower bound of S : $\forall s \in S, \ell^* \leq s$;
- (ii) ℓ^* is the *greatest* lower bound: if ℓ is *any* lower bound of S , then $\ell \leq \ell^*$.

Remark (Standard quantified statement). $u^* = \sup S \leftrightarrow (\forall s \in S, s \leq u^*) \wedge (\forall u \in A, (\forall s \in S, s \leq u) \rightarrow u^* \leq u)$.

$\ell^* = \inf S \leftrightarrow (\forall s \in S, \ell^* \leq s) \wedge (\forall \ell \in A, (\forall s \in S, \ell \leq s) \rightarrow \ell \leq \ell^*)$.

Remark (Dependencies).

- [Upper and Lower Bounds](#)

- [Partial Order](#)
- [Subset](#)

Remark (How to read these definitions). Condition (i) says u^* is a *candidate*: it stands above everything in S . Condition (ii) says u^* is the *best* candidate: it is at most as large as any other upper bound. Together they say: u^* is the unique element of A that is simultaneously an upper bound of S and no larger than any other upper bound.

A useful restatement of condition (ii) is its contrapositive: if $v < u^*$, then v is *not* an upper bound of S , meaning there exists some $s \in S$ with $v < s$. This is how the condition is typically used in proofs in analysis: to verify that $u^* = \sup S$, you show (i) u^* is an upper bound, and (ii) for every $\varepsilon > 0$, the value $u^* - \varepsilon$ is *not* an upper bound, i.e., there exists $s \in S$ with $s > u^* - \varepsilon$.

Proposition

Proposition 10.1.20 (Uniqueness of Supremum and Infimum). *Let $(A, \leq)$ be a poset and $S \subseteq A$. If $\sup S$ exists, it is unique. If $\inf S$ exists, it is unique.*

Remark (Standard quantified statement). $\forall u^*, v^* \in A$: if both u^* and v^* satisfy the two conditions in the definition of $\sup S$, then $u^* = v^*$.

Remark (Dependencies).

- [Supremum and Infimum](#)
- [Partial Order](#)
- [Subset](#)

Remark (Proof idea). If u^* and v^* are both suprema, then u^* is an upper bound, so the minimality of v^* gives $v^* \leq u^*$; and symmetrically $u^* \leq v^*$. Antisymmetry of the partial order then forces $u^* = v^*$. This is a standard antisymmetry argument: when two elements are mutually $\leq$ -related in a partial order, they must be equal.

Remark (Supremum need not belong to S). The set $S = (0, 1) \subseteq \mathbb{R}$ is open: it contains neither endpoint. Yet $\sup S = 1$ and $\inf S = 0$. Neither 0 nor 1 belongs to S .

When the supremum does happen to lie in S , it is called the *maximum* of S , written $\max S$. Thus $\max S = \sup S$ when $\sup S \in S$, and $\max S$ is undefined when $\sup S \notin S$. Every maximum is a supremum, but the converse fails: a supremum may exist without being a maximum. The same relationship holds between infimum and minimum.

This distinction is critical in analysis. A continuous function on a closed bounded interval always achieves its maximum (the Extreme Value Theorem), but on an open interval it may approach a supremum without ever attaining it.

Remark (Supremum need not exist). Being bounded above is necessary but not sufficient for a supremum to exist. Consider $S = \{r \in \mathbb{Q} : r^2 < 2\}$ in the poset $(\mathbb{Q}, \leq)$. Every element of S satisfies $r < 2$, so $2 \in \mathbb{Q}$ is an upper bound. But the candidate supremum would need to be $\sqrt{2}$, and $\sqrt{2} \notin \mathbb{Q}$. Any rational upper bound $q > \sqrt{2}$ is not the least upper bound,

because $\frac{q+\sqrt{2}}{2}$ is a smaller upper bound (and is also irrational, which means we can always find a rational strictly between any rational upper bound and $\sqrt{2}$, and that rational is still an upper bound). In short: S is bounded above in $\mathbb{Q}$, but $\sup S$ does not exist in $\mathbb{Q}$.

This is the precise failure that the *completeness axiom* for $\mathbb{R}$ corrects: it asserts that every nonempty subset of $\mathbb{R}$ that is bounded above has a supremum in $\mathbb{R}$. The rational numbers satisfy every other ordered field axiom but fail this one.

Example (Sup and inf in divisibility). Order the set $D = \{1, 2, 3, 4, 6, 12\}$ by divisibility: $a \leq b$ iff $a \mid b$. Take $S = \{4, 6\}$.

Upper bounds of S : an element $u \in D$ must satisfy $4 \mid u$ and $6 \mid u$, i.e., u must be a common multiple of 4 and 6 in D . The only such element is 12, so $\sup S = 12 = \text{lcm}(4, 6)$.

Lower bounds of S : an element $\ell \in D$ must satisfy $\ell \mid 4$ and $\ell \mid 6$, i.e., ℓ must be a common divisor of 4 and 6 in D . The common divisors are $\{1, 2\}$; the greatest is 2, so $\inf S = 2 = \text{gcd}(4, 6)$.

This example reveals a deep pattern: in any divisibility poset, $\sup$ computes the least common multiple and $\inf$ computes the greatest common divisor. Order theory and number theory are speaking the same language.

Example (Sup and inf for function spaces). Let $B(X)$ be the set of bounded real-valued functions on a set X , ordered pointwise: $f \leq g$ iff $f(x) \leq g(x)$ for all $x \in X$. For a collection $\{f_\alpha\}$ of bounded functions, the pointwise supremum is $(\sup_\alpha f_\alpha)(x) = \sup_\alpha f_\alpha(x)$. In measure theory and integration theory, one constantly takes suprema and infima of sequences of functions; this example shows that such operations are instances of the abstract $\sup/\inf$ defined here.

Proposition

Proposition 10.1.21 (Characterisation of Supremum). *Let $(A, \leq)$ be a poset, $S \subseteq A$, and $u^* \in A$. Then $u^* = \sup S$ if and only if:*

- (i) u^* is an upper bound of S ; and
- (ii) for every $v \in A$ with $v < u^*$, there exists $s \in S$ with $v < s$.

Remark (Standard quantified statement). $u^* = \sup S \leftrightarrow (\forall s \in S, s \leq u^*) \wedge (\forall v \in A, v < u^* \rightarrow \exists s \in S, v < s)$.

Remark (Dependencies).

- [Supremum and Infimum](#)
- [Partial Order](#)
- [Upper and Lower Bounds](#)
- [Subset](#)

Remark (Why this characterisation is useful). Condition (ii) here is the contrapositive of the minimality condition in the definition. In analysis this is the working form: to check that

u^* is the supremum, you verify that nothing strictly below u^* is an upper bound. In $\mathbb{R}$ this becomes: for every $\varepsilon > 0$, there exists $s \in S$ with $s > u^* - \varepsilon$. This is the exact form used in the proof that sup interacts correctly with limits, integrals, and measures.

Proposition

Proposition 10.1.22 (Duality of Supremum and Infimum). *Let $(A, \leq)$ be a poset and $S \subseteq A$. Then $\inf_{(A, \leq)} S = \sup_{(A, \geq)} S$, where $(A, \geq)$ denotes the dual poset. In particular: if every nonempty bounded-above subset of $(A, \leq)$ has a supremum, then every nonempty bounded-below subset of $(A, \leq)$ has an infimum.*

Remark (Standard quantified statement). $\forall S \subseteq A$: ℓ^* is the infimum of S in $(A, \leq) \leftrightarrow \ell^*$ is the supremum of S in $(A, \geq)$.

Remark (Dependencies).

- [Supremum and Infimum](#)
- [Partial Order](#)
- [Duality Principle for Posets](#)
- [Subset](#)

Remark (Consequence for $\mathbb{R}$). This proposition explains why the completeness axiom for $\mathbb{R}$ is stated only for suprema: the infimum statement comes for free by duality. The axiom says every nonempty bounded-above subset has a supremum; the proposition immediately gives that every nonempty bounded-below subset has an infimum. You will use this constantly without comment in Volume II.

Remark (Sequences and sup/inf). A sequence $(a_n)_{n=1}^\infty$ in $\mathbb{R}$ is a function $\mathbb{N} \rightarrow \mathbb{R}$, so its range $\{a_n : n \in \mathbb{N}\}$ is a subset of $\mathbb{R}$. The quantities $\sup_n a_n$ and $\inf_n a_n$ are the supremum and infimum of this range set in the poset $(\mathbb{R}, \leq)$.

More subtle are the *limit superior* and *limit inferior*:

$$\limsup_{n \rightarrow \infty} a_n = \inf_{n \geq 1} \sup_{k \geq n} a_k, \quad \liminf_{n \rightarrow \infty} a_n = \sup_{n \geq 1} \inf_{k \geq n} a_k.$$

These are iterated suprema and infima. The fact that they always exist in $\overline{\mathbb{R}}$ (the extended reals, where $\pm\infty$ are adjoined to supply missing bounds) is a direct application of the completeness of $\mathbb{R}$. Mastering the abstract definitions here makes those formulas transparent rather than mysterious.

Remark (Transition). Supremum and infimum are defined for arbitrary subsets of a poset, but they need not always exist. When *every* two-element subset $\{a, b\}$ has both a supremum and an infimum, the poset has enough structure to define binary operations $a \vee b = \sup\{a, b\}$ and $a \wedge b = \inf\{a, b\}$. This additional structure is called a *lattice*, and it is developed in the next section. Lattices are the precise abstract setting for the σ -algebras and topologies encountered in measure theory and topology.

Concept	Meaning
Lattice	Poset with $\sup\{a, b\}$ and $\inf\{a, b\}$ for all a, b
Join / Meet	$a \vee b = \sup\{a, b\}$; $a \wedge b = \inf\{a, b\}$
Complete lattice	Every subset has sup and inf
Distributive lattice	$a \wedge (b \vee c) = (a \wedge b) \vee (a \wedge c)$
Boolean lattice	Complemented distributive lattice
$\mathcal{P}(X)$	Complete Boolean lattice under $\subseteq$
σ -algebras	Sub-lattices of $\mathcal{P}(X)$ closed under countable ops
Knaster–Tarski	Every monotone map on a complete lattice has a fixed point

10.1.3 Lattices

The preceding section defined supremum and infimum for arbitrary subsets of a poset. In many posets that appear in practice, every pair of elements already has a supremum and an infimum. When this is the case, the binary operations $a \vee b = \sup\{a, b\}$ and $a \wedge b = \inf\{a, b\}$ are well-defined everywhere, giving the poset the algebraic structure of a *lattice*.

Lattices are ubiquitous:

- The power set $\mathcal{P}(X)$ ordered by $\subseteq$ is a complete lattice, with join = union and meet = intersection.
- Every σ -algebra is a sub-lattice of $\mathcal{P}(X)$ that is additionally closed under countable joins and meets and under complementation.
- Every topology is a sub-structure of $\mathcal{P}(X)$ that is closed under arbitrary joins and finite meets.
- In functional analysis, the lattice of closed subspaces of a Hilbert space is a complete lattice under inclusion.

Understanding lattices here means that when σ -algebras, topologies, and sub- σ -algebras are encountered downstream, their closure conditions will be recognisable instances of a single abstract pattern.

Definition (Lattice)

Definition 10.1.23 (Lattice). A *lattice* is a poset $(L, \leq)$ in which every two-element subset $\{a, b\} \subseteq L$ has both a supremum and an infimum in L .

The supremum $\sup\{a, b\}$ is called the *join* of a and b , written $a \vee b$. The infimum $\inf\{a, b\}$ is called the *meet* of a and b , written $a \wedge b$.

Remark (Standard quantified statement). $(L, \leq)$ is a lattice $\leftrightarrow \forall a, b \in L, \exists a \vee b \in L$ and $\exists a \wedge b \in L$, where $a \vee b$ and $a \wedge b$ satisfy the sup and inf conditions of [Definition \(Supremum and Infimum\)](#).

Remark (Join and meet as operations). In a lattice, $\vee$ and $\wedge$ are binary operations $L \times L \rightarrow L$. They satisfy, for all $a, b, c \in L$:

- (i) *Commutativity*: $a \vee b = b \vee a$ and $a \wedge b = b \wedge a$.
- (ii) *Associativity*: $(a \vee b) \vee c = a \vee (b \vee c)$ and $(a \wedge b) \wedge c = a \wedge (b \wedge c)$.
- (iii) *Idempotency*: $a \vee a = a$ and $a \wedge a = a$.
- (iv) *Absorption*: $a \vee (a \wedge b) = a$ and $a \wedge (a \vee b) = a$.

Properties (i)–(iv) follow directly from the order-theoretic definitions of join and meet. Conversely, any set with two binary operations satisfying (i)–(iv) can be given a partial order making it a lattice: define $a \leq b$ iff $a \wedge b = a$. The two approaches — order-theoretic and algebraic — are equivalent.

Example (Power set lattice). For any set X , the power set $(\mathcal{P}(X), \subseteq)$ is a lattice: $A \vee B = A \cup B$ and $A \wedge B = A \cap B$. The algebraic laws of the Set Algebra section (commutativity, associativity, distributivity, absorption) are exactly the lattice laws in this example. This is not a coincidence: Boolean algebra is a special kind of lattice.

Example (Divisibility lattice). The divisibility order on $\mathbb{N}^+ = \{1, 2, 3, \dots\}$ makes it a lattice: $a \vee b = \text{lcm}(a, b)$ and $a \wedge b = \text{gcd}(a, b)$, as we noted in Example 4.1.2.

Example (Not every poset is a lattice). Consider the poset $(\{a, b, c, d\}, \leq)$ with $a < c$, $a < d$, $b < c$, $b < d$, and a, b incomparable. The subset $\{a, b\}$ has two minimal upper bounds, c and d , which are incomparable. Hence $\sup\{a, b\}$ does not exist, and this poset is not a lattice.

The power set $\mathcal{P}(\{1, 2, 3\})$ restricted to subsets of cardinality $\neq 1$ provides a cleaner example: $\{1, 2\}$ and $\{1, 3\}$ have no meet in this restricted collection.

Definition (Complete Lattice)

Definition 10.1.24 (Complete Lattice). A lattice $(L, \leq)$ is a *complete lattice* if every subset $S \subseteq L$ (including the empty set and L itself) has both a supremum and an infimum in L :

$$\forall S \subseteq L, \quad \sup S \in L \quad \text{and} \quad \inf S \in L.$$

Remark (Standard quantified statement). $(L, \leq)$ is a complete lattice $\leftrightarrow \forall S \subseteq L, \exists u^* \in L, \exists \ell^* \in L$ satisfying the sup and inf conditions of Definition (Supremum and Infimum).

Remark (Dependencies).

- [Partial Order](#)
- [Subset](#)
- [Supremum and Infimum](#)

Remark (The empty set forces top and bottom). When $S = \emptyset$, the sup condition says: u^* is an upper bound of the empty set (which is vacuously satisfied by every element of L), and u^*

is at most every other upper bound. This forces $u^* = \inf L$, the *bottom element* of L (often written $\perp$). Symmetrically, $\inf \emptyset = \sup L$, the *top element* (often written $\top$).

Concretely: in $(\mathcal{P}(X), \subseteq)$, we have $\sup \emptyset = \emptyset$ (the empty union) and $\inf \emptyset = X$ (the empty intersection equals the whole universe, by convention). Every complete lattice therefore has a top element $\top = \sup L$ and a bottom element $\perp = \inf L$.

Example ($\mathcal{P}(X)$ is a complete lattice). For any set X , the power set $(\mathcal{P}(X), \subseteq)$ is a complete lattice: for any family $\{A_\alpha\}_{\alpha \in I} \subseteq \mathcal{P}(X)$,

$$\sup_{\alpha \in I} A_\alpha = \bigcup_{\alpha \in I} A_\alpha, \quad \inf_{\alpha \in I} A_\alpha = \bigcap_{\alpha \in I} A_\alpha.$$

The bottom element is $\emptyset$ and the top element is X . This is the canonical example of a complete lattice, and every σ -algebra on X is a complete sub-lattice of $(\mathcal{P}(X), \subseteq)$.

Example ($[0, \infty]$ is a complete lattice). The extended positive half-line $[0, \infty] = [0, \infty) \cup \{+\infty\}$ with the usual order is a complete lattice. Every nonempty subset has a supremum (which may be $+\infty$) and an infimum (which may be 0). This is the value space for measures: a measure assigns to each measurable set a value in $[0, \infty]$, and the key properties of measures (countable additivity, monotone convergence) are stated in terms of sups and infs in this complete lattice.

Definition (Distributive Lattice)

Definition 10.1.25 (Distributive Lattice). A lattice $(L, \leq)$ is *distributive* if for all $a, b, c \in L$:

$$a \wedge (b \vee c) = (a \wedge b) \vee (a \wedge c).$$

Remark (Dependencies).

- [Lattice](#)
- [Subset](#)
- [Supremum and Infimum](#)

Remark (One law implies the other). By the duality principle for lattices, distributivity of $\wedge$ over $\vee$ is equivalent to distributivity of $\vee$ over $\wedge$: $a \vee (b \wedge c) = (a \vee b) \wedge (a \vee c)$. Proving one gives the other for free.

Remark (Why distributivity matters). The distributive law is what makes a lattice behave like set algebra. In the power set $\mathcal{P}(X)$, the distributive laws are exactly the set identities $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$ from Theorem 3.22. Distributivity is the condition that separates lattices that “look like sets” from those that do not (e.g., the lattice of closed subspaces of a Hilbert space is not distributive).

Definition (Complement and Boolean Lattice)

Definition 10.1.26 (Complement and Boolean Lattice). Let $(L, \leq)$ be a bounded distributive lattice with top element $\top$ and bottom element $\perp$. An element $a' \in L$ is a *complement* of $a \in L$ if

$$a \vee a' = \top \quad \text{and} \quad a \wedge a' = \perp.$$

A *Boolean lattice* (or *Boolean algebra*) is a bounded distributive lattice in which every element has a complement.

Remark (The power set is the canonical Boolean lattice). In $\mathcal{P}(X)$, the complement of a set A is $A^c = X \setminus A$, since $A \cup A^c = X = \top$ and $A \cap A^c = \emptyset = \perp$. The power set is therefore a complete Boolean lattice. Every σ -algebra on X is a Boolean sub-algebra of $\mathcal{P}(X)$: it is closed under complements (by definition) and under finite meets and joins (since every σ -algebra is an algebra).

Remark (σ -algebras as sub-lattices). A σ -algebra $\mathcal{M}$ on X is, from the lattice-theoretic viewpoint, a sub-structure of $(\mathcal{P}(X), \subseteq)$ that is:

- a complete sub-lattice (closed under all countable joins and meets);
- a Boolean sub-algebra (closed under complementation).

The condition “closed under countable joins” is exactly the σ -algebra axiom that countable unions of measurable sets are measurable. The condition “closed under meets” for countable families follows from De Morgan together with closure under countable unions and complements.

This perspective makes the definition of σ -algebra feel natural: it is exactly a complete Boolean sub-algebra of the power set lattice, where “complete” is interpreted countably (countable joins and meets), not set-theoretically.

Remark (Topologies as join-sub-structures). A topology τ on X is *not* a sub-lattice of $\mathcal{P}(X)$ in general, because it is not closed under arbitrary meets (i.e., arbitrary intersections of open sets need not be open). However, τ is closed under arbitrary joins (arbitrary unions of open sets are open) and finite meets (finite intersections of open sets are open). This makes τ an *inf-semilattice* for finite meets and a complete *join-semilattice*. The failure of closure under infinite meets is fundamental: it is precisely what allows open sets to “separate” points, and it is the reason topology is structurally different from measure theory.

Theorem (Knaster–Tarski Fixed-Point Theorem)

Let $(L, \leq)$ be a complete lattice and $f : L \rightarrow L$ a monotone function (i.e., $a \leq b \Rightarrow f(a) \leq f(b)$). Then f has a least fixed point and a greatest fixed point in L . Specifically:

$$\text{lfp}(f) = \inf\{a \in L : f(a) \leq a\}, \quad \text{gfp}(f) = \sup\{a \in L : a \leq f(a)\}.$$

Remark (Standard quantified statement). $\forall$ complete lattices $(L, \leq)$, $\forall$ monotone $f : L \rightarrow L$: $\exists p, q \in L$ such that $f(p) = p$, $f(q) = q$, and $\forall r \in L$ with $f(r) = r$: $p \leq r \leq q$.

Remark (Why this theorem appears here). The Knaster–Tarski theorem is not needed for most of Volume I, but it is placed here because its hypotheses — complete lattice, monotone function — are now in place and the statement is short. Its significance reaches further:

- In set theory, it is used to prove the Schröder–Bernstein theorem without the Axiom of Choice (the monotone map sends $A \mapsto X \setminus g(Y \setminus f(A))$ on $\mathcal{P}(X)$).
- In measure theory, it is used to construct the smallest σ -algebra containing a given class of sets (the generated σ -algebra is a fixed point of the “closure under countable operations” operator).
- In theoretical computer science and domain theory, it is the foundation for denotational semantics.

For now, note that $(\mathcal{P}(X), \subseteq)$ is a complete lattice and any set operation that is monotone (such as $A \mapsto \overline{A}$, closure in a topological space, or $A \mapsto \sigma(A)$, generated σ -algebra) has a fixed point by this theorem.

Remark (Transition). With the notion of lattice in hand, the Hasse diagram becomes a more powerful visualisation tool: the Hasse diagram of a lattice has a unique bottom element ($\inf L$) and a unique top element ($\sup L$), and the join and meet of any two elements can be read off as the unique path structure in the diagram. The next section introduces Hasse diagrams, the duality principle, and the connection between duality and the passage from $\sup$ to $\inf$.

Remark (Dependencies).

- [Distributive Lattice](#)
- [Complete Lattice](#)

Hasse Diagrams and Duality — Quick Reference	
Concept	Meaning
Cover relation	$a \lessdot b$: $a < b$ with nothing strictly between
Hasse diagram	Graph encoding a poset; edges only for cover relations
Dual poset	$(A, \geq)$ obtained by reversing the order on $(A, \leq)$
Duality principle	Every theorem about posets yields a dual theorem for free

10.1.4 Hasse Diagrams and the Duality Principle

A partial order can be an enormous set of pairs, impossible to visualise directly. The *Hasse diagram* is a compression: it retains only the pairs that are “immediate” in the order, discarding those implied by transitivity and reflexivity. From the compressed diagram the full order can be recovered by taking transitive closure. Hasse diagrams are the standard visual language for finite posets; knowing how to read and draw them is essential for understanding the order examples in topology and lattice theory.

Definition (Cover Relation)

Definition 10.1.27 (Cover Relation). Let $(A, \leq)$ be a poset and $a, b \in A$. We say b *covers* a , written $a \triangleleft b$, if $a < b$ and there is no $c \in A$ with $a < c < b$.

Remark (Standard quantified statement). $a \triangleleft b \leftrightarrow (a < b) \wedge \neg(\exists c \in A, a < c \wedge c < b)$.

Remark (Intuition). The cover relation strips the partial order down to its “essential edges.” If $a \triangleleft b$, then b is the *immediate successor* of a : you cannot insert anything between them. In a finite poset, every strict relation $a < b$ factors as a finite chain of cover steps: $a = a_0 \triangleleft a_1 \triangleleft \dots \triangleleft a_k = b$. In an infinite poset, this factorisation may fail (e.g., in $(\mathbb{R}, \leq)$, no two distinct real numbers are in a cover relation because there is always a rational strictly between them).

Remark (Dependencies).

- [Partial Order](#)
- [Strict Order](#)

Definition (Hasse Diagram)

Definition 10.1.28 (Hasse Diagram). The *Hasse diagram* of a finite poset $(A, \leq)$ is the directed graph whose vertices are the elements of A , with an upward edge from a to b whenever $a \triangleleft b$. Three graphical conventions apply:

- (i) Reflexive loops ($a \leq a$) are suppressed.
- (ii) Edges implied by transitivity are suppressed: if $a \triangleleft b$ and $b \triangleleft c$, no direct edge from a to c is drawn.
- (iii) Direction is encoded by height: $a \triangleleft b$ is drawn with b strictly above a , so arrowheads are unnecessary.

Remark (Dependencies).

- [Partial Order](#)
- [Cover Relation](#)
- [Finite and Infinite Sets](#)

Remark (How to recover the full order). The full order can be read back from the Hasse diagram: $a \leq b$ if and only if either $a = b$ or there is an upward path in the diagram from a to b (a sequence of cover edges going strictly upward). The diagram is therefore a lossless encoding of the partial order, compressed by removing the redundant reflexive and transitive edges.

Example (Divisibility on $\{1, 2, 3, 4, 6, 12\}$). Order the set $D = \{1, 2, 3, 4, 6, 12\}$ by divisibility ($a \leq b$ iff $a \mid b$). The covers are:

$$1 \lessdot 2, \quad 1 \lessdot 3, \quad 2 \lessdot 4, \quad 2 \lessdot 6, \quad 3 \lessdot 6, \quad 4 \lessdot 12, \quad 6 \lessdot 12.$$

The Hasse diagram places 1 at the bottom and 12 at the top. The relation $1 \leq 4$ holds in the poset but the edge $1 \rightarrow 4$ is *not* drawn because $1 \lessdot 2 \lessdot 4$ provides an upward path. Drawing the direct edge would be redundant.

Notice also: 4 and 6 are incomparable, so there is no edge between them — they sit at the same level with no path connecting them. The join $4 \vee 6 = 12 = \text{lcm}(4, 6)$ and the meet $4 \wedge 6 = 2 = \text{gcd}(4, 6)$ are readable from the diagram as the lowest common ancestor and the highest common descendant, respectively.

Example (Power set $\mathcal{P}(\{a, b, c\})$ under inclusion). The Hasse diagram of $(\mathcal{P}(\{a, b, c\}), \subseteq)$ has four levels: $\emptyset$ at the bottom; the three singletons $\{a\}, \{b\}, \{c\}$ one level up; the three two-element sets $\{a, b\}, \{a, c\}, \{b, c\}$ above those; and $\{a, b, c\}$ at the top. Each singleton is covered by the two two-element sets containing it. This diagram is the Boolean lattice B_3 : it has a bottom ($\emptyset$), a top ($\{a, b, c\}$), and every element has a complement ($A' = \{a, b, c\} \setminus A$).

Remark (Limitation to finite posets). For infinite posets such as $(\mathbb{N}, \leq)$, the cover relation is well-defined ($n \lessdot n+1$) but the full diagram cannot be drawn. In $(\mathbb{R}, \leq)$ or $(\mathbb{Q}, \leq)$, the cover relation is empty: no two elements are in a cover relation. Hasse diagrams are a practical visualisation tool for finite or schematic examples and should not be used as a proof device in infinite settings.

Definition (Dual Poset)

Definition 10.1.29 (Dual Poset). Let $(A, \leq)$ be a poset. The *dual poset* is $(A, \geq)$, where $a \geq b$ is defined to mean $b \leq a$. The dual is also written $(A, \leq^{\text{op}})$.

Remark (Dependencies).

- [Partial Order](#)
- [Relation](#)

Remark (Intuition). The dual poset is obtained by flipping the Hasse diagram upside down. Every structural feature is preserved but reflected: elements at the top are now at the bottom, minimal elements become maximal, upper bounds become lower bounds, and suprema become infima and vice versa (as established in Proposition 4.1.22).

Proposition

Proposition 10.1.30 (Duality Principle for Posets). Let Φ be any first-order statement about a poset $(A, \leq)$ expressed using only the relation $\leq$. Let Φ^* be the statement obtained from Φ by replacing every occurrence of $\leq$ with $\geq$ (equivalently, working in the dual poset). Then:

$$(A, \leq) \models \Phi \iff (A, \geq) \models \Phi.$$

In particular, if Φ is a theorem about all posets, then so is Φ^* .

Remark (Standard quantified statement). $\forall$ posets $(A, \leq)$, $\forall$ first-order sentences Φ in the language of $\leq$: $(A, \leq) \models \Phi \leftrightarrow (A, \geq) \models \Phi^*$, where Φ^* substitutes $\geq$ for every $\leq$.

Remark (Dependencies).

- [Partial Order](#)
- [Relation](#)

Remark (Theorems come in pairs at no extra cost). The duality principle means every proved theorem about posets automatically yields a second theorem — its dual — without any additional work. You will encounter this repeatedly in analysis. Here are the most important instances:

- *Supremum $\leftrightarrow$ Infimum.* Every theorem about sup yields a theorem about inf by duality. In particular, the completeness axiom for $\mathbb{R}$ (every nonempty bounded-above subset has a supremum) immediately implies that every nonempty bounded-below subset has an infimum — not a separate axiom but a consequence by duality.
- *Minimal $\leftrightarrow$ Maximal.* A statement about minimal elements dualises to a statement about maximal elements. Zorn’s Lemma (every poset in which every chain is bounded above has a maximal element) dualises to: every poset in which every chain is bounded below has a minimal element.
- *Well-ordering is not self-dual.* $(\mathbb{N}, \leq)$ is well-ordered (every nonempty subset has a least element); its dual $(\mathbb{N}, \geq)$ is not well-ordered because the whole set has no greatest element. Duality preserves the form of a definition but not whether a particular structure satisfies it.
- *Join $\leftrightarrow$ Meet in lattices.* Every lattice identity involving $\vee$ and $\wedge$ dualises by swapping the two operations. The distributive law $a \wedge (b \vee c) = (a \wedge b) \vee (a \wedge c)$ dualises to $a \vee (b \wedge c) = (a \vee b) \wedge (a \vee c)$: proving one gives the other for free.

Remark (Transition). The order vocabulary developed across the order sections — partial orders, total orders, well-orders, bounds, suprema, infima, lattices, directed sets, and duality — is the complete toolkit for order-theoretic reasoning in analysis. In Volume II, the construction of $\mathbb{R}$ requires only the completeness axiom (existence of suprema) and the Dedekind cut or Cauchy sequence constructions, both of which use only the vocabulary established here. In Volume III, metric space arguments, the definition of compactness, and the construction of Lebesgue measure all draw on this vocabulary repeatedly.

Order Extensions — Quick Reference

Concept	Meaning
Preorder	Reflexive, transitive (not necessarily antisymmetric)
Loaset / toset	Synonym for totally ordered set (total = linear order)
Symmetric part	$xIy \iff xRy \wedge yRx$
Asymmetric part	$xPy \iff xRy \wedge \neg(yRx)$
Transitive closure	Smallest transitive relation containing R
Extension	$\succsim'$ agrees with $\succsim$ on strict and weak ranks
OWC	$xT(\succsim)y \Rightarrow \neg(y \succ x)$
Szpilrajn's Thm	Every partial order extends to a linear order

10.1.5 Order Extensions

Definition (Symmetric and Asymmetric Parts)

Definition 10.1.31 (Symmetric and Asymmetric Parts). For any binary relation R on X , define:

- the *symmetric part*: $xIy \iff xRy \text{ and } yRx$
- the *asymmetric part*: $xPy \iff xRy \text{ but not } yRx$

Note that $R = P \cup I$.

Remark (Dependencies).

- [Relation](#)
- [Union](#)
- [Set Difference](#)

Remark (Preference-theoretic reading). In decision theory, R encodes a preference relation: xRy means “ x is at least as good as y .” Then I is the *indifference* relation (equally good) and P is the *strict preference* relation (strictly better). This decomposition is fundamental to utility representation theory.

Remark (Preorder and loaset are defined elsewhere). The order relations this section uses are defined in the Relations chapter: a [preorder](#) is reflexive and transitive, and a [linear order](#) is a total (connex) partial order. The corresponding *structure* — the [loaset](#), i.e. the linearly/totally ordered set — is catalogued with the order structures in the Ordered Sets section. Nothing is restated here, so no definition can drift.

Remark (Preorder vs. partial order). A preorder need not be antisymmetric: two distinct elements may satisfy xRy and yRx simultaneously (they are “indifferent” but not equal). Adding antisymmetry promotes a preorder to a partial order. Every partial order is a preorder, but not conversely.

Definition (Maximal Elements and Upper Bounds)

Definition 10.1.32 (Maximal Elements and Upper Bounds). Let $\succsim$ be a binary relation on X .

- The set of *maximal elements* of X is

$$\text{Max}(X, \succsim) = \{x \in X \mid y \succsim x \text{ for no } y \in X \text{ with } y \neq x\}.$$

- The set of *upper bounds* of X is

$$M(X, \succsim) = \{x \in X \mid x \succsim y \text{ for all } y \in X\}.$$

Remark (Dependencies).

- [Relation](#)
- [Upper and Lower Bounds](#)
- [Strict Order](#)

Remark (Maximal vs. greatest). A maximal element has nothing strictly above it; an upper bound (greatest element) is above everything. In a partial order, the greatest element is unique if it exists and is always maximal, but maximal elements need not be greatest. In a linear order, maximal and greatest coincide.

Remark (Transitive closure (defined in the Relations chapter)). The *transitive closure* $T(R)$ of a binary relation R on X — the [smallest transitive relation containing \$R\$](#) — is defined in the Relations chapter. We recall it here only to use it, and do not restate the definition, so the two chapters cannot drift apart. (For the explicit construction, see the description below.)

Remark (Dependencies).

- [Transitive Closure \(Relations\)](#)
- [Transitive Relation](#)

Remark (Constructive description). Define $R^0 = R$ and $xR^m y$ if there exist $z_1, \dots, z_m \in X$ such that $xRz_1R \cdots Rz_mRy$. Then

$$T(R) = R \cup \bigcup_{m \in \mathbb{N}} R^m.$$

Existence follows from the fact that the intersection of any collection of transitive relations is transitive, so the intersection of all transitive supersets of R is the smallest such relation.

Definition (Extension of a Preorder)

Definition 10.1.33 (Extension of a Preorder). Let $\lesssim$ be a preorder on X . A preorder $\lesssim'$ is an *extension* of $\lesssim$ if

$$x \lesssim y \Rightarrow x \lesssim' y, \quad x \succ y \Rightarrow x \succ' y.$$

A *complete extension* is an extension that is also complete.

Remark (Dependencies).

- [Preorder](#)
- [Relation](#)

Remark (What extensions do). An extension of $\lesssim$ “fills in” the comparisons left undecided by $\lesssim$ without reversing any existing strict comparison. The goal is to promote a partial ranking to a total ranking while respecting the original judgements.

Theorem (Szpilrajn, 1930)

For any nonempty set X and partial order $\lesssim$ on X , there exists a linear order that is an extension of $\lesssim$.

Remark (Proof sketch via Hausdorff Maximum Principle). Let T_X be the set of all partial orders on X extending $\lesssim$, ordered by inclusion. By the Hausdorff Maximum Principle (equivalent to AC), T_X has a maximal chain; its union $\lesssim^*$ is a partial order extending $\lesssim$. If $\lesssim^*$ were not complete, one could enlarge it via transitive closure, contradicting maximality. Hence $\lesssim^*$ is a linear order extending $\lesssim$.

Dependence on AC. The theorem is equivalent to the Axiom of Choice for infinite sets; no proof avoiding AC is known.

Corollary (Complete preorder extension)

Corollary 10.1.34 (Complete preorder extension). *For any nonempty set X and preorder $\lesssim$ on X , there exists a complete preorder that is an extension of $\lesssim$.*

Remark (Proof). Proof details.

Remark (Standard quantified statement). $\forall X \neq \emptyset, \forall \lesssim$ preorder on X : $\exists \lesssim'$ complete preorder on X s.t. $(x \lesssim y \rightarrow x \lesssim' y) \wedge (x \succ y \rightarrow x \succ' y)$.

Remark (Dependencies).

- [Preorder](#)
- [Extension of a Preorder](#)
- [Quotient Set](#)

- [Symmetric and Asymmetric Parts](#)

Remark (Proof). Pass to the quotient $X/\sim$ (where $\sim$ is the symmetric part), apply Szpilrajn’s Theorem to obtain a linear order on $X/\sim$, then pull back to a complete preorder on X .

Definition and Proposition (Only Weak Cycles)

Definition 10.1.35 (and Proposition (Only Weak Cycles)). Let $\succsim$ be a binary relation on X with asymmetric part $\succ$ and transitive closure $T(\succsim)$. We say $\succsim$ satisfies *only weak cycles* (OWC) if

$$xT(\succsim)y \Rightarrow \neg(y \succ x).$$

Proposition. A binary relation $\succsim$ on a nonempty set X can be extended to a complete preorder if and only if it satisfies OWC.

Remark (Dependencies).

- [Relation](#)
- [Symmetric and Asymmetric Parts](#)
- [Transitive Closure](#)
- [Extension of a Preorder](#)

Remark (Intuition for OWC). OWC prohibits the following: there is a “chain” of weak preferences $x_1 \succsim x_2 \succsim \cdots \succsim x_n$ but a strict reversal $x_n \succ x_1$. Such a configuration cannot be extended to a complete preorder because the chain forces $x_1 \succsim' x_n$ in any extension, but strict preference $x_n \succ x_1$ in the extension would mean $x_n \succsim' x_1$ but not $x_1 \succsim' x_n$ — contradicting $x_1 \succsim' x_n$.

Remark (Transition to AC). Szpilrajn’s Theorem relies on Zorn’s Lemma / Hausdorff Maximum Principle, which are equivalent to the Axiom of Choice. The next section develops these equivalences directly.

Induced Orders and Order Embeddings — Quick Reference

Concept	Meaning
Induced preorder	$x \leq_f y \Leftrightarrow f(x) \leq' f(y)$; always a preorder
Induced partial order	Induced order is a partial order iff f is injective
f-indistinguishable	$x \sim_f y$: both $x \leq_f y$ and $y \leq_f x$
Order embedding	Injective, order-preserving, and order-reflecting
Embedding vs. isomorphism	Embedding is iso onto image; isomorphism is surjective too
Suborder	Restriction of $\leq'$ to a subset $S \subseteq B$
<i>Key results:</i>	
$\leq_f$ preorder: always. Antisymmetry: iff f injective.	
f order embedding $\Rightarrow (A, \leq_f) \cong (f(A), \leq')$.	

10.1.6 Induced Orders and Order Embeddings

The simplest way to put an order on a set A is to compare elements of A *indirectly* through a function into an already-ordered set. This construction, called the *induced order* or *pullback order*, is ubiquitous: it explains how the usual order on $\mathbb{Z}$ restricts to $\mathbb{N}$, how functions on a common domain acquire a pointwise order, and how subsets inherit the order of the ambient space.

Definition (Induced Order)

Definition 10.1.36 (Induced Order). Let $(B, \leq')$ be a partially ordered set and let $f : A \rightarrow B$ be a function. The *order induced by f* (or *pullback order along f*) is the relation $\leq_f$ on A defined by:

$$x \leq_f y \iff f(x) \leq' f(y).$$

Remark (Intuition). Two elements of A are compared by sending them through f and comparing their images in $(B, \leq')$. The order on A is entirely inherited from B via f : we never compare elements of A directly, only their f -values.

Remark (Standard quantified statement). $\forall x, y \in A, \quad x \leq_f y \iff f(x) \leq' f(y)$.

Remark (Dependencies).

- [Partial Order](#)
- [Function](#)
- [Relation](#)

Proposition

Proposition 10.1.37 ($\leq_f$ is always a preorder). *For any function $f : A \rightarrow B$ and partial order $(B, \leq')$, the relation $\leq_f$ is a preorder on A : it is reflexive and transitive.*

Remark (Standard quantified statement). $\forall x \in A : f(x) \leq' f(x)$ (reflexivity); $\forall x, y, z \in A : f(x) \leq' f(y) \wedge f(y) \leq' f(z) \rightarrow f(x) \leq' f(z)$ (transitivity).

Remark (Dependencies).

- [Induced Order](#)
- [Preorder](#)
- [Function](#)
- [Partial Order](#)

Remark (Proof strategy). Both properties are inherited directly from $(B, \leq')$ by pulling back through f : reflexivity at x uses reflexivity at $f(x)$, and transitivity along x, y, z uses transitivity along $f(x), f(y), f(z)$.

Definition (f -Indistinguishable Elements)

Definition 10.1.38 (f -Indistinguishable Elements). Let $f : A \rightarrow B$ and $\leq_f$ be the induced order. Two elements $x, y \in A$ are f -indistinguishable, written $x \sim_f y$, if both $x \leq_f y$ and $y \leq_f x$, i.e. if

$$f(x) \leq' f(y) \quad \text{and} \quad f(y) \leq' f(x).$$

Remark (Dependencies).

- [Induced Order](#)
- [Function](#)
- [Partial Order](#)

Remark (Intuition). Two elements are f -indistinguishable when they map to the same position in the $\leq'$ -order — not necessarily to the same *point*, but to mutually comparable points. When $\leq'$ is a partial order (antisymmetric), $x \sim_f y$ forces $f(x) = f(y)$, making f non-injective the only obstruction to antisymmetry.

Proposition

Proposition 10.1.39 ($\leq_f$ is a partial order iff f is injective). Let $(B, \leq')$ be a partially ordered set and $f : A \rightarrow B$. The induced order $\leq_f$ is a partial order on A if and only if f is injective.

Remark (Standard quantified statement). $\leq_f$ is a partial order $\leftrightarrow \forall x, y \in A, f(x) \leq' f(y) \wedge f(y) \leq' f(x) \rightarrow x = y \leftrightarrow \forall x, y \in A, f(x) = f(y) \rightarrow x = y$ (injectivity of f , since $\leq'$ is antisymmetric).

Remark (Dependencies).

- [Induced Order](#)
- [Partial Order](#)
- [Injective Function](#)

Remark (Common error). It is tempting to think the induced order is automatically a partial order “because $\leq'$ is one.” It is not. The issue is antisymmetry: if f collapses two distinct points $x \neq y$ to the same or comparable images, we get $x \leq_f y$ and $y \leq_f x$ with $x \neq y$, violating antisymmetry. Reflexivity and transitivity lift freely; antisymmetry does not.

Example (Non-injective f destroys antisymmetry). Let $B = (\mathbb{N}, \leq)$ and let $f : \{a, b\} \rightarrow \mathbb{N}$ be the constant function $f(a) = f(b) = 0$. Then $a \leq_f b$ (since $0 \leq 0$) and $b \leq_f a$ (since $0 \leq 0$), but $a \neq b$. So $\leq_f$ is not antisymmetric, confirming that non-injectivity forces a failure.

Example (Injective f gives a genuine partial order). Let $B = (\mathcal{P}(\{1, 2, 3\}), \subseteq)$ and let $A = \{x, y, z\}$ with $f(x) = \{1\}$, $f(y) = \{2\}$, $f(z) = \{1, 2\}$. Then f is injective. The induced order has $x \leq_f z$ (since $\{1\} \subseteq \{1, 2\}$) and $y \leq_f z$ (since $\{2\} \subseteq \{1, 2\}$), but x and y are incomparable. This is a genuine partial order on A .

Definition (Suborder / Restriction)

Definition 10.1.40 (Suborder / Restriction). Let $(B, \leq')$ be a partially ordered set and $S \subseteq B$. The *suborder* on S (or *induced order on S*) is the relation $\leq'_S$ defined by:

$$x \leq'_S y \iff x \leq' y, \quad \text{for } x, y \in S.$$

Remark (Dependencies).

- [Partial Order](#)
- [Subset](#)
- [Relation](#)

Remark (Connection to induced orders). The suborder on $S \subseteq B$ is exactly the order induced by the inclusion map $\iota : S \hookrightarrow B$, $\iota(x) = x$. Since ι is injective, the suborder is always a partial order whenever $(B, \leq')$ is. This justifies speaking of “ S with the inherited order” without further verification.

Definition (Order Embedding)

Definition 10.1.41 (Order Embedding). Let $(A, \leq)$ and $(B, \leq')$ be partially ordered sets. A function $f : A \rightarrow B$ is an *order embedding* if for all $x, y \in A$:

$$x \leq y \iff f(x) \leq' f(y).$$

Remark (Dependencies).

- [Ordered Set](#)
- [Function](#)

Remark (Two directions, one condition). The $(\Rightarrow)$ direction says f is order-preserving (see [Definition \(Order-Preserving Map\)](#)). The $(\Leftarrow)$ direction — $f(x) \leq' f(y) \Rightarrow x \leq y$ — is called *order-reflection*. An order embedding both preserves and reflects the order, so f is a perfect local copy of $(A, \leq)$ inside $(B, \leq')$.

Remark (Relation to the induced order). If $f : A \rightarrow B$ is an order embedding from $(A, \leq)$ to $(B, \leq')$, then $\leq$ coincides with the induced order $\leq_f$:

$$x \leq y \iff f(x) \leq' f(y) \iff x \leq_f y.$$

In other words, an order embedding is exactly an injective function whose induced order agrees with the existing order on A .

Proposition (Order embeddings are injective)

Proposition 10.1.42 (Order embeddings are injective). *Every order embedding $f : (A, \leq) \rightarrow (B, \leq')$ is injective. Go to proof.*

Remark (Standard quantified statement). $\forall x, y \in A : f(x) = f(y) \rightarrow x = y$.

Remark (Dependencies).

- [Order Embedding](#)
- [Injective Function](#)
- [Partial Order](#)

Remark (Proof strategy). Injectivity falls out of the reflection direction: if two elements have the same image, reflection forces them to be mutually $\leq$ -related, and antisymmetry collapses them to equality.

Proposition (Order embedding is isomorphism onto image)

Proposition 10.1.43 (Order embedding is isomorphism onto image). *If $f : (A, \leq) \rightarrow (B, \leq')$ is an order embedding, then f is an order isomorphism from $(A, \leq)$ to the suborder $(f(A), \leq'_{f(A)})$. Go to proof.*

Remark (Standard quantified statement). $\forall x, y \in A : x \leq y \leftrightarrow f(x) \leq'_{f(A)} f(y)$, i.e. f is an order isomorphism $(A, \leq) \cong (f(A), \leq')$.

Remark (Dependencies).

- [Order Embedding](#)
- [Image of a Set](#)
- [Suborder](#)
- [Partial Order](#)

Remark (Consequence). This proposition is the order-theoretic analogue of the first isomorphism theorem: an order embedding always realises $(A, \leq)$ as an isomorphic copy of a subposet of $(B, \leq')$. When f is also surjective, the embedding becomes a full order isomorphism (see [Definition \(Order Isomorphism\)](#)).

Example (Standard embeddings). Each of the following is an order embedding:

- The inclusion $\mathbb{N} \hookrightarrow \mathbb{Z}$ with standard $\leq$ on both: $m \leq n \iff m \leq n$ trivially. The natural numbers sit inside the integers as an isomorphic copy of $(\mathbb{N}, \leq)$.
- The function $f : \mathbb{N} \rightarrow \mathbb{N}$, $f(n) = 2n$, embeds $(\mathbb{N}, \leq)$ into itself: $m \leq n \iff 2m \leq 2n$. The image is the even numbers with the inherited order.
- The map $A \mapsto A$ from $(\mathcal{P}(X), \subseteq)$ to itself is an order isomorphism (the identity), but any injective $g : \mathcal{P}(X) \rightarrow \mathcal{P}(X)$ that preserves and reflects $\subseteq$ is an order embedding.

Remark (Transition). Induced orders and embeddings are the order-theoretic counterpart of substructures in algebra. Just as a subgroup is a subset closed under the group operations, a sub-poset is a subset with the inherited order — and an order embedding is the morphism that witnesses this. In analysis, these ideas appear when restricting an order from $\mathbb{R}$ to subsets (intervals, sequences, function spaces), and when the completeness property is transferred between ordered structures.

10.2 Zorn

AC Equivalences — Quick Reference

Statement	Role
Axiom of Choice (AC)	Choice function from any collection of nonempty sets
Product form of AC	Cartesian product of nonempty sets is nonempty
Zorn's Lemma	Poset with every chain bounded $\Rightarrow$ maximal element
Hausdorff Maximum Principle	Every poset has a $\subseteq$ -maximal chain
Well-Ordering Theorem	Every set admits a well-ordering

All five statements are equivalent (over ZF set theory).

10.2.1 Zorn's Lemma and the Axiom of Choice

Remark (Why AC is non-constructive). For finite collections, the axiom is provable without extra assumptions (see SRF-STO-C08-S8.1). For infinite collections, no construction can produce the choice function: AC asserts its existence without exhibiting it. This non-constructive character is both AC's power and the source of the foundational controversy surrounding it.

Zorn's Lemma

Let $(P, \leq)$ be a poset in which every chain (loset) has an upper bound in P . Then P has at least one maximal element.

Remark (Reading Zorn's Lemma). The hypothesis “every chain has an upper bound” does *not* require the bound to lie in the chain itself. The conclusion gives a maximal element: an element above which nothing in P sits. In a partial order, there may be many maximal elements; Zorn guarantees at least one.

Remark (Typical proof pattern using Zorn's Lemma). To show an object of type X exists with a maximal property:

1. Partially order candidate objects by inclusion (or extension).
2. Verify every chain of candidates has an upper bound (usually its union).
3. Conclude by Zorn that a maximal candidate exists.

4. Show the maximal candidate has the desired property (often: if it lacked it, it could be enlarged, contradicting maximality).

This pattern recurs throughout algebra and analysis: maximal ideals, algebraic bases (Hamel bases), maximal consistent sets, and Szpilrajn's Theorem.

Hausdorff Maximum Principle

In every poset $(P, \leq)$ there exists a $\subseteq$ -maximal chain; that is, a chain $C \subseteq P$ such that no chain $C' \subseteq P$ strictly contains C .

Remark (Relation to Zorn). The Hausdorff Maximum Principle is an immediate corollary of Zorn's Lemma: take the poset of chains ordered by inclusion; every chain of chains is bounded by its union; Zorn gives a maximal chain. Conversely, Hausdorff implies Zorn. Both are equivalent to AC over ZF.

Remark (Application: Szpilrajn via Hausdorff). In the proof of Szpilrajn's Theorem ([↑ Thm](#)), let T_X be the set of all partial orders on X extending $\preceq$, ordered by $\subseteq$. By the Hausdorff Maximum Principle, T_X has a maximal chain A ; its union $\preceq^* = \bigcup A$ is a partial order extending $\preceq$. If $\preceq^*$ were not total, there would exist incomparable x, y and one could extend $\preceq^*$ by adding (x, y) (closing under transitivity), contradicting maximality of A . Hence $\preceq^*$ is a linear order. $\square$

Remark (Transition). Zorn's Lemma is the workhorse for existence proofs throughout abstract algebra and analysis. It first appears in the project context here and at the proof stubs in §11 of the Chapter VIII exercises.

Chapter 11

Cardinality



11.1 Cardinality

Cardinality — Quick Reference	
Concept	Meaning
Same cardinality	Bijection $f : A \rightarrow B$ exists; written $A \sim B$
Finite set	$A \sim \{1, \dots, n\}$ for some $n \in \mathbb{N}$, or $A = \emptyset$
Countably infinite	$A \sim \mathbb{N}$
Countable	Finite or countably infinite
Uncountable	Infinite but not countable
$ A \leq B $	Injection $A \hookrightarrow B$ exists
$\mathbb{Q}$ countable	Diagonal enumeration
$\mathbb{R}$ uncountable	Cantor diagonal argument
Schröder–Bernstein	$ A \leq B $ and $ B \leq A \Rightarrow A \sim B$
Cantor’s theorem	$ A < \mathcal{P}(A) $ for every set A
Countable union	Countable union of countable sets is countable

11.1.1 Cardinality and Infinite Sets

The intuitive idea of “size” works perfectly for finite sets: $\{a, b, c\}$ has three elements, $\{1, 2, 3, 4\}$ has four, and any two finite sets can be compared by counting. For infinite sets, counting breaks down, and a different foundation is needed. The key insight, due to Cantor, is that two sets have the “same size” if and only if their elements can be matched perfectly — one-to-one and onto — without any element left over on either side. This single idea, made precise through the notion of a bijection, gives a theory of size that applies uniformly to both finite and infinite sets, and produces genuinely surprising results: the integers and the

rational numbers turn out to have the same size as the natural numbers, while the real numbers are provably larger.

Definition (Same Cardinality)

Definition 11.1.1 (Same Cardinality). Two sets A and B have the *same cardinality*, written $A \sim B$, if there exists a bijection $f : A \rightarrow B$.

Remark (Standard quantified statement). $A \sim B \leftrightarrow \exists f : A \rightarrow B, (\forall a_1, a_2 \in A, f(a_1) = f(a_2) \rightarrow a_1 = a_2) \wedge (\forall b \in B, \exists a \in A, f(a) = b)$.

Remark (Dependencies).

- [Bijective Function](#)

Remark (Same cardinality is an equivalence relation). The relation $\sim$ is an equivalence relation on the class of all sets:

- (i) *Reflexive*: $A \sim A$ via id_A .
- (ii) *Symmetric*: if $f : A \rightarrow B$ is a bijection then $f^{-1} : B \rightarrow A$ is a bijection, so $A \sim B$ implies $B \sim A$.
- (iii) *Transitive*: if $f : A \rightarrow B$ and $g : B \rightarrow C$ are bijections then $g \circ f : A \rightarrow C$ is a bijection, so $A \sim B$ and $B \sim C$ imply $A \sim C$.

This is a genuine equivalence relation, so all the theory of equivalence classes developed in the previous section applies. The equivalence class of A under $\sim$ is called the *cardinality* of A , written $|A|$.

Example (Infinite sets can be equinumerous with proper subsets). Let $E = \{2, 4, 6, \dots\}$ be the set of positive even integers. The function $f : \mathbb{N} \rightarrow E$ defined by $f(n) = 2n$ is a bijection, so $\mathbb{N} \sim E$. This seems paradoxical: E is a proper subset of $\mathbb{N}$, yet the two sets have the same cardinality. This is not a contradiction but rather one of the defining characteristics of infinite sets — a phenomenon that simply cannot occur for finite sets.

Remark (Dedekind's characterization of infinite sets). The phenomenon in Example 5.1.1 is not an accident. An infinite set can always be matched bijectively with some proper subset of itself. This is sometimes taken as the *definition* of an infinite set (Dedekind's definition): a set is infinite if and only if it is equinumerous with a proper subset of itself.

Definition (Finite and Infinite Sets)

Definition 11.1.2 (Finite and Infinite Sets). A set A is *finite* if $A = \emptyset$ or $A \sim \{1, 2, \dots, n\}$ for some $n \in \mathbb{N}$. A set is *infinite* if it is not finite.

Remark (Standard quantified statement). A is finite $\leftrightarrow A = \emptyset \vee \exists n \in \mathbb{N}, \exists f : A \rightarrow \{1, \dots, n\}, f$ bijective. A is infinite $\leftrightarrow \neg(A \text{ is finite})$.

Remark (Dependencies).

- [Empty Set](#)
- [Same Cardinality](#)
- Natural Numbers

Definition (Countable and Uncountable Sets)

Definition 11.1.3 (Countable and Uncountable Sets). A set A is *countably infinite* if $A \sim \mathbb{N}$. A set is *countable* if it is finite or countably infinite. A set is *uncountable* if it is infinite but not countable.

Remark (Standard quantified statement). A is countably infinite $\leftrightarrow \exists f : A \rightarrow \mathbb{N}$, f bijective. A is countable $\leftrightarrow A$ finite $\vee A \sim \mathbb{N}$. A is uncountable $\leftrightarrow A$ infinite $\wedge \neg(A \sim \mathbb{N})$.

Remark (Dependencies).

- [Finite and Infinite Sets](#)
- [Same Cardinality](#)
- Natural Numbers

Remark (Countable means listable). A set A is countably infinite if and only if its elements can be arranged into an infinite list $a_1, a_2, a_3, \dots$ with no repetitions and every element of A appearing somewhere in the list. The bijection $f : \mathbb{N} \rightarrow A$ is precisely such a listing: $a_n = f(n)$. This is why countability is often informally described as “can be listed.”

Definition (Comparing Cardinalities)

Definition 11.1.4 (Comparing Cardinalities). Write $|A| \leq |B|$ if there exists an injection $f : A \hookrightarrow B$. Write $|A| < |B|$ if $|A| \leq |B|$ but $A \not\sim B$.

Remark (Standard quantified statement). $|A| \leq |B| \leftrightarrow \exists f : A \rightarrow B, \forall a_1, a_2 \in A, f(a_1) = f(a_2) \rightarrow a_1 = a_2$ (injection exists).

Remark (Dependencies).

- [Injective Function](#)
- [Same Cardinality](#)

Remark (Why injections, not surjections). An injection $A \hookrightarrow B$ places a “copy” of A inside B without overlap, which formalizes “ A is no larger than B .” The direction matters: an injection from A to B is not the same as an injection from B to A .

Theorem ($\mathbb{Q}$ is countable)

Theorem 11.1.5 ($\mathbb{Q}$ is countable). *The set $\mathbb{Q}$ of rational numbers is countable. Go to proof.*

Remark (Dependencies).

- [Countable Set](#)
- Natural Numbers
- [Bijective Function](#)

Remark (Proof strategy — diagonal enumeration). The argument arranges all positive rationals into an infinite two-dimensional array: place p/q (in lowest terms, $p, q > 0$) in row p , column q . Then traverse the array along successive diagonals — the anti-diagonal $p + q = k$ for $k = 2, 3, 4, \dots$ — skipping any fraction already seen. This produces an explicit list of all positive rationals, and one then interleaves the negative rationals and zero. The precise details are left as a guided proof exercise. The core idea is: a doubly infinite list can be converted into a singly infinite list by reading diagonals.

Remark (What this means). $\mathbb{Q}$ is dense in $\mathbb{R}$ — between any two real numbers there is a rational — yet $\mathbb{Q}$ is countable. This means the rationals are in some precise sense “thin” even though they are everywhere. The uncountability of $\mathbb{R}$ that follows shows the reals are genuinely larger.

Theorem (Countable union of countable sets is countable)

Theorem 11.1.6 (Countable union of countable sets is countable). *Let $\{A_n\}_{n \in \mathbb{N}}$ be a countable family of countable sets. Then $\bigcup_{n=1}^{\infty} A_n$ is countable. Go to proof.*

Remark (Dependencies).

- [Countable Set](#)
- Natural Numbers
- [Indexed Family of Sets](#)
- [Union](#)

Remark (Proof strategy). The argument again uses a diagonal enumeration. Arrange the elements of each A_n as a row in an infinite array (padding with repetitions if A_n is finite), then read the array along anti-diagonals. This produces a surjection from $\mathbb{N}$ onto $\bigcup A_n$, which is enough to establish countability. The formal proof is a standard exercise.

Remark (Why this theorem matters downstream). This result is used constantly. In topology: a countable union of nowhere-dense sets is called a *meager* set, and Baire’s theorem says a complete metric space is not meager — the proof requires knowing that countable unions behave predictably. In measure theory: a countable union of sets of measure zero has measure zero, by countable additivity. Whenever an argument says “take the union over all $n \in \mathbb{N}$ ” and needs the result to remain manageable in size, this theorem is the justification.

Theorem ($\mathbb{R}$ is uncountable)

Theorem 11.1.7 ($\mathbb{R}$ is uncountable). *The set $\mathbb{R}$ of real numbers is not countable. Go to proof.*

Remark (Dependencies).

- [Countable Set](#)
- [Finite and Infinite Sets](#)
- [Countable Union of Countable Sets](#)

Remark (Proof strategy — Cantor’s diagonal argument). This is one of the most important proof techniques in all of mathematics. Suppose, for contradiction, that $\mathbb{R}$ were countable, so its elements could be listed: $r_1, r_2, r_3, \dots$. Write each r_n in decimal expansion. Construct a new real number x by choosing the n -th decimal digit of x to differ from the n -th decimal digit of r_n (and avoiding 9 to prevent the $0.999\dots = 1$ ambiguity). Then $x \neq r_n$ for every n , because x and r_n differ in the n -th decimal place. So x is not on the list — contradicting the assumption that the list contains every real number.

The key move is the diagonal construction: to defeat every row r_n simultaneously, make x disagree with row n at position n . This “diagonal sweep” is a technique that reappears in logic (Gödel’s incompleteness theorem), computability theory (the halting problem), and the proof of Cantor’s theorem below.

Remark (Consequence). Since $\mathbb{Q}$ is countable and $\mathbb{R}$ is uncountable, the irrational numbers $\mathbb{R} \setminus \mathbb{Q}$ must be uncountable: if they were countable, then $\mathbb{R} = \mathbb{Q} \cup (\mathbb{R} \setminus \mathbb{Q})$ would be a countable union of countable sets, hence countable — contradiction.

Theorem (Cantor’s Theorem)

Theorem 11.1.8 (Cantor’s Theorem). *For any set A , there is no surjection $A \rightarrow \mathcal{P}(A)$. In particular, $|A| < |\mathcal{P}(A)|$. Go to proof.*

Remark (Standard quantified statement). $\forall A : \neg \exists f : A \rightarrow \mathcal{P}(A), \forall S \in \mathcal{P}(A), \exists a \in A, f(a) = S$. Equivalently: $\forall f : A \rightarrow \mathcal{P}(A), \exists D \in \mathcal{P}(A), \forall a \in A, f(a) \neq D$.

Remark (Dependencies).

- [Function](#)
- [Surjective Function](#)
- [Power Set](#)
- [Axiom Schema of Separation](#)

Remark (Proof strategy — the same diagonal idea). Suppose $f : A \rightarrow \mathcal{P}(A)$ is any function. Construct the set

$$D = \{a \in A \mid a \notin f(a)\}.$$

D is a well-defined subset of A , so $D \in \mathcal{P}(A)$. Claim: D is not in the image of f . For suppose $D = f(a_0)$ for some $a_0 \in A$. Then:

- if $a_0 \in D$, then by definition $a_0 \notin f(a_0) = D$ — contradiction.
- if $a_0 \notin D$, then by definition $a_0 \in f(a_0) = D$ — contradiction.

Either case is impossible, so $D \neq f(a_0)$ for every $a_0 \in A$, and f is not surjective. Since f was arbitrary, no surjection $A \rightarrow \mathcal{P}(A)$ exists.

Notice the structure: D is defined by “ $a \in D$ iff $a \notin f(a)$,” which is a diagonal construction — the membership of a in D is decided by looking at position (a, a) in the “membership table” of f , and then disagreeing. This is the abstract form of the same move used in the proof that $\mathbb{R}$ is uncountable.

Remark (A hierarchy of infinities). Cantor’s theorem implies there is no largest cardinality: for any set A , the power set $\mathcal{P}(A)$ is strictly larger. Starting from $\mathbb{N}$, the sets

$$\mathbb{N}, \quad \mathcal{P}(\mathbb{N}), \quad \mathcal{P}(\mathcal{P}(\mathbb{N})), \quad \dots$$

form a strictly increasing sequence of cardinalities. One can show $|\mathcal{P}(\mathbb{N})| = |\mathbb{R}|$ via the characteristic function bijection (described below), so the second level is already the cardinality of the continuum.

Definition (Function Space B^A)

Definition 11.1.9 (Function Space B^A). For sets A and B , the *function space* B^A denotes the set of all functions from A to B :

$$B^A := \{f : A \rightarrow B\}.$$

Remark (Dependencies).

- [Function](#)
- [Set and Membership](#)

Remark (Why the exponential notation). For finite sets with $|A| = m$ and $|B| = n$, the number of functions $A \rightarrow B$ is n^m (there are n choices for each of the m inputs). The notation B^A reflects this count. In particular, $|\{0, 1\}^A| = 2^{|A|}$ for finite A , which matches $|\mathcal{P}(A)| = 2^{|A|}$.

Remark (The characteristic function bijection). For any set A , the power set $\mathcal{P}(A)$ and the function space $\{0, 1\}^A$ have the same cardinality via the *characteristic function* bijection:

$$\mathcal{P}(A) \rightarrow \{0, 1\}^A, \quad S \mapsto \mathbf{1}_S,$$

where $\mathbf{1}_S(a) = 1$ if $a \in S$ and $\mathbf{1}_S(a) = 0$ if $a \notin S$. This bijection is why Cantor’s theorem is sometimes stated as: there is no surjection $A \rightarrow \{0, 1\}^A$. The notation 2^A is used interchangeably with $\{0, 1\}^A$, and $|\mathcal{P}(A)| = 2^{|A|}$ is the general statement.

In functional analysis, spaces of the form B^A — functions from A to B — are the central objects of study. Knowing they are sets (with a definite cardinality when A and B are concrete) is the set-theoretic foundation for everything that follows.

Theorem (Schröder–Bernstein Theorem)

Theorem 11.1.10 (Schröder–Bernstein Theorem). *If $|A| \leq |B|$ and $|B| \leq |A|$, then $A \sim B$.*

Equivalently: if there exist injections $f : A \hookrightarrow B$ and $g : B \hookrightarrow A$, then there exists a bijection $h : A \rightarrow B$. Go to proof.

Remark (Standard quantified statement). $(\exists f : A \hookrightarrow B) \wedge (\exists g : B \hookrightarrow A) \rightarrow \exists h : A \rightarrow B$, h bijective.

Remark (Dependencies).

- [Comparing Cardinalities](#)
- [Injective Function](#)
- [Bijective Function](#)

Remark (Why this theorem is indispensable). In practice, proving $A \sim B$ by exhibiting an explicit bijection can be very difficult. Schröder–Bernstein gives a two-injection strategy: find any injection each way, and the theorem guarantees a bijection exists (even if you cannot describe it explicitly). This is a powerful tool for cardinality computations. For example, to show $|(0, 1)| = |\mathbb{R}|$, find an injection $(0, 1) \hookrightarrow \mathbb{R}$ (inclusion) and an injection $\mathbb{R} \hookrightarrow (0, 1)$ (e.g., $x \mapsto \frac{1}{\pi} \arctan(x) + \frac{1}{2}$); the theorem does the rest.

Remark (Proof idea). The proof constructs h by partitioning A into two pieces: those elements whose “ancestry” under repeated application of $g \circ f$ is finite (they go through f), and those whose ancestry is infinite (they go through g^{-1}). The formal argument is a standard exercise; the key insight is that the two injections together cover A in a structured enough way to stitch together a bijection.

Remark (Sequences as functions). A *sequence* in a set X is a function $f : \mathbb{N} \rightarrow X$. The element $f(n)$ is written x_n , and the sequence is written $(x_n)_{n \in \mathbb{N}}$ or simply (x_n) . This is not a new object: it is an element of the function space $X^{\mathbb{N}}$. When metric spaces and topology discuss convergence of sequences $x_1, x_2, x_3, \dots$ in an arbitrary space X , the formal object is always a function $\mathbb{N} \rightarrow X$, and all the function machinery — injectivity, composition, preimages — applies to it directly.

Remark (Transition). Cardinality distinguishes between the countable (manageable, listable) and the uncountable (genuinely larger). This distinction organizes much of what follows. In metric spaces: separable spaces have countable dense subsets, and separability is a strong structural property. In measure theory: σ -algebras are closed under *countable* unions — not arbitrary ones — precisely because countable operations preserve the kind of control that makes measure theory work. In functional analysis: L^2 spaces have countable orthonormal bases (Hilbert spaces are separable), while non-separable spaces behave very differently. The line between countable and uncountable is not a technicality; it is one of the organizing principles of analysis.

Chapter 12

Euclidean Geometry



12.1 Foundations

Definitions

Remark (Source note). The following definitions are Euclid’s Book I definitions in the Heath translation [Euclid, 1956, Book I, Definitions 1–23].

Definition

Definition 12.1.1 (Point). A point is that which has no part.

Remark (Standard quantified statement). This is a classical Euclidean source-text clause. In this chapter it is treated as a named primitive statement rather than expanded into a modern first-order formalization.

Definition

Definition 12.1.2 (Line). A line is breadthless length.

Remark (Standard quantified statement). This is a classical Euclidean source-text clause. In this chapter it is treated as a named primitive statement rather than expanded into a modern first-order formalization.

Definition

Definition 12.1.3 (Ends of a line). The ends of a line are points.

Remark (Standard quantified statement). This is a classical Euclidean source-text clause. In this chapter it is treated as a named primitive statement rather than expanded into a

modern first-order formalization.

Definition

Definition 12.1.4 (Straight line). A straight line is a line which lies evenly with the points on itself.

Remark (Standard quantified statement). This is a classical Euclidean source-text clause. In this chapter it is treated as a named primitive statement rather than expanded into a modern first-order formalization.

Definition

Definition 12.1.5 (Surface). A surface is that which has length and breadth only.

Remark (Standard quantified statement). This is a classical Euclidean source-text clause. In this chapter it is treated as a named primitive statement rather than expanded into a modern first-order formalization.

Definition

Definition 12.1.6 (Edges of a surface). The edges of a surface are lines.

Remark (Standard quantified statement). This is a classical Euclidean source-text clause. In this chapter it is treated as a named primitive statement rather than expanded into a modern first-order formalization.

Definition

Definition 12.1.7 (Plane surface). A plane surface is a surface which lies evenly with the straight lines on itself.

Remark (Standard quantified statement). This is a classical Euclidean source-text clause. In this chapter it is treated as a named primitive statement rather than expanded into a modern first-order formalization.

Definition

Definition 12.1.8 (Plane angle). A plane angle is the inclination to one another of two lines in a plane which meet one another and do not lie in a straight line.

Remark (Standard quantified statement). This is a classical Euclidean source-text clause. In this chapter it is treated as a named primitive statement rather than expanded into a modern first-order formalization.

Definition

Definition 12.1.9 (Rectilinear angle). And when the lines containing the angle are straight, the angle is called rectilinear.

Remark (Standard quantified statement). This is a classical Euclidean source-text clause. In this chapter it is treated as a named primitive statement rather than expanded into a modern first-order formalization.

Definition

Definition 12.1.10 (Right angle and perpendicular). When a straight line standing on a straight line makes the adjacent angles equal to one another, each of the equal angles is right, and the straight line standing on the other is called a perpendicular to that on which it stands.

Remark (Standard quantified statement). This is a classical Euclidean source-text clause. In this chapter it is treated as a named primitive statement rather than expanded into a modern first-order formalization.

Definition

Definition 12.1.11 (Obtuse angle). An obtuse angle is an angle greater than a right angle.

Remark (Standard quantified statement). This is a classical Euclidean source-text clause. In this chapter it is treated as a named primitive statement rather than expanded into a modern first-order formalization.

Definition

Definition 12.1.12 (Acute angle). An acute angle is an angle less than a right angle.

Remark (Standard quantified statement). This is a classical Euclidean source-text clause. In this chapter it is treated as a named primitive statement rather than expanded into a modern first-order formalization.

Definition

Definition 12.1.13 (Boundary). A boundary is that which is an extremity of anything.

Remark (Standard quantified statement). This is a classical Euclidean source-text clause. In this chapter it is treated as a named primitive statement rather than expanded into a modern first-order formalization.

Definition

Definition 12.1.14 (Figure). A figure is that which is contained by any boundary or boundaries.

Remark (Standard quantified statement). This is a classical Euclidean source-text clause. In this chapter it is treated as a named primitive statement rather than expanded into a modern first-order formalization.

Definition

Definition 12.1.15 (Circle). A circle is a plane figure contained by one line such that all the straight lines falling upon it from one point among those lying within the figure equal one another.

Remark (Standard quantified statement). This is a classical Euclidean source-text clause. In this chapter it is treated as a named primitive statement rather than expanded into a modern first-order formalization.

Definition

Definition 12.1.16 (Center of a circle). And the point is called the center of the circle.

Remark (Standard quantified statement). This is a classical Euclidean source-text clause. In this chapter it is treated as a named primitive statement rather than expanded into a modern first-order formalization.

Definition

Definition 12.1.17 (Diameter). A diameter of the circle is any straight line drawn through the center and terminated in both directions by the circumference of the circle, and such a straight line also bisects the circle.

Remark (Standard quantified statement). This is a classical Euclidean source-text clause. In this chapter it is treated as a named primitive statement rather than expanded into a modern first-order formalization.

Definition

Definition 12.1.18 (Semicircle). A semicircle is the figure contained by the diameter and the circumference cut off by it. And the center of the semicircle is the same as that of the circle.

Remark (Standard quantified statement). This is a classical Euclidean source-text clause. In this chapter it is treated as a named primitive statement rather than expanded into a modern first-order formalization.

Definition

Definition 12.1.19 (Rectilinear figures). Rectilinear figures are those which are contained by straight lines, trilateral figures being those contained by three, quadrilateral those contained by four, and multilateral those contained by more than four straight lines.

Remark (Standard quantified statement). This is a classical Euclidean source-text clause. In this chapter it is treated as a named primitive statement rather than expanded into a modern first-order formalization.

Definition

Definition 12.1.20 (Equilateral, isosceles, and scalene triangles). Of trilateral figures, an equilateral triangle is that which has its three sides equal, an isosceles triangle that which has two of its sides alone equal, and a scalene triangle that which has its three sides unequal.

Remark (Standard quantified statement). This is a classical Euclidean source-text clause. In this chapter it is treated as a named primitive statement rather than expanded into a modern first-order formalization.

Definition

Definition 12.1.21 (Right-angled, obtuse-angled, and acute-angled triangles). Further, of trilateral figures, a right-angled triangle is that which has a right angle, an obtuse-angled triangle that which has an obtuse angle, and an acute-angled triangle that which has its three angles acute.

Remark (Standard quantified statement). This is a classical Euclidean source-text clause. In this chapter it is treated as a named primitive statement rather than expanded into a modern first-order formalization.

Definition

Definition 12.1.22 (Quadrilateral classes). Of quadrilateral figures, a square is that which is both equilateral and right-angled; an oblong that which is right-angled but not equilateral; a rhombus that which is equilateral but not right-angled; and a rhomboid that which has its opposite sides and angles equal to one another but is neither equilateral nor right-angled. And let quadrilaterals other than these be called trapezia.

Remark (Standard quantified statement). This is a classical Euclidean source-text clause. In this chapter it is treated as a named primitive statement rather than expanded into a modern first-order formalization.

Definition

Definition 12.1.23 (Parallel straight lines). Parallel straight lines are straight lines which, being in the same plane and being produced indefinitely in both directions, do not meet one another in either direction.

Remark (Standard quantified statement). This is a classical Euclidean source-text clause. In this chapter it is treated as a named primitive statement rather than expanded into a modern first-order formalization.

Postulates

Remark (Source note). The following are Euclid's five postulates from Book I of the *Elements* in the Heath translation [Euclid, 1956, Book I, Postulates 1–5].

Remark (Exposition). Postulates are geometry-specific starting assumptions. They say which constructions are permitted and which basic geometric facts may be used without proof. In Euclid's system, these are not conclusions reached from earlier results; they are part of the foundation from which the theory begins.

Later propositions are proved from the definitions, the postulates, the common notions, and propositions already established. Thus the postulates are the rules of play for Euclidean geometry: they determine what can be constructed, what may be assumed about straight lines and circles, and where the theory's geometric content enters before proof begins.

Axiom

Axiom 12.1.24 (Euclid Postulate I: Drawing a straight line). Let it have been postulated to draw a straight line from any point to any point.

Remark (Standard quantified statement). This is a classical Euclidean source-text clause. In this chapter it is treated as a named primitive statement rather than expanded into a modern first-order formalization.

Axiom

Axiom 12.1.25 (Euclid Postulate II: Producing a finite straight line). Let it have been postulated to produce a finite straight line continuously in a straight line.

Remark (Standard quantified statement). This is a classical Euclidean source-text clause. In this chapter it is treated as a named primitive statement rather than expanded into a modern first-order formalization.

Axiom

Axiom 12.1.26 (Euclid Postulate III: Describing a circle). Let it have been postulated to describe a circle with any center and distance.

Remark (Standard quantified statement). This is a classical Euclidean source-text clause. In this chapter it is treated as a named primitive statement rather than expanded into a modern first-order formalization.

Axiom

Axiom 12.1.27 (Euclid Postulate IV: Equality of right angles). Let it have been postulated that all right angles are equal to one another.

Remark (Standard quantified statement). This is a classical Euclidean source-text clause. In this chapter it is treated as a named primitive statement rather than expanded into a modern first-order formalization.

Axiom

Axiom 12.1.28 (Euclid Postulate V: Parallel postulate). Let it have been postulated that, if a straight line falling on two straight lines makes the interior angles on the same side less than two right angles, the two straight lines, if produced indefinitely, meet on that side on which are the angles less than the two right angles.

Remark (Standard quantified statement). This is a classical Euclidean source-text clause. In this chapter it is treated as a named primitive statement rather than expanded into a modern first-order formalization.

Axioms

Remark (Source note). Euclid calls the following principles *common notions*. They function as general axioms governing equality and comparison in Book I of the *Elements* [Euclid, 1956, Book I, Common Notions 1–5].

Axiom

Axiom 12.1.29 (Euclid Common Notion I: Equality is transitive through a common equal). Things which are equal to the same thing are also equal to one another.

Remark (Standard quantified statement). This is a classical Euclidean source-text clause. In this chapter it is treated as a named primitive statement rather than expanded into a modern first-order formalization.

Axiom

Axiom 12.1.30 (Euclid Common Notion II: Adding equals to equals). If equals be added to equals, the wholes are equal.

Remark (Standard quantified statement). This is a classical Euclidean source-text clause. In this chapter it is treated as a named primitive statement rather than expanded into a modern first-order formalization.

Axiom

Axiom 12.1.31 (Euclid Common Notion III: Subtracting equals from equals). If equals be subtracted from equals, the remainders are equal.

Remark (Standard quantified statement). This is a classical Euclidean source-text clause. In this chapter it is treated as a named primitive statement rather than expanded into a modern first-order formalization.

Axiom

Axiom 12.1.32 (Euclid Common Notion IV: Coincidence implies equality). Things which coincide with one another are equal to one another.

Remark (Standard quantified statement). This is a classical Euclidean source-text clause. In this chapter it is treated as a named primitive statement rather than expanded into a modern first-order formalization.

Axiom

Axiom 12.1.33 (Euclid Common Notion V: Whole and part). The whole is greater than the part.

Remark (Standard quantified statement). This is a classical Euclidean source-text clause. In this chapter it is treated as a named primitive statement rather than expanded into a modern first-order formalization.

Remark (Exposition). Common notions are general principles of reasoning rather than specifically geometric assumptions. They express basic behavior of equality, comparison, and logical transfer: equal things may be substituted for equal things, adding or subtracting equals preserves equality, and a whole exceeds its part.

Because these principles are not tied to points, lines, circles, or planes, they can be used throughout mathematics, not only in geometry. In modern language, they play the role of background axioms governing how equality and comparison may be used inside proofs.

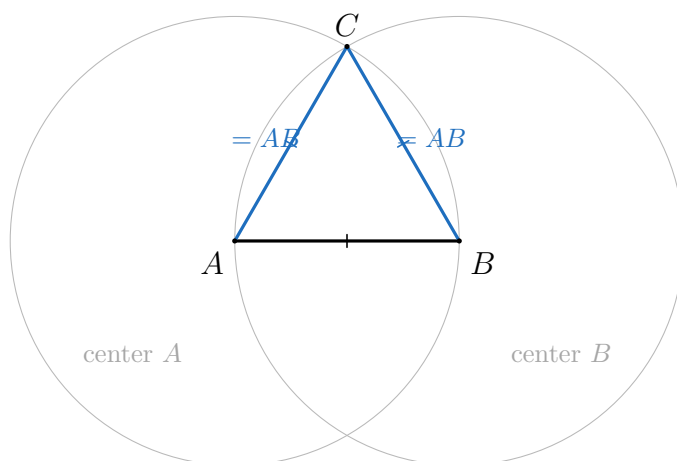
12.2 Compass-and-Straightedge Constructions

Propositions

Remark (Source note). The following propositions are selected from Euclid's Book I in modern theorem form, following the Heath translation of the *Elements* [Euclid, 1956, Book I]. The selection is curated for the trigonometric construction: congruence, parallel-angle control, triangle angle-sum, area comparison, square construction, and the Pythagorean theorem.

Theorem

Theorem 12.2.1 (Euclid I.1: Constructing an equilateral triangle). *Given a finite straight line segment AB , there exists an equilateral triangle ABC constructed on AB .*

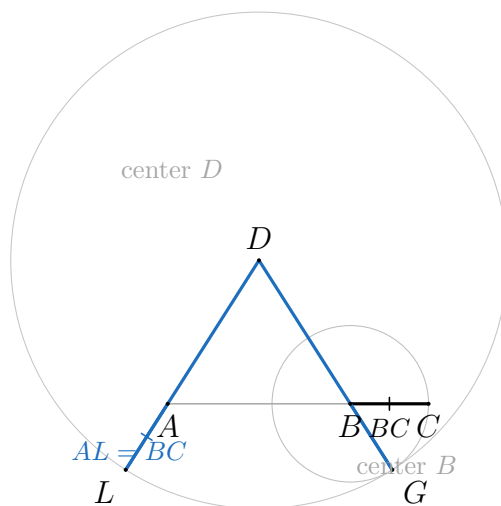


Remark (Proof). Go to proof.

Remark (Standard quantified statement). This is a classical Euclidean source-text clause. In this chapter it is treated as a named primitive statement rather than expanded into a modern first-order formalization.

Theorem

Theorem 12.2.2 (Euclid I.2: Copying a segment from a given point). *Given a point A and a finite straight line segment BC , there exists a straight line segment beginning at A and equal to BC .*

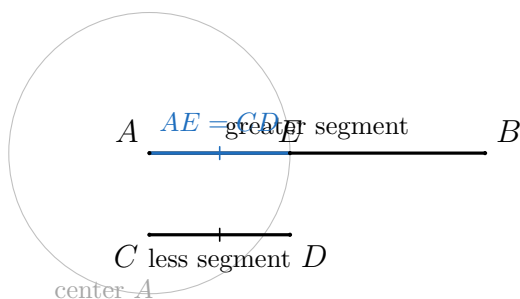


Remark (Proof). Go to proof.

Remark (Standard quantified statement). This is a classical Euclidean source-text clause. In this chapter it is treated as a named primitive statement rather than expanded into a modern first-order formalization.

Theorem

Theorem 12.2.3 (Euclid I.3: Cutting off an equal segment). *Given two unequal finite straight line segments, there exists a construction which cuts off from the greater a straight line segment equal to the less.*

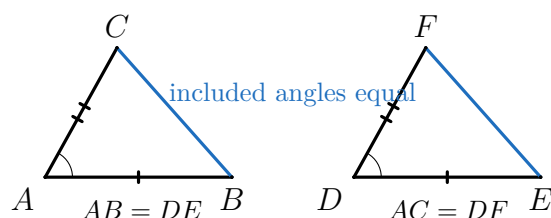


Remark (Proof). Go to proof.

Remark (Standard quantified statement). This is a classical Euclidean source-text clause. In this chapter it is treated as a named primitive statement rather than expanded into a modern first-order formalization.

Theorem

Theorem 12.2.4 (Euclid I.4: Side-angle-side congruence). *If two triangles have two sides equal to two sides respectively, and have the angles contained by the equal straight lines equal, then they also have the base equal to the base, the triangle equal to the triangle, and the remaining angles equal to the remaining angles respectively.*

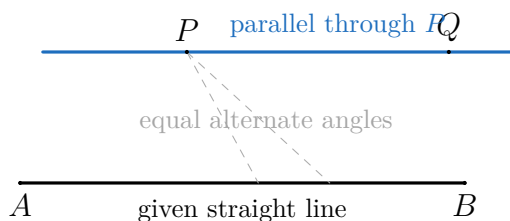


Remark (Proof). Go to proof.

Remark (Standard quantified statement). This is a classical Euclidean source-text clause. In this chapter it is treated as a named primitive statement rather than expanded into a modern first-order formalization.

Theorem

Theorem 12.2.5 (Euclid I.31: Drawing a parallel through a point). *Through a given point, to draw a straight line parallel to a given straight line.*

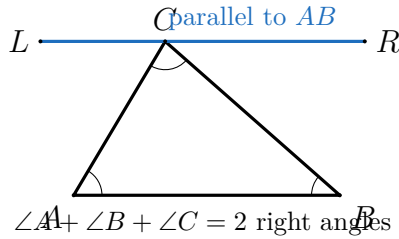


Remark (Proof). Go to proof.

Remark (Standard quantified statement). This is a classical Euclidean source-text clause. In this chapter it is treated as a named primitive statement rather than expanded into a modern first-order formalization.

Theorem

Theorem 12.2.6 (Euclid I.32: Triangle angle sum). *In any triangle, if one of the sides is produced, then the exterior angle is equal to the two interior and opposite angles, and the three interior angles of the triangle are equal to two right angles.*

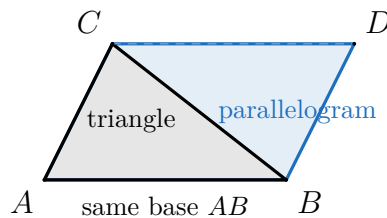


Remark (Proof). Go to proof.

Remark (Standard quantified statement). This is a classical Euclidean source-text clause. In this chapter it is treated as a named primitive statement rather than expanded into a modern first-order formalization.

Theorem

Theorem 12.2.7 (Euclid I.41: Triangle and parallelogram on the same base). *If a parallelogram has the same base with a triangle and is in the same parallels, then the parallelogram is double the triangle.*

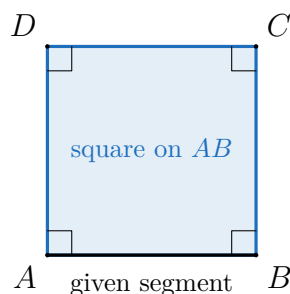


Remark (Proof). Go to proof.

Remark (Standard quantified statement). This is a classical Euclidean source-text clause. In this chapter it is treated as a named primitive statement rather than expanded into a modern first-order formalization.

Theorem

Theorem 12.2.8 (Euclid I.46: Constructing a square on a segment). *On a given straight line, to describe a square.*

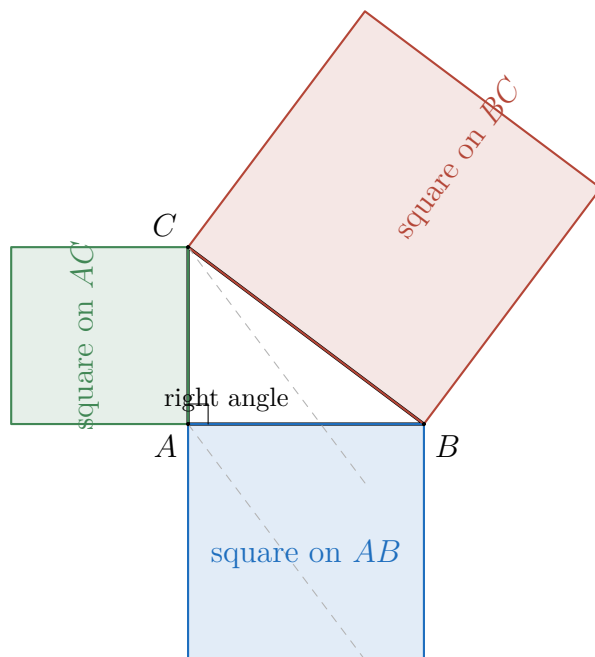


Remark (Proof). Go to proof.

Remark (Standard quantified statement). This is a classical Euclidean source-text clause. In this chapter it is treated as a named primitive statement rather than expanded into a modern first-order formalization.

Theorem

Theorem 12.2.9 (Euclid I.47: Pythagorean theorem). *In right-angled triangles, the square on the side subtending the right angle is equal to the squares on the sides containing the right angle.*



Remark (Proof). Go to proof.

Remark (Placement). This proposition is pulled forward as the Book I capstone needed by the trigonometric definitions. Its professional proof depends on later Book I propositions that are not yet developed in this chapter; the proof file records those pending Euclidean nodes explicitly.

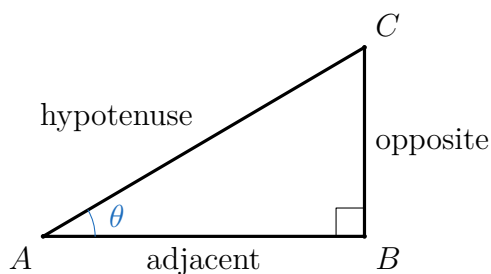
Chapter 13

Trigonometry



13.0.1 Trigonometric Functions from Euclidean Geometry

Remark (Geometric intuition). A right triangle fixes three side lengths relative to an acute angle: opposite, adjacent, and hypotenuse. Similar right triangles may be larger or smaller, but their corresponding side ratios are unchanged. This invariance is what makes a trigonometric ratio a function of the angle rather than a feature of one drawing.



Definition

Definition 13.0.1 (Right-triangle trigonometric ratios). Let T be a right triangle and let θ be one of its acute angles. Write

$$\text{opp}(T, \theta), \quad \text{adj}(T, \theta), \quad \text{hyp}(T)$$

for the side lengths opposite θ , adjacent to θ , and opposite the right angle, respectively. Define

$$\begin{aligned} \sin \theta &= \frac{\text{opp}(T, \theta)}{\text{hyp}(T)}, & \cos \theta &= \frac{\text{adj}(T, \theta)}{\text{hyp}(T)}, & \tan \theta &= \frac{\text{opp}(T, \theta)}{\text{adj}(T, \theta)}, \\ \cot \theta &= \frac{\text{adj}(T, \theta)}{\text{opp}(T, \theta)}, & \sec \theta &= \frac{\text{hyp}(T)}{\text{adj}(T, \theta)}, & \csc \theta &= \frac{\text{hyp}(T)}{\text{opp}(T, \theta)}. \end{aligned}$$

Remark (Standard quantified statement). For every right triangle T and every acute angle θ of T , the six displayed ratios are defined whenever the denominator in the displayed quotient is nonzero.

Remark (Primitive and derived ratios). The primitive geometric ratios are sine and cosine. Tangent, cotangent, secant, and cosecant are convenient quotient or reciprocal ratios built from the same three side lengths.

Remark (Dependencies).

- [Right-angled triangle](#)
- [Right angle and perpendicular](#)

Definition

Definition 13.0.2 (Sine and cosine). For an acute Euclidean angle θ , choose any right triangle T having θ as an acute angle and set

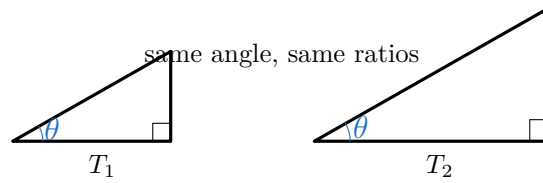
$$\sin \theta = \frac{\text{opp}(T, \theta)}{\text{hyp}(T)}, \quad \cos \theta = \frac{\text{adj}(T, \theta)}{\text{hyp}(T)}.$$

Remark (Standard quantified statement). For every acute Euclidean angle θ , if T is a right triangle having θ as an acute angle, then $\sin \theta$ and $\cos \theta$ are the opposite-over-hypotenuse and adjacent-over-hypotenuse ratios of T .

Remark (Well-definedness debt). This definition depends on the similarity-invariance theorem below. Until that theorem is invoked, the displayed formulas describe a ratio in a chosen triangle, not yet a function of θ alone.

Remark (Dependencies).

- [Right-triangle trigonometric ratios](#)



Theorem

Theorem 13.0.3 (Similarity invariance of trigonometric ratios). *If two right triangles have the same acute angle θ , then their corresponding opposite-to-hypotenuse and adjacent-to-hypotenuse ratios are equal.*

Remark (Proof). Go to proof.

Remark (Standard quantified statement). For all right triangles T_1, T_2 and every acute angle θ , if θ is an acute angle of both triangles, then

$$\frac{\text{opp}(T_1, \theta)}{\text{hyp}(T_1)} = \frac{\text{opp}(T_2, \theta)}{\text{hyp}(T_2)} \quad \text{and} \quad \frac{\text{adj}(T_1, \theta)}{\text{hyp}(T_1)} = \frac{\text{adj}(T_2, \theta)}{\text{hyp}(T_2)}.$$

Remark (Dependencies).

- [Euclid I.32](#)

Theorem

Theorem 13.0.4 (Well-definedness of sine and cosine). *For each acute Euclidean angle θ , the values $\sin \theta$ and $\cos \theta$ are independent of the right triangle chosen to compute them.*

Remark (Proof). Go to proof.

Remark (Dependencies).

- [Sine and cosine](#)
- [Similarity invariance of trigonometric ratios](#)

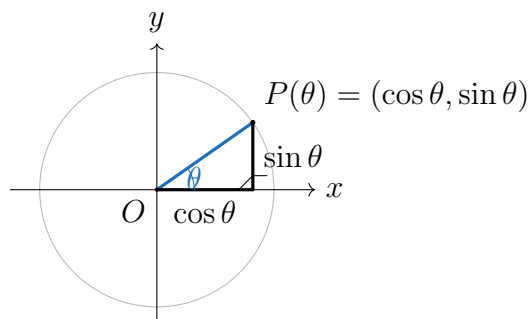
Theorem

Theorem 13.0.5 (Six trigonometric functions). *The six right-triangle trigonometric ratios define functions of an acute Euclidean angle, subject to the denominator restrictions in their defining quotients.*

Remark (Proof). Go to proof.

Remark (Dependencies).

- [Right-triangle trigonometric ratios](#)
- [Well-definedness of sine and cosine](#)



Definition

Definition 13.0.6 (Unit-circle sine and cosine). Let θ be an angle in standard position, and let $P(\theta) = (x_\theta, y_\theta)$ be the point where its terminal ray meets the unit circle. Define

$$\cos \theta = x_\theta, \quad \sin \theta = y_\theta.$$

Remark (Debt flag). This definition extends sine and cosine beyond acute angles. Treating θ as an arbitrary real-valued radian input requires arc length and rectifiability machinery not yet developed in this volume.

Remark (Dependencies).

- [Circle](#)
- [Cartesian Product](#)
- [Well-definedness of sine and cosine](#)

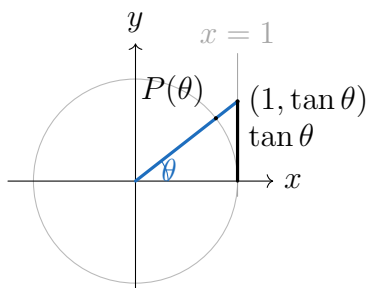
Theorem

Theorem 13.0.7 (Unit-circle agreement with right-triangle trigonometry). *For $0 < \theta < 90^\circ$, the unit-circle definitions of sine and cosine agree with the right-triangle definitions.*

Remark (Proof). Go to proof.

Remark (Dependencies).

- [Sine and cosine](#)
- [Unit-circle sine and cosine](#)



Definition

Definition 13.0.8 (Extended tangent, cotangent, secant, and cosecant). Where the displayed denominators are nonzero, define

$$\tan \theta = \frac{\sin \theta}{\cos \theta}, \quad \cot \theta = \frac{\cos \theta}{\sin \theta}, \quad \sec \theta = \frac{1}{\cos \theta}, \quad \csc \theta = \frac{1}{\sin \theta}.$$

Remark (Geometric reading). The tangent of θ is the signed height where the terminal ray meets the vertical tangent line $x = 1$.

Remark (Dependencies).

- [Unit-circle sine and cosine](#)
- [Unit-circle agreement with right-triangle trigonometry](#)

Definition

Definition 13.0.9 (Radian measure). One radian is the angle that cuts off arc length 1 on the unit circle. A full turn has measure 2π radians.

Remark (Debt flag). This definition uses arc length intuitively. Its rigorous foundation belongs after rectifiable curves and the construction of π from circumference.

Remark (Dependencies).

- [Unit-circle sine and cosine](#)
- [Circle](#)

Theorem

Theorem 13.0.10 (Symmetry and periodicity of sine and cosine). *For every angle in the unit-circle domain,*

$$\cos(-\theta) = \cos \theta, \quad \sin(-\theta) = -\sin \theta,$$

and one full turn returns the same sine and cosine values.

Remark (Proof). Go to proof.

Theorem

Theorem 13.0.11 (Pythagorean identity). *For every angle in the unit-circle domain,*

$$\cos^2 \theta + \sin^2 \theta = 1.$$

Remark (Proof). Go to proof.

Remark (Dependencies).

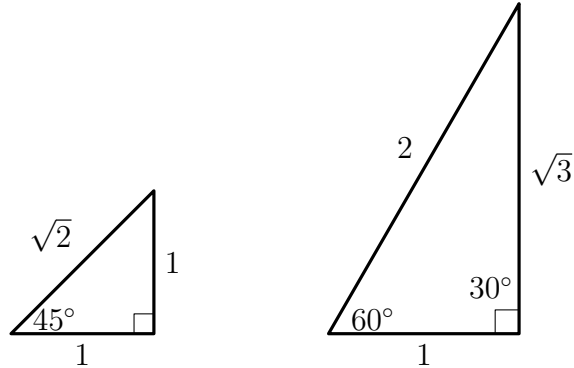
- [Euclid I.47](#)

- Unit-circle sine and cosine

Theorem

Theorem 13.0.12 (Angle-sum and difference identities). *The sine and cosine of a sum or difference of angles are determined by the standard angle-sum and angle-difference formulas.*

Remark (Proof). Go to proof.



Remark (Special angles). The 45° and 30° – 60° values come from the isosceles right triangle and the bisected equilateral triangle. The radian entries use the radian convention above and inherit its arc-length debt.

Chapter 14

Analytical Geometry



14.1 Analytical Geometry

Remark (Status). Notes, proofs, and exercises will appear here in a future revision.

14.2 Lines

Definition

Definition 14.2.1 (Point-slope form). Let $P_1 = (x_1, y_1)$ be a point in the Euclidean plane and let $m \in \mathbb{R}$. The point-slope form of the line through P_1 with slope m is

$$y - y_1 = m(x - x_1).$$

Remark (Standard quantified statement). For every point $P_1 = (x_1, y_1) \in \mathbb{R}^2$ and every $m \in \mathbb{R}$, the corresponding line is the set of all $(x, y) \in \mathbb{R}^2$ satisfying $y - y_1 = m(x - x_1)$.

Remark (Derivation). The formula records the condition that the slope from P_1 to an arbitrary point (x, y) on the line is constant and equal to m .

Remark (Dependencies).

- Real Numbers
- [Cartesian Product](#)
- Subtraction on $\mathbb{R}$
- Multiplication on $\mathbb{R}$

Definition

Definition 14.2.2 (Two-point form). Let $P_1 = (x_1, y_1)$ and $P_2 = (x_2, y_2)$ be distinct points with $x_1 \neq x_2$. The two-point form of the line through P_1 and P_2 is

$$y - y_1 = \left(\frac{y_2 - y_1}{x_2 - x_1} \right) (x - x_1).$$

Remark (Standard quantified statement). For all distinct points $P_1 = (x_1, y_1)$ and $P_2 = (x_2, y_2)$ in $\mathbb{R}^2$ with $x_1 \neq x_2$, the line through P_1 and P_2 is the set of all $(x, y) \in \mathbb{R}^2$ satisfying the displayed two-point equation.

Remark (Dependencies).

- [Point-Slope Form](#)
- Subtraction on $\mathbb{R}$
- $\mathbb{R}$ Is a Field

Definition

Definition 14.2.3 (General linear form). The general linear form of a line in $\mathbb{R}^2$ is

$$Ax + By + C = 0,$$

where $A, B, C \in \mathbb{R}$ and $A^2 + B^2 \neq 0$.

Remark (Standard quantified statement). For all $A, B, C \in \mathbb{R}$ with $A^2 + B^2 \neq 0$, the equation $Ax + By + C = 0$ determines a nondegenerate line in $\mathbb{R}^2$.

Remark (Interpretation). The vector (A, B) is normal to the line. This form includes vertical lines and therefore avoids the exceptional cases present in slope-based forms.

Remark (Dependencies).

- Real Numbers
- Addition on $\mathbb{R}$
- Multiplication on $\mathbb{R}$
- $\mathbb{R}$ Is a Field

Definition

Definition 14.2.4 (Vector parametric form). A line in $\mathbb{R}^2$ may be represented as the image of $\mathbf{r} : \mathbb{R} \rightarrow \mathbb{R}^2$ given by

$$\mathbf{r}(t) = \mathbf{r}_0 + t\mathbf{v},$$

where $\mathbf{r}_0 \in \mathbb{R}^2$ is a point on the line and $\mathbf{v} \in \mathbb{R}^2$ is a nonzero direction vector.

Remark (Standard quantified statement). For every $\mathbf{r}_0 \in \mathbb{R}^2$ and every nonzero $\mathbf{v} \in \mathbb{R}^2$, the set $\{\mathbf{r}_0 + t\mathbf{v} : t \in \mathbb{R}\}$ is a line in $\mathbb{R}^2$.

Remark (Dependencies).

- Real Numbers
- [Cartesian Product](#)
- [Function](#)
- Addition on $\mathbb{R}$
- Multiplication on $\mathbb{R}$

Definition

Definition 14.2.5 (Intercept form). If a line meets the x -axis at $(a, 0)$ and the y -axis at $(0, b)$ with $a, b \neq 0$, its intercept form is

$$\frac{x}{a} + \frac{y}{b} = 1.$$

Remark (Standard quantified statement). For all nonzero $a, b \in \mathbb{R}$, the line with intercepts $(a, 0)$ and $(0, b)$ is the set of all $(x, y) \in \mathbb{R}^2$ satisfying $x/a + y/b = 1$.

Remark (Derivation). The intercept form follows by applying the two-point form to the points $(a, 0)$ and $(0, b)$ and then rearranging the resulting equation.

Remark (Dependencies).

- [Two-Point Form](#)
- $\mathbb{R}$ Is a Field

Definition

Definition 14.2.6 (Affine parametric line in $\mathbb{R}^n$). Given distinct points $\mathbf{x}_1, \mathbf{x}_2 \in \mathbb{R}^n$, the line through them is

$$\mathbf{x}(t) = \mathbf{x}_1 + t(\mathbf{x}_2 - \mathbf{x}_1), \quad t \in \mathbb{R}.$$

Remark (Standard quantified statement). For all distinct $\mathbf{x}_1, \mathbf{x}_2 \in \mathbb{R}^n$, the set $\{\mathbf{x}_1 + t(\mathbf{x}_2 - \mathbf{x}_1) : t \in \mathbb{R}\}$ is the affine line through $\mathbf{x}_1$ and $\mathbf{x}_2$.

Remark (Interpretation). For $0 \leq t \leq 1$, the same formula describes the line segment joining $\mathbf{x}_1$ and $\mathbf{x}_2$. For arbitrary real t , it describes the entire affine line.

Remark (Dependencies).

- Real Numbers
- [Cartesian Product](#)

- Subtraction on $\mathbb{R}$
- Addition on $\mathbb{R}$
- Multiplication on $\mathbb{R}$

Definition

Definition 14.2.7 (Normal form). A line in $\mathbb{R}^2$ may be written in normal form as

$$x \cos \alpha + y \sin \alpha - p = 0,$$

where $p \geq 0$ is the distance from the origin to the line.

Remark (Standard quantified statement). For every $p \geq 0$ and every real angle α , the equation $x \cos \alpha + y \sin \alpha - p = 0$ describes a line whose unit normal direction is $(\cos \alpha, \sin \alpha)$.

Remark (Dependencies).

- [General Linear Form](#)
- [Sine and Cosine](#)
- Order on $\mathbb{R}$

Definition

Definition 14.2.8 (Symmetric form). Given a point (x_1, y_1) and direction cosines $(\cos \theta, \sin \theta)$, the symmetric form of a line is

$$\frac{x - x_1}{\cos \theta} = \frac{y - y_1}{\sin \theta} = r.$$

Remark (Standard quantified statement). For every point (x_1, y_1) and every direction with nonzero displayed denominators, the symmetric equation describes the same line as the parametric equations $x = x_1 + r \cos \theta$ and $y = y_1 + r \sin \theta$.

Remark (Dependencies).

- [Vector Parametric Form](#)
- [Sine and Cosine](#)
- $\mathbb{R}$ Is a Field

Appendix A

Lean 4 Formalization



This generated appendix is reserved for Lean 4 statements and exposition emitted from the `lra-lean` overlay.

Bibliography

Euclid. *The Thirteen Books of Euclid's Elements*. Dover Publications, 1956.

Alfred Tarski. *Introduction to Logic and to the Methodology of Deductive Sciences*. Oxford University Press, 1941.